

软件分析

南京大学

计算机科学与技术系

程序设计语言与

静态分析研究组

李棣 谭添

Static Program Analysis

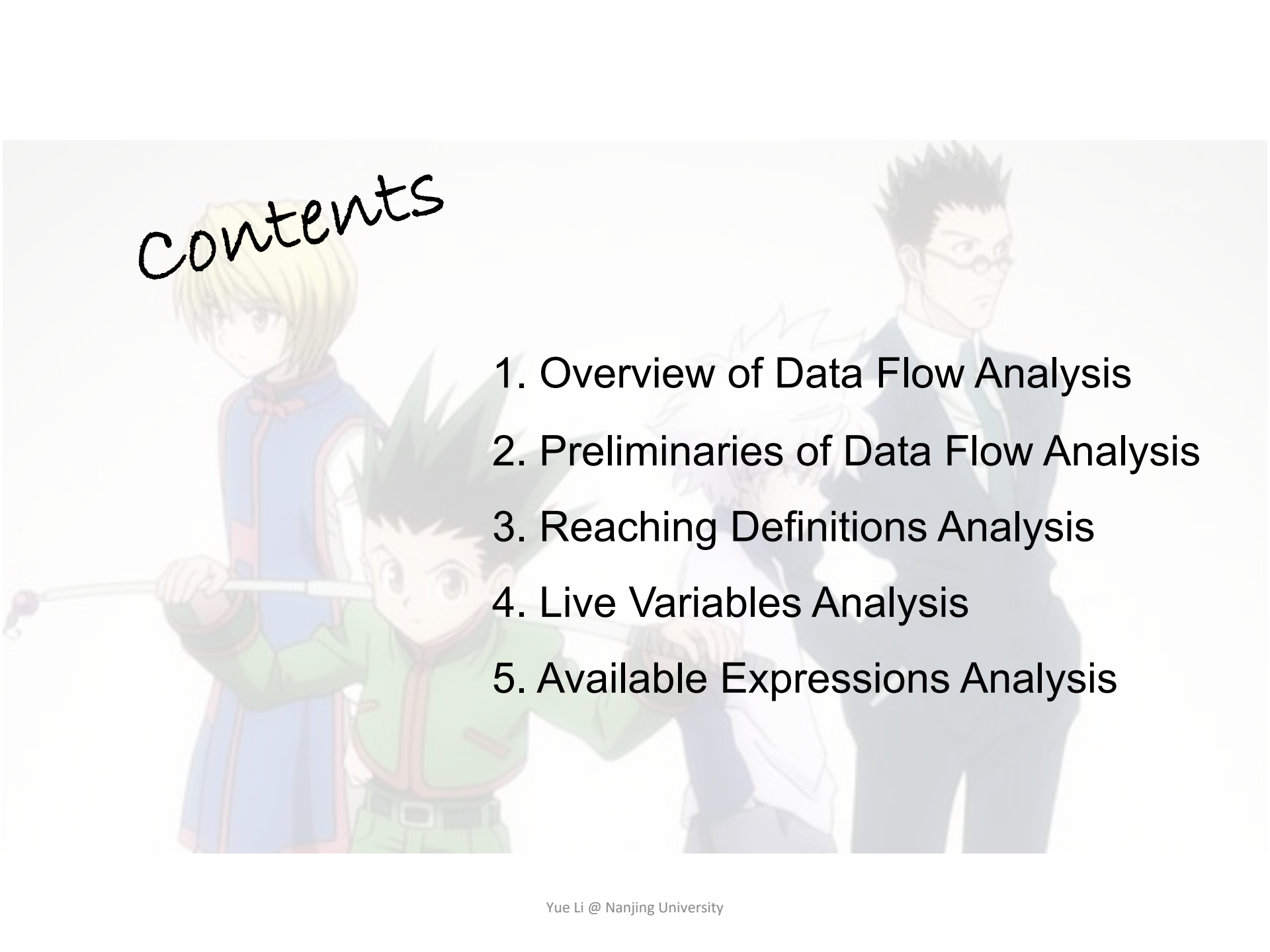
Data Flow Analysis — Applications

Nanjing University

Yue Li

2022

Contents



1. Overview of Data Flow Analysis
2. Preliminaries of Data Flow Analysis
3. Reaching Definitions Analysis
4. Live Variables Analysis
5. Available Expressions Analysis

Data Flow Analysis

Data Flow Analysis

How Data Flows on CFG?

Data Flow Analysis

How Data Flows on CFG?

How Data
Flows through the

CFG (a program)?

Data Flow Analysis

How Data Flows on CFG?

How application-specific Data
Flows through the

CFG (a program)?

Data Flow Analysis

How Data Flows on CFG?

How application-specific Data

Flows through the

Nodes (BBs/statements) and

Edges (control flows) of

CFG (a program)?

Recall 1st
lecture

Data Flow Analysis

How Data Flows on CFG?

How application-specific Data ← Abstraction

Over-approximation → Flows through the

Nodes (BBs/statements) and

Edges (control flows) of

CFG (a program)?

Recall 1st
lecture

Data Flow Analysis

How Data Flows on CFG?

How application-specific Data ← Abstraction

Over-approximation → Flows through the

for most static analyses
(may analysis)

Nodes (BBs/statements) and
Edges (control flows) of
CFG (a program)?

Recall 1st
lecture

Data Flow Analysis

How Data Flows on CFG?

How application-specific Data ← Abstraction

Over-approximation → Flows through the

for most static analyses
(may analysis)

Nodes (BBs/statements) and
Edges (control flows) of
CFG (a program)?

may analysis:

outputs information that may be true (over-approximation)

must analysis:

outputs information that must be true (under-approximation)

Over- and under-approximations are both for safety of analysis

Data Flow Analysis

How Data Flows on CFG?

How application-specific Data ← Abstraction

Safe-approximation → Flows through the

may analysis: safe=over

must analysis: safe=under

Nodes (BBs/statements) and

Edges (control flows) of
CFG (a program)?

Data Flow Analysis

How Data Flows on CFG?

How application-specific Data ← Abstraction

Safe-approximation → Flows through the

Transfer function → Nodes (BBs/statements) and

Control-flow handling → Edges (control flows) of
CFG (a program)?

Recall 1st
lecture

Data Flow Analysis

How Data Flows on CFG?

How application-specific Data ← Abstraction

Safe-approximation → Flows through the

Transfer function → Nodes (BBs/statements) and

Control-flow handling → Edges (control flows) of
CFG (a program)?

Recall 1st
lecture

Data Flow Analysis

How Data Flows on CFG?

$+, -, 0, \perp, \top$

How application-specific Data $\leftarrow$ Abstraction

Safe-approximation $\rightarrow$ Flows through the

Transfer function $\longrightarrow$ Nodes (BBs/statements) and

Control-flow handling $\rightarrow$ Edges (control flows) of
CFG (a program)?

Recall 1st
lecture

Data Flow Analysis

How Data Flows on CFG?

$+, -, 0, \perp, \top$

How application-specific Data $\leftarrow$ Abstraction

Safe-approximation $\rightarrow$ Flows through the

Transfer function $\longrightarrow$ Nodes (BBs/statements) and

$+ \text{ op } - = - ; + \text{ op } + = +$

Control-flow handling $\rightarrow$ Edges (control flows) of
CFG (a program)?

Recall 1st
lecture

Data Flow Analysis

How Data Flows on CFG?

$+, -, 0, \perp, \top$

How application-specific Data $\leftarrow$ Abstraction

Safe-approximation $\rightarrow$ Flows through the

Transfer function $\longrightarrow$ Nodes (BBs/statements) and

$+ \text{ op } - = - ; + \text{ op } + = +$

Control-flow handling $\rightarrow$

Edges (control flows) of
CFG (a program)?

union the signs at merges

Recall 1st
lecture

Data Flow Analysis

How Data Flows on CFG?

$+, -, 0, \perp, \top$

How application-specific Data $\leftarrow$ Abstraction

Safe-approximation $\rightarrow$ Flows through the

Transfer function $\longrightarrow$ Nodes (BBs/statements) and

$+ \text{ op } - = - ; + \text{ op } + = +$

Control-flow handling $\rightarrow$ Edges (control flows) of
CFG (a program)?

union the signs at merges

different data-flow analysis applications have
different data abstraction and
different flow safe-approximation strategies, i.e.,
different transfer functions and control-flow handlings

Recall 1st
lecture

Data Flow Analysis

How Data Flows on CFG?

$+, -, 0, \perp, \top$

How application-specific Data $\leftarrow$ Abstraction

Safe-approximation $\rightarrow$ Flows through the

Transfer function $\longrightarrow$ Nodes (BBs/statements) and

$+ \text{ op } - = - ; + \text{ op } + = +$

Control-flow handling $\rightarrow$ Edges (control flows) of
CFG (a program)?

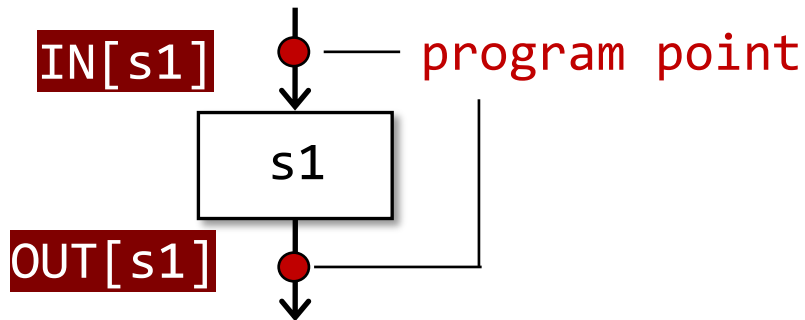
union the signs at merges

different data-flow analysis applications have
different data abstraction and
different flow safe-approximation strategies, i.e.,
different transfer functions and control-flow handlings

Preliminaries of Data Flow Analysis

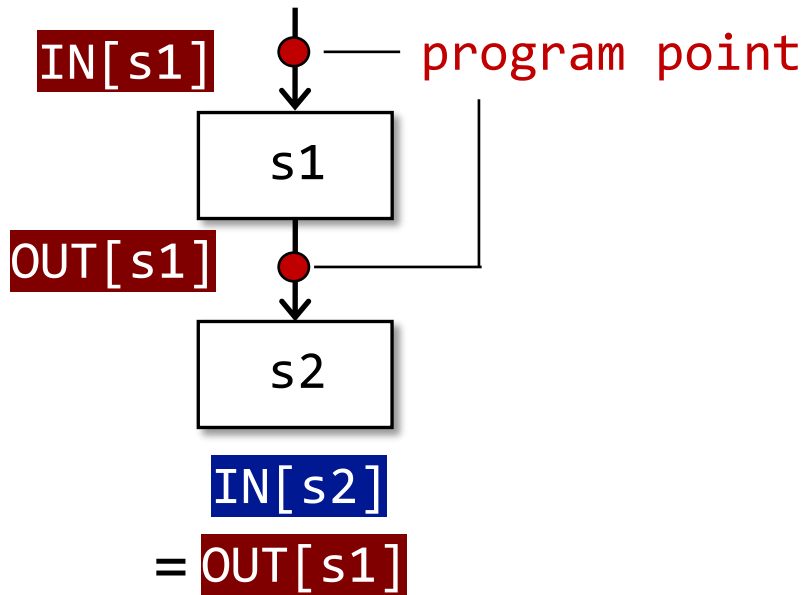
Input and Output States

- Each execution of an IR statement transforms an **input state** to a new **output state**
- The input (output) state is associated with the **program point** before (after) the statement



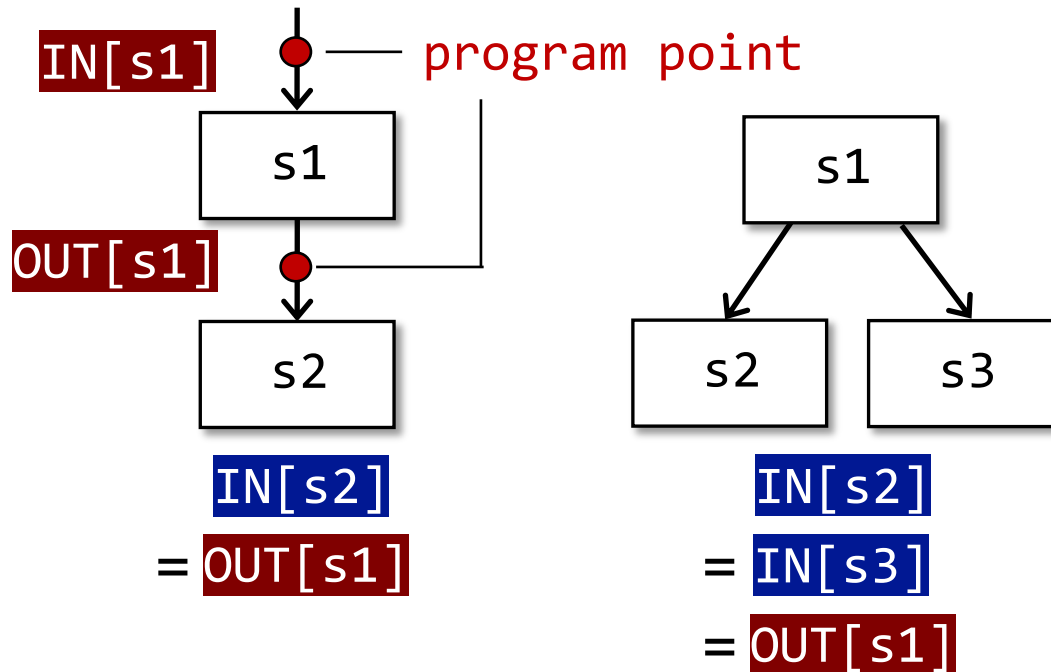
Input and Output States

- Each execution of an IR statement transforms an **input state** to a new **output state**
- The input (output) state is associated with the **program point** before (after) the statement



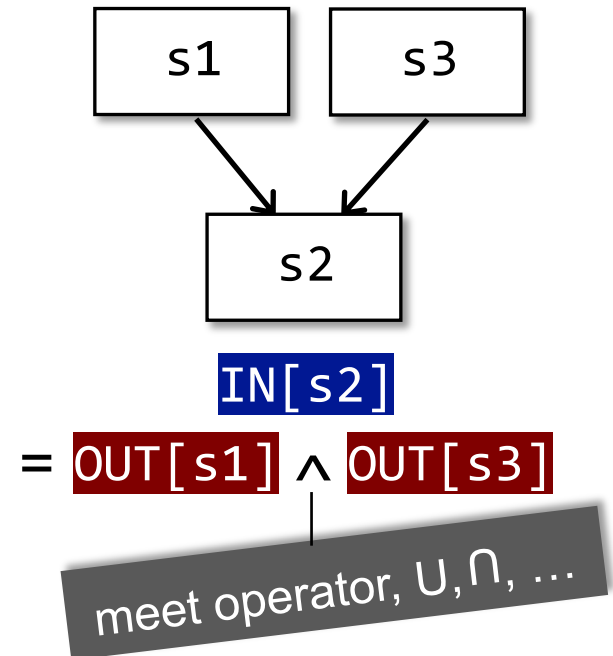
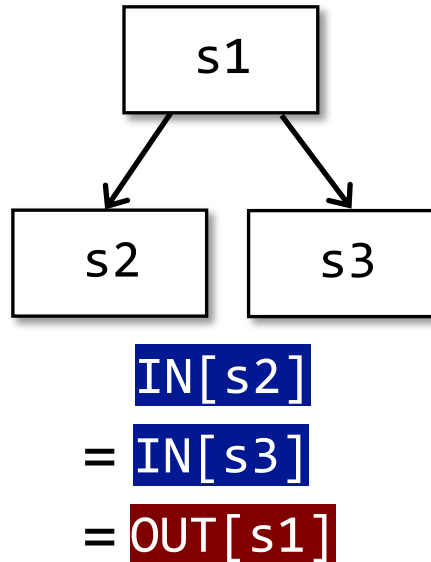
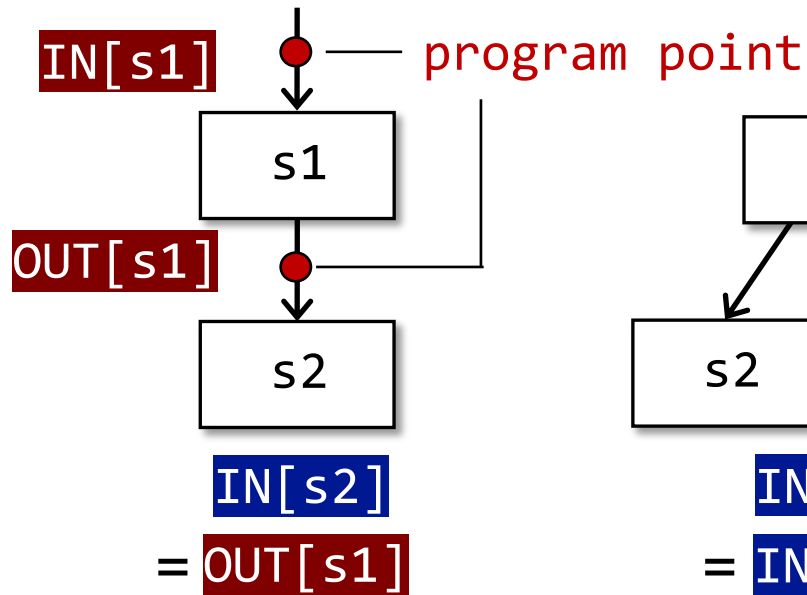
Input and Output States

- Each execution of an IR statement transforms an **input state** to a new **output state**
- The input (output) state is associated with the **program point** before (after) the statement



Input and Output States

- Each execution of an IR statement transforms an **input state** to a new **output state**
- The input (output) state is associated with the **program point** before (after) the statement



In each data-flow analysis application, we associate with every program point a **data-flow value** that represents an *abstraction* of the set of all possible **program states** that can be observed for that point.

In each data-flow analysis application, we associate with every program point a **data-flow value** that represents an *abstraction* of the set of all possible **program states** that can be observed for that point.

```
x = 10;  
•  
y = -1;  
•  
x = y;  
•  
x = x / 6;  
•
```

Recall 1st
lecture

In each data-flow analysis application, we associate with every program point a **data-flow value** that represents an *abstraction* of the set of all possible **program states** that can be observed for that point.

$x = 10;$



$y = -1;$



$x = y;$



$x = x / y;$



$x = \boxed{+} \quad y = \boxed{\perp}$

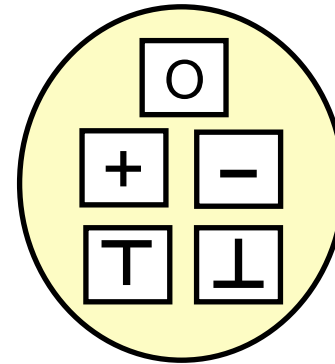
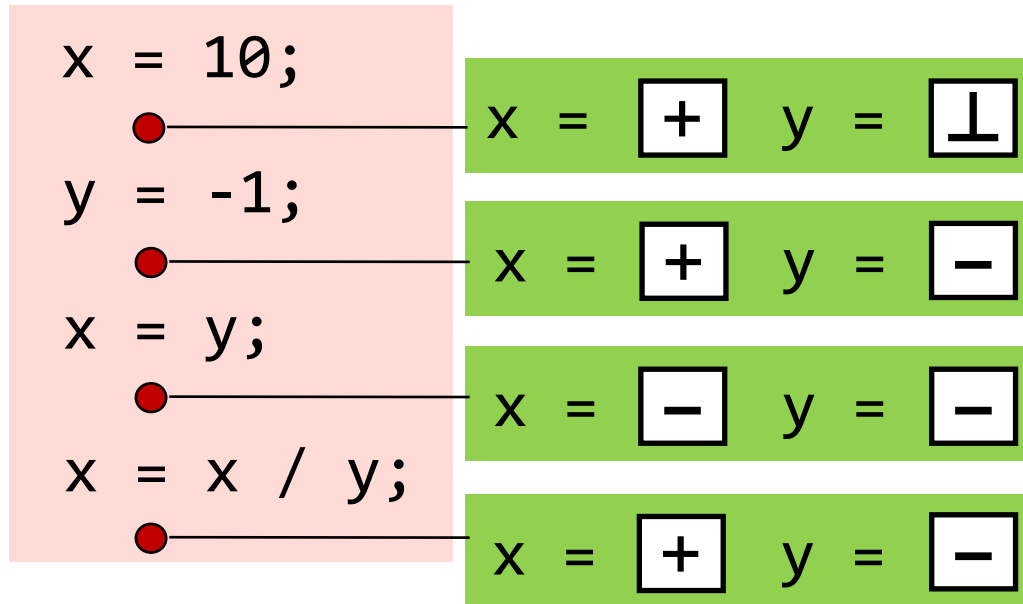
$x = \boxed{+} \quad y = \boxed{-}$

$x = \boxed{-} \quad y = \boxed{-}$

$x = \boxed{+} \quad y = \boxed{-}$

Recall 1st
lecture

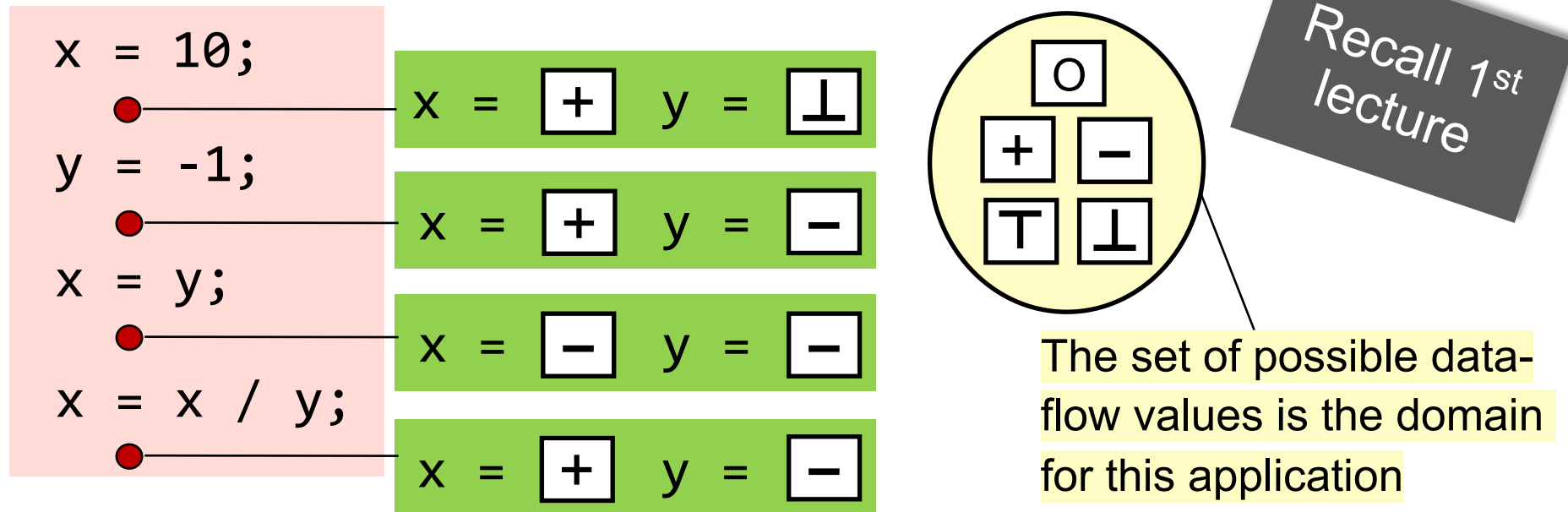
In each data-flow analysis application, we associate with every program point a **data-flow value** that represents an *abstraction* of the set of all possible **program states** that can be observed for that point.



Recall 1st lecture

The set of possible data-flow values is the domain for this application

In each data-flow analysis application, we associate with every program point a **data-flow value** that represents an *abstraction* of the set of all possible **program states** that can be observed for that point.



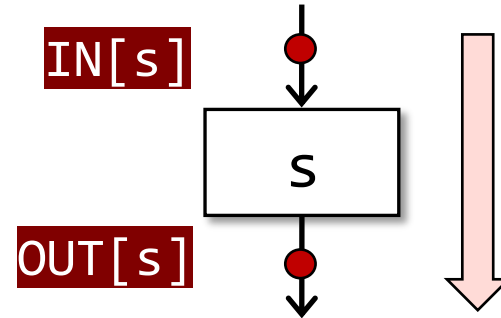
Data-flow analysis is to **find a solution** to a set of *safe-approximation-directed constraints* on the $IN[s]$'s and $OUT[s]$'s, for **all statements** s .

- *constraints* based on semantics of statements (*transfer functions*)
- *constraints* based on the *flows of control*

Notations for Transfer Function's Constraints

- Forward Analysis

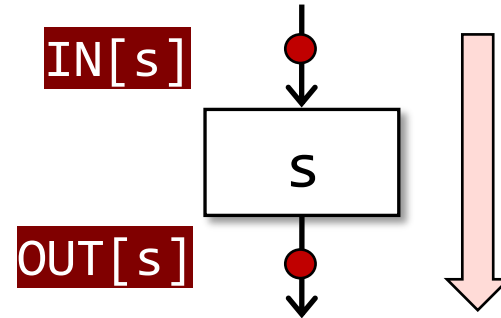
$$\text{OUT}[s] = f_s(\text{IN}[s])$$



Notations for Transfer Function's Constraints

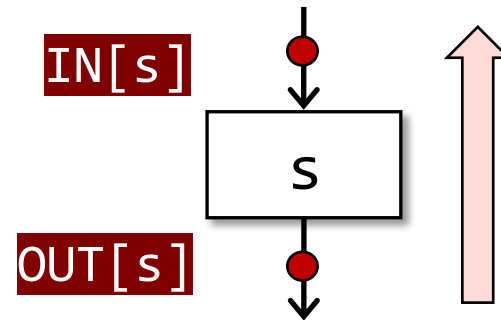
- Forward Analysis

$$\text{OUT}[s] = f_s(\text{IN}[s])$$



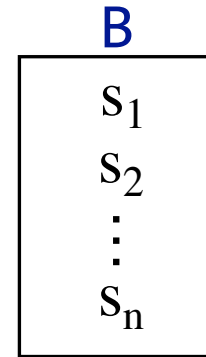
- Backward Analysis

$$\text{IN}[s] = f_s(\text{OUT}[s])$$



Notations for Control Flow's Constraints

- Control flow within a BB
- Control flow among BBs



Notations for Control Flow's Constraints

- Control flow within a BB

$$\text{IN}[s_{i+1}] = \text{OUT}[s_i], \text{ for all } i = 1, 2, \dots, n-1$$

- Control flow among BBs

B

s_1

s_2

$\vdots$

s_n

Notations for Control Flow's Constraints

- Control flow within a BB

$$\text{IN}[s_{i+1}] = \text{OUT}[s_i], \text{ for all } i = 1, 2, \dots, n-1$$

- Control flow among BBs

$$\text{IN}[B] = \text{IN}[s_1]$$

$$\text{OUT}[B] = \text{OUT}[s_n]$$

B

s_1

s_2

$\vdots$

s_n

Notations for Control Flow's Constraints

- Control flow within a BB

$$\text{IN}[s_{i+1}] = \text{OUT}[s_i], \text{ for all } i = 1, 2, \dots, n-1$$

- Control flow among BBs

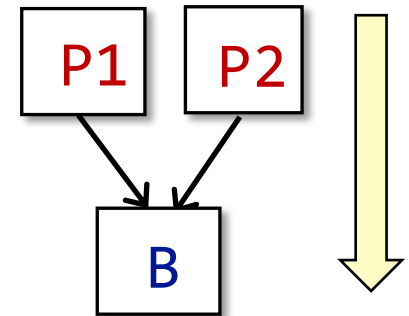
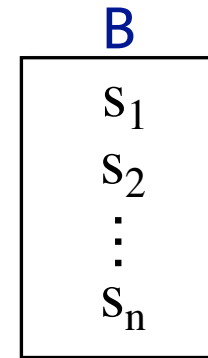
$$\text{IN}[B] = \text{IN}[s_1]$$

$$\text{OUT}[B] = \text{OUT}[s_n]$$

$$\text{OUT}[B] = f_B(\text{IN}[B]), f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1}$$

$$\text{IN}[B] = \bigwedge_{P \text{ a predecessor of } B} \text{OUT}[P]$$

The meet operator $\bigwedge$ is used to summarize the contributions from different paths at the confluence of those paths



Notations for Control Flow's Constraints

- Control flow within a BB

$$\text{IN}[s_{i+1}] = \text{OUT}[s_i], \text{ for all } i = 1, 2, \dots, n-1$$

- Control flow among BBs

$$\text{IN}[B] = \text{IN}[s_1]$$

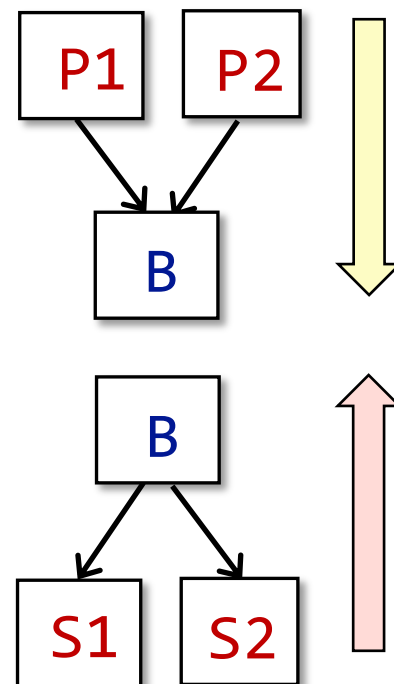
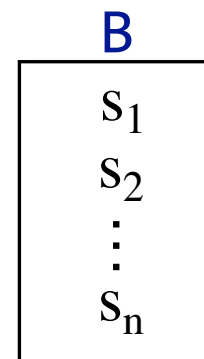
$$\text{OUT}[B] = \text{OUT}[s_n]$$

$$\text{OUT}[B] = f_B(\text{IN}[B]), f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1}$$

$$\text{IN}[B] = \bigwedge_{P \text{ a predecessor of } B} \text{OUT}[P]$$

$$\text{IN}[B] = f_B(\text{OUT}[B]), f_B = f_{s_1} \circ \dots \circ f_{s_{n-1}} \circ f_{s_n}$$

$$\text{OUT}[B] = \bigwedge_{S \text{ a successor of } B} \text{IN}[S]$$



Data Flow Analysis Applications

(I) Reaching Definitions Analysis

(II) Live Variables Analysis

(III) Available Expressions Analysis

Issues Not Covered

- Method Calls
 - Intra-procedural CFG
 - Will be introduced in lecture: Inter-procedural Analysis
- Aliases
 - Variables have no aliases
 - Will be introduced in lecture: Pointer Analysis

Reaching Definitions

A **definition d** at program point p *reaches* a point q if there is a path from p to q such that **d** is not “killed” along that path

Reaching Definitions

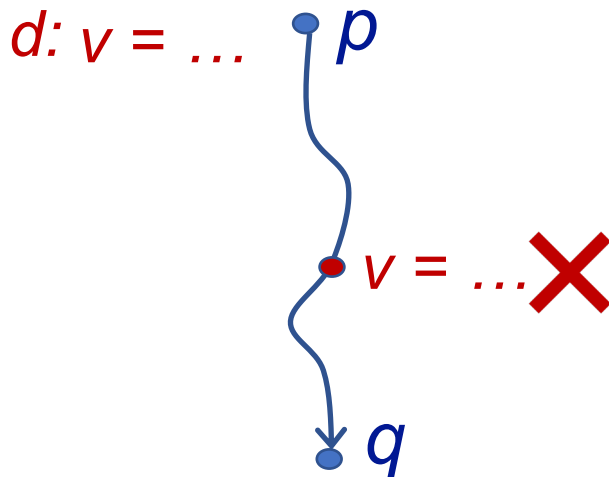
A **definition** d at program point p *reaches* a point q if there is a path from p to q such that d is not “killed” along that path

- A **definition of a variable** v is a statement that assigns a value to v

Reaching Definitions

A **definition** d at program point p *reaches* a point q if there is a path from p to q such that d is not “killed” along that path

- A **definition of a variable** v is a statement that assigns a value to v
- Translated as: definition of variable v at program point p reaches point q if there is a path from p to q such that no new definition of v appears on that path



Reaching Definitions

A **definition** **d** at program point **p** *reaches* a point **q** if there is a path from **p** to **q** such that **d** is not “killed” along that path

- A **definition of a variable** **v** is a statement that assigns a value to **v**
- Translated as: definition of variable **v** at program point **p** reaches point **q** if there is a path from **p** to **q** such that no new definition of **v** appears on that path
- Reaching definitions can be used to **detect possible undefined variables**. e.g., introduce a dummy definition for each variable **v** at the entry of CFG, and if the dummy definition of **v** reaches a point **p** where **v** is used, then **v** may be used before definition (as *undefined reaches v*)

Understanding Reaching Definitions

Abstraction

- Data Flow Values/Facts
 - The definitions of all the variables in a program

Understanding Reaching Definitions

Abstraction

- Data Flow Values/Facts
 - The definitions of all the variables in a program
 - Can be represented by bit vectors

e.g., D1, D2, D3, D4, ..., D100 (100 definitions)

00000...0
└──────────┘
100 bits

Bit i from the left represents definition D_i

Understanding Reaching Definitions

Safe-approximation

- Transfer Function
- Control Flow

Understanding Reaching Definitions

$$D: v = x \text{ op } y$$

Safe-approximation

This statement “**generates**” a definition **D** of variable v and “**kills**” all the other definitions in the program that define variable v , while leaving the remaining incoming definitions unaffected.

- Transfer Function
- Control Flow

Understanding Reaching Definitions

$$D: v = x \text{ op } y$$

Safe-approximation

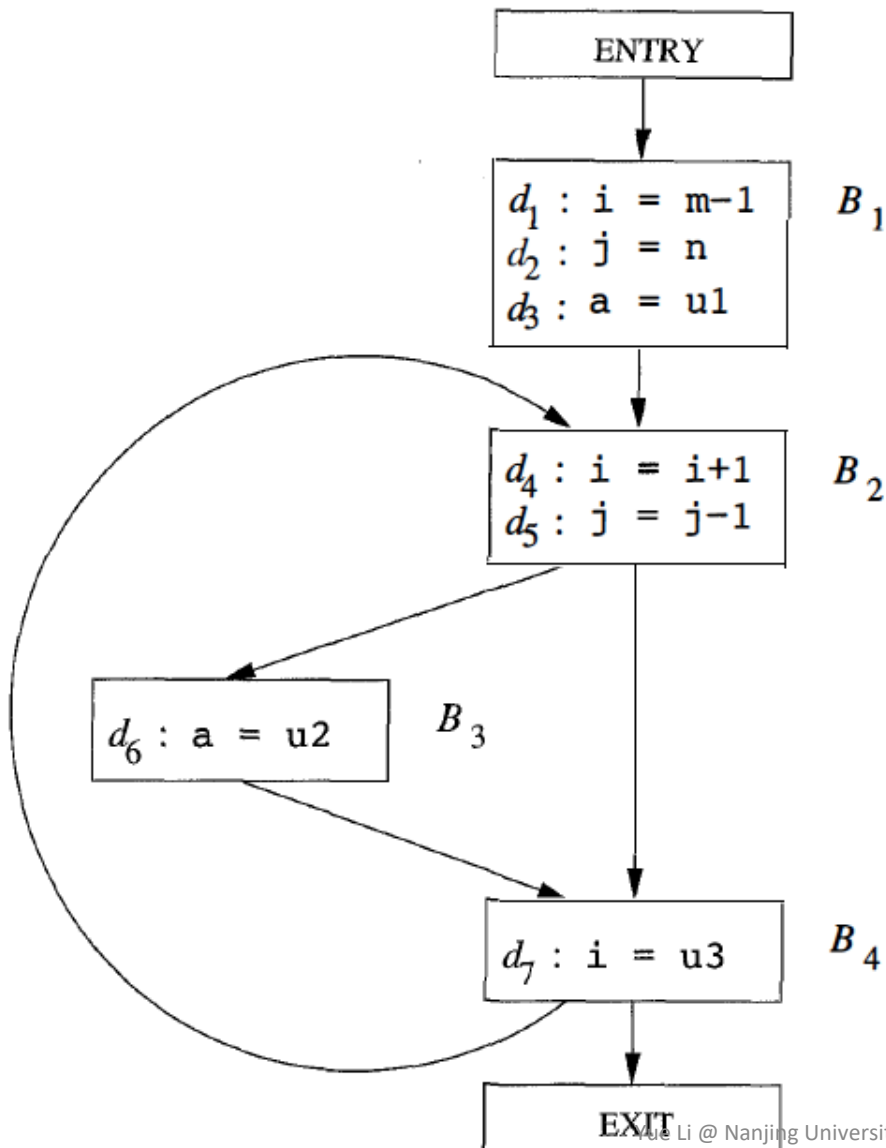
This statement “**generates**” a **definition D** of variable v and “**kills**” **all the other definitions** in the program that define variable v , while leaving the remaining incoming definitions unaffected.

- Transfer Function

$$\text{OUT}[B] = \text{gen}_B \cup (\text{IN}[B] - \text{kill}_B)$$

- Control Flow

$$\text{OUT}[B] = \text{gen}_B \cup (\text{IN}[B] - \text{kill}_B)$$



$$\text{gen}_{B_1} = \{ d_1, d_2, d_3 \}$$

$$\text{kill}_{B_1} = \{ d_4, d_5, d_6, d_7 \}$$

$$\text{gen}_{B_2} = \{ d_4, d_5 \}$$

$$\text{kill}_{B_2} = \{ d_1, d_2, d_7 \}$$

$$\text{gen}_{B_3} = \{ d_6 \}$$

$$\text{kill}_{B_3} = \{ d_3 \}$$

$$\text{gen}_{B_4} = \{ d_7 \}$$

$$\text{kill}_{B_4} = \{ d_1, d_4 \}$$

Understanding Reaching Definitions

$$D: v = x \text{ op } y$$

Safe-approximation

This statement “**generates**” a **definition D** of variable v and “**kills**” **all the other definitions** in the program that define variable v , while leaving the remaining incoming definitions unaffected.

- Transfer Function

$$\text{OUT}[B] = \text{gen}_B \cup (\text{IN}[B] - \text{kill}_B)$$

- Control Flow

Understanding Reaching Definitions

$$D: v = x \text{ op } y$$

Safe-approximation

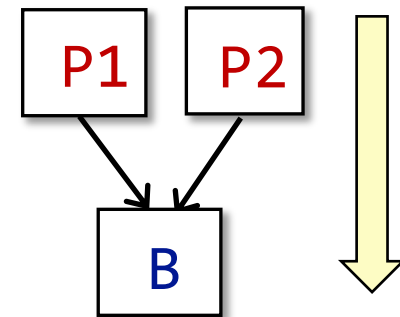
This statement “**generates**” a **definition D** of variable v and “**kills**” **all the other definitions** in the program that define variable v , while leaving the remaining incoming definitions unaffected.

- Transfer Function

$$\text{OUT}[B] = \text{gen}_B \cup (\text{IN}[B] - \text{kill}_B)$$

- Control Flow

$$\text{IN}[B] = \bigcup_{P \text{ a predecessor of } B} \text{OUT}[P]$$



Understanding Reaching Definitions

D: $v = x \text{ op } y$

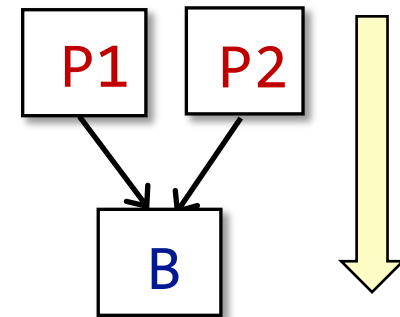
Safe-approximation

This statement “**generates**” a **definition D** of variable v and “**kills**” **all the other definitions** in the program that define variable v , while leaving the remaining incoming definitions unaffected.

A definition reaches a program point as long as there exists at least one path along which the definition reaches.

- Control Flow

$$IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$$



Understanding Reaching Definitions

$$D: v = x \text{ op } y$$

Safe-approximation

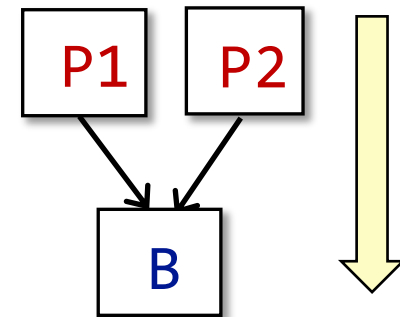
This statement “**generates**” a **definition D** of variable v and “**kills**” **all the other definitions** in the program that define variable v , while leaving the remaining incoming definitions unaffected.

- Transfer Function

$$\text{OUT}[B] = \text{gen}_B \cup (\text{IN}[B] - \text{kill}_B)$$

- Control Flow

$$\text{IN}[B] = \bigcup_{P \text{ a predecessor of } B} \text{OUT}[P]$$



Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

these are two boundary conditions, all of these can be understood by using semantic meanings

METHOD: Here, entry is a starting point, has no data flow into it. Before starting analysis, all basic block do not have any data flow into it, this is an iterative method.

$OUT[entry] = \emptyset;$

for (each basic block $B \setminus entry$)

$OUT[B] = \emptyset;$

while (changes to any OUT occur)

for (each basic block $B \setminus entry$) {

$IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P];$

$OUT[B] = gen_B \cup (IN[B] - kill_B);$

}

Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
OUT[entry] =  $\emptyset$ ; ?  
for (each basic block  $B \setminus entry$ )  
    OUT[B] =  $\emptyset$ ;  
    while (changes to any OUT occur)  
        for (each basic block  $B \setminus entry$ ) {  
             $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$ ;  
             $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
        }
```

Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ ) ?  
    OUT[B] =  $\emptyset$ ;  
    while (changes to any OUT occur)  
        for (each basic block  $B \setminus entry$ ) {  
             $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$ ;  
             $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
        }
```

Why entry is excluded?

Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ )  
    OUT[B] =  $\emptyset$ ; ?  
    while (changes to any OUT occur)  
        for (each basic block  $B \setminus entry$ ) {  
             $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$ ;  
             $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
        }
```

Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ )  
    OUT[B] =  $\emptyset$ ;  
while (changes to any OUT occur) ?  
    for (each basic block  $B \setminus entry$ ) {  
         $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$ ;  
         $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
    }
```

Why this iterative algorithm
can finally stop?

Algorithm of Reaching Definitions Analysis

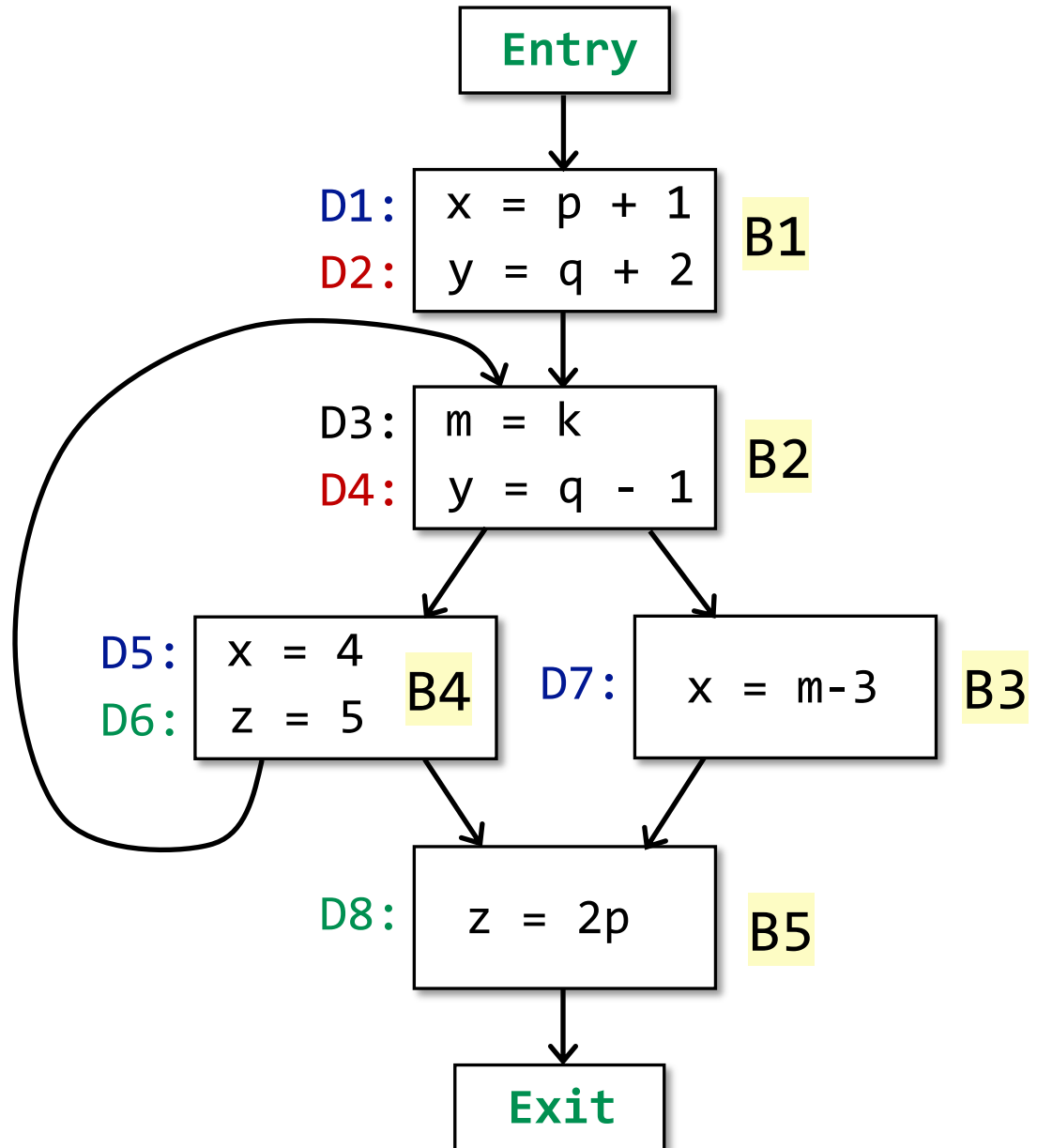
INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ )  
    OUT[B] =  $\emptyset$ ;  
    while (changes to any OUT occur)  
        for (each basic block  $B \setminus entry$ ) {  
             $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$ ;  
             $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
        }
```

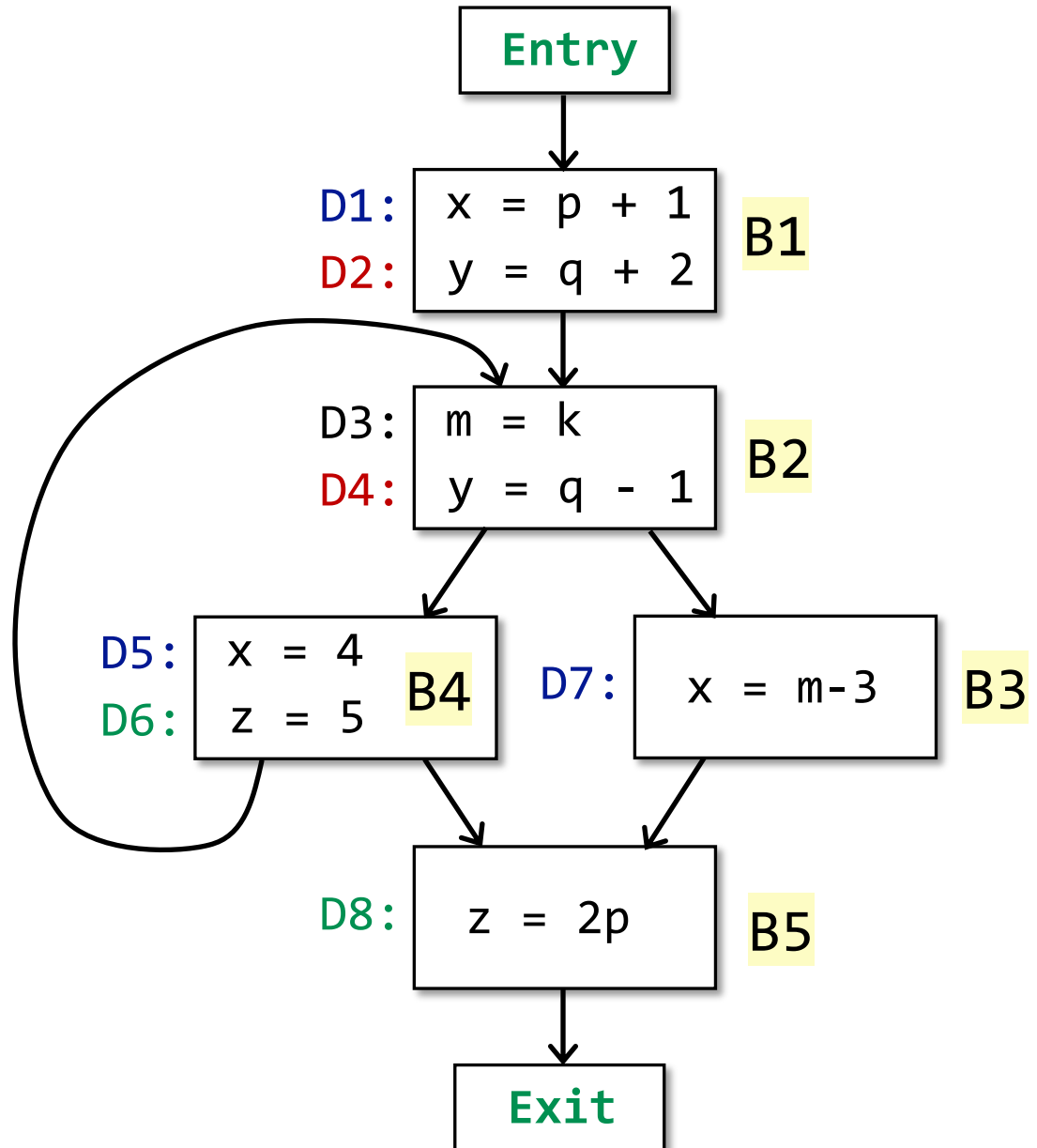




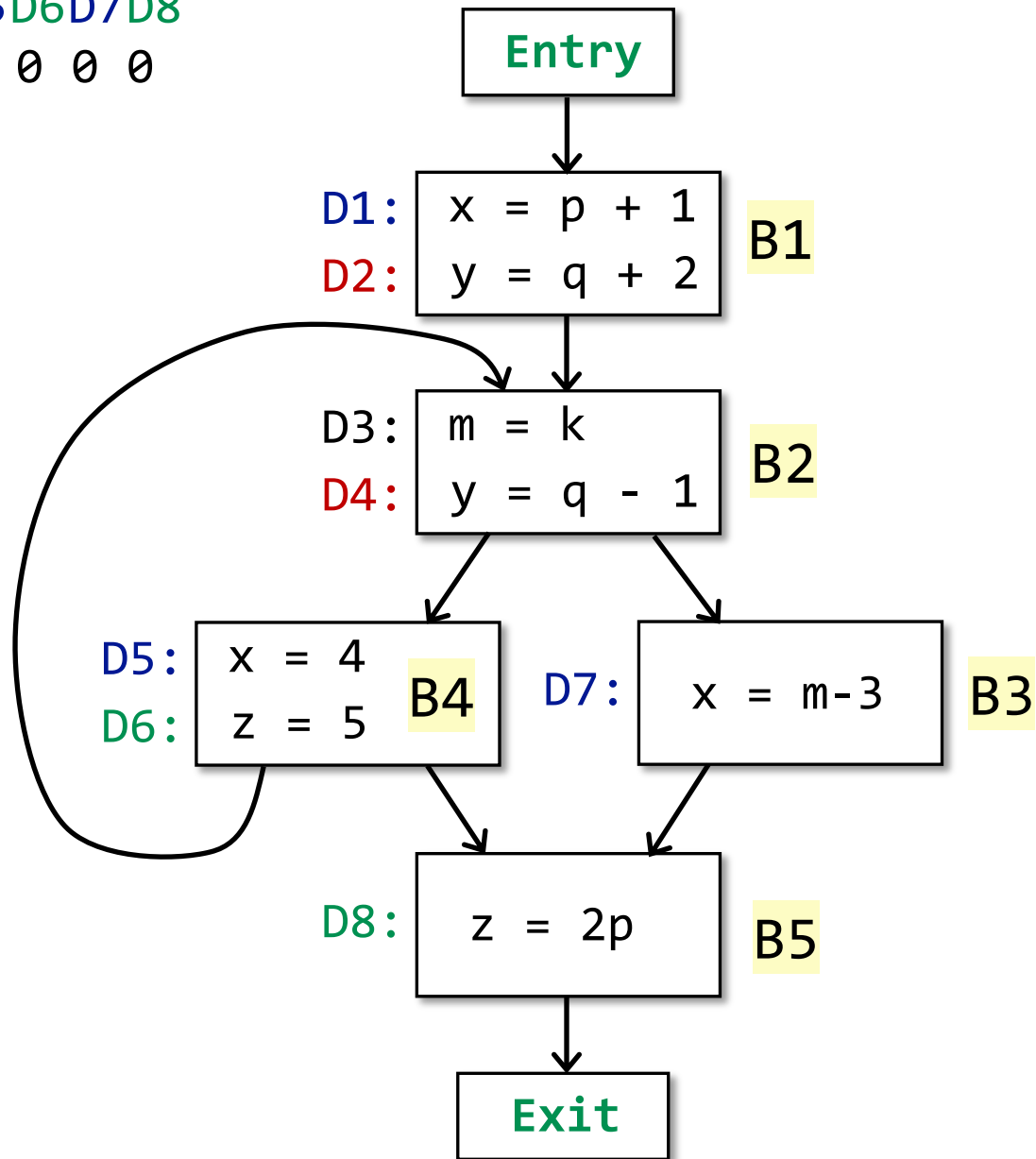
```

D1
D2
do {
  D3
  D4
  if(...) {
    D5
    D6
  } else {
    D7
    break
  }
} while(...)
D8

```



D1D2D3D4D5D6D7D8
0 0 0 0 0 0 0 0



Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

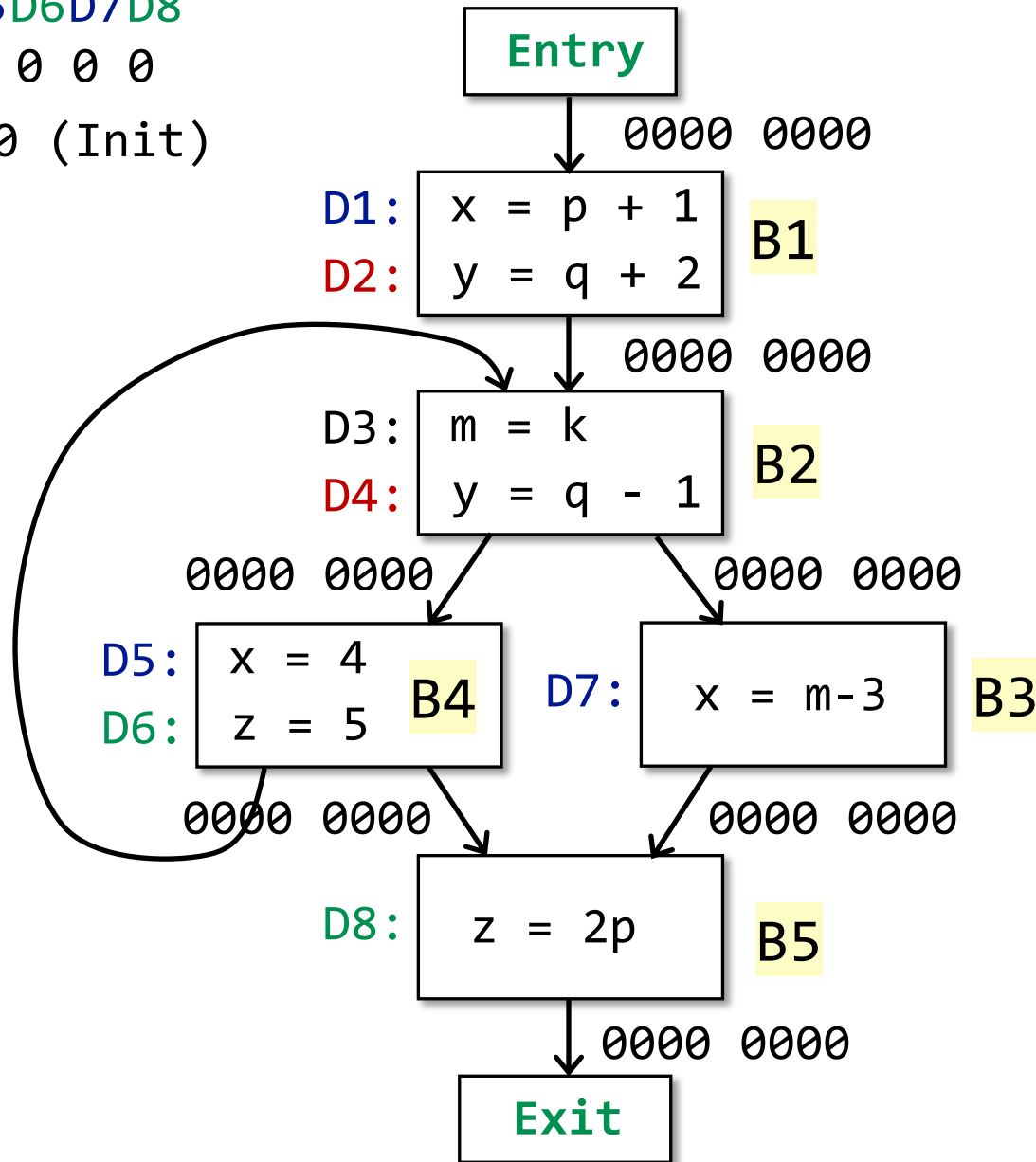
```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ )  
    OUT[B] =  $\emptyset$ ;  
while (changes to any OUT occur)  
    for (each basic block  $B \setminus entry$ ) {  
         $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$ ;  
         $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
    }
```



D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)



Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ )  
    OUT[B] =  $\emptyset$ ;  
    while (changes to any OUT occur)  
        for (each basic block  $B \setminus entry$ ) {  
             $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$ ;  
             $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
        }
```

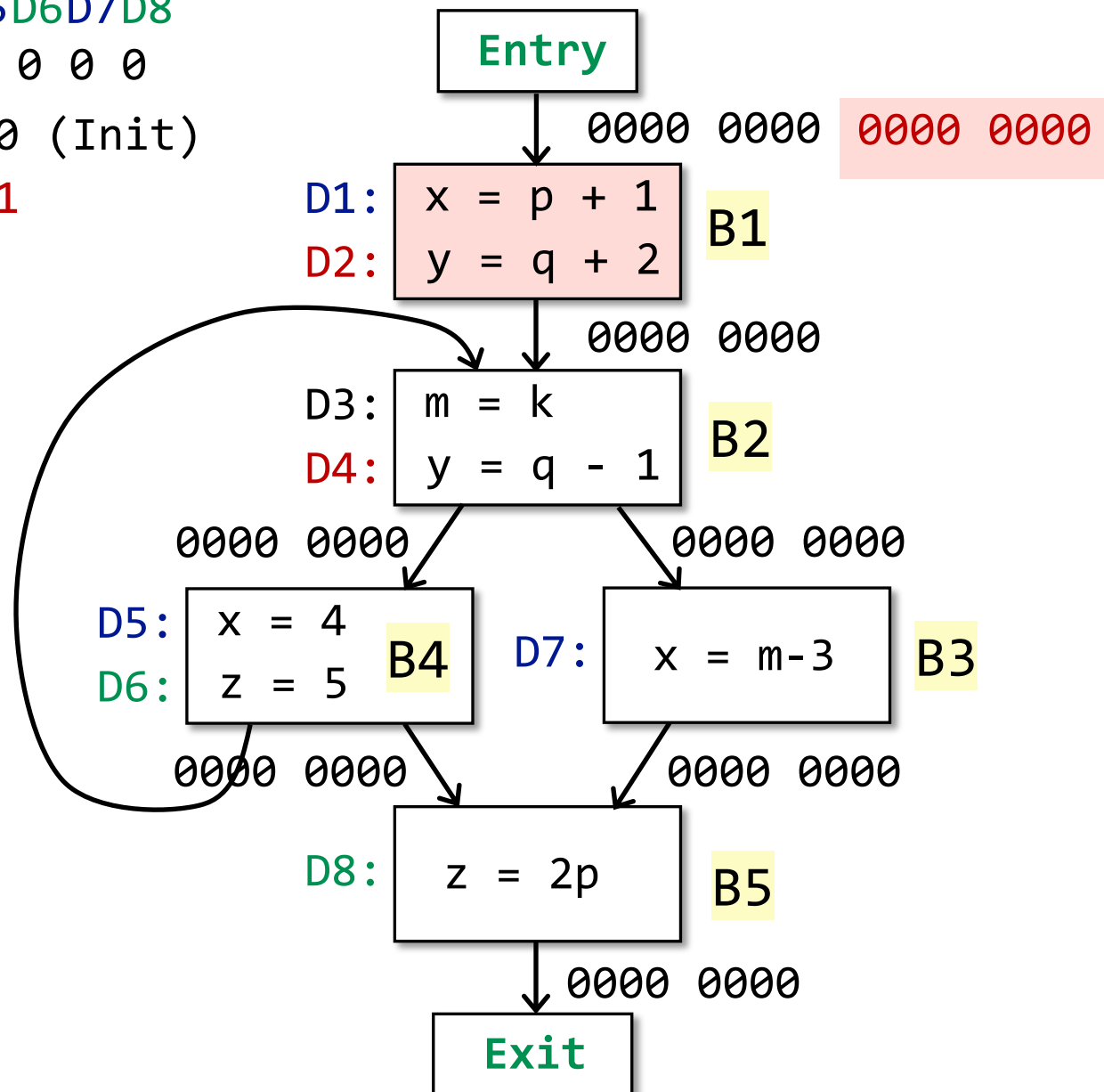


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1



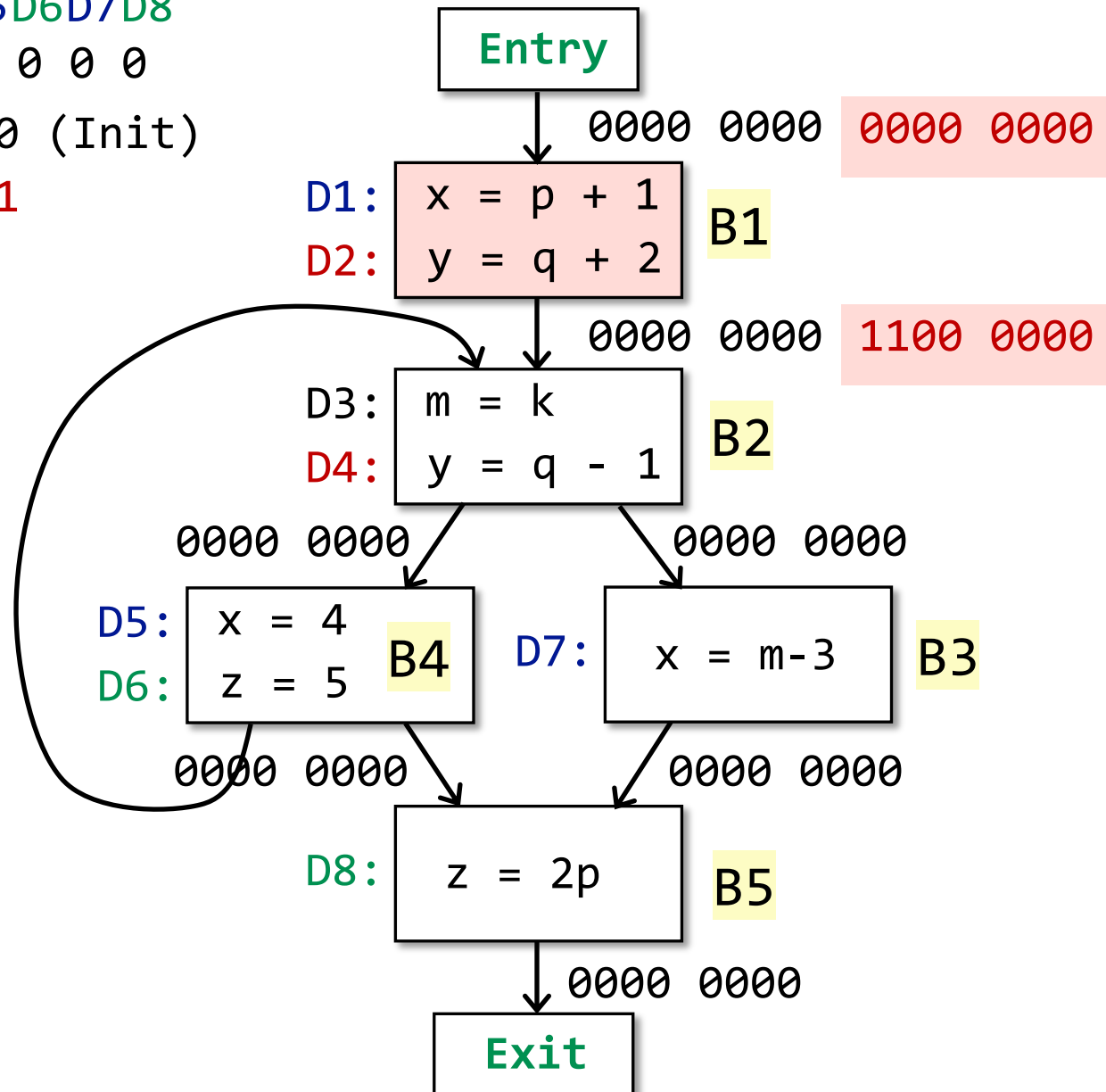
$$\text{OUT}[B] = \text{gen}_B \cup (\text{IN}[B] - \text{kill}_B);$$

D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

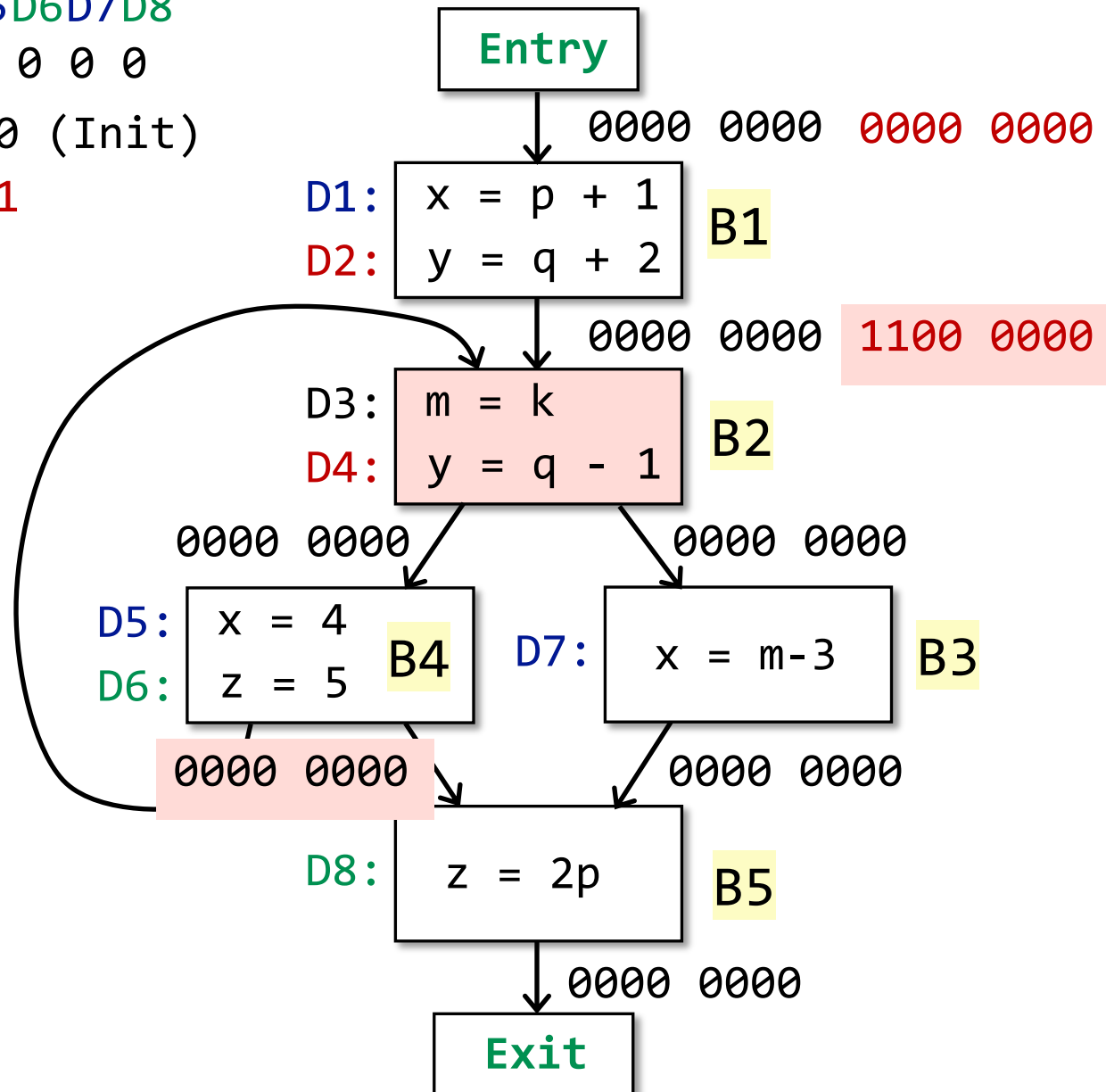


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1



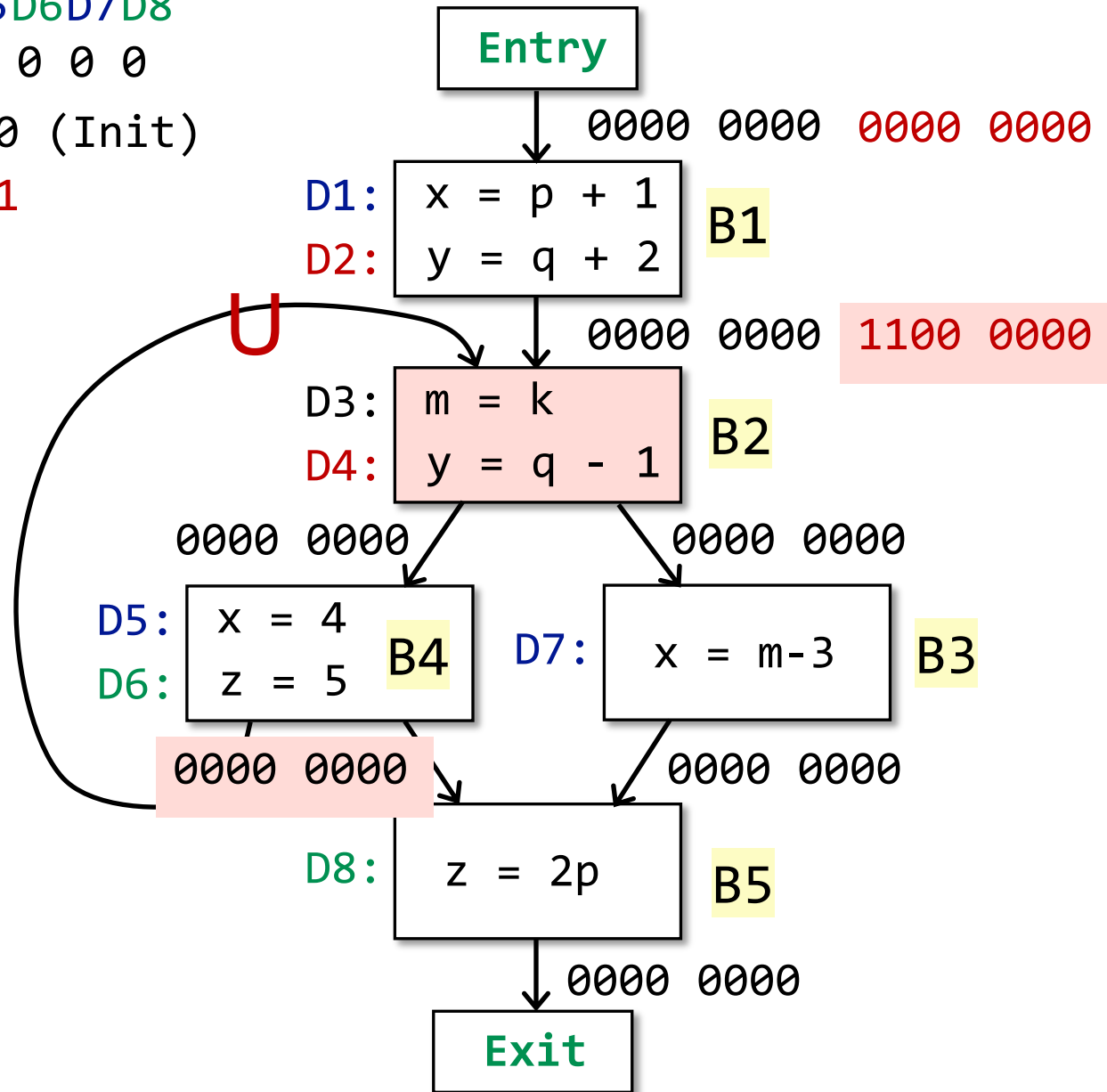
$IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P];$ iversity

D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

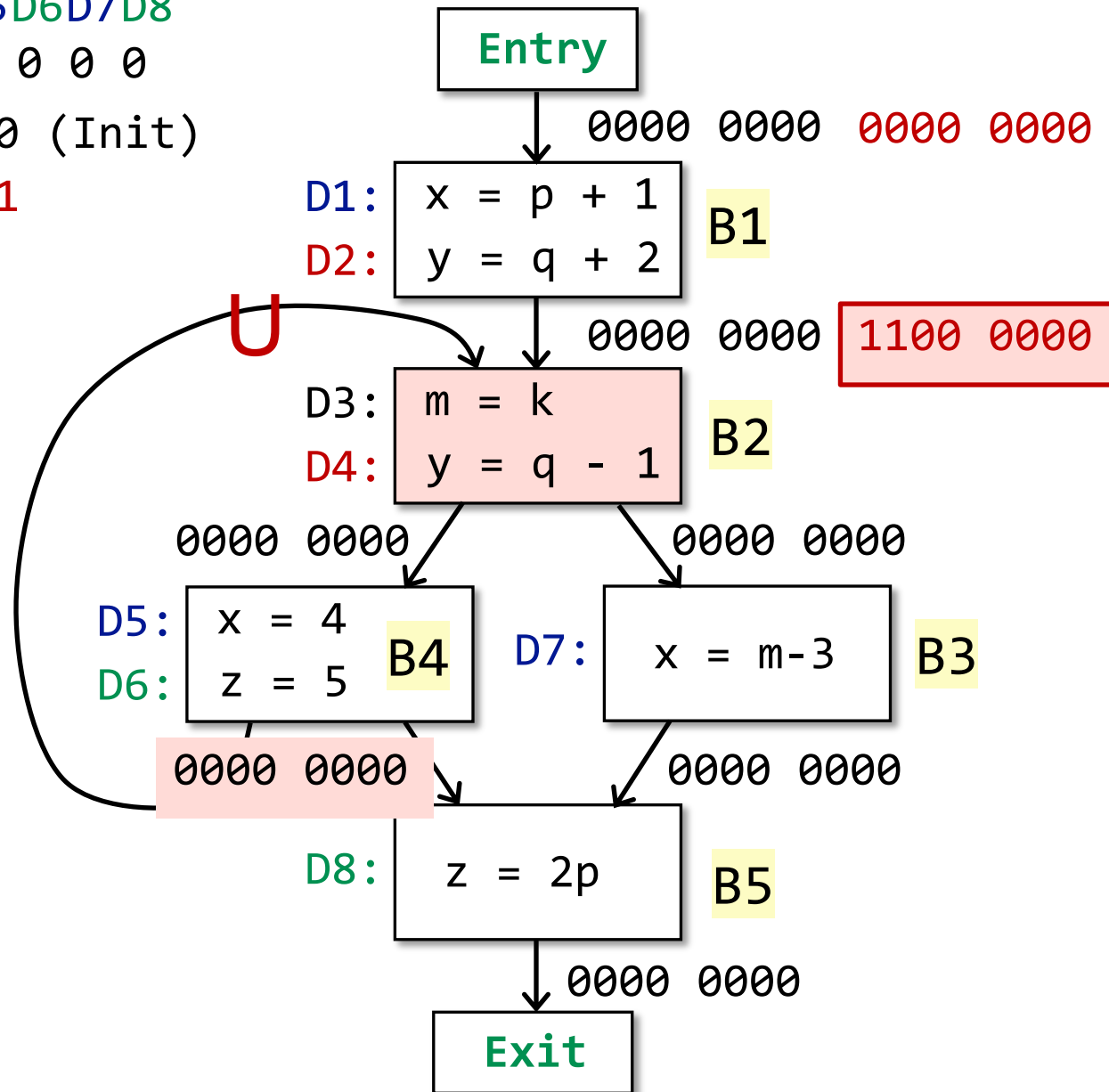


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

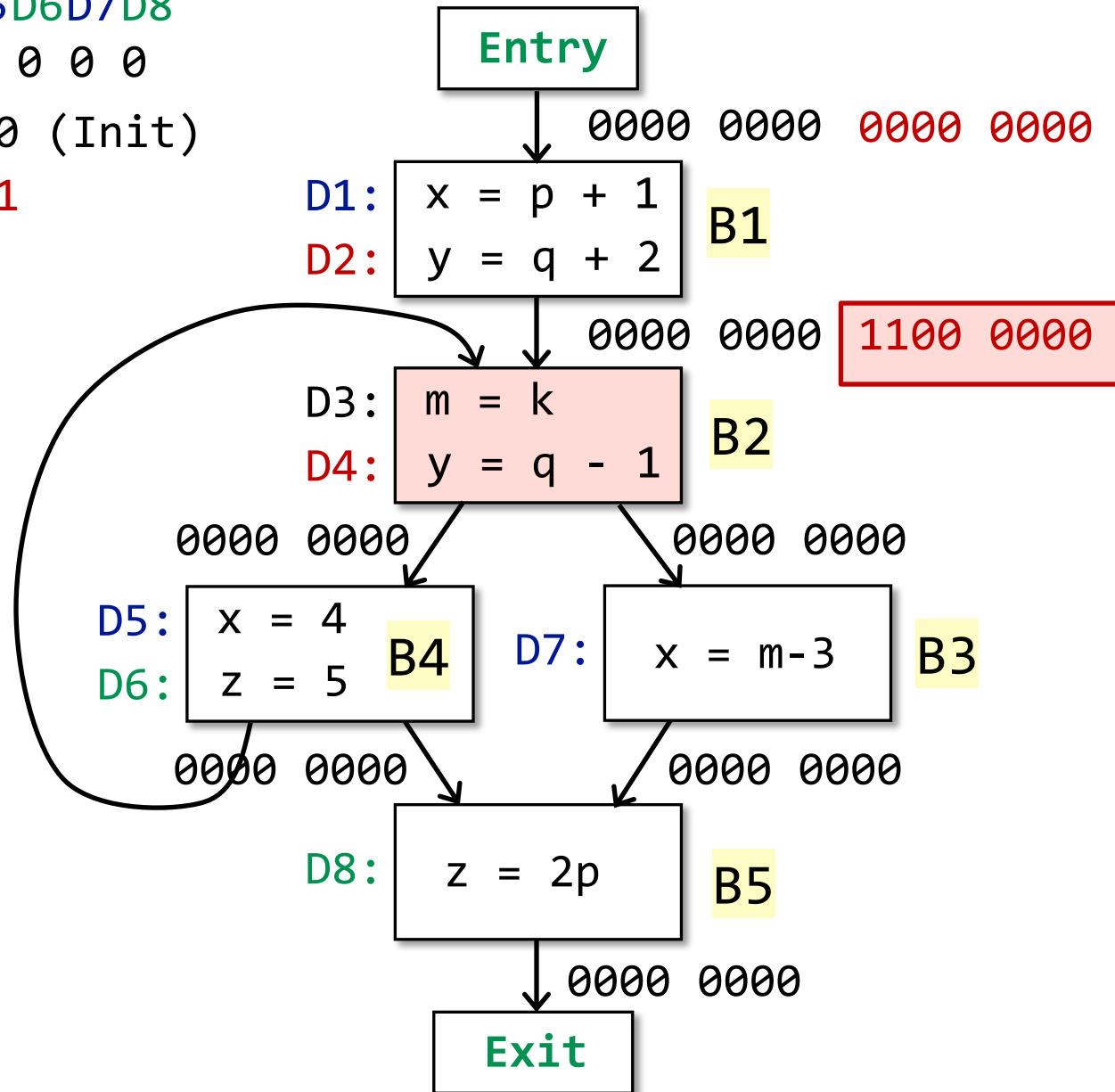


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

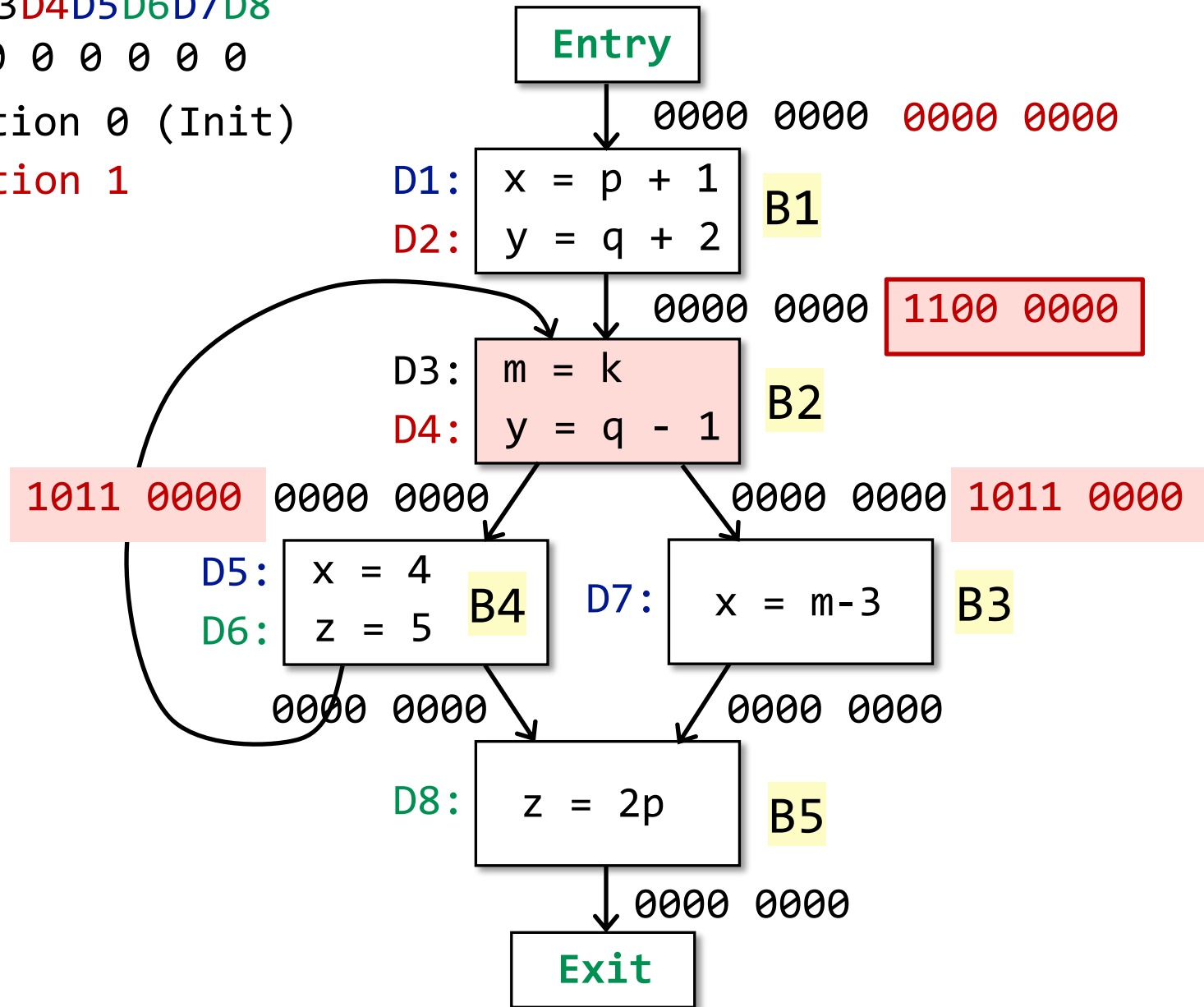


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

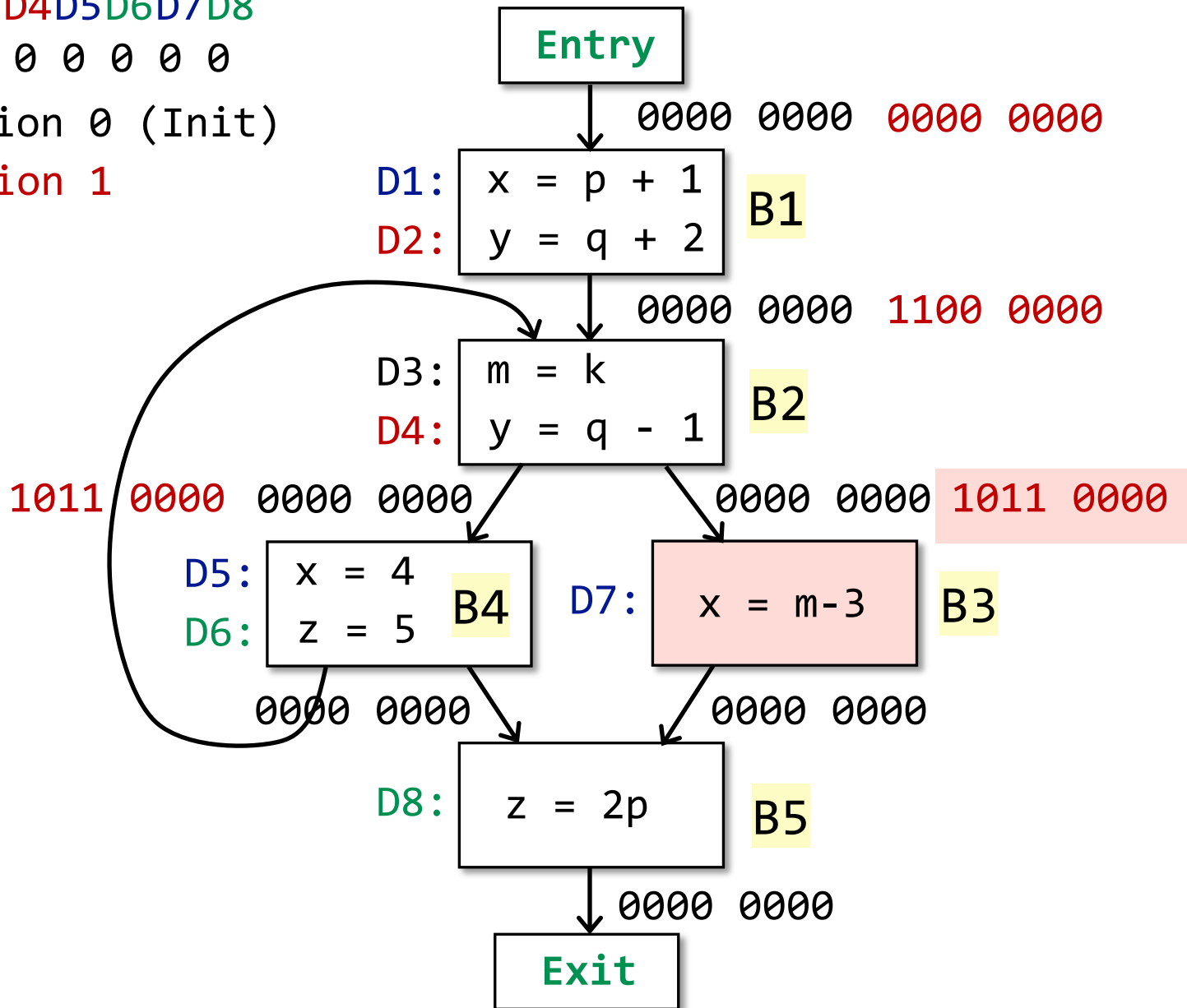


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

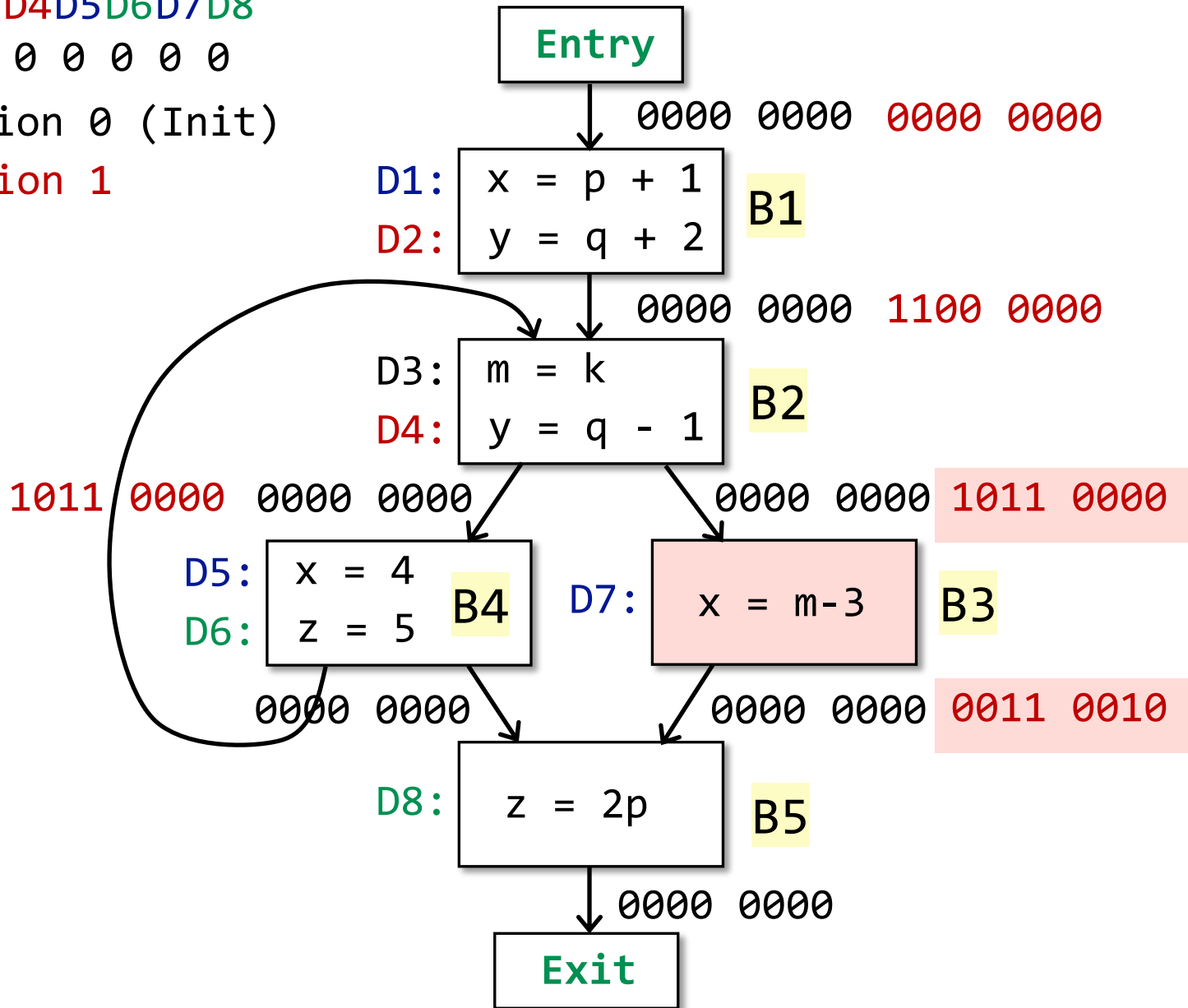


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

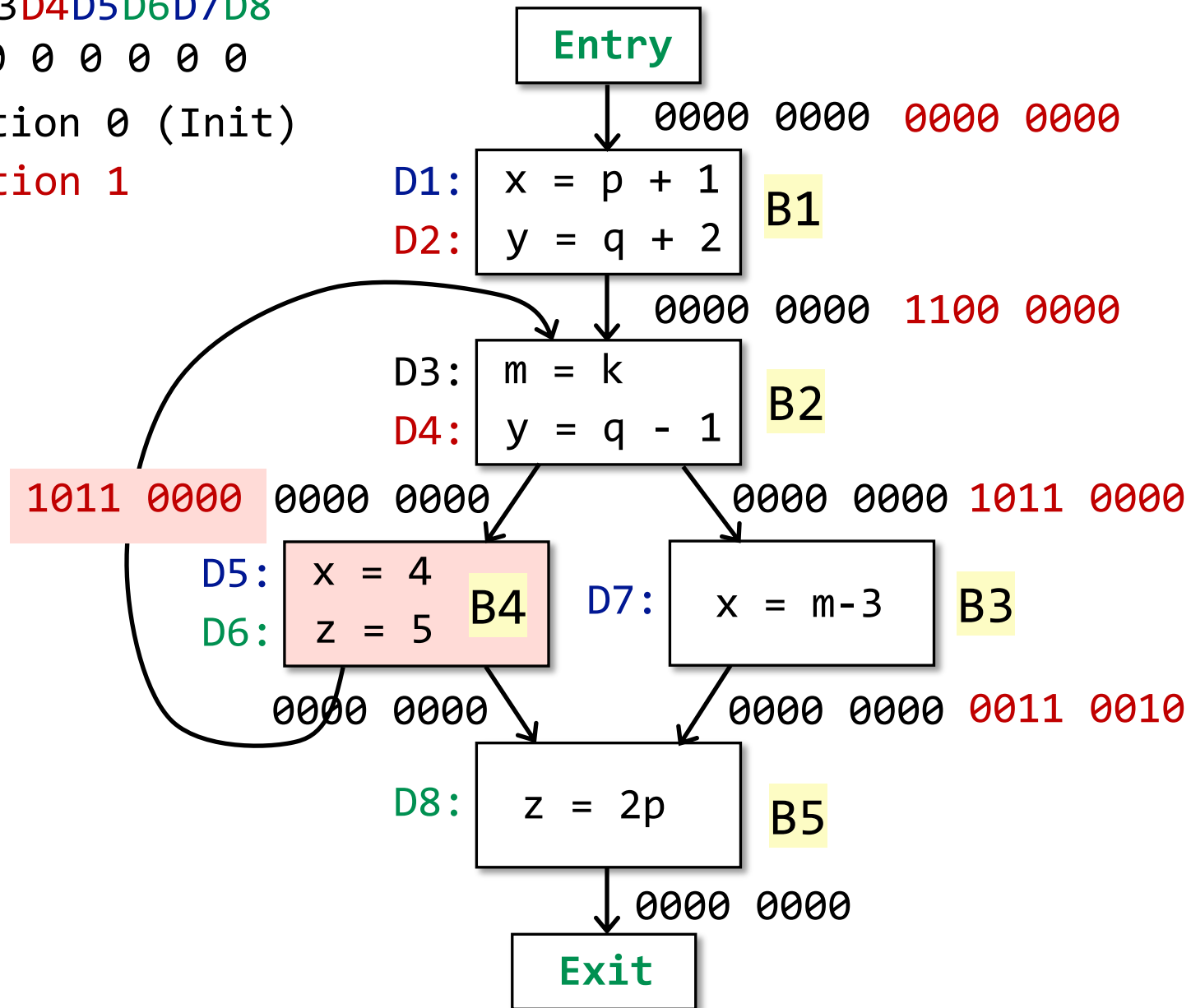


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

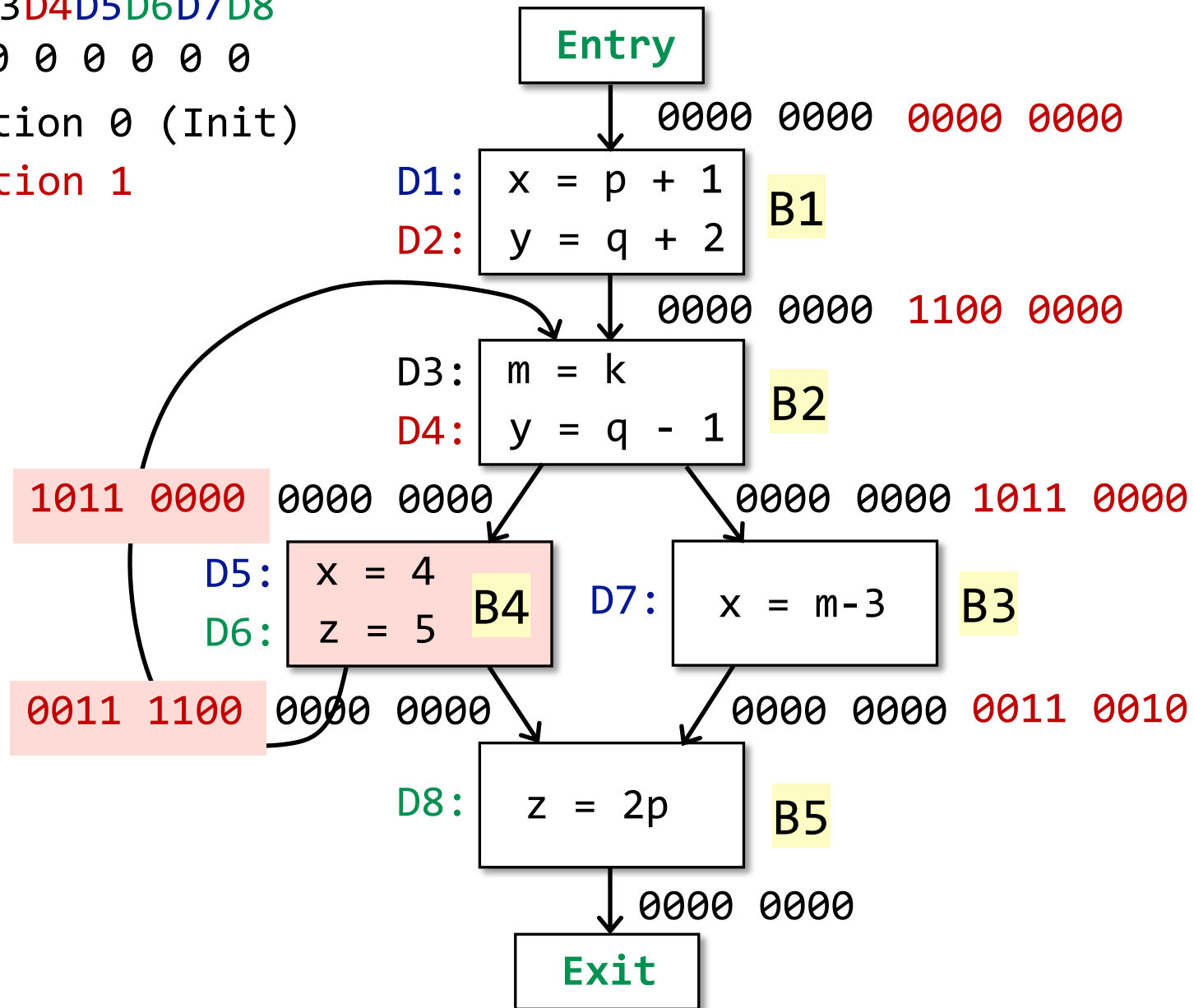


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

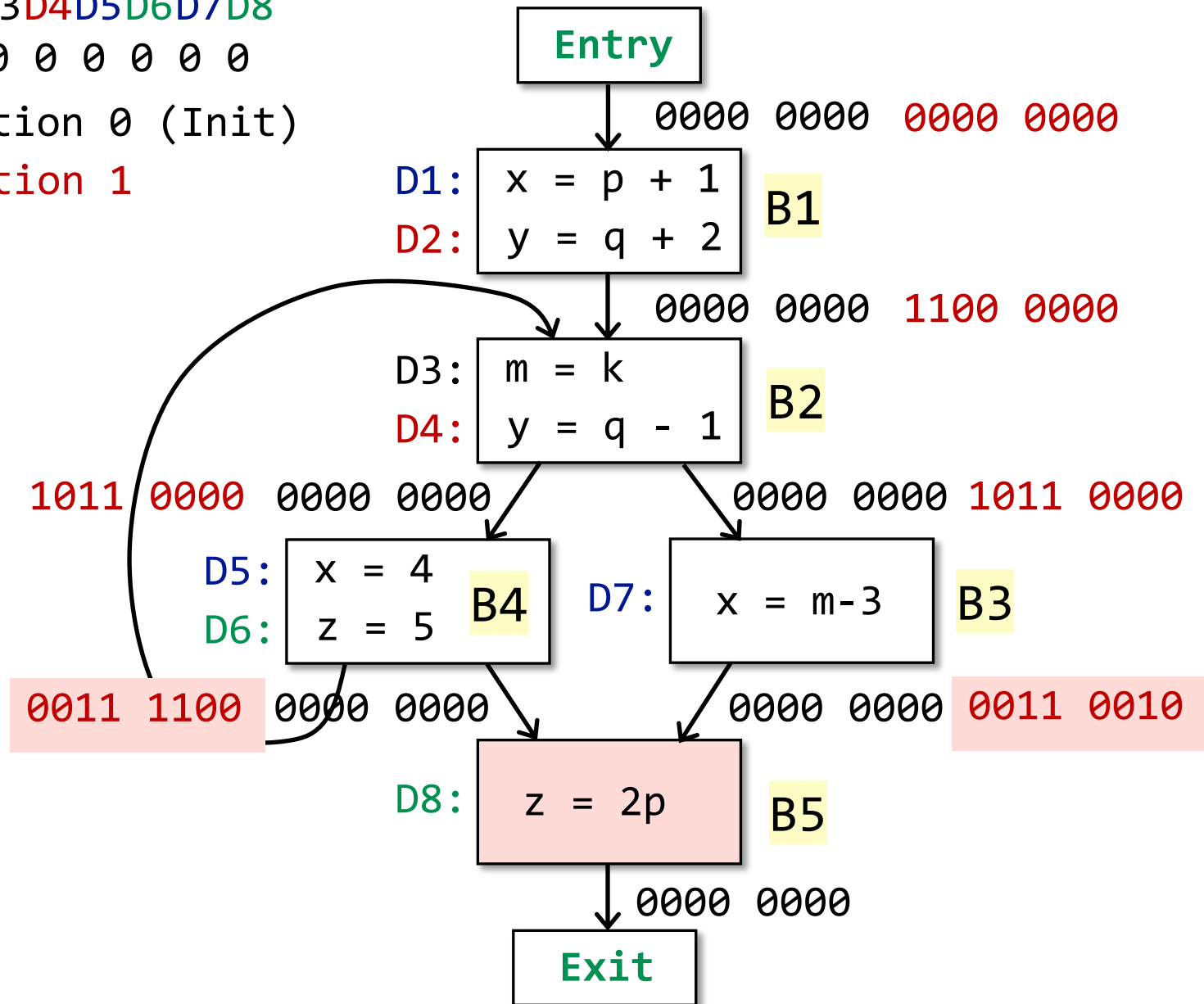


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

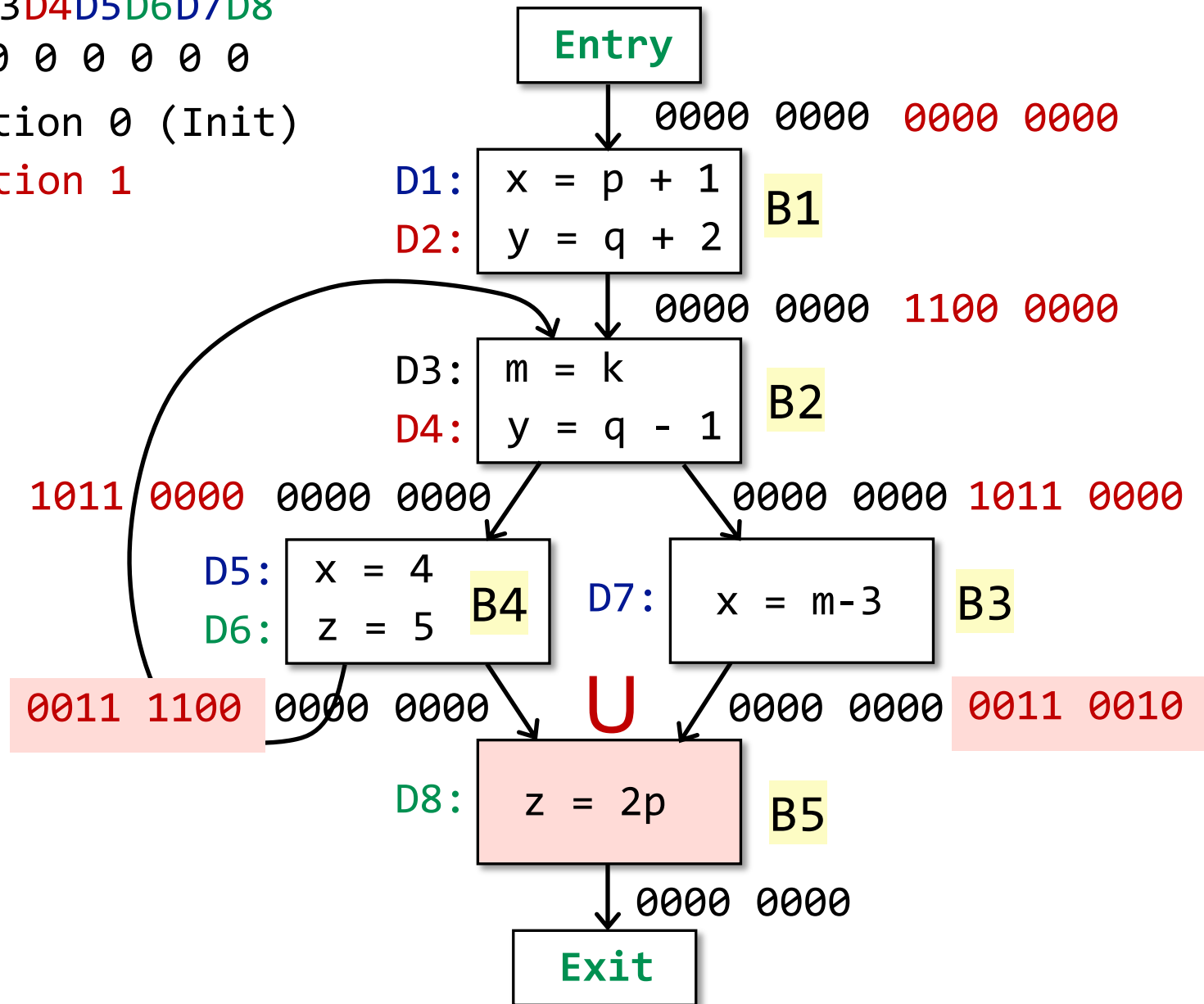


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

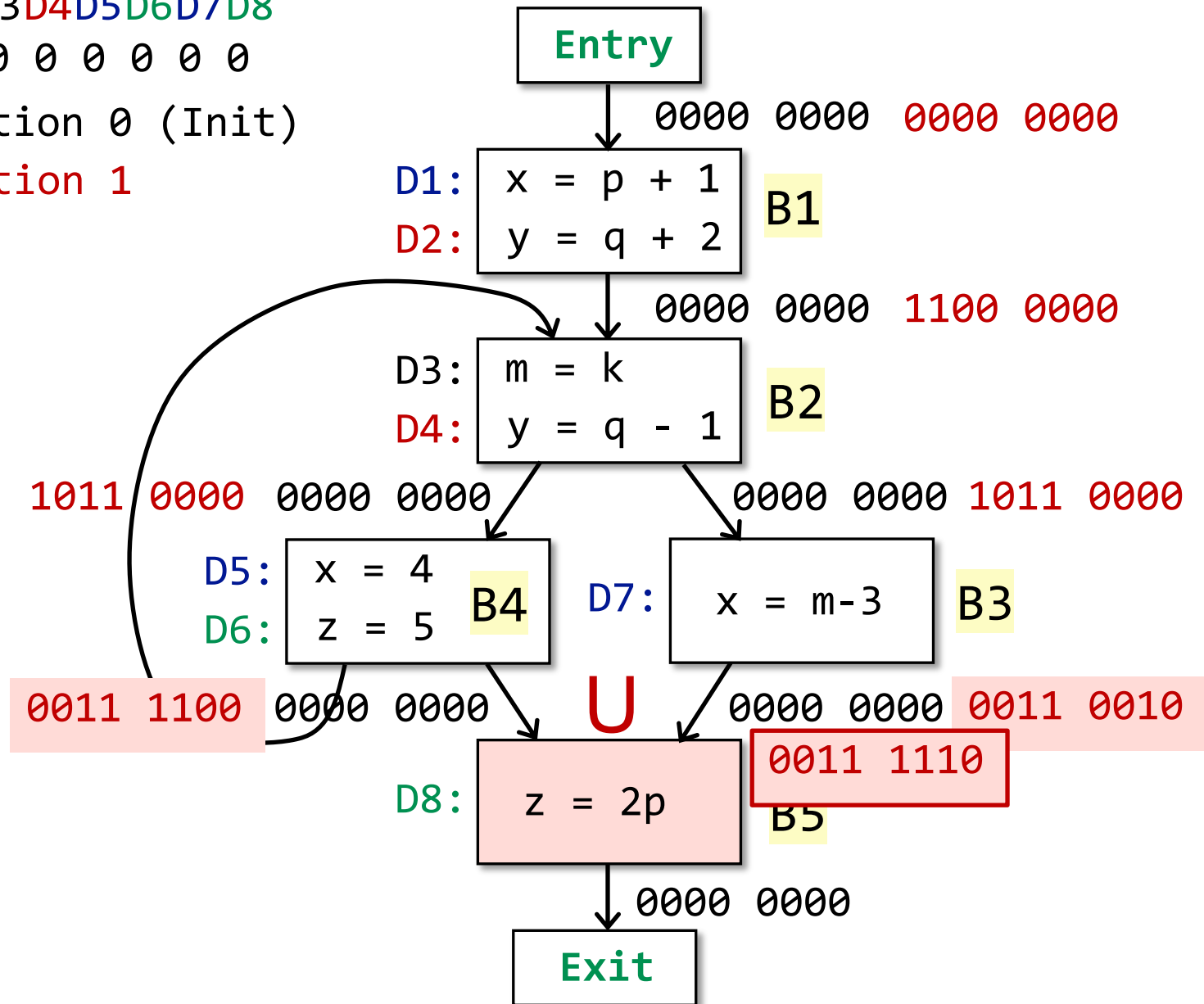


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

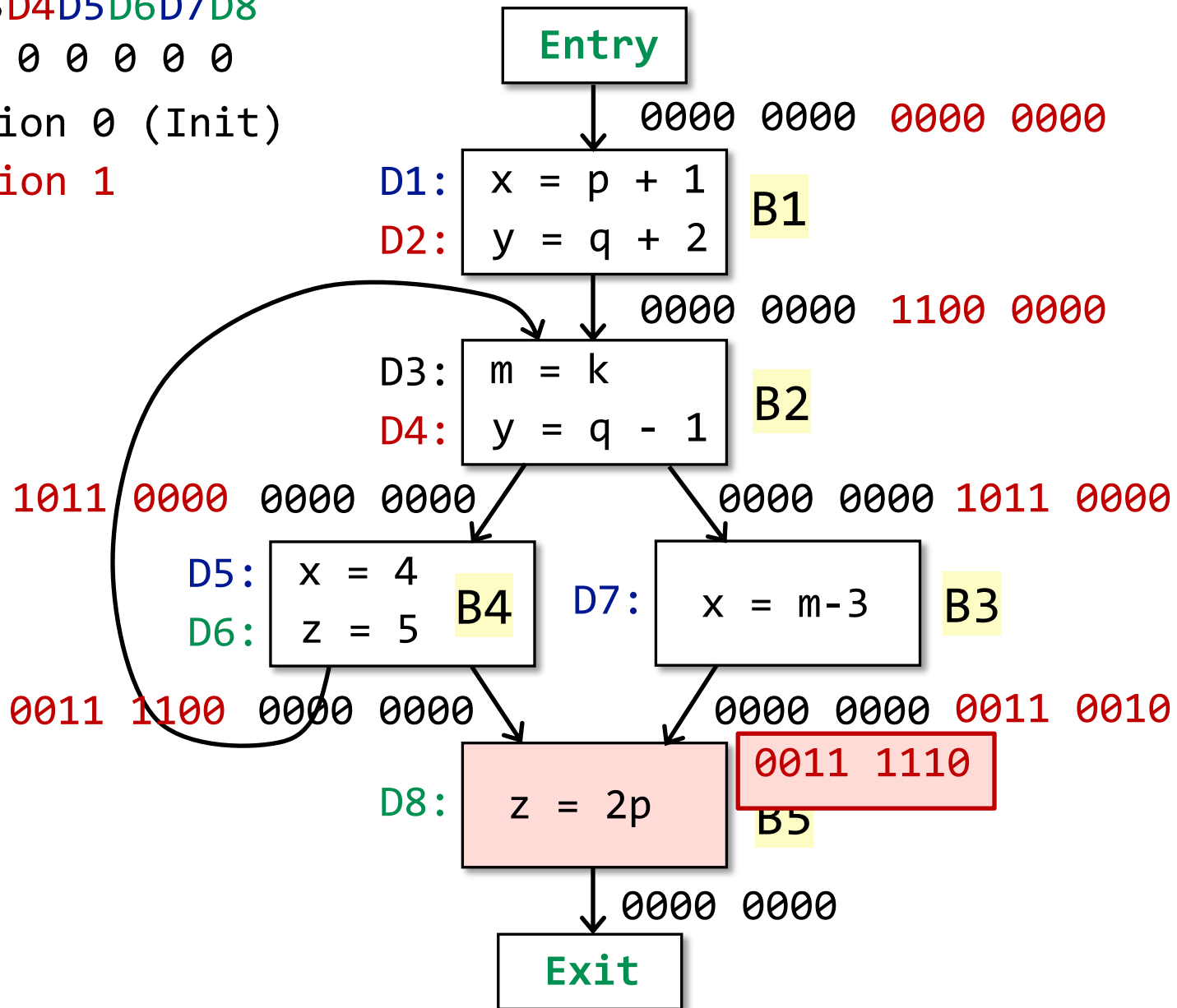


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

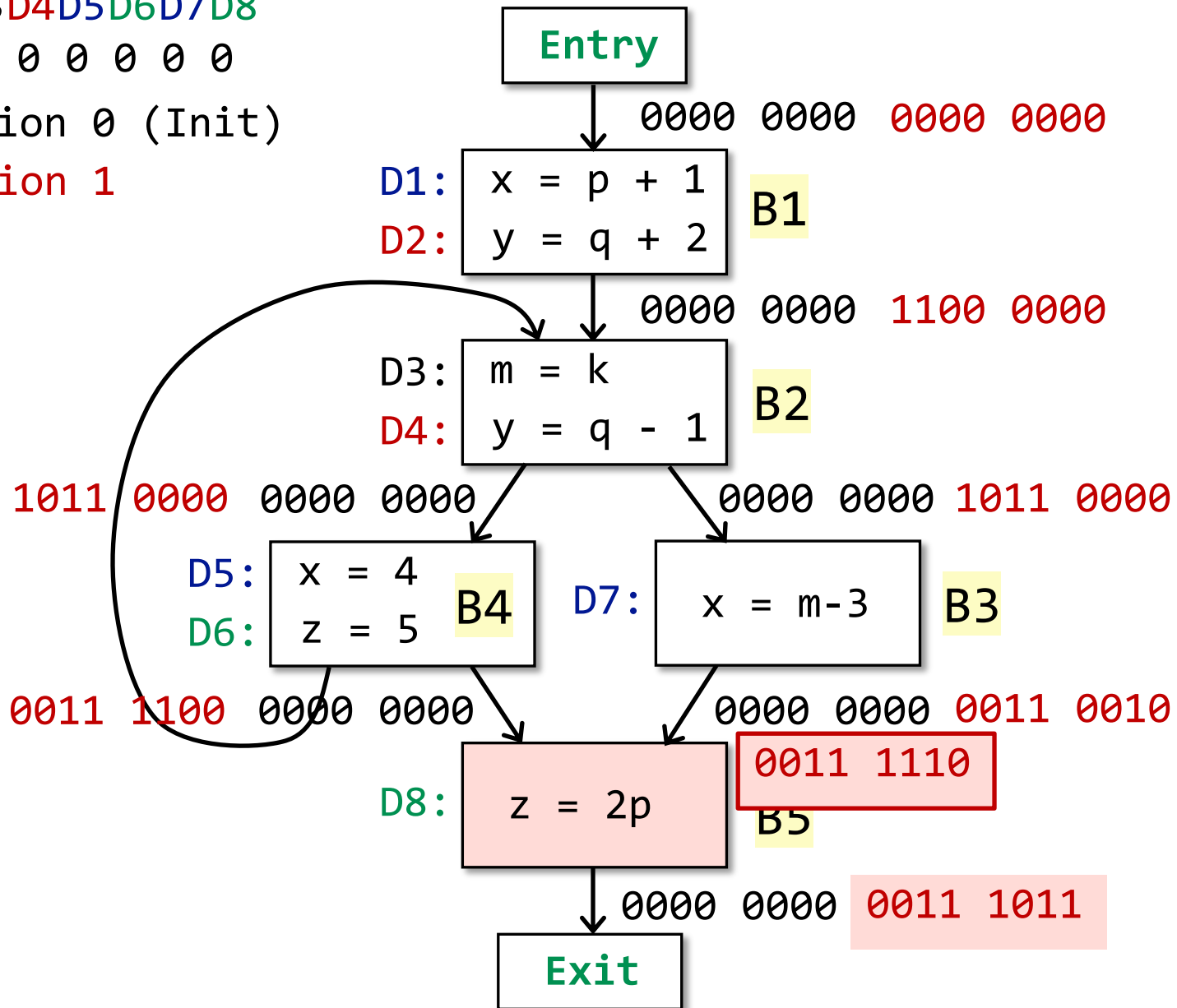


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

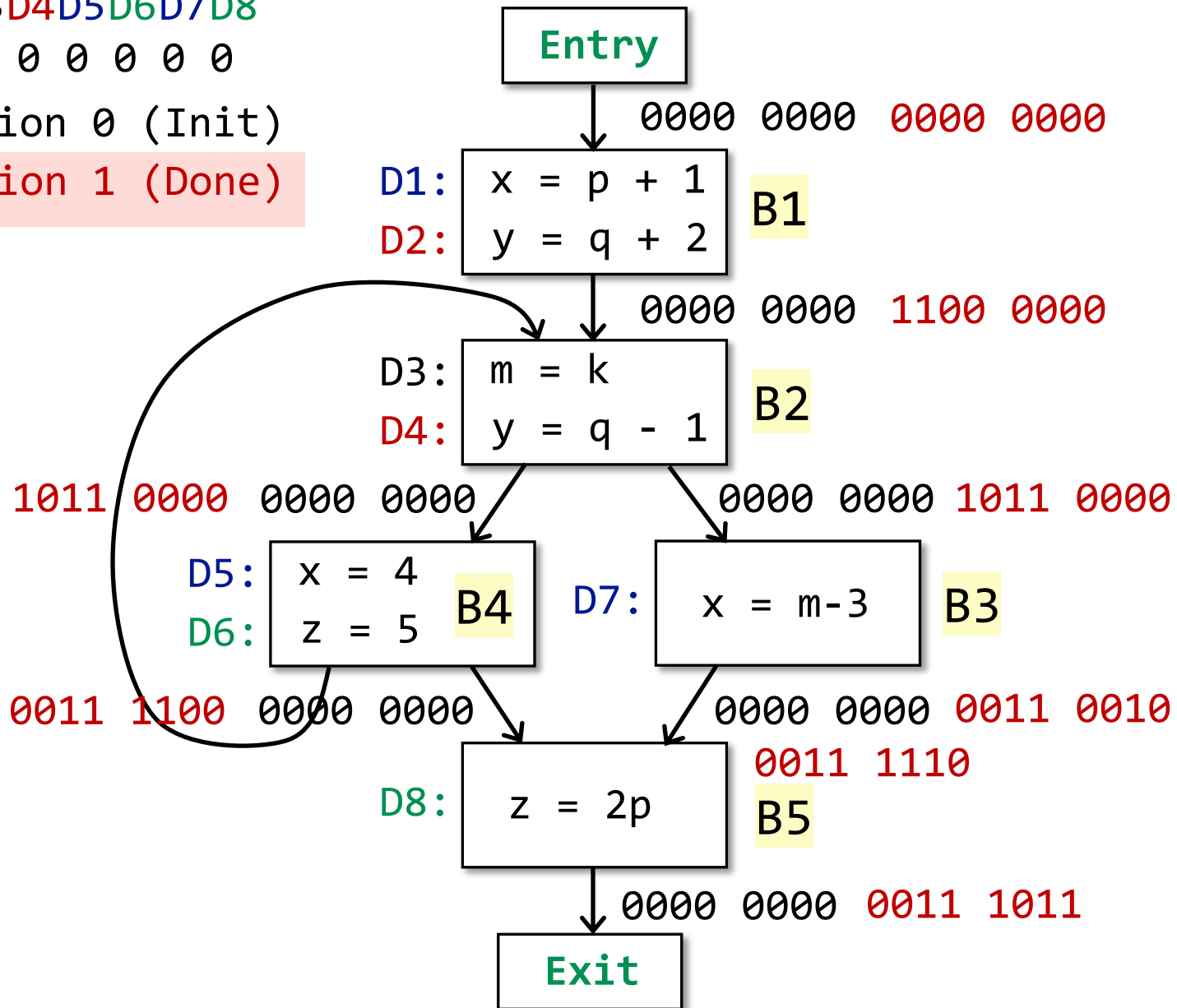


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)



Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

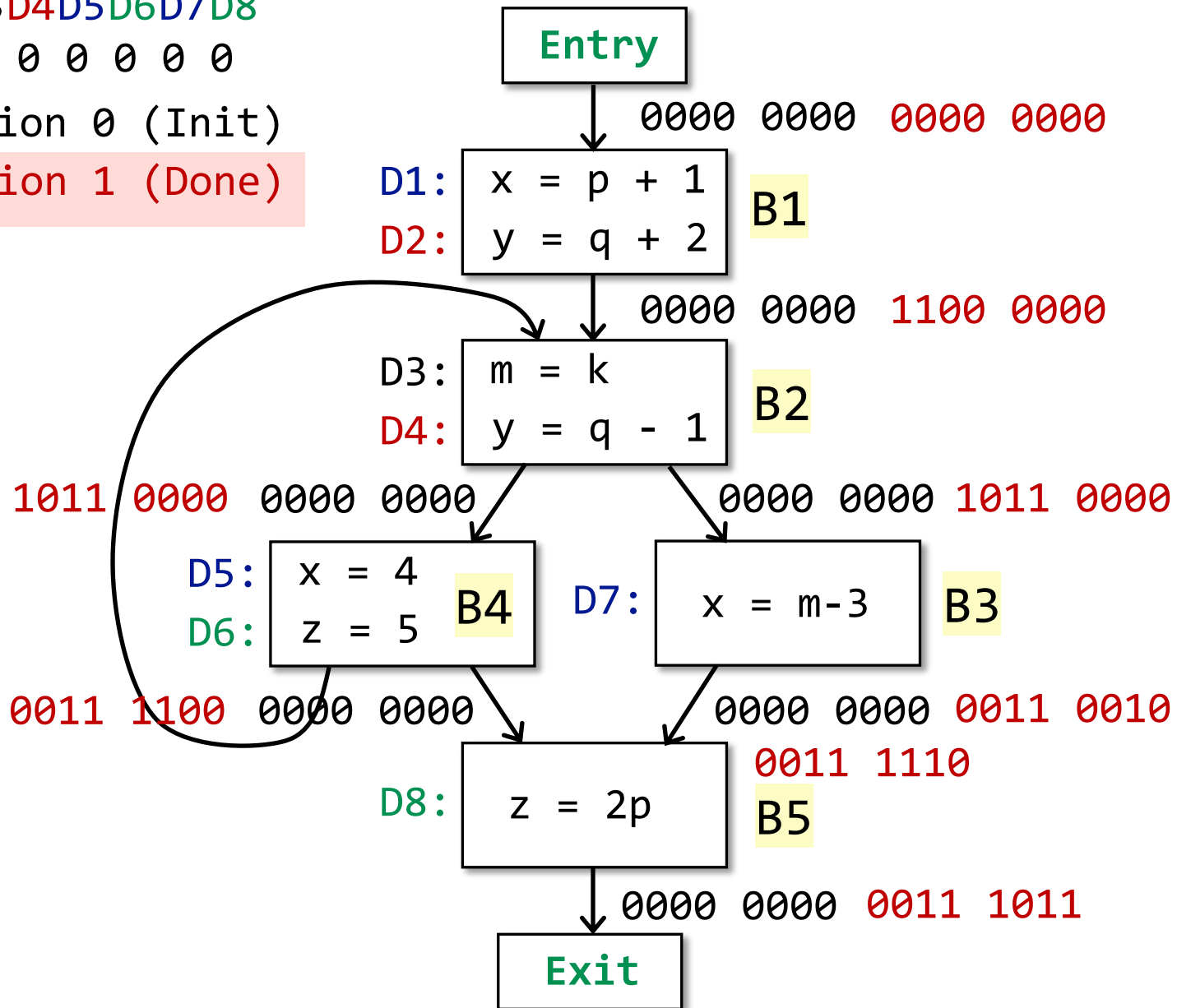
```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ )  
    OUT[B] =  $\emptyset$ ;  
    while (changes to any OUT occur)  
        for (each basic block  $B \setminus entry$ ) {  
             $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$ ;  
             $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
        }
```

D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

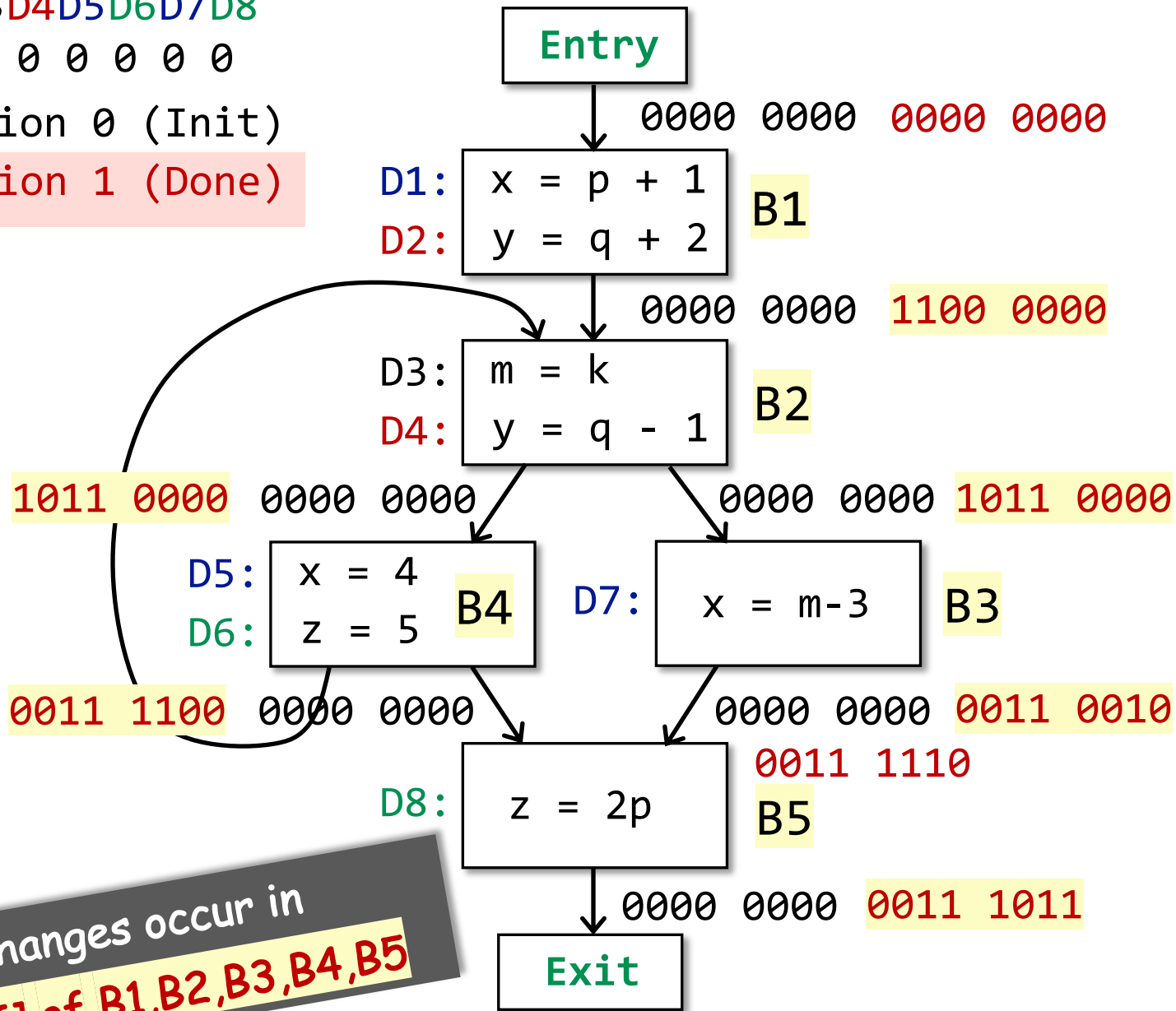


D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)



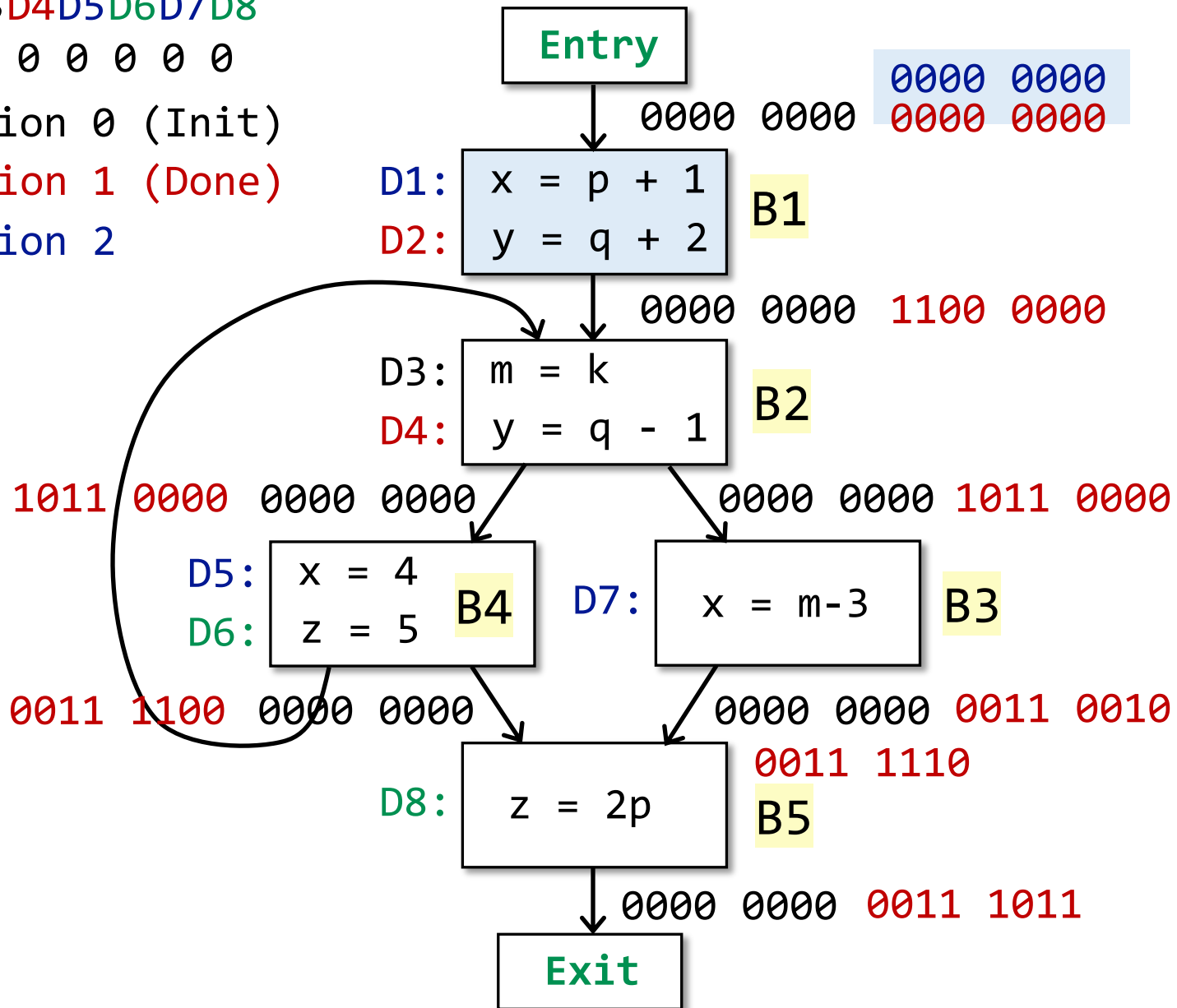
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



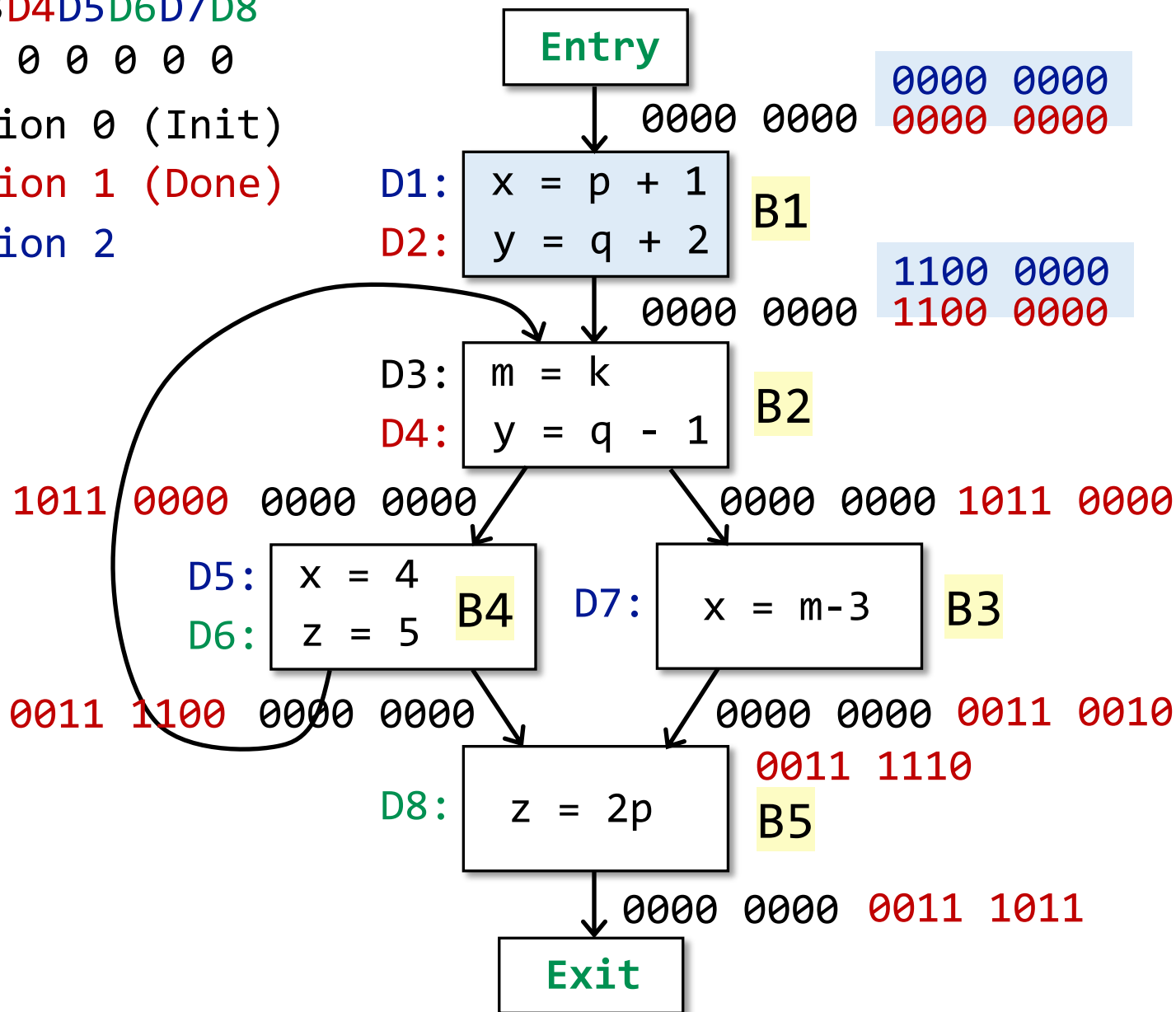
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



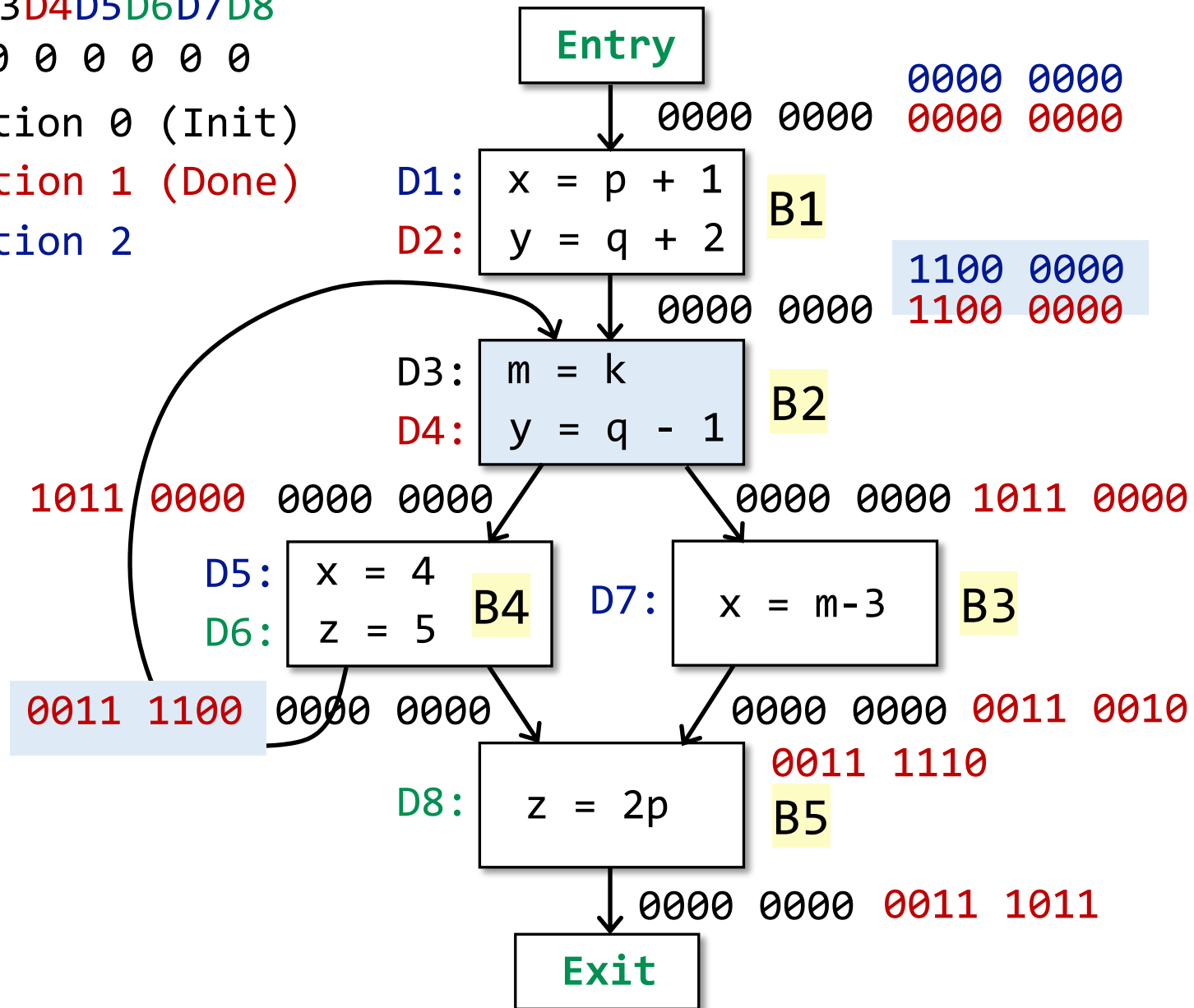
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



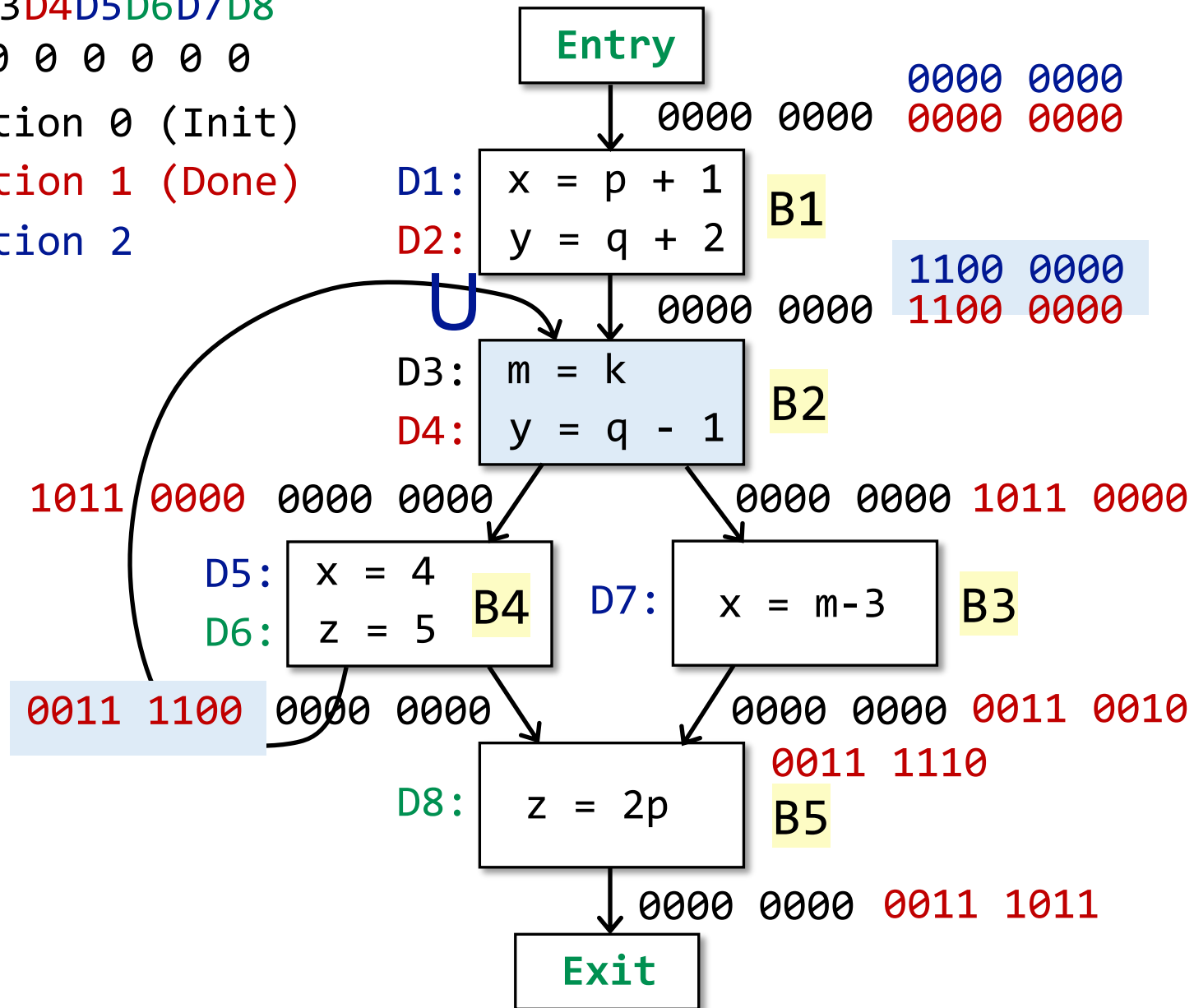
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



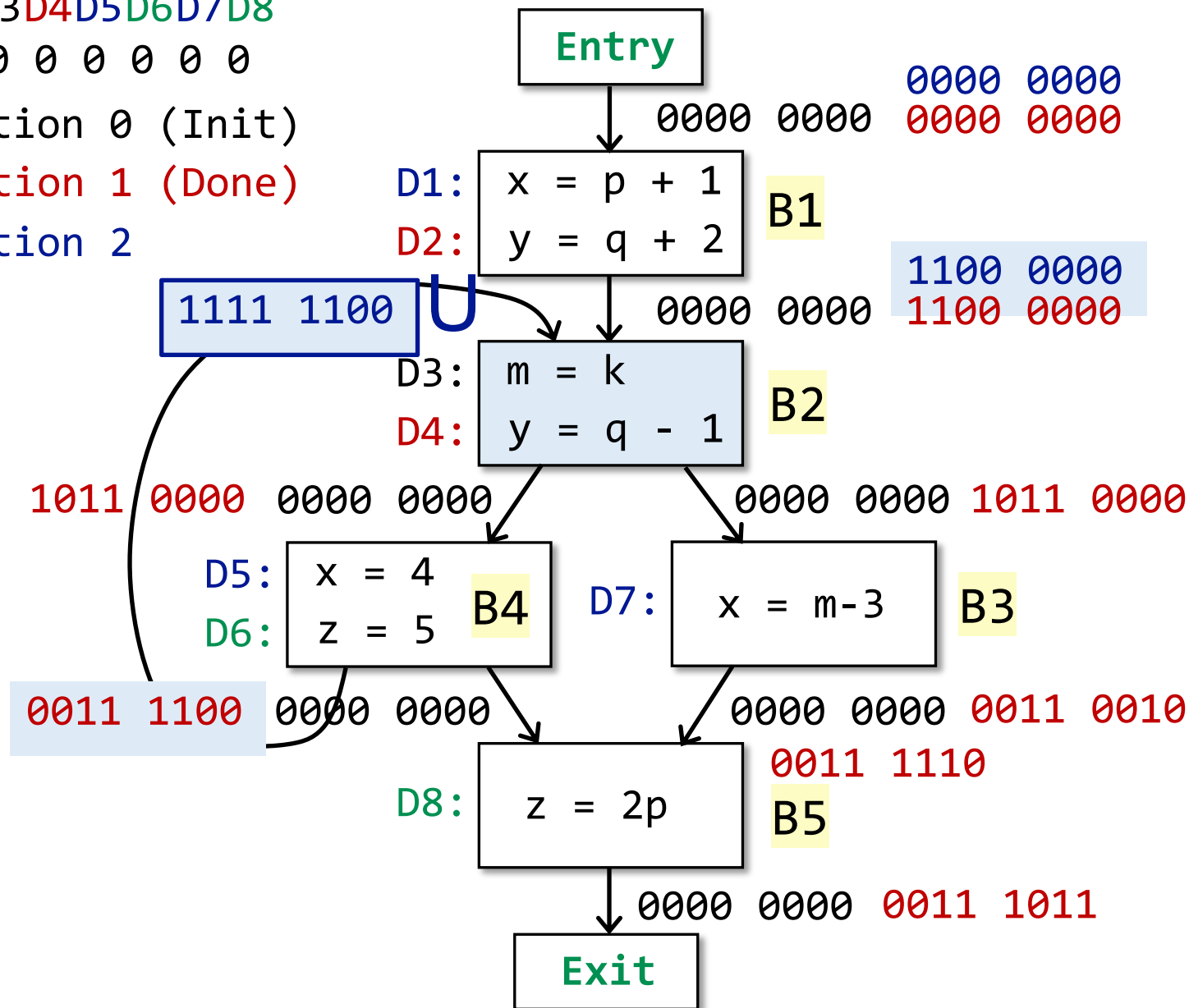
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



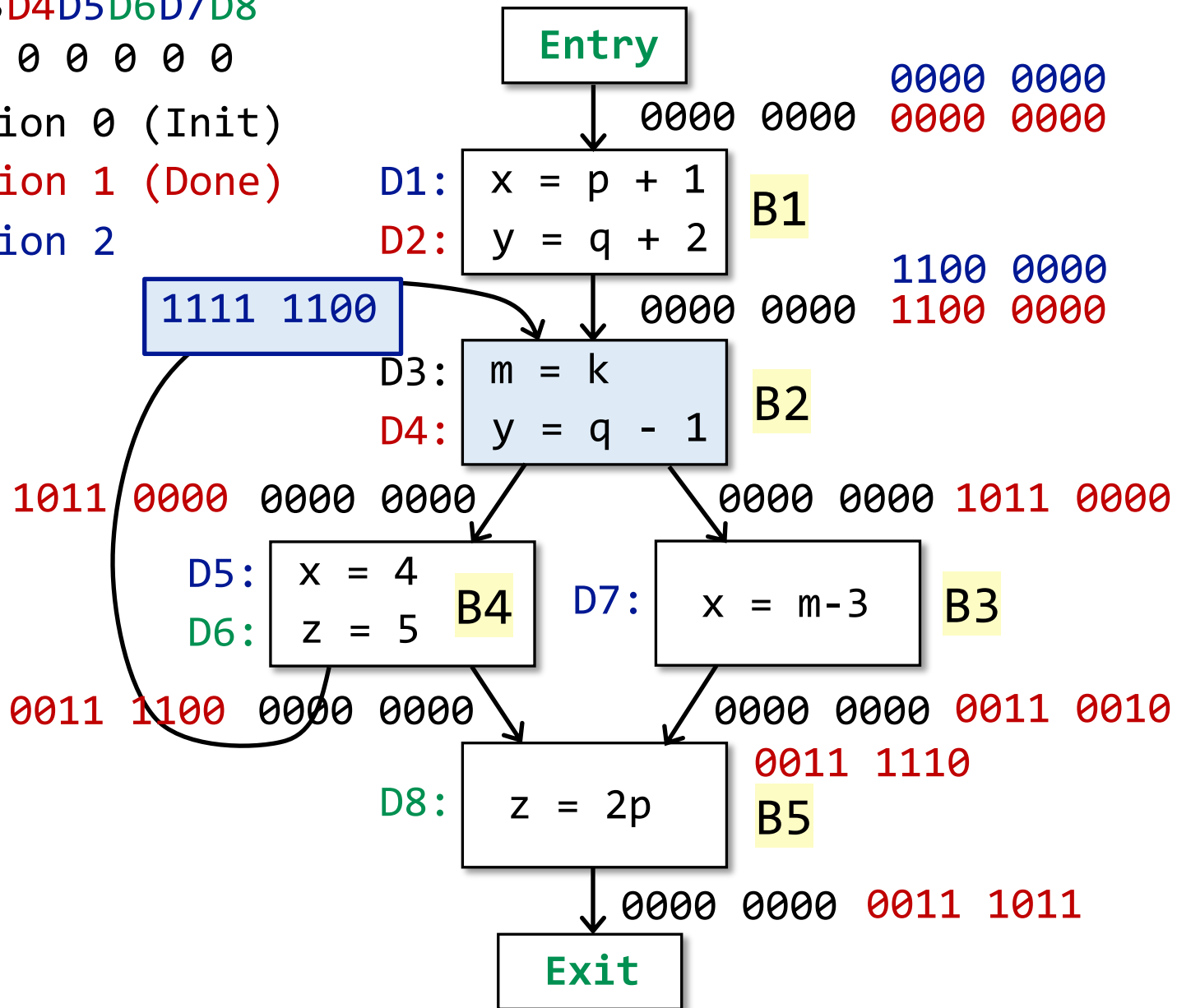
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



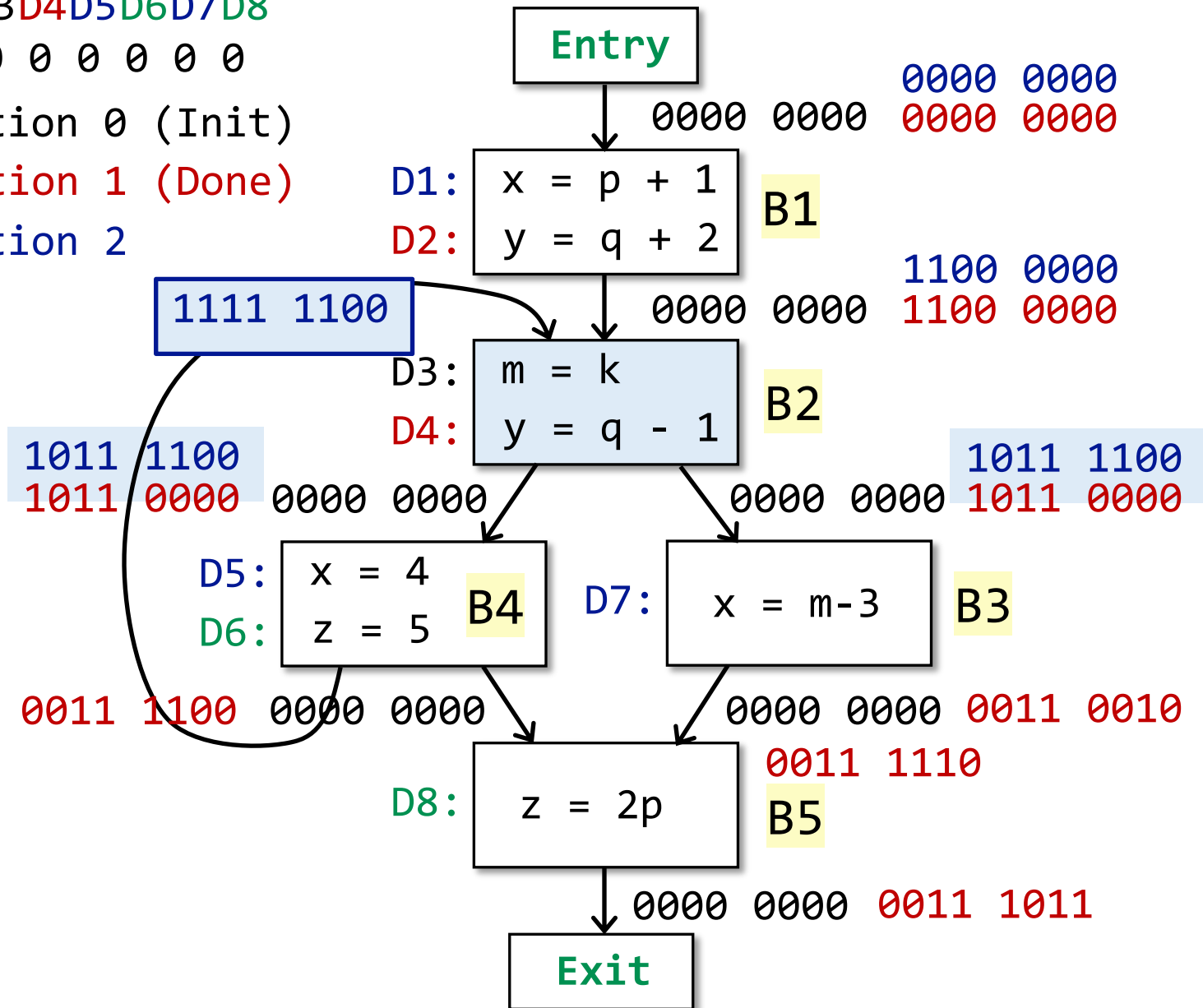
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



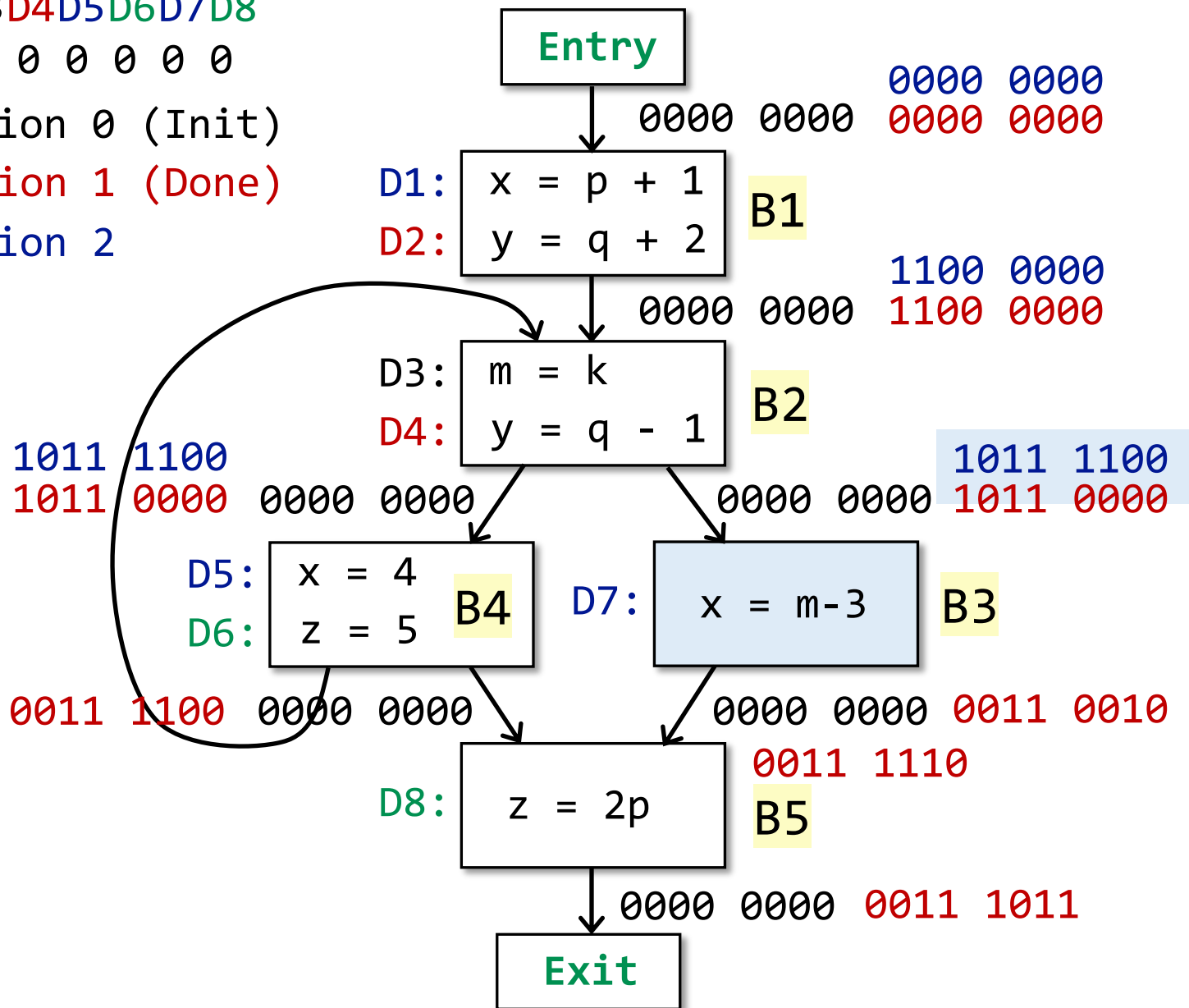
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



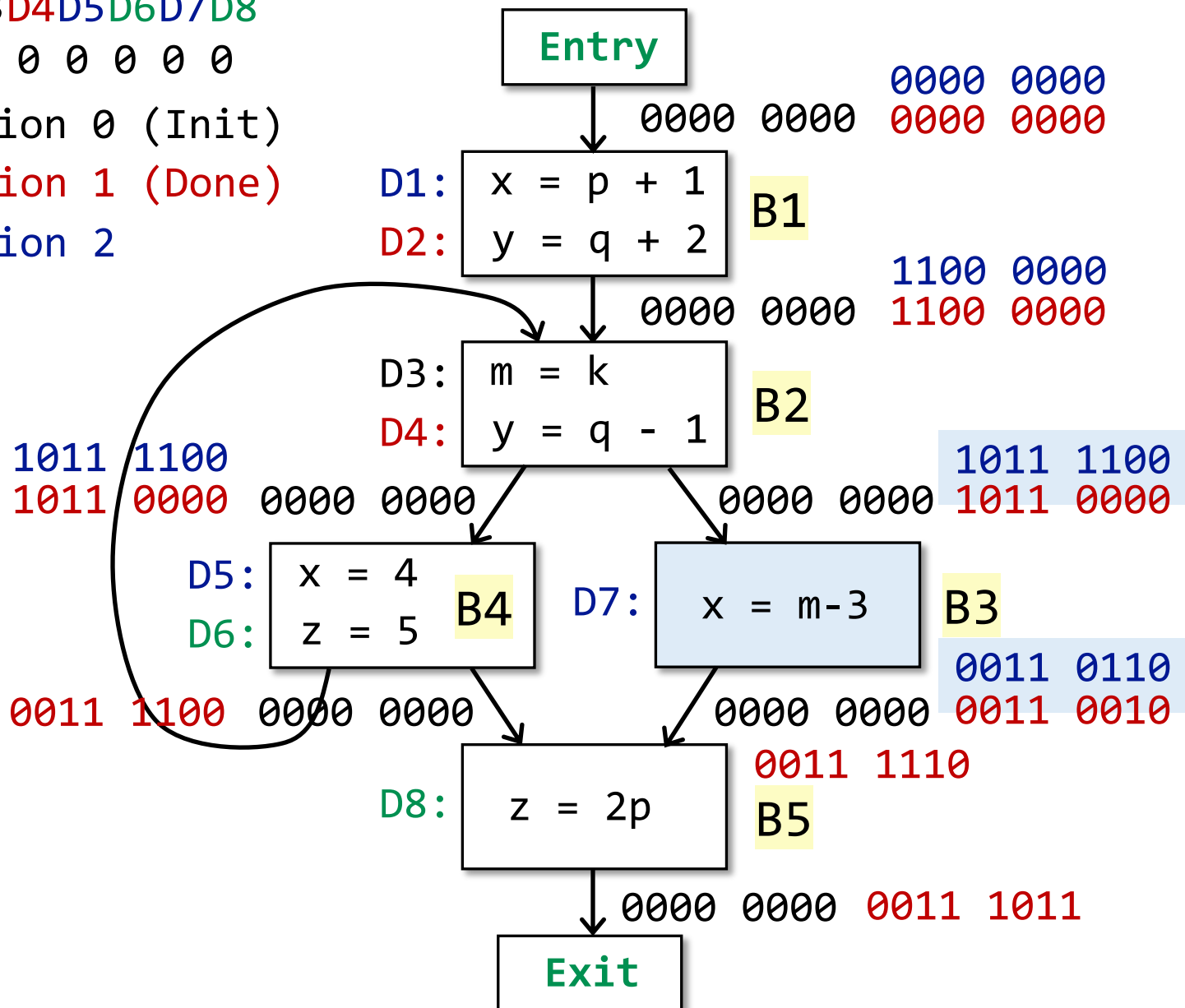
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



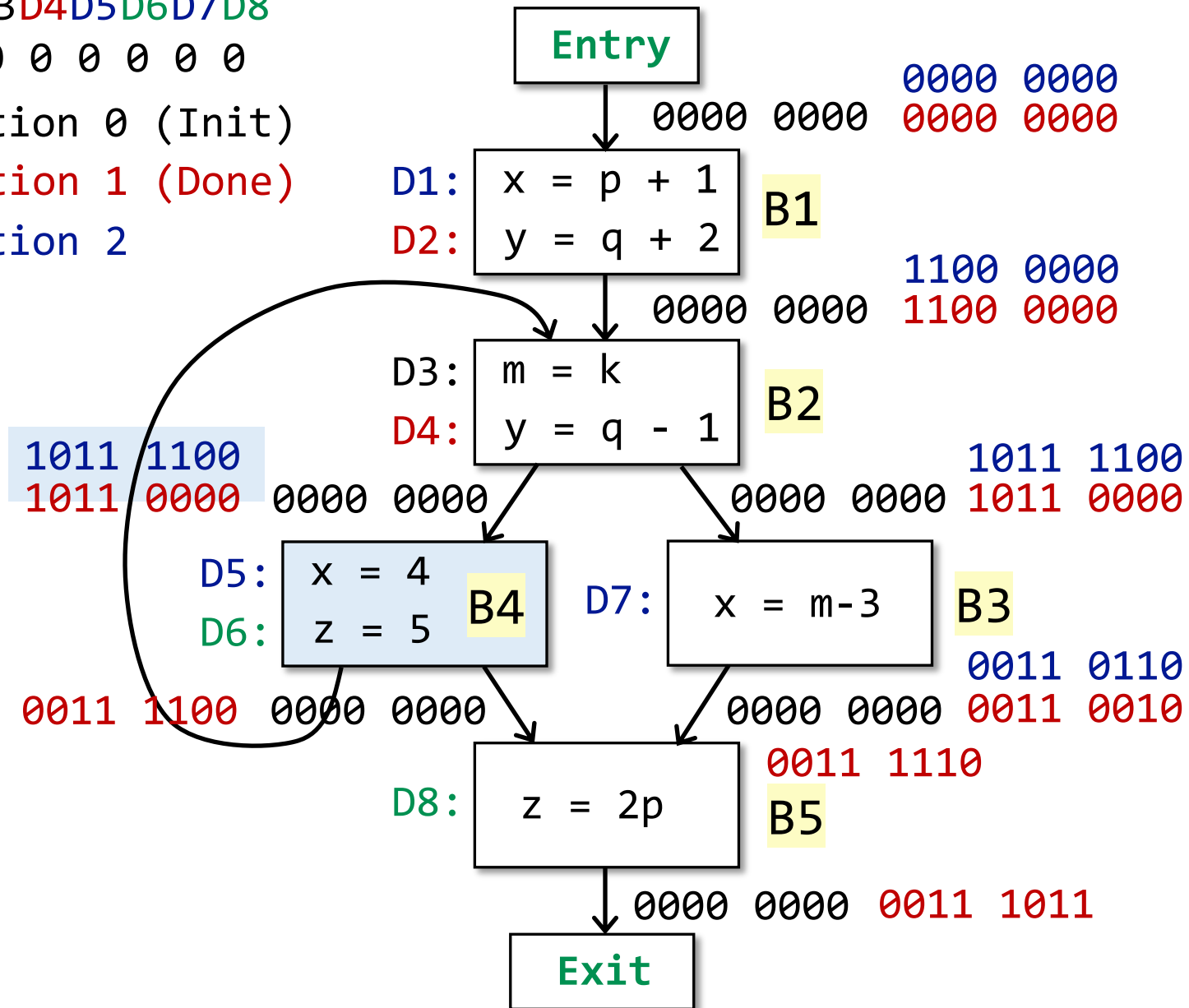
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



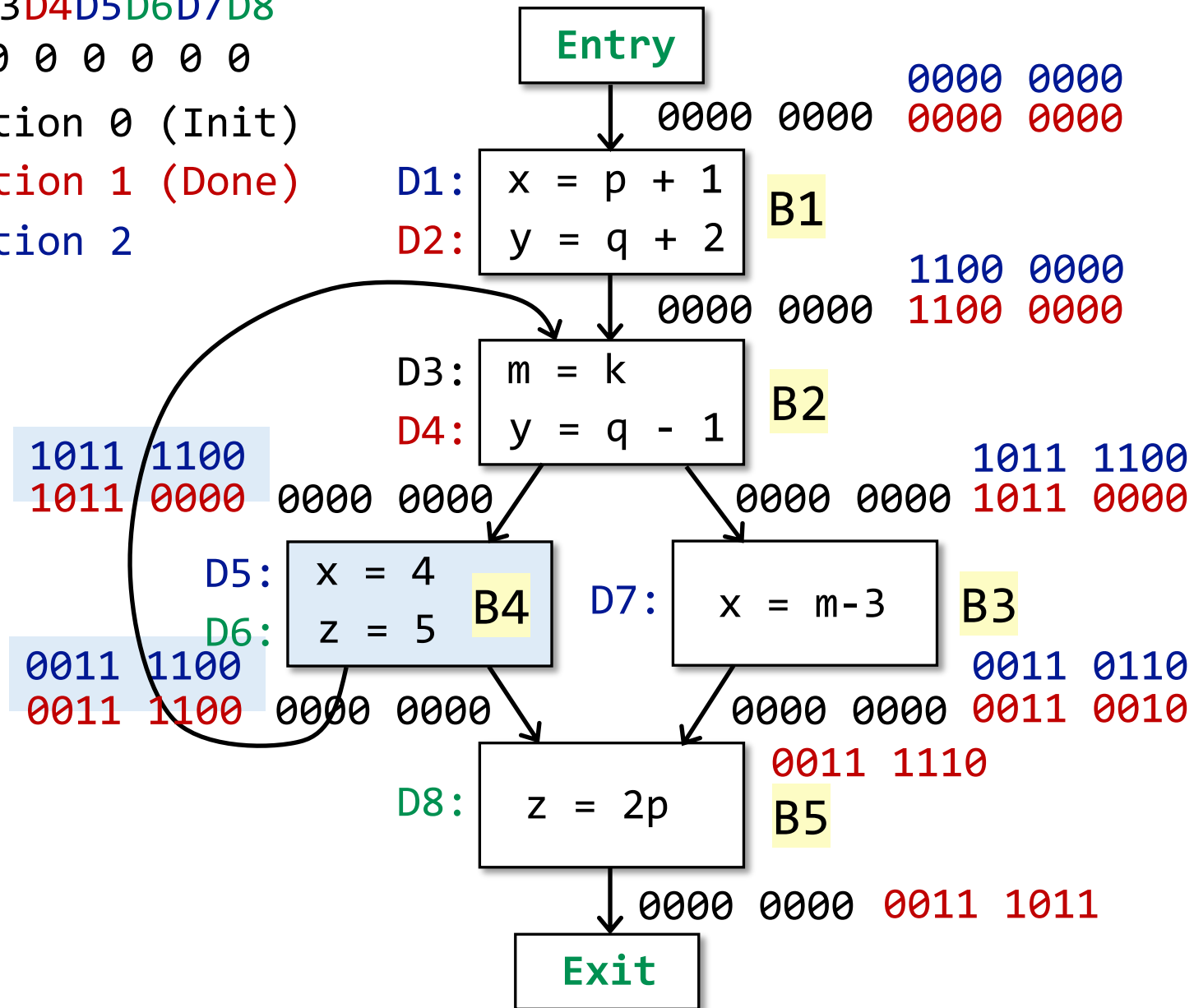
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



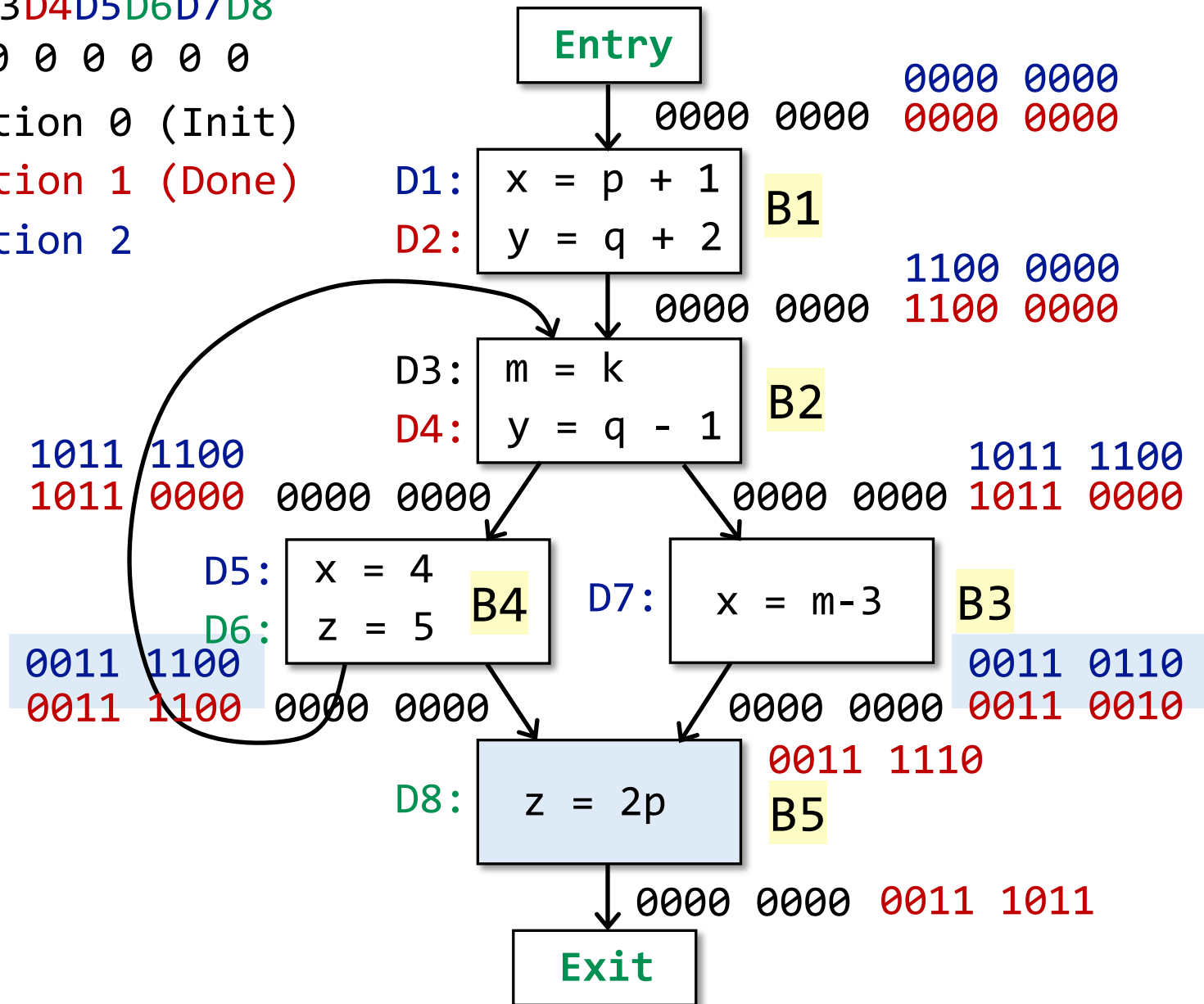
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



0 0 0 0 0 0 0 0

Iteration 1 (Done)



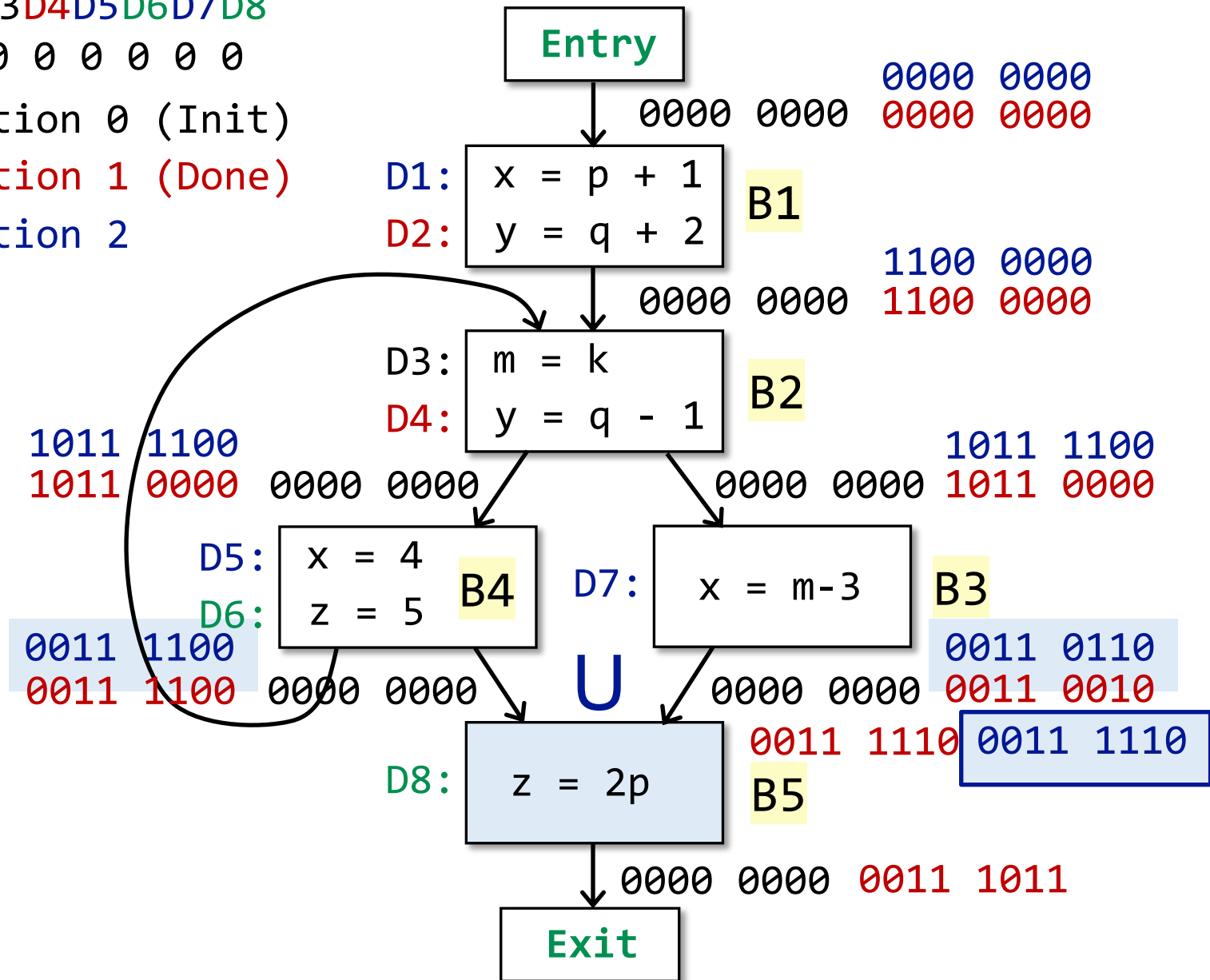
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



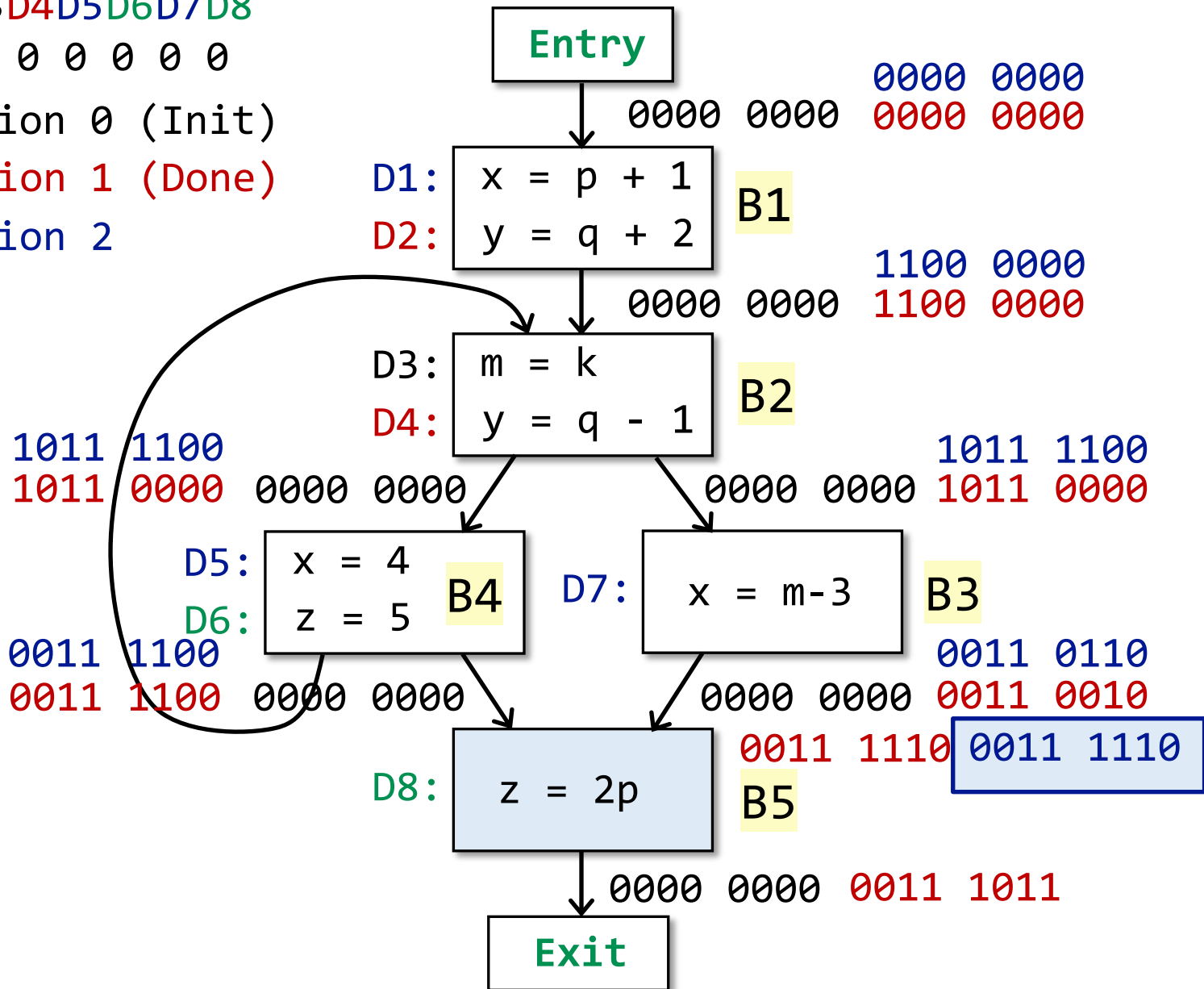
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



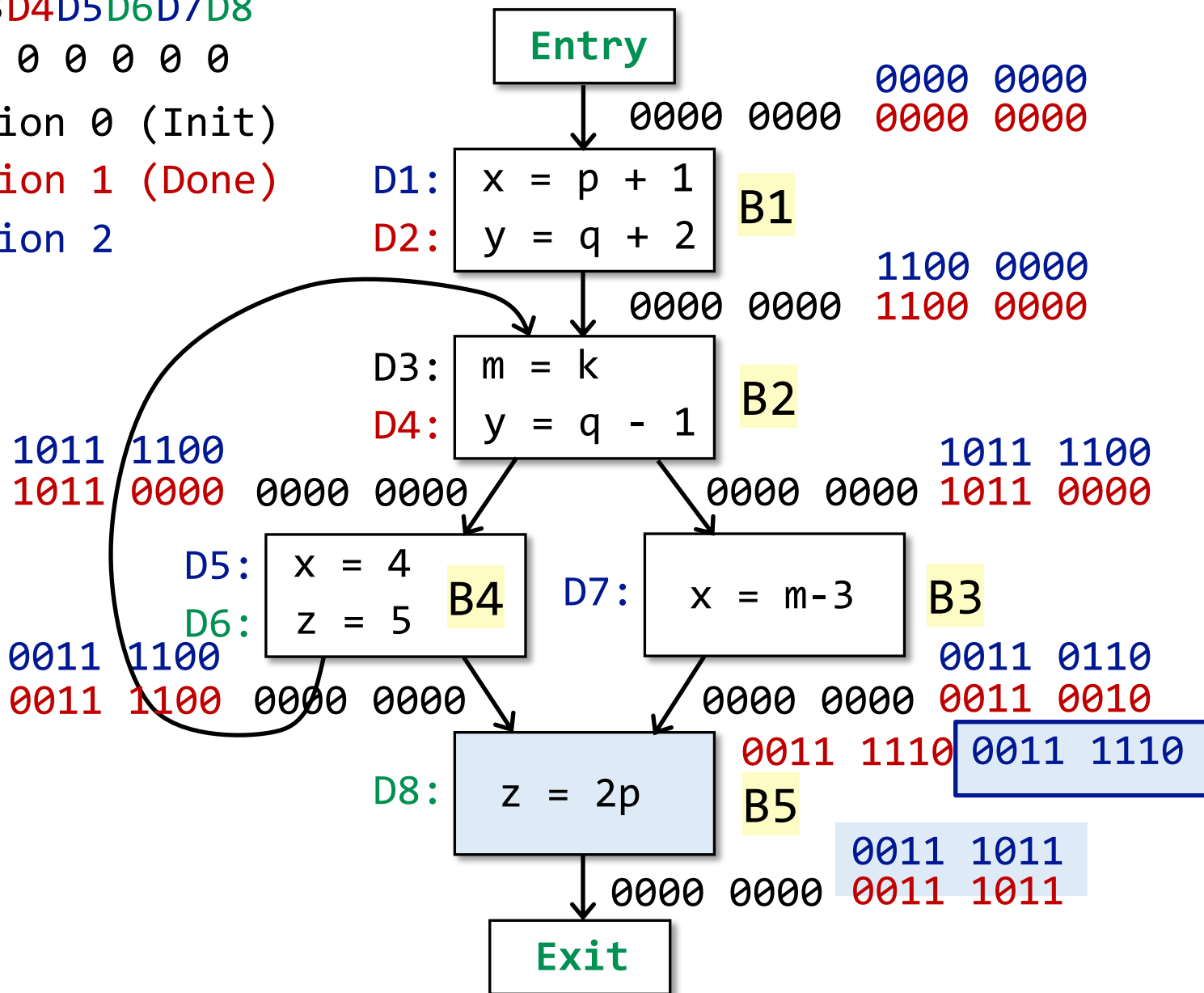
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



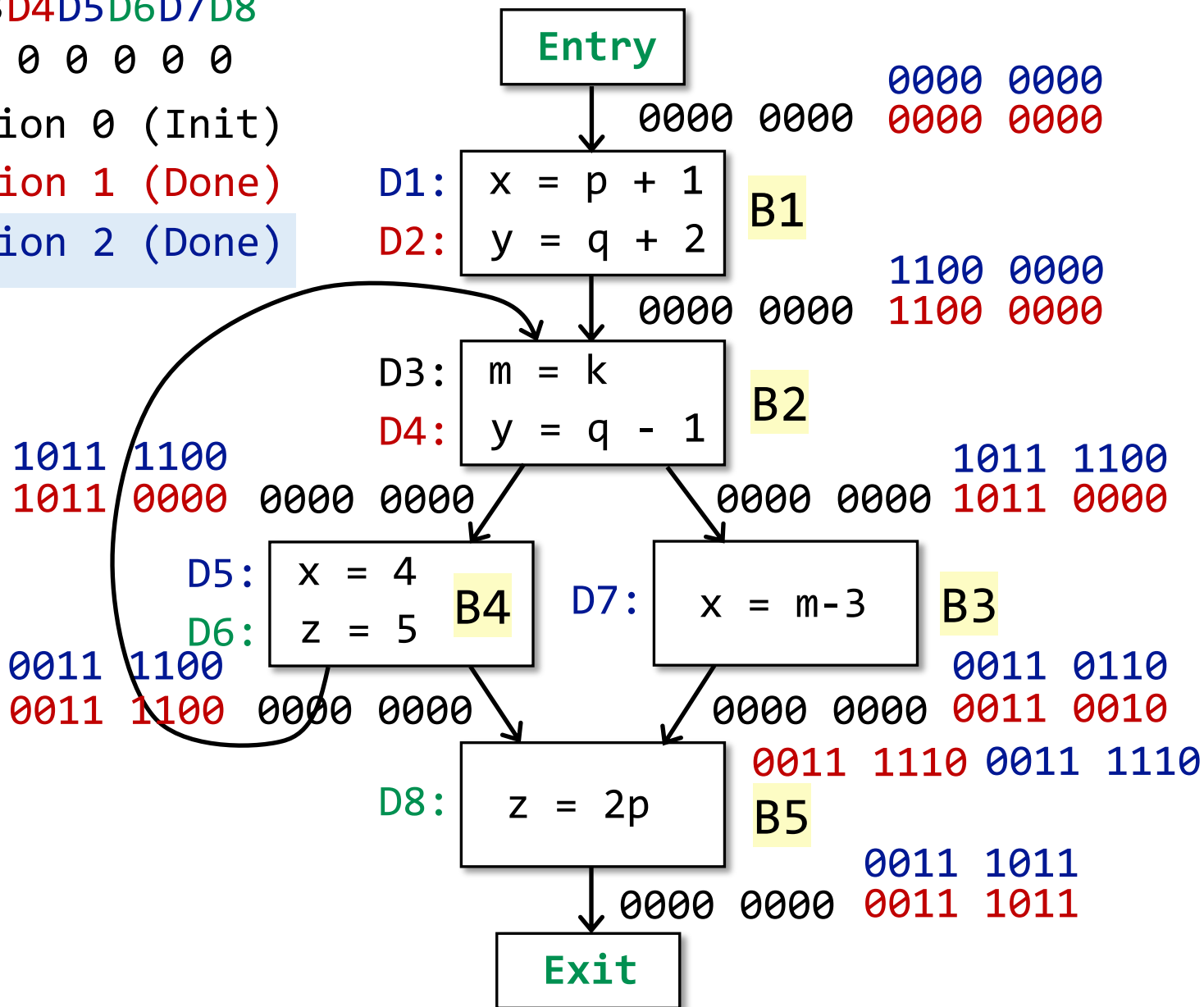
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)



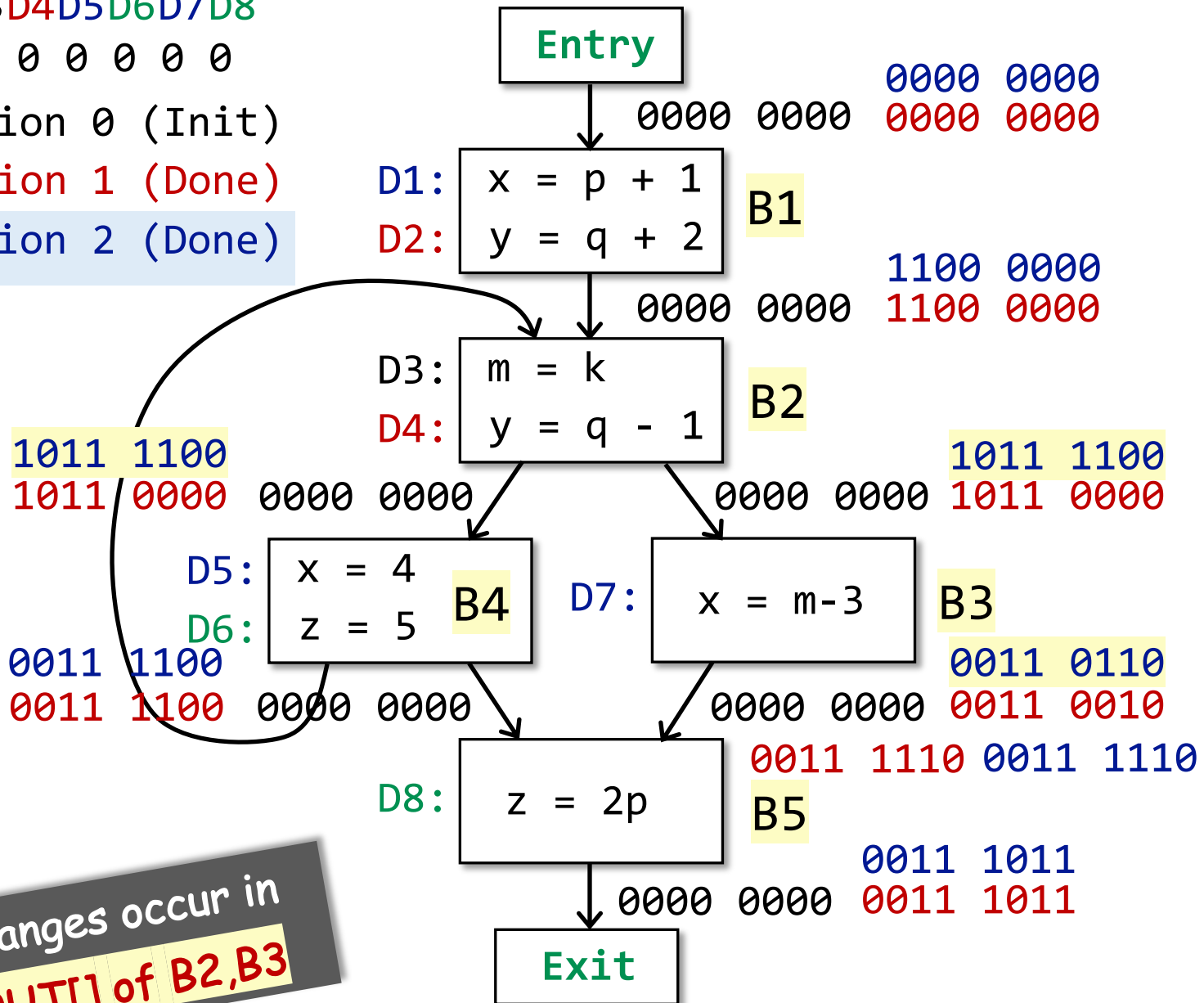
D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)



Changes occur in
OUT[] of B2, B3

D1D2D3D4D5D6D7D8

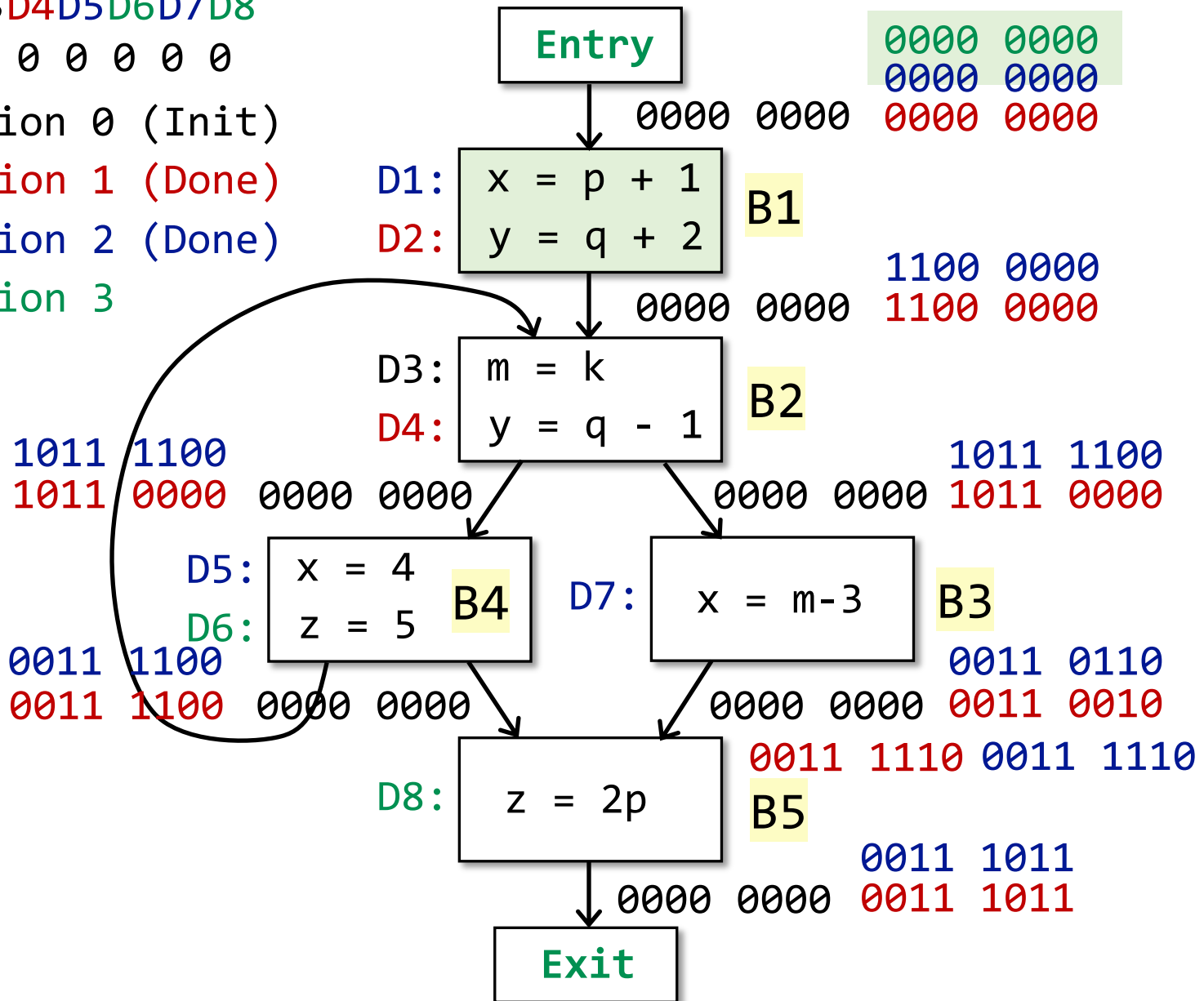
0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



D1D2D3D4D5D6D7D8

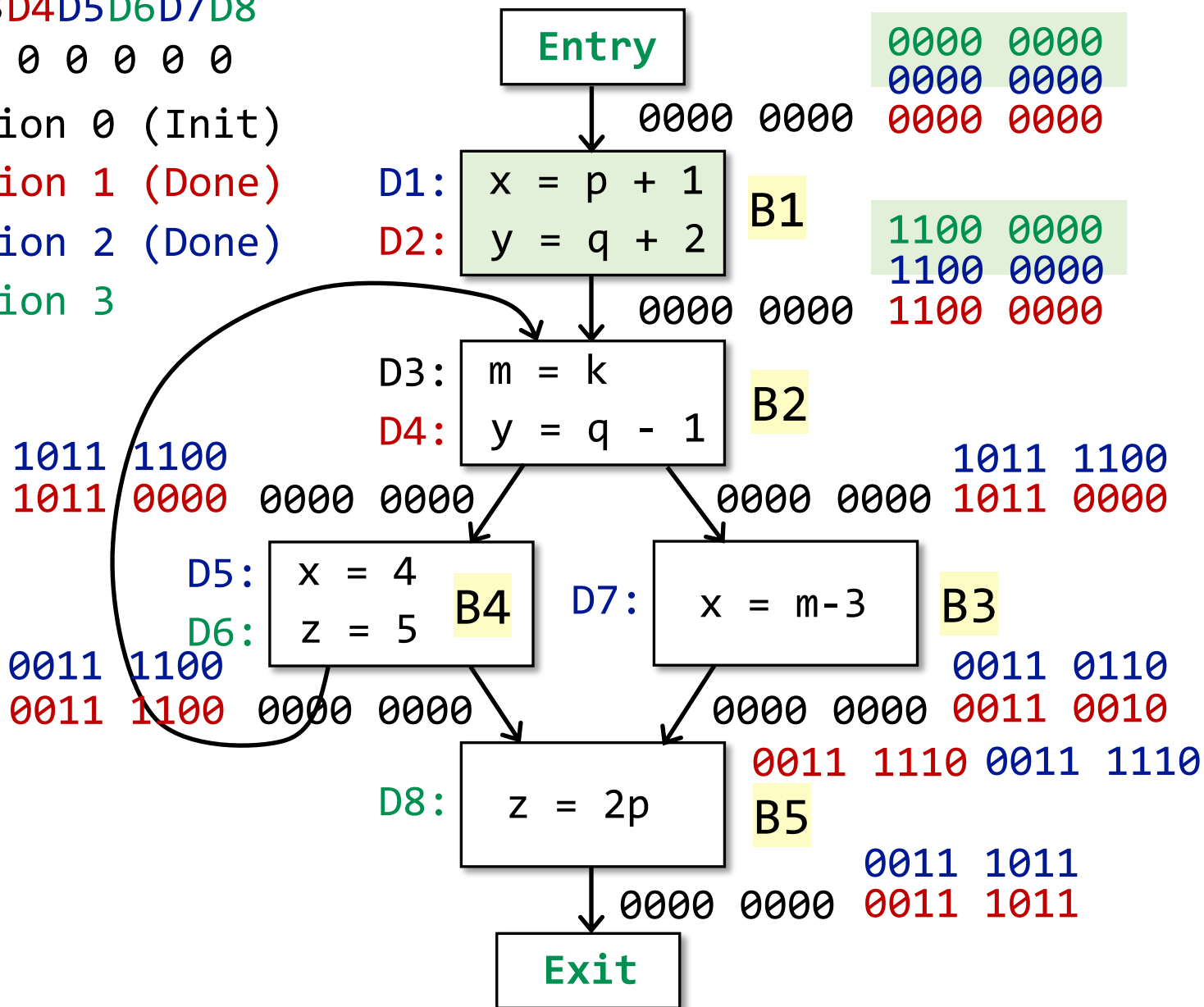
0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



0 0 0 0 0 0 0 0

Iteration 1 (Done)

Iteration 3



D1D2D3D4D5D6D7D8

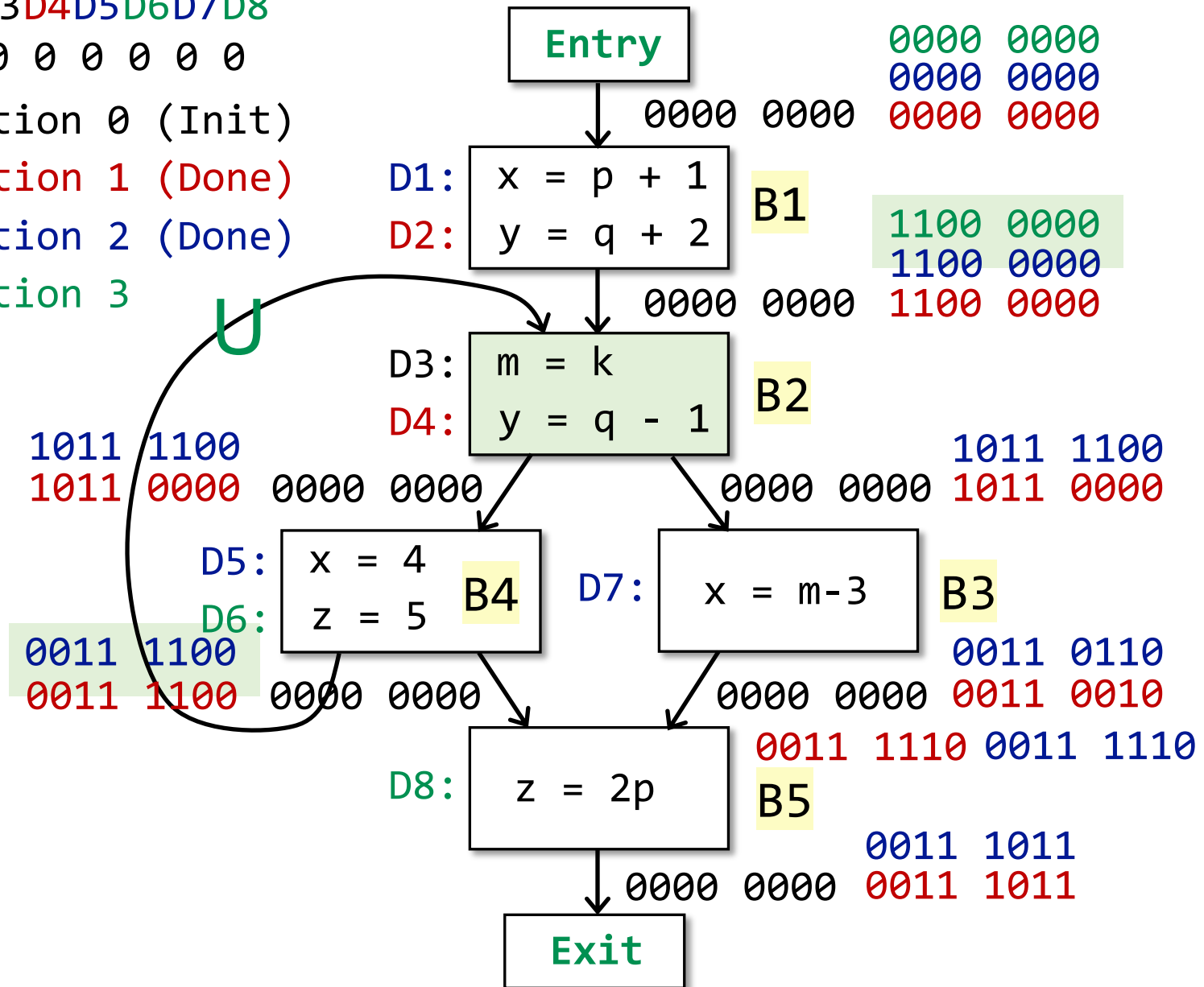
0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



0 0 0 0 0 0 0 0

Iteration 3



0 0 0 0 0 0 0 0

Iteration 1 (Done)

Iteration 3



0 0 0 0 0 0 0 0

Iteration 3



D1D2D3D4D5D6D7D8

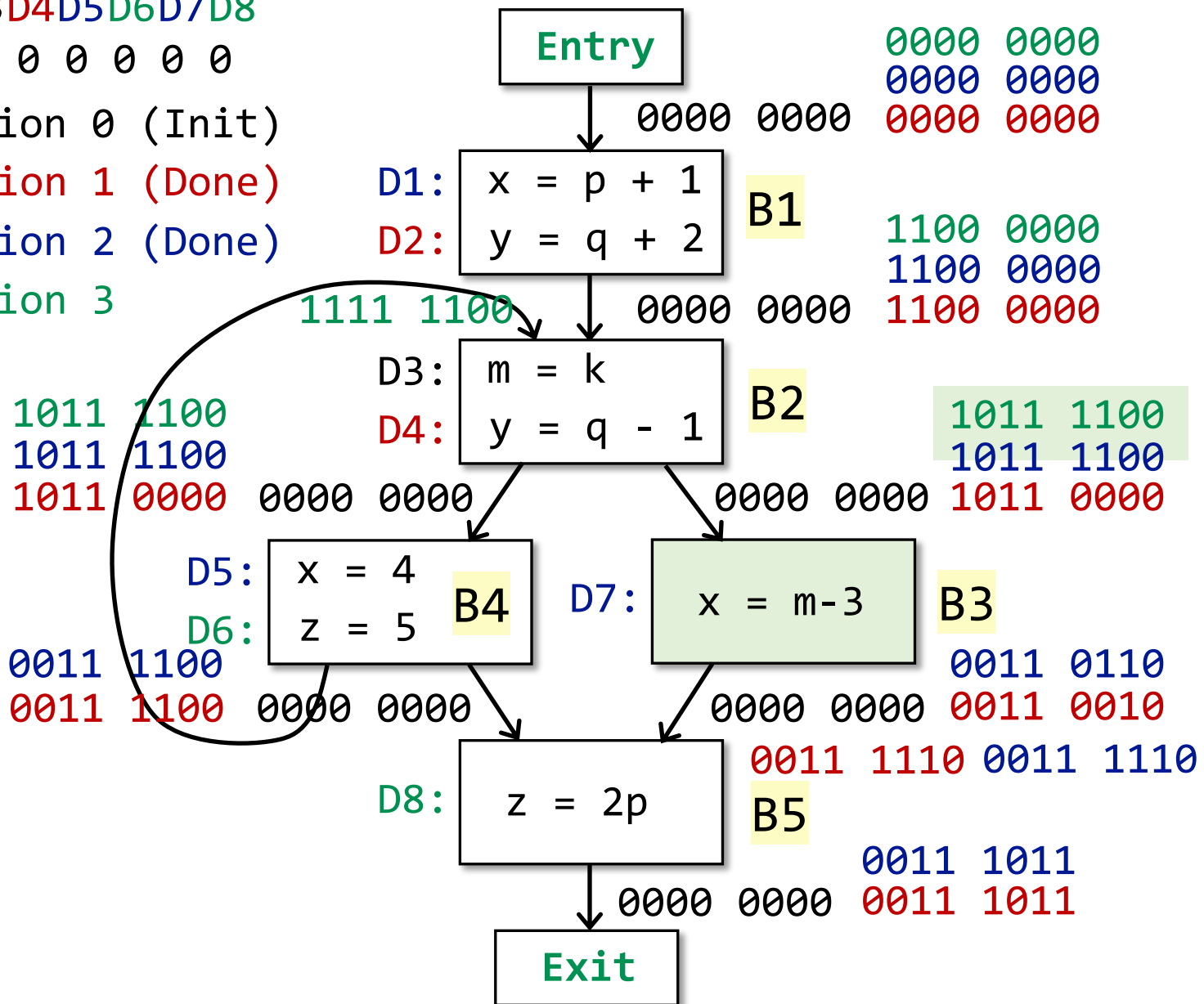
0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



D1D2D3D4D5D6D7D8

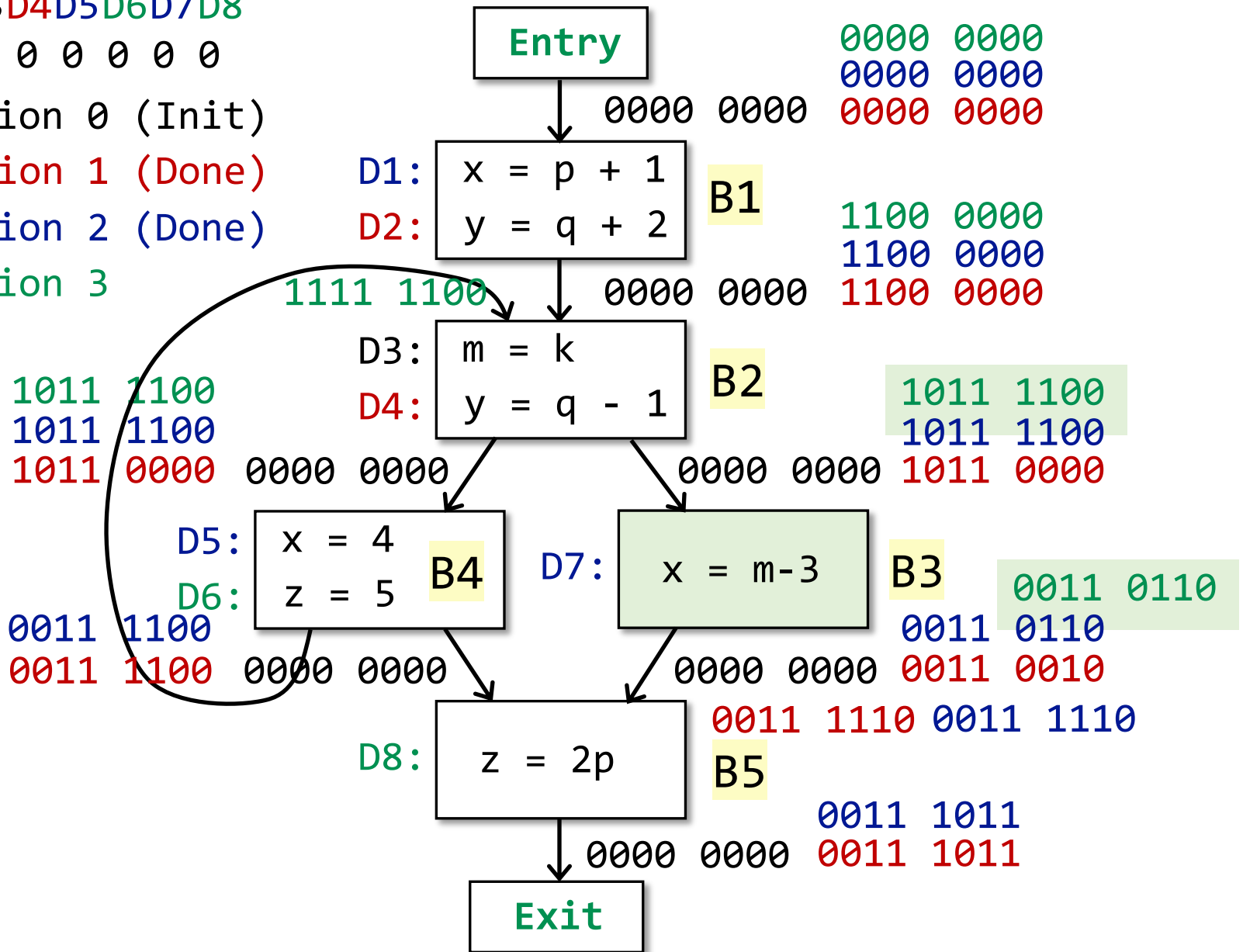
0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



D1D2D3D4D5D6D7D8

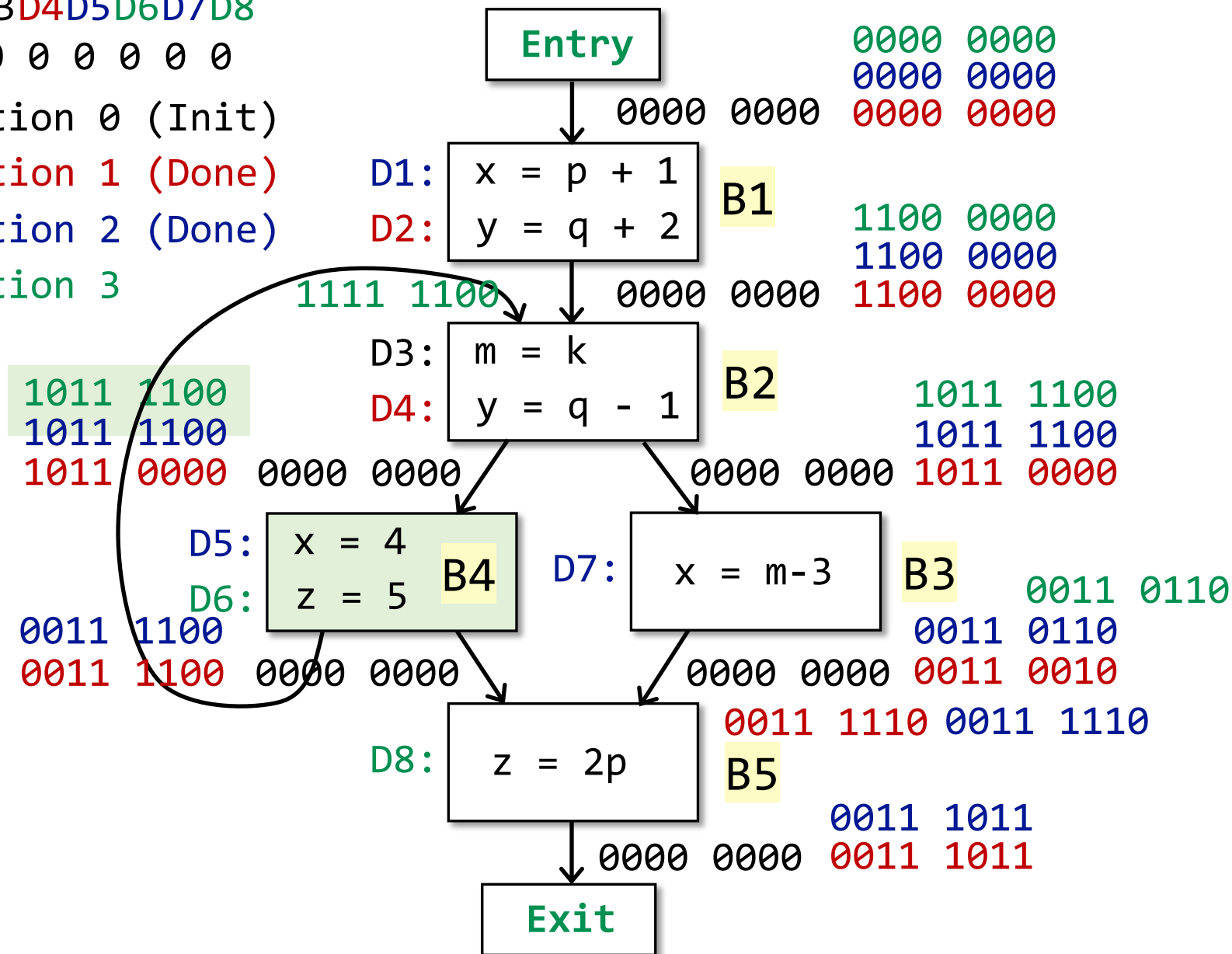
0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

0000 0000

0000 0000
0000 0000
0000 0000

D1: $x = p + 1$

B1

D2: $y = q + 2$

0000 0000

1100 0000
1100 0000
1100 0000

D3: $m = k$

B2

D4: $y = q - 1$

0000 0000

1011 1100
1011 1100
1011 0000

D5: $x = 4$

B4

D6: $z = 5$

D7: $x = m - 3$

B3

0011 0110

1011 1100
1011 1100
1011 0000

0011 1100
0011 1100
0011 1100

0000 0000

0000 0000

0011 0110
0011 0110
0011 0010

D8: $z = 2p$

B5

0000 0000

0011 1011
0011 1011

Exit

D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

0000 0000

0000 0000
0000 0000
0000 0000

D1: $x = p + 1$

B1

D2: $y = q + 2$

0000 0000

1100 0000
1100 0000
1100 0000

D3: $m = k$

B2

D4: $y = q - 1$

0000 0000

1011 1100
1011 1100
1011 0000

D5: $x = 4$

B4

D6: $z = 5$

D7: $x = m - 3$

B3

0011 1100

0011 1100

0011 1100

0000 0000

$x = m - 3$

0000 0000

0011 0110

0011 0110

0011 0010

D8: $z = 2p$

B5

0000 0000

0011 1011
0011 1011

Exit

0 0 0 0 0 0 0 0

Iteration 1 (Done)

Iteration 3



D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

0000 0000

0000 0000
0000 0000
0000 0000

D1: $x = p + 1$

B1

D2: $y = q + 2$

0000 0000

1100 0000
1100 0000
1100 0000

D3: $m = k$

B2

D4: $y = q - 1$

0000 0000

1011 1100
1011 1100
1011 0000

D5: $x = 4$

B4

D6: $z = 5$

D7: $x = m - 3$

B3

0011 1100
0011 1100
0011 1100

0011 1100
0011 1100
0011 1100

0011 1110

0000 0000

U

0000 0000

0011 0110
0011 0110
0011 0010

0011 0110

0011 1110 0011 1110

D8: $z = 2p$

B5

0000 0000

0011 1011
0011 1011

Exit

D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

0000 0000

0000 0000
0000 0000
0000 0000

D1: $x = p + 1$

B1

D2: $y = q + 2$

0000 0000

1100 0000
1100 0000
1100 0000

D3: $m = k$

B2

D4: $y = q - 1$

0000 0000

1011 1100
1011 1100
1011 0000

D5: $x = 4$

B4

D6: $z = 5$

D7: $x = m - 3$

B3

0011 1100
0011 1100
0011 1100

0011 0110
0011 0110
0011 0010

0011 1110

D8: $z = 2p$

B5

0000 0000

0011 1011
0011 1011

Exit

D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

0000 0000

0000 0000
0000 0000
0000 0000

D1: $x = p + 1$

B1

D2: $y = q + 2$

0000 0000

1100 0000
1100 0000
1100 0000

D3: $m = k$

B2

D4: $y = q - 1$

0000 0000

1011 1100
1011 1100
1011 0000

D5: $x = 4$

B4

D6: $z = 5$

D7: $x = m - 3$

B3

0011 1100
0011 1100
0011 1100

0011 1110

0000 0000

0000 0000

0011 0110
0011 0110
0011 0010

D8: $z = 2p$

B5

0000 0000

0011 1011
0011 1011
0011 1011

Exit

D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3 (Done)

Entry

0000 0000

0000 0000
0000 0000
0000 0000

D1: $x = p + 1$

B1

D2: $y = q + 2$

1100 0000
1100 0000
1100 0000

D3: $m = k$

B2

D4: $y = q - 1$

1011 1100
1011 1100
1011 0000

D5: $x = 4$

B4

D6: $z = 5$

D7: $x = m - 3$

B3

0011 0110

0011 1100
0011 1100
0011 1100

0011 1110

D8: $z = 2p$

B5

0011 1011
0011 1011
0011 1011

0000 0000

Exit

D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3 (Done)

Entry

0000 0000

0000 0000
0000 0000
0000 0000

D1: $x = p + 1$

B1

D2: $y = q + 2$

1100 0000
1100 0000
1100 0000

D3: $m = k$

B2

D4: $y = q - 1$

1011 1100
1011 1100
1011 0000

D5: $x = 4$

B4

D6: $z = 5$

D7: $x = m - 3$

B3

0011 0110

0011 1100
0011 1100
0011 1100

0011 1110

D8: $z = 2p$

B5

0011 1011
0011 1011
0011 1011

0011 0110
0011 0010

0011 1110 0011 1110

Exit

No changes occur
in any OUT[]

D1D2D3D4D5D6D7D8

0 0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3 (Done)

Entry

0000 0000

0000 0000
0000 0000
0000 0000

D1: $x = p + 1$

B1

D2: $y = q + 2$

1100 0000
1100 0000
1100 0000

D3: $m = k$

B2

D4: $y = q - 1$

1011 1100
1011 1100
1011 0000

D5: $x = 4$

B4

D6: $z = 5$

D7: $x = m - 3$

B3

0011 0110
0011 0110
0011 0010

D8: $z = 2p$

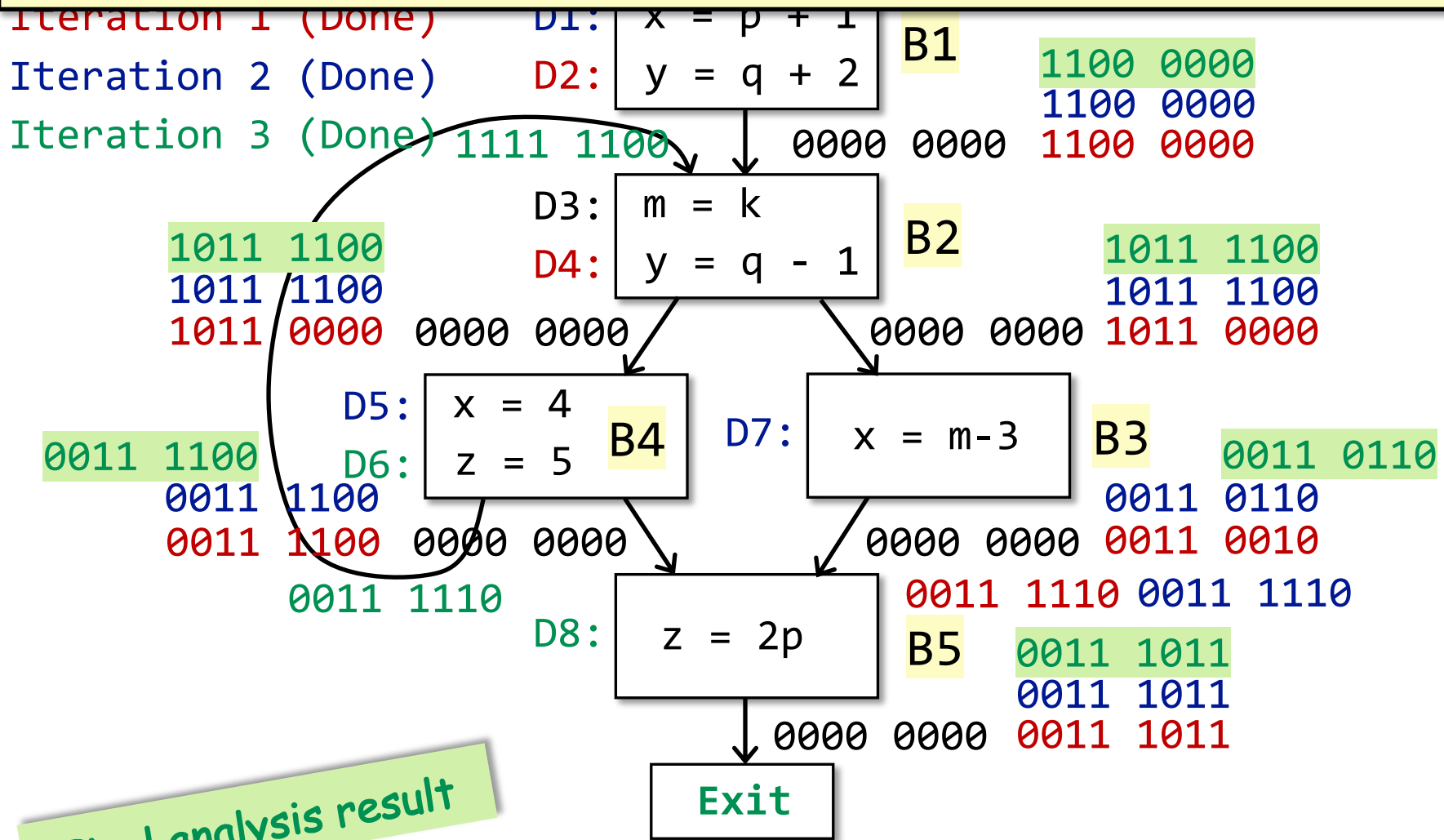
B5

0011 1011
0011 1011
0011 1011

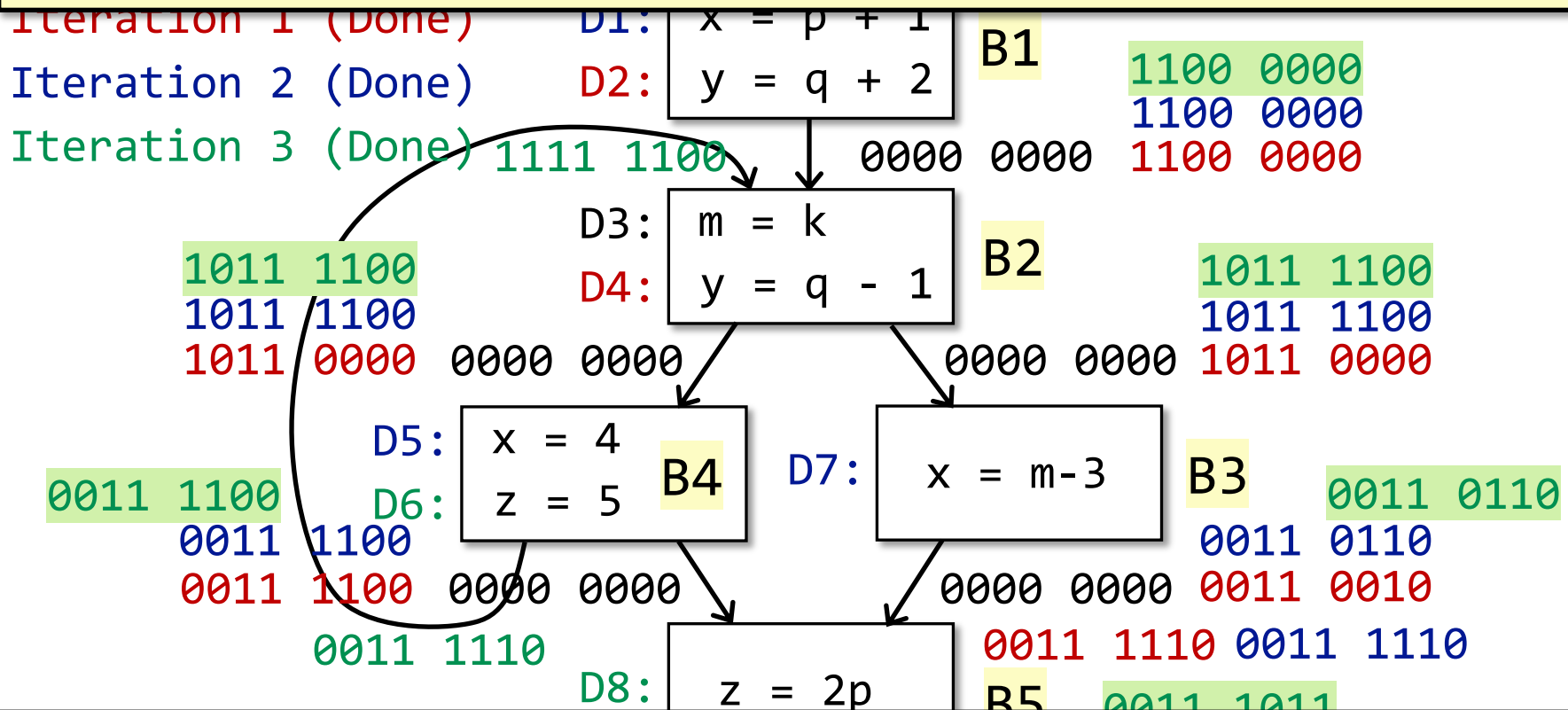
Exit

Final analysis result

In each data-flow analysis application, we associate with every program point a **data-flow value** that represents an *abstraction* of the set of all possible **program states** that can be observed for that point.



In each data-flow analysis application, we associate with every program point a **data-flow value** that represents an *abstraction* of the set of all possible **program states** that can be observed for that point.



Data-flow analysis is to **find a solution** to a set of *safe-approximation-directed constraints* on the $IN[s]$'s and $OUT[s]$'s, for **all** statements s .

- *constraints* based on semantics of statements (*transfer functions*)
- *constraints* based on the *flows of control*

Algorithm of Reaching Definitions Analysis

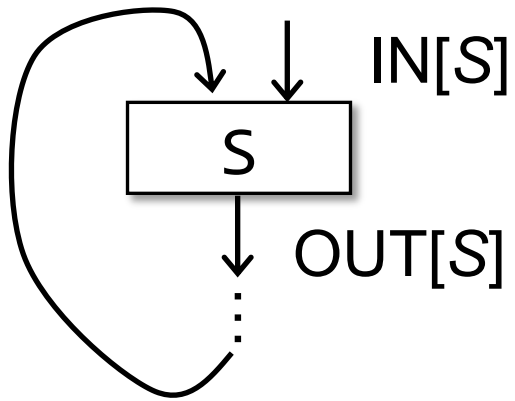
INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

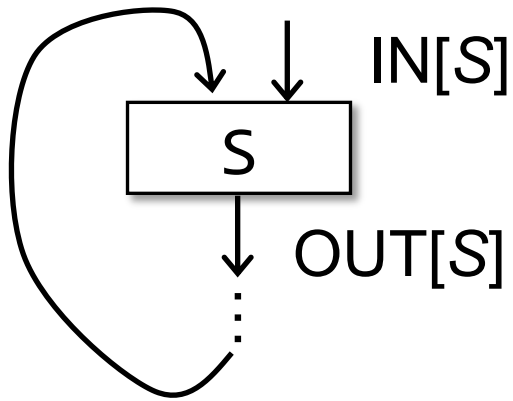
METHOD:

```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B$ )  
    OUT[B] =  $\emptyset$ ;  
while (changes to any OUT occur)  
    for (each basic block  $B \setminus entry$ ) {  
         $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$ ;  
         $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
    }
```

Why this iterative algorithm
can finally stop?

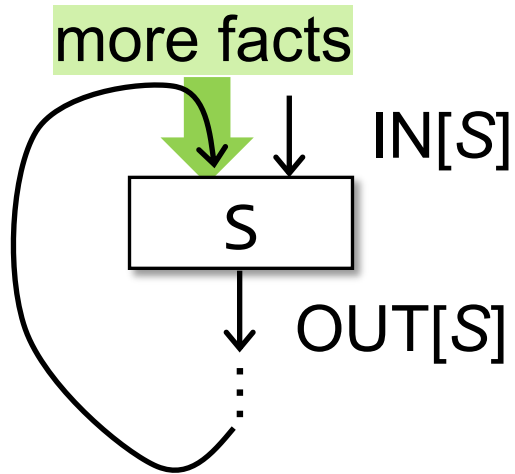


$$OUT[S] = gen_S \cup (IN[S] - kill_S);$$



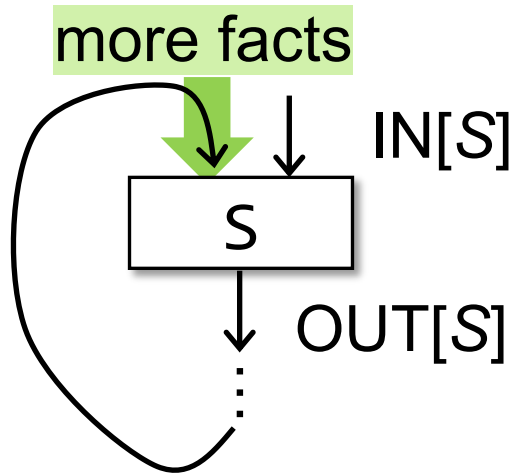
$$\text{OUT}[S] = \text{gen}_S \cup (\text{IN}[S] - \text{kill}_S);$$

- gen_S and kill_S remain unchanged



$$\text{OUT}[S] = \text{gen}_S \cup (\text{IN}[S] - \text{kill}_S);$$

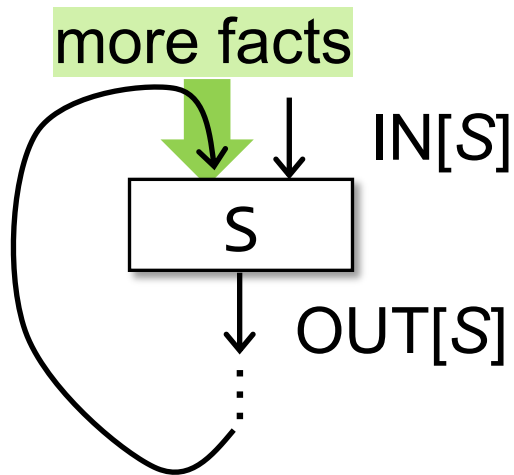
- gen_S and kill_S remain unchanged
- When **more facts** flow in $\text{IN}[S]$, the “**more facts**” either



$$\text{OUT}[S] = \text{gen}_S \cup (\text{IN}[S] - \text{kill}_S);$$

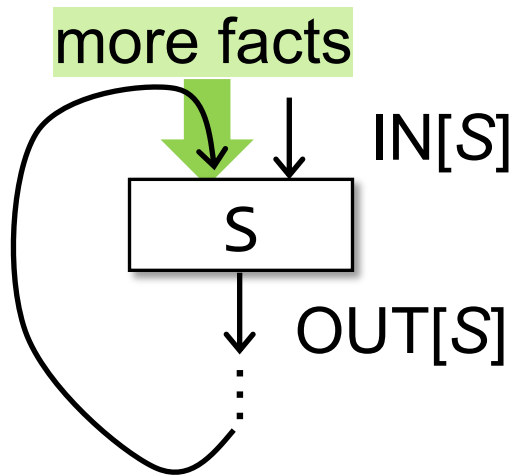
survivor_S

- gen_S and kill_S remain unchanged
- When **more facts** flow in $\text{IN}[S]$, the “**more facts**” either
 - is killed, or
 - flows to $\text{OUT}[S]$ (**survivor_S**)



$$\text{OUT}[S] = \text{gen}_S \cup \underbrace{(\text{IN}[S] - \text{kill}_S)}_{\text{survivor}_S};$$

- gen_S and kill_S remain unchanged
- When **more facts** flow in $\text{IN}[S]$, the “**more facts**” either
 - is killed, or
 - flows to $\text{OUT}[S]$ (**survivor_S**)
- When a fact is added to $\text{OUT}[S]$, through either gen_S , or **survivor_S**, it stays there forever

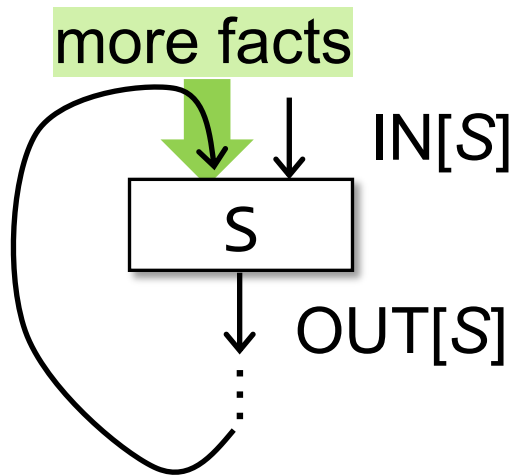


$$\text{OUT}[S] = \text{gen}_S \cup (\text{IN}[S] - \text{kill}_S);$$

}

survivor_S

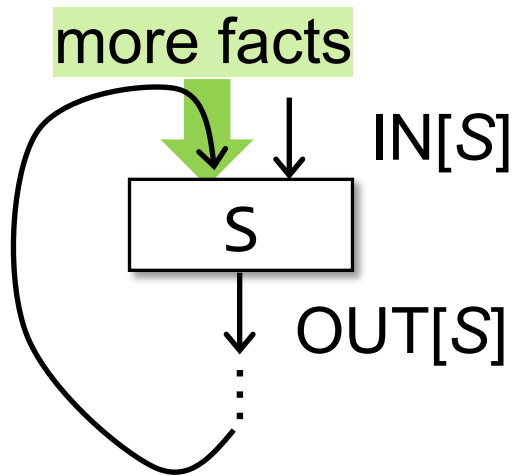
- gen_S and kill_S remain unchanged
- When **more facts** flow in $\text{IN}[S]$, the “**more facts**” either
 - is killed, or
 - flows to $\text{OUT}[S]$ (**survivor_S**)
- When a fact is added to $\text{OUT}[S]$, through either gen_S , or **survivor_S**, it stays there forever
- Thus $\text{OUT}[S]$ never shrinks (e.g., $0 \rightarrow 1$, or $1 \rightarrow 1$)



$$\text{OUT}[S] = \text{gen}_S \cup (\text{IN}[S] - \text{kill}_S);$$

survivor_S

- gen_S and kill_S remain unchanged
- When **more facts** flow in $\text{IN}[S]$, the “**more facts**” either
 - is killed, or
 - flows to $\text{OUT}[S]$ (**survivor_S**)
- When a fact is added to $\text{OUT}[S]$, through either gen_S , or **survivor_S**, it stays there forever
- Thus $\text{OUT}[S]$ never shrinks (e.g., $0 \rightarrow 1$, or $1 \rightarrow 1$)
- As the set of facts is finite (e.g., all definitions in the program),



$$\text{OUT}[S] = \text{gen}_S \cup (\text{IN}[S] - \text{kill}_S);$$

survivor_S

- gen_S and kill_S remain unchanged
- When **more facts** flow in $\text{IN}[S]$, the “**more facts**” either
 - is killed, or
 - flows to $\text{OUT}[S]$ (**survivor_S**)
- When a fact is added to $\text{OUT}[S]$, through either gen_S , or **survivor_S**, it stays there forever
- Thus $\text{OUT}[S]$ never shrinks (e.g., $0 \rightarrow 1$, or $1 \rightarrow 1$)
- As the set of facts is finite (e.g., all definitions in the program), there must exist a pass of iteration during which nothing is added to any OUT , and then the algorithm terminates

Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

$OUT[entry] = \emptyset;$

for (each basic block $B \neq entry$)

$OUT[B] = \emptyset;$

while (changes to any OUT occur)

for (each basic block $B \neq entry$) {

$IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P];$

$OUT[B] = gen_B \cup (IN[B] - kill_B);$

}

Safe to terminate
by this condition?

Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

$OUT[entry] = \emptyset;$

for (each basic block $B \neq entry$)

$OUT[B] = \emptyset;$

while (changes to any OUT occur)

for (each basic block $B \neq entry$) {

$IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P];$

$OUT[B] = gen_B \cup (IN[B] - kill_B);$

}

Safe to terminate
by this condition?

IN's will not change if
OUT's do not change

Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

$OUT[entry] = \emptyset;$

for (each basic block $B \neq entry$)

$OUT[B] = \emptyset;$

while (changes to any OUT occur)

for (each basic block $B \neq entry$) {

IN's will not change if
OUT's do not change

$IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P];$

OUT's will not change
if IN's do not change

$OUT[B] = gen_B \cup (IN[B] - kill_B);$

Safe to terminate
by this condition?

Algorithm of Reaching Definitions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

$OUT[entry] = \emptyset;$

for (each basic block $B \neq entry$)

$OUT[B] = \emptyset;$

while (changes to any OUT occur)

for (each basic block $B \neq entry$) {

$IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P];$

$OUT[B] = gen_B \cup (IN[B] - kill_B);$

Safe to terminate
by this condition?

IN's will not change if
OUT's do not change

OUT's will not change
if IN's do not change

Reach a **fixed point**
Also related with **monotonicity**
(**next lectures**)

Data Flow Analysis Applications

(I) Reaching Definitions Analysis

(II) Live Variables Analysis

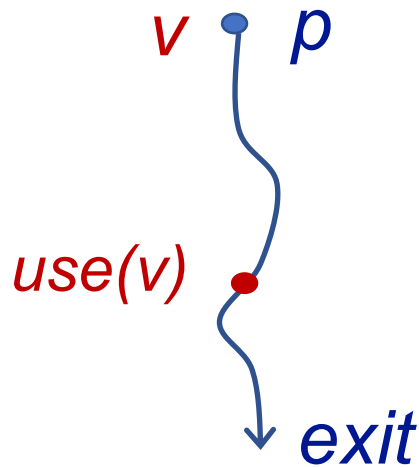
(III) Available Expressions Analysis

Live Variables Analysis

Live variables analysis tells whether the value of **variable v** at **program point p** could be used along some path in CFG starting at p . If so, v is live at p ; otherwise, v is dead at p .

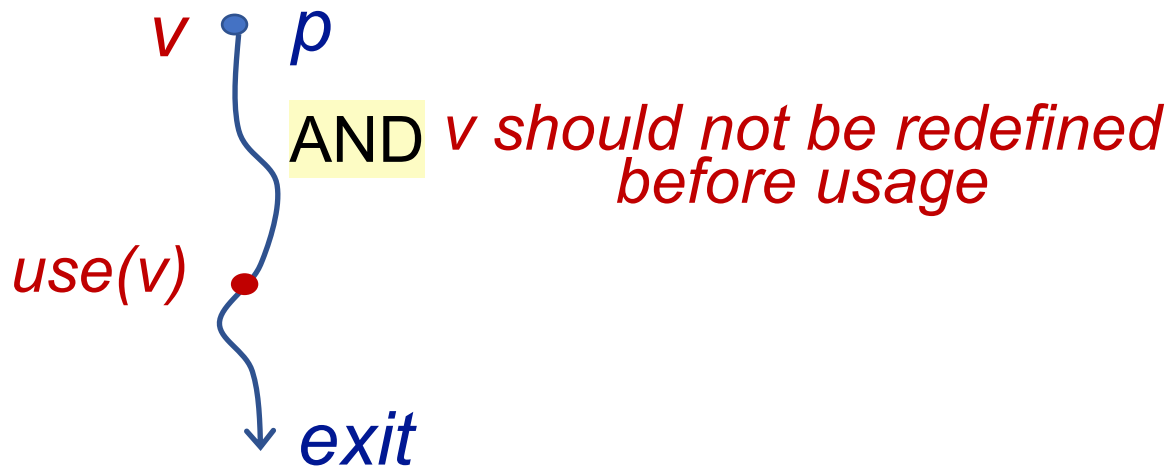
Live Variables Analysis

Live variables analysis tells whether the value of **variable v** at **program point p** could be used along some path in CFG starting at p . If so, v is live at p ; otherwise, v is dead at p .



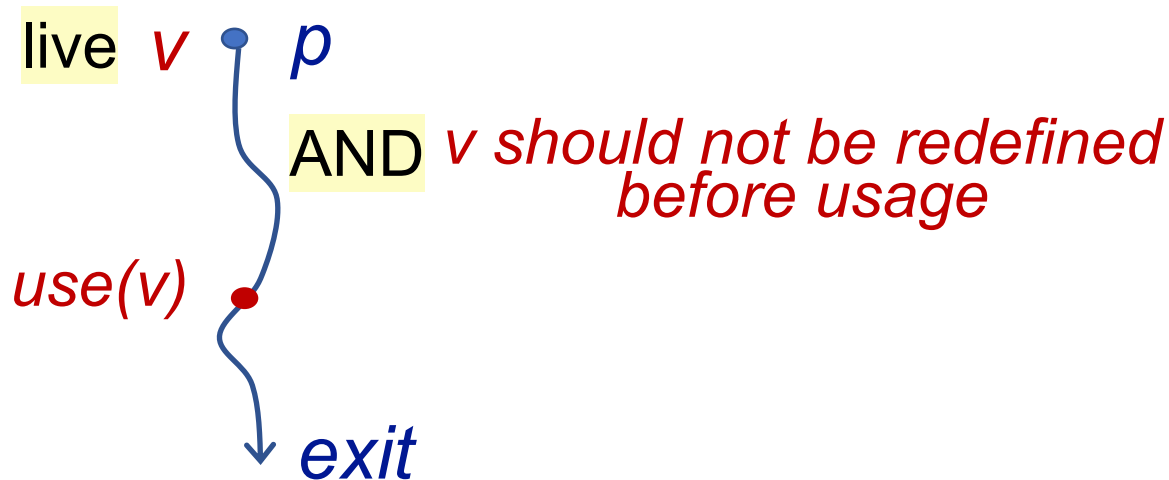
Live Variables Analysis

Live variables analysis tells whether the value of **variable v** at **program point p** could be used along some path in CFG starting at p . If so, v is live at p ; otherwise, v is dead at p .



Live Variables Analysis

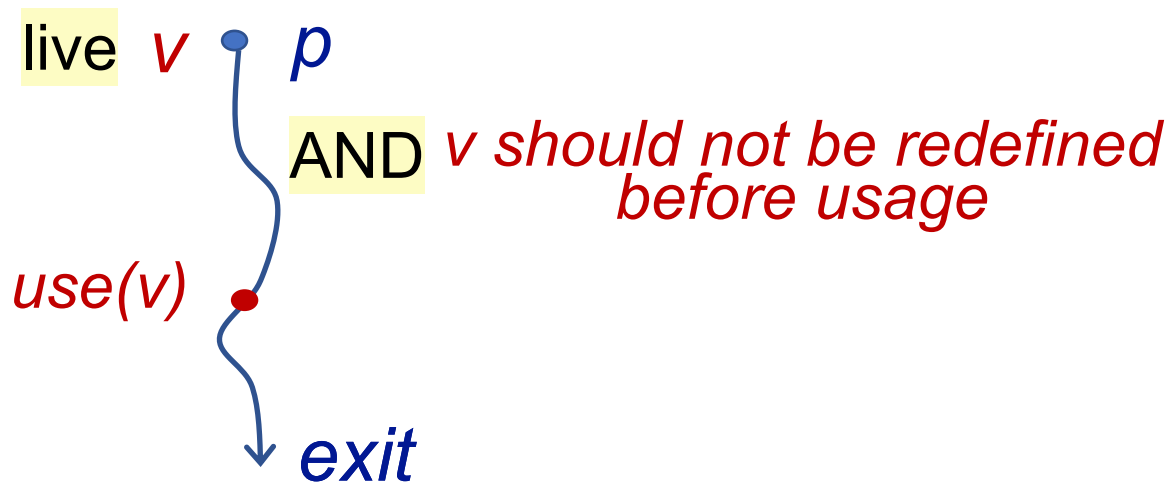
Live variables analysis tells whether the value of **variable v** at **program point p** could be used along some path in CFG starting at p . If so, v is live at p ; otherwise, v is dead at p .



Live Variables Analysis

Live variables analysis tells whether the value of **variable v** at **program point p** could be used along some path in CFG starting at p . If so, v is live at p ; otherwise, v is dead at p .

- Information of live variables can be used for register allocations. e.g., at some point all registers are full and we need to use one, then we should favor using a register with a dead value.



Understanding Live Variables Analysis

Abstraction

- Data Flow Values/Facts
 - All the variables in a program
 - Can be represented by bit vectors

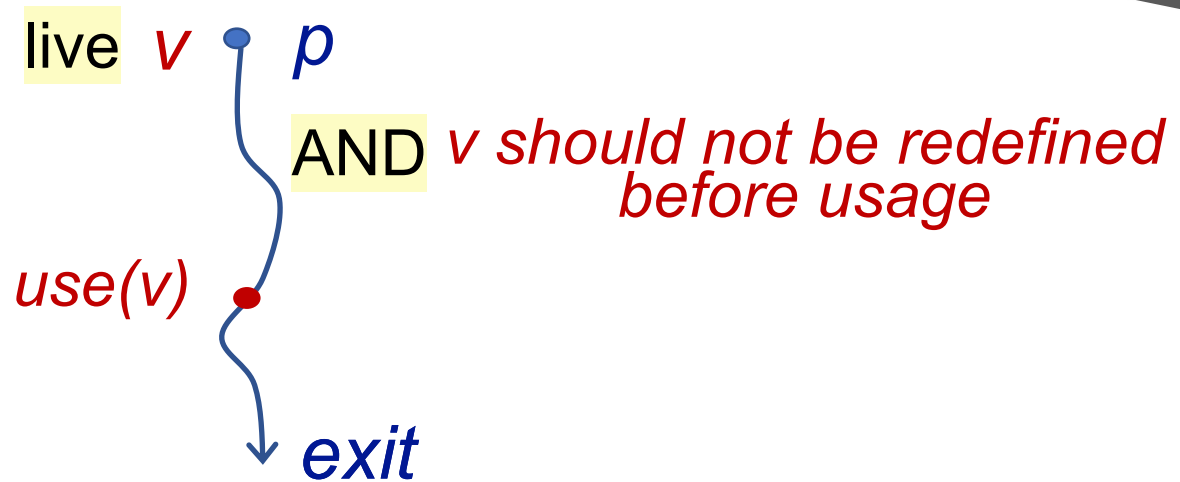
e.g., $V_1, V_2, V_3, V_4, \dots, V_{100}$ (100 variables)

00000...0
└──────────┘
100 bits

Bit i from the left represents variable V_i

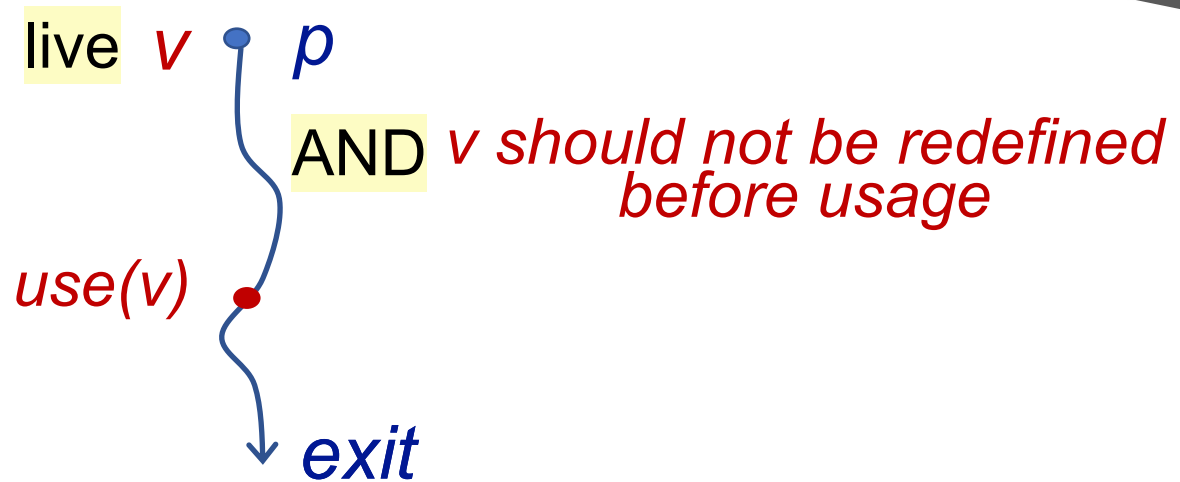
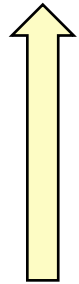
Understanding Live Variables Analysis

Safe-approximation



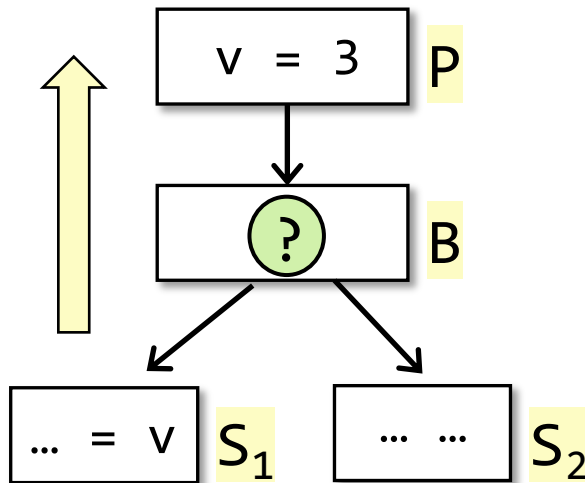
Understanding Live Variables Analysis

Safe-approximation

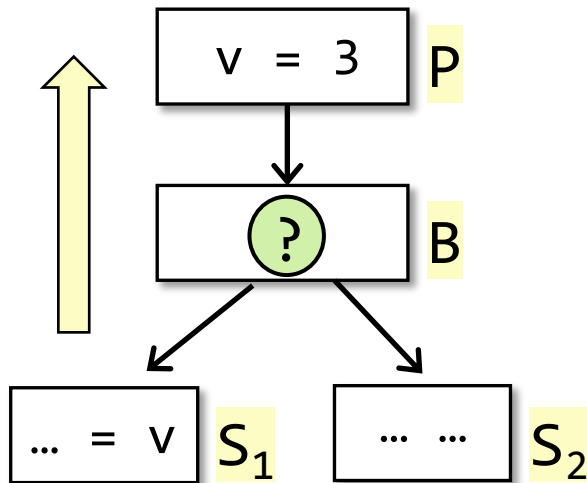


Understanding Live Variables Analysis

Safe-approximation

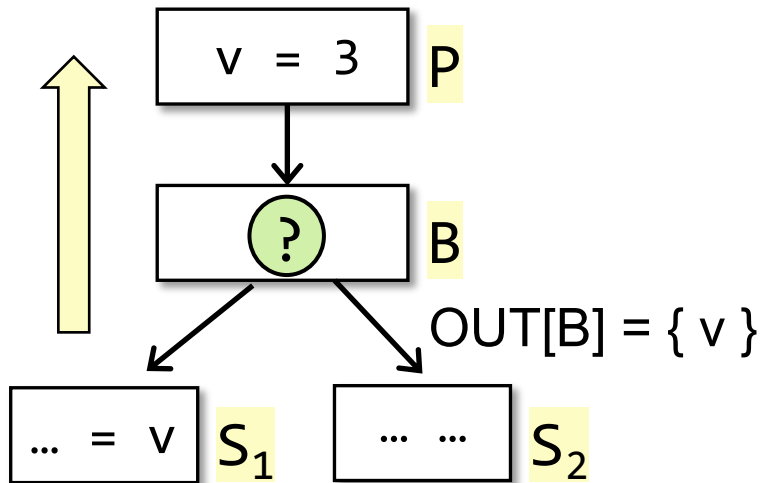


Understanding Live Variables Analysis



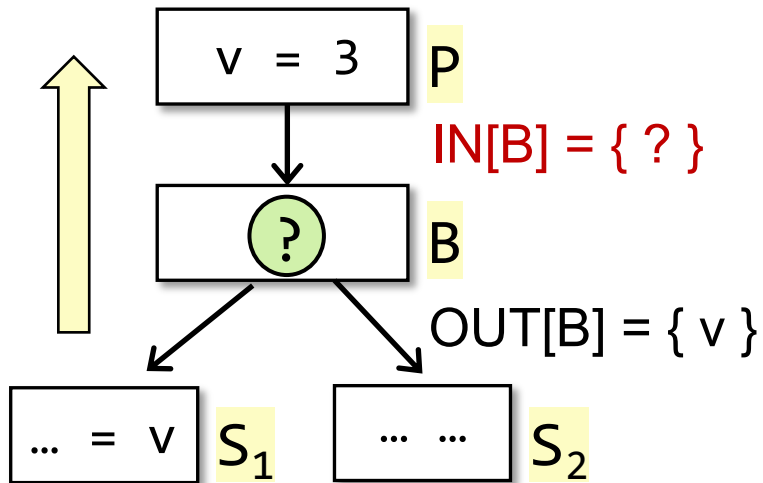
$$\text{OUT}[B] = \bigcup_{S \text{ a successor of } B} \text{IN}[S]$$

Understanding Live Variables Analysis



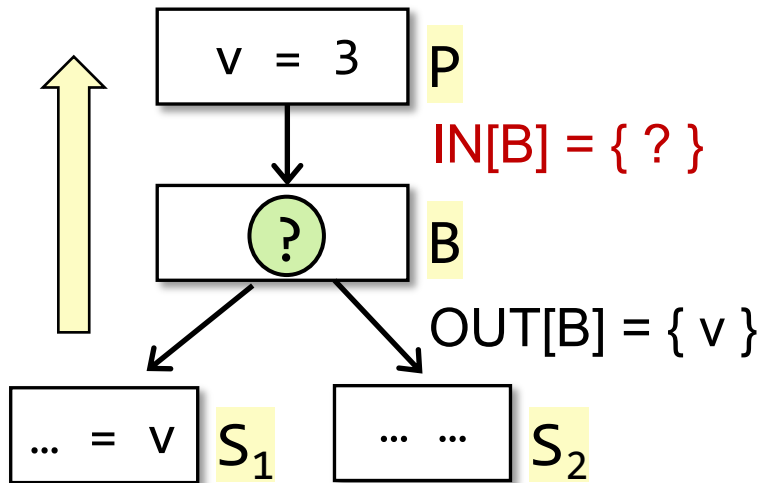
$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis

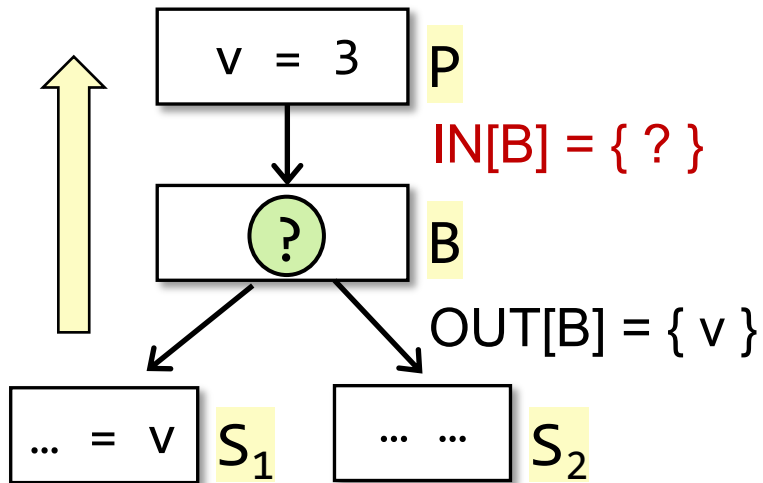


Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of** $IN[B]$?

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



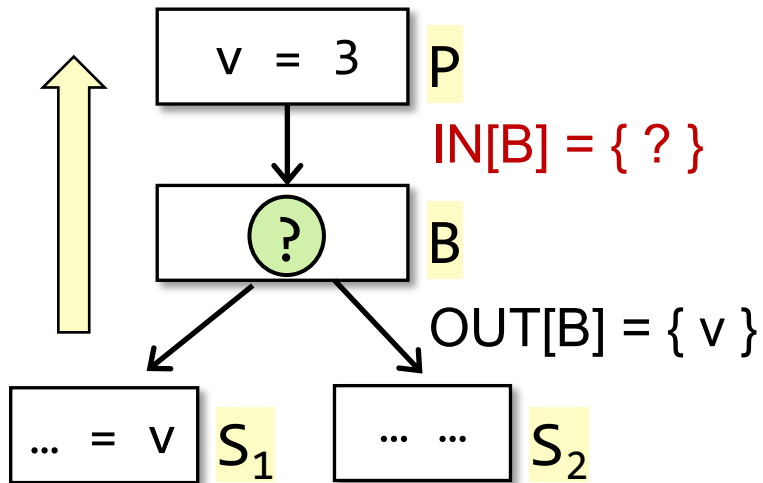
Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of** $IN[B]$?

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

① $k = n$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



$IN[B] = \{ ? \}$

$OUT[B] = \{ v \}$

Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of $IN[B]$?**

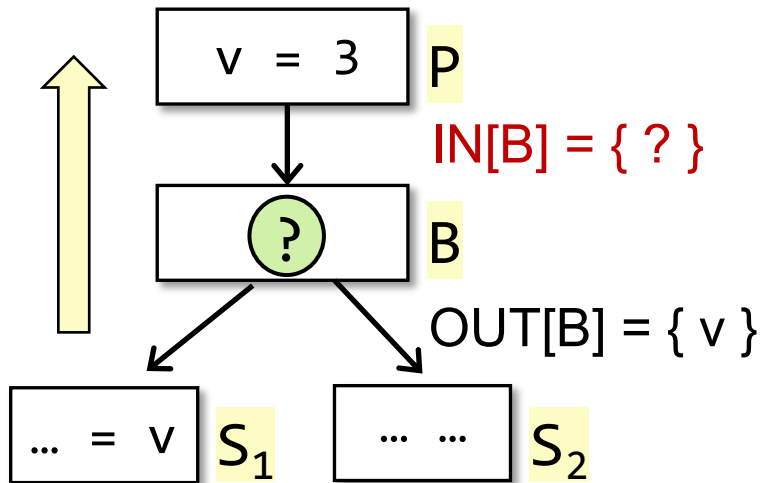
Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

① $k = n$

$IN[B] = \{ v \}$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



$IN[B] = \{ ? \}$

$OUT[B] = \{ v \}$

Tip: determine **whether** the variable `v` in some register `R` **is live**, or should we delete the value 3 of `v` in `R`, **at the point of $IN[B]$** ?

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

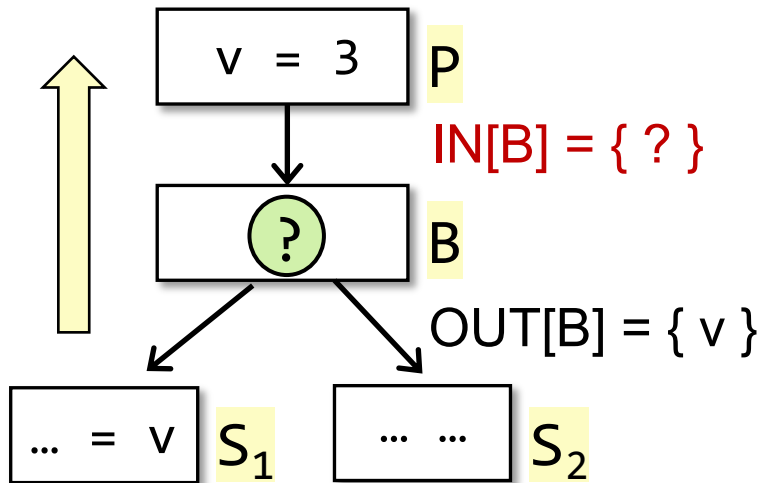
① $k = n$

② $k = v$

$IN[B] = \{ v \}$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



$IN[B] = \{ ? \}$

$OUT[B] = \{ v \}$

Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of $IN[B]$?**

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

① $k = n$

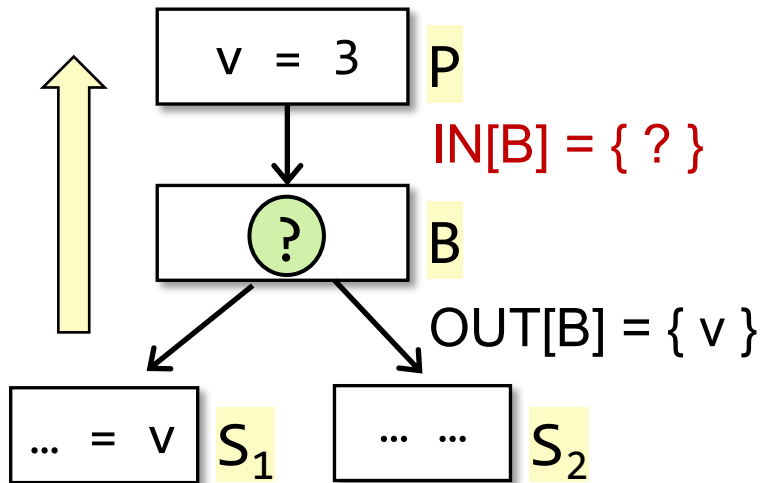
$IN[B] = \{ v \}$

② $k = v$

$IN[B] = \{ v \}$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



$IN[B] = \{ ? \}$

$OUT[B] = \{ v \}$

Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of $IN[B]$?**

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

① $k = n$

$IN[B] = \{ v \}$

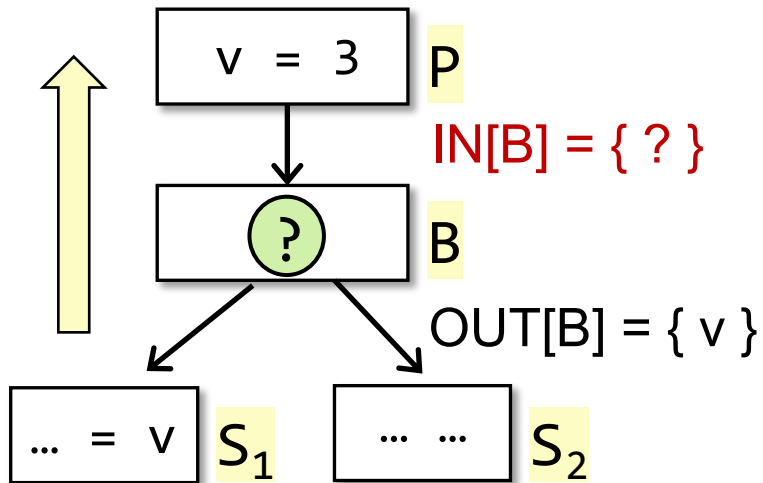
② $k = v$

$IN[B] = \{ v \}$

③ $v = 2$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



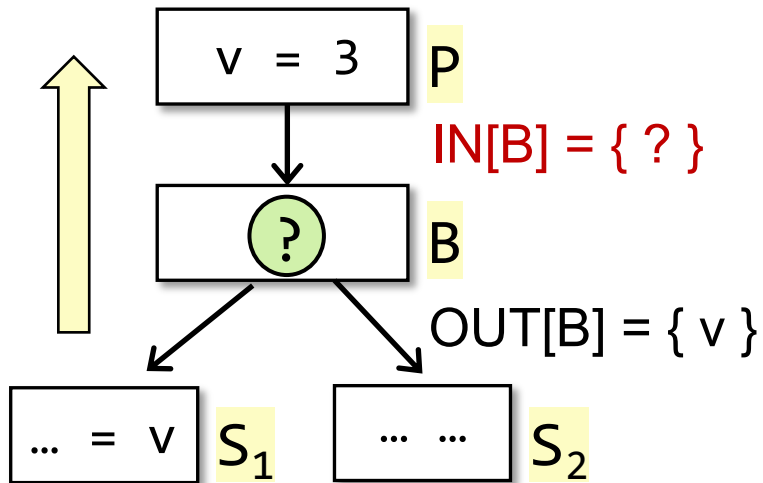
Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of IN[B]?**

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

- | | | |
|-------------------|-------------------|-----------------|
| ① $k = n$ | ② $k = v$ | ③ $v = 2$ |
| $IN[B] = \{ v \}$ | $IN[B] = \{ v \}$ | $IN[B] = \{ \}$ |

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



$IN[B] = \{ ? \}$

$OUT[B] = \{ v \}$

Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of $IN[B]$?**

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

① $k = n$

$IN[B] = \{ v \}$

② $k = v$

$IN[B] = \{ v \}$

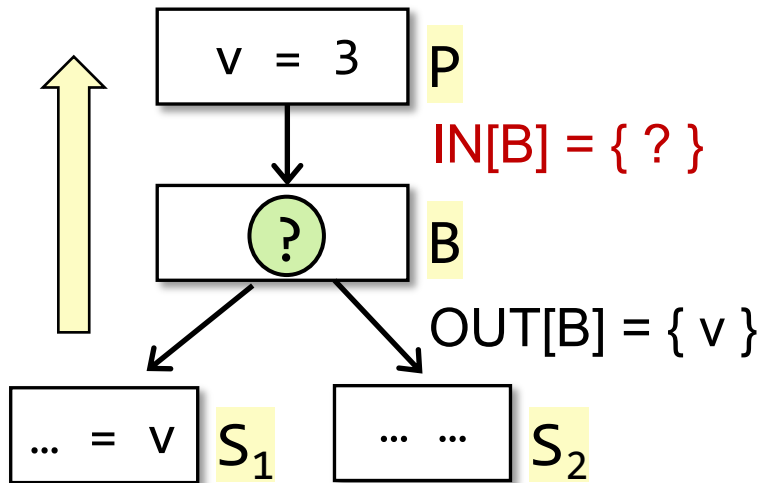
③ $v = 2$

$IN[B] = \{ \}$

④ $v = v - 1$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



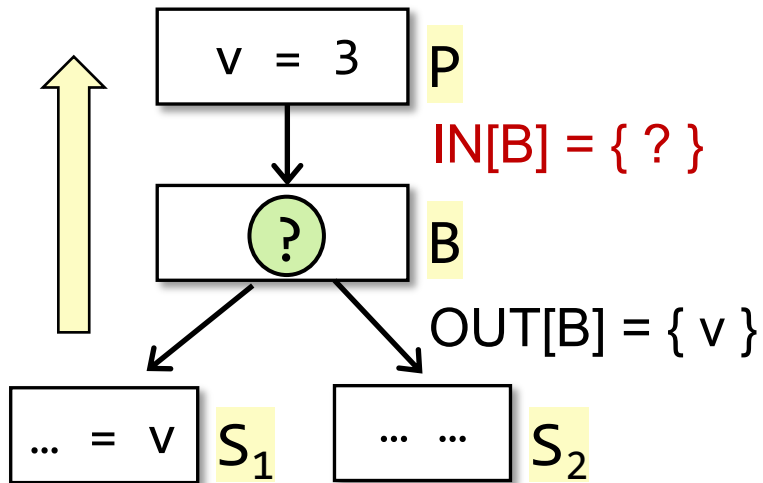
Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of $IN[B]$?**

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

- ① $k = n$
 $IN[B] = \{ v \}$
- ② $k = v$
 $IN[B] = \{ v \}$
- ③ $v = 2$
 $IN[B] = \{ \}$
- ④ $v = v - 1$
 $IN[B] = \{ v \}$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



$IN[B] = \{ ? \}$

$OUT[B] = \{ v \}$

Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of $IN[B]$?**

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

① $k = n$

$IN[B] = \{ v \}$

② $k = v$

$IN[B] = \{ v \}$

③ $v = 2$

$IN[B] = \{ \}$

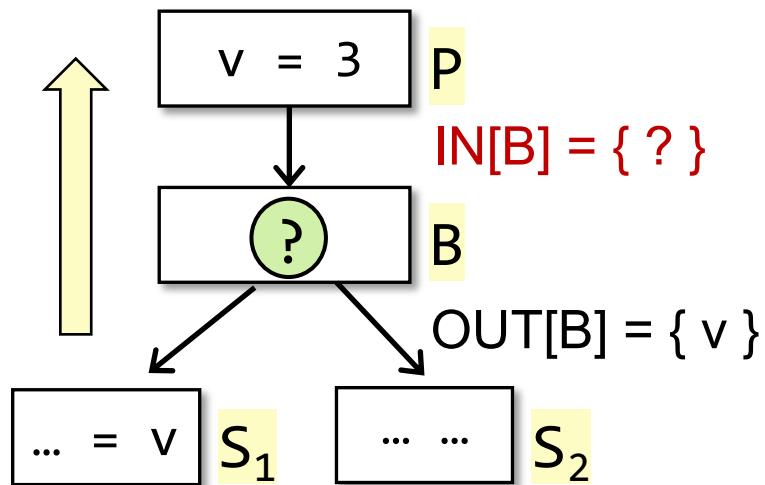
④ $v = v - 1$

$IN[B] = \{ v \}$

⑤ $\begin{matrix} v = 2 \\ k = v \end{matrix}$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



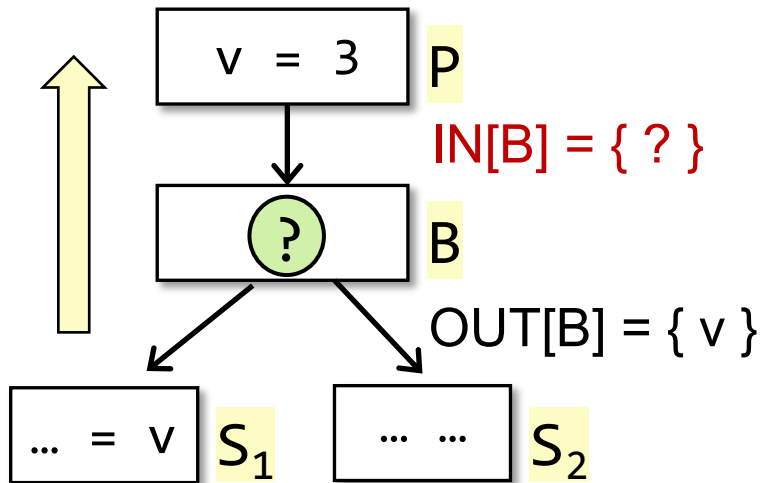
Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of $IN[B]$?**

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

- | | | |
|------------------------------------|---|------------------------------|
| ① $k = n$
$IN[B] = \{ v \}$ | ② $k = v$
$IN[B] = \{ v \}$ | ③ $v = 2$
$IN[B] = \{ \}$ |
| ④ $v = v - 1$
$IN[B] = \{ v \}$ | ⑤ $\begin{matrix} v = 2 \\ k = v \end{matrix}$
$IN[B] = \{ \}$ | |

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



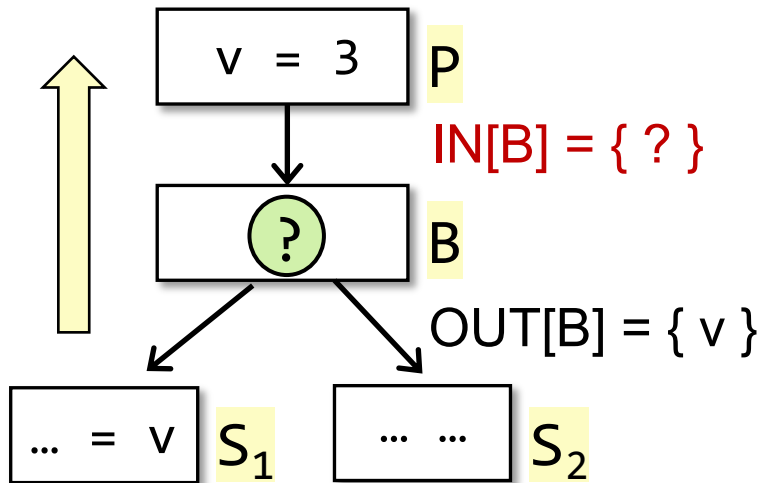
Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of $IN[B]$?**

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

- | | | |
|------------------------------------|---|--|
| ① $k = n$
$IN[B] = \{ v \}$ | ② $k = v$
$IN[B] = \{ v \}$ | ③ $v = 2$
$IN[B] = \{ \}$ |
| ④ $v = v - 1$
$IN[B] = \{ v \}$ | ⑤ $\begin{matrix} v = 2 \\ k = v \end{matrix}$
$IN[B] = \{ \}$ | ⑥ $\begin{matrix} k = v \\ v = 2 \end{matrix}$ |

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



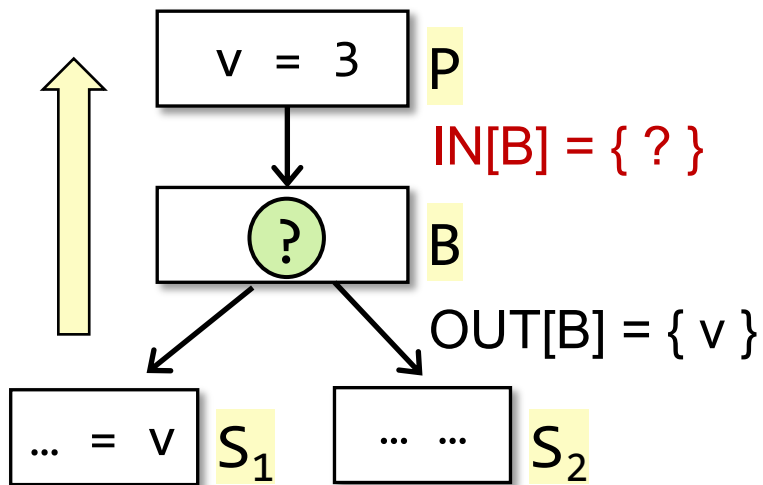
Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of $IN[B]$?**

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

- | | | |
|------------------------------------|---|---|
| ① $k = n$
$IN[B] = \{ v \}$ | ② $k = v$
$IN[B] = \{ v \}$ | ③ $v = 2$
$IN[B] = \{ \}$ |
| ④ $v = v - 1$
$IN[B] = \{ v \}$ | ⑤ $\begin{matrix} v = 2 \\ k = v \end{matrix}$
$IN[B] = \{ \}$ | ⑥ $\begin{matrix} k = v \\ v = 2 \end{matrix}$
$IN[B] = \{ v \}$ |

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

Understanding Live Variables Analysis



Tip: determine **whether** the variable v in some register R **is live**, or should we delete the value 3 of v in R , **at the point of $IN[B]$?**

Yes: $IN[B] = \{ v \}$ No: $IN[B] = \{ \}$

- | | | |
|------------------------------------|---|---|
| ① $k = n$
$IN[B] = \{ v \}$ | ② $k = v$
$IN[B] = \{ v \}$ | ③ $v = 2$
$IN[B] = \{ \}$ |
| ④ $v = v - 1$
$IN[B] = \{ v \}$ | ⑤ $\begin{matrix} v = 2 \\ k = v \end{matrix}$
$IN[B] = \{ \}$ | ⑥ $\begin{matrix} k = v \\ v = 2 \end{matrix}$
$IN[B] = \{ v \}$ |

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

$$IN[B] = use_B \cup (OUT[B] - def_B)$$

Algorithm of Live Variables Analysis

INPUT: CFG (def_B and use_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
IN[exit] =  $\emptyset$ ;  
for (each basic block  $B \setminus exit$ )  
    IN[B] =  $\emptyset$ ;  
    while (changes to any IN occur)  
        for (each basic block  $B \setminus exit$ ) {  
            OUT[B] =  $\bigcup_{S \text{ a successor of } B} IN[S]$ ;  
            IN[B] =  $use_B \cup (OUT[B] - def_B)$ ;  
        }
```


Algorithm of Live Variables Analysis

INPUT: CFG (def_B and use_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

$IN[exit] = \emptyset;$

for (each basic block $B \setminus exit$)

$IN[B] = \emptyset;$

while (changes to any IN occur)

for (each basic block $B \setminus exit$) {

$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S];$

$IN[B] = use_B \cup (OUT[B] - def_B);$

}

Algorithm of Live Variables Analysis

INPUT: CFG (def_B and use_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
IN[exit] =  $\emptyset$ ;  
for (each basic block  $B \setminus exit$ )  
    IN[B] =  $\emptyset$ ;  
    while (changes to any IN occur)  
        for (each basic block  $B \setminus exit$ ) {  
            OUT[B] =  $\bigcup_{S \text{ a successor of } B} IN[S]$ ;  
            IN[B] =  $use_B \cup (OUT[B] - def_B)$ ;  
        }
```

Algorithm of Live Variables Analysis

INPUT: CFG (def_B and use_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
IN[exit] =  $\emptyset$ ;  
for (each basic block  $B \setminus exit$ )  
    IN[B] =  $\emptyset$ ;  
    while (changes to any IN occur)  
        for (each basic block  $B \setminus exit$ ) {  
            OUT[B] =  $\bigcup_{S \text{ a successor of } B} IN[S]$ ;  
            IN[B] =  $use_B \cup (OUT[B] - def_B)$ ;  
        }
```

Algorithm of Live Variables Analysis

INPUT: CFG (def_B and use_B computed for each basic block B)

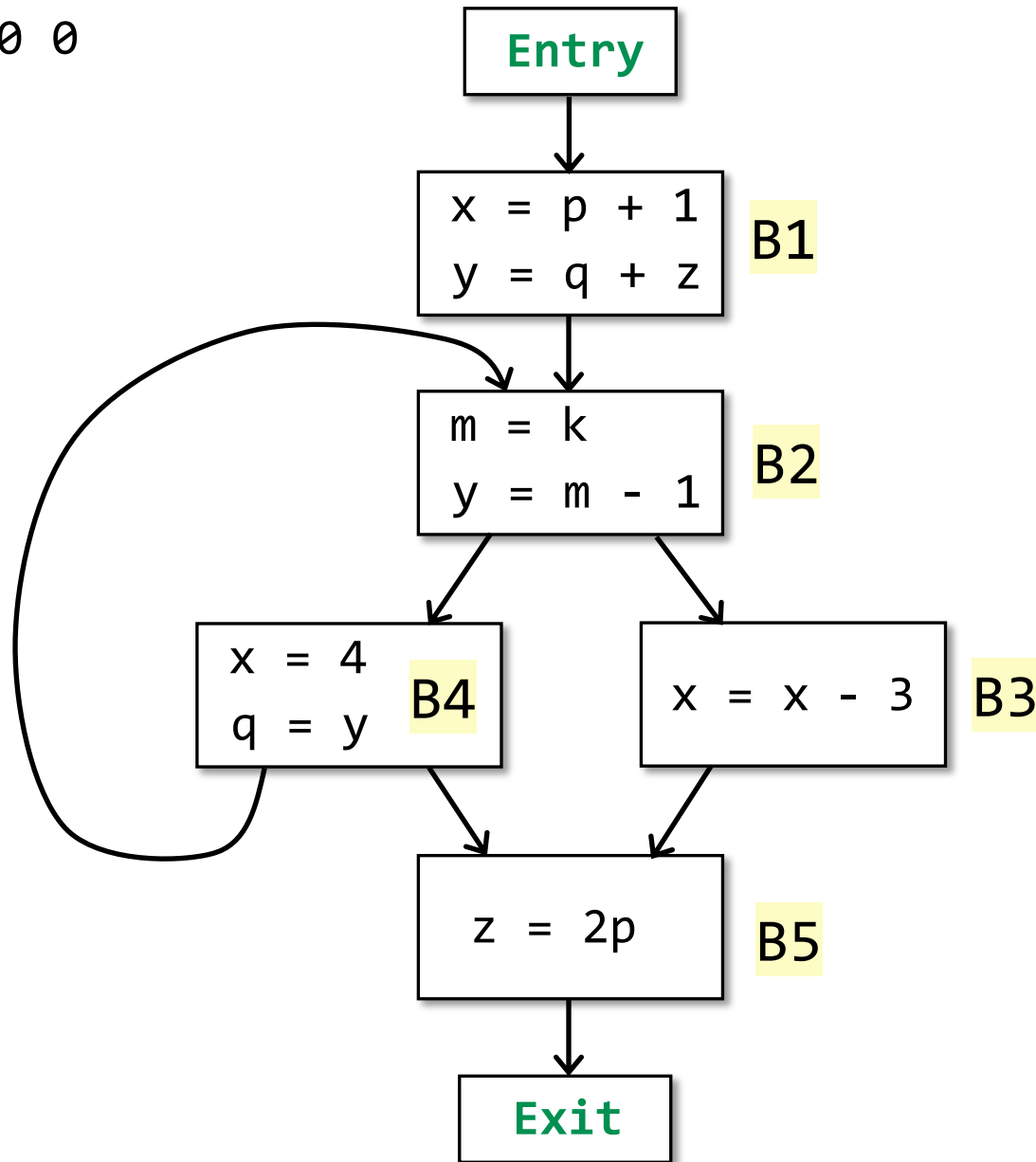
OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
IN[exit] =  $\emptyset$ ;  
for (each basic block  $B \setminus exit$ )  
    IN[B] =  $\emptyset$ ;  
    while (changes to any IN occur)  
        for (each basic block  $B \setminus exit$ ) {  
            OUT[B] =  $\bigcup_{S \text{ a successor of } B} IN[S]$ ;  
            IN[B] =  $use_B \cup (OUT[B] - def_B)$ ;  
        }
```



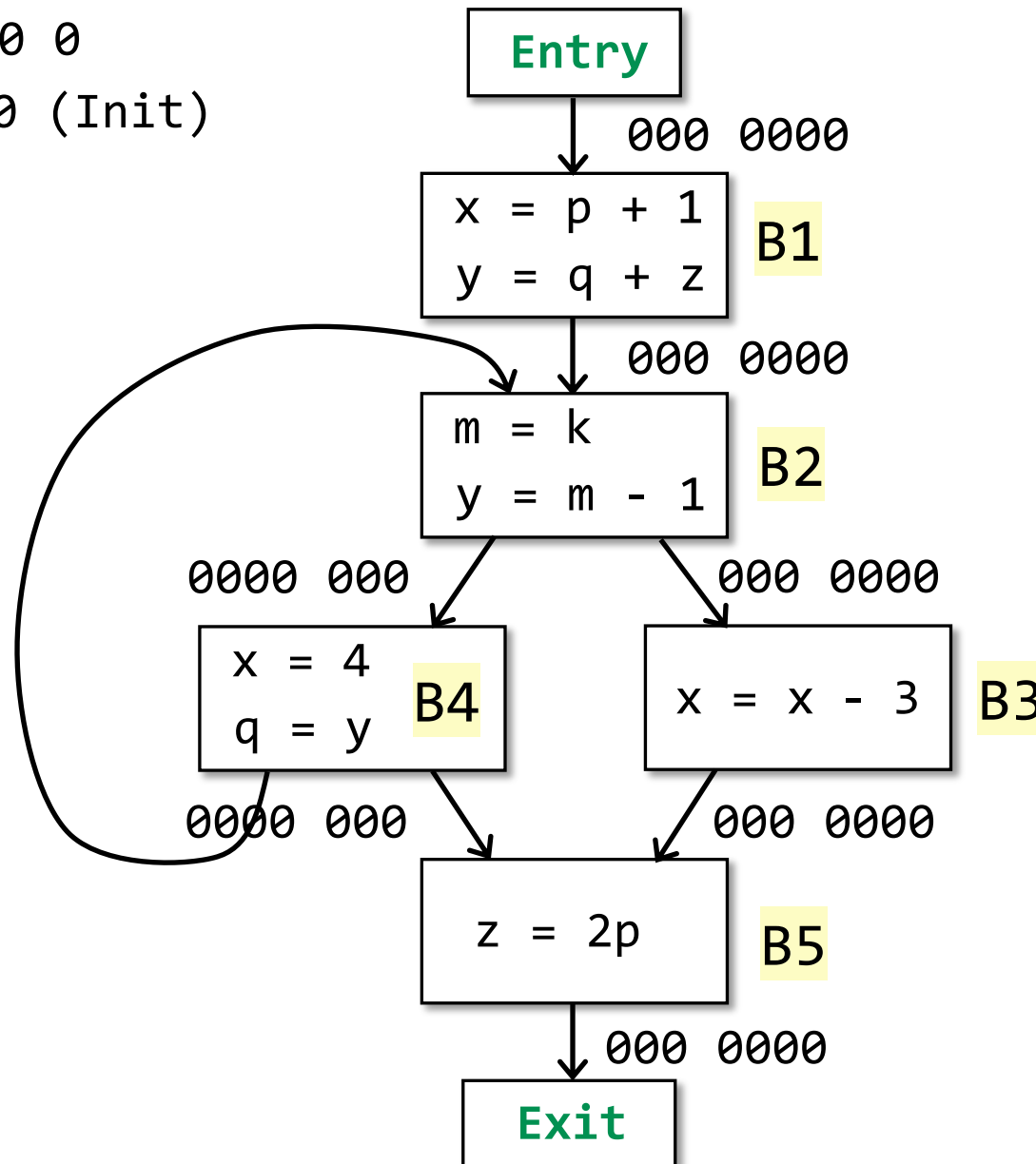
x	y	z	p	q	m	k
0	0	0	0	0	0	0



x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

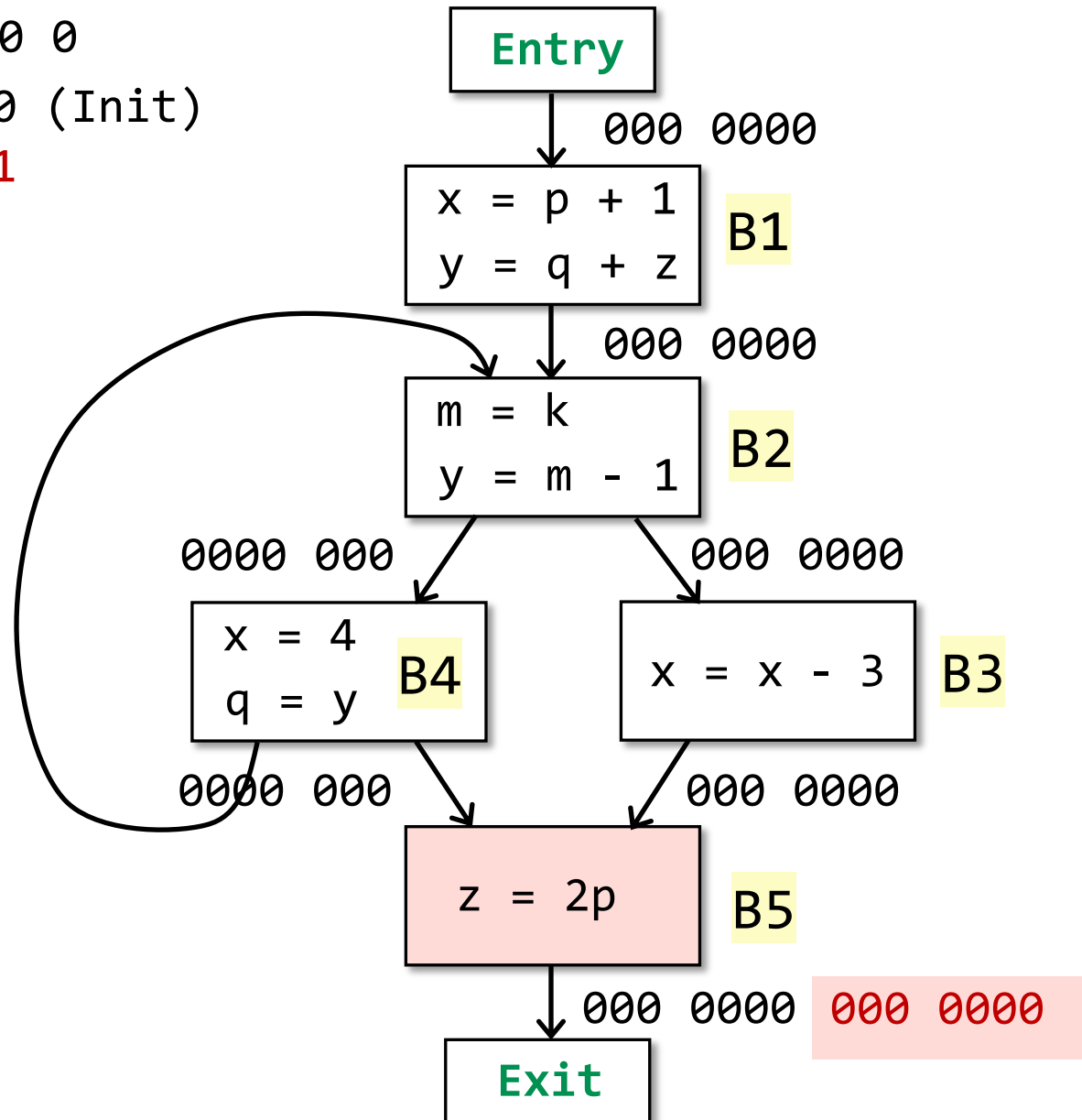


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1



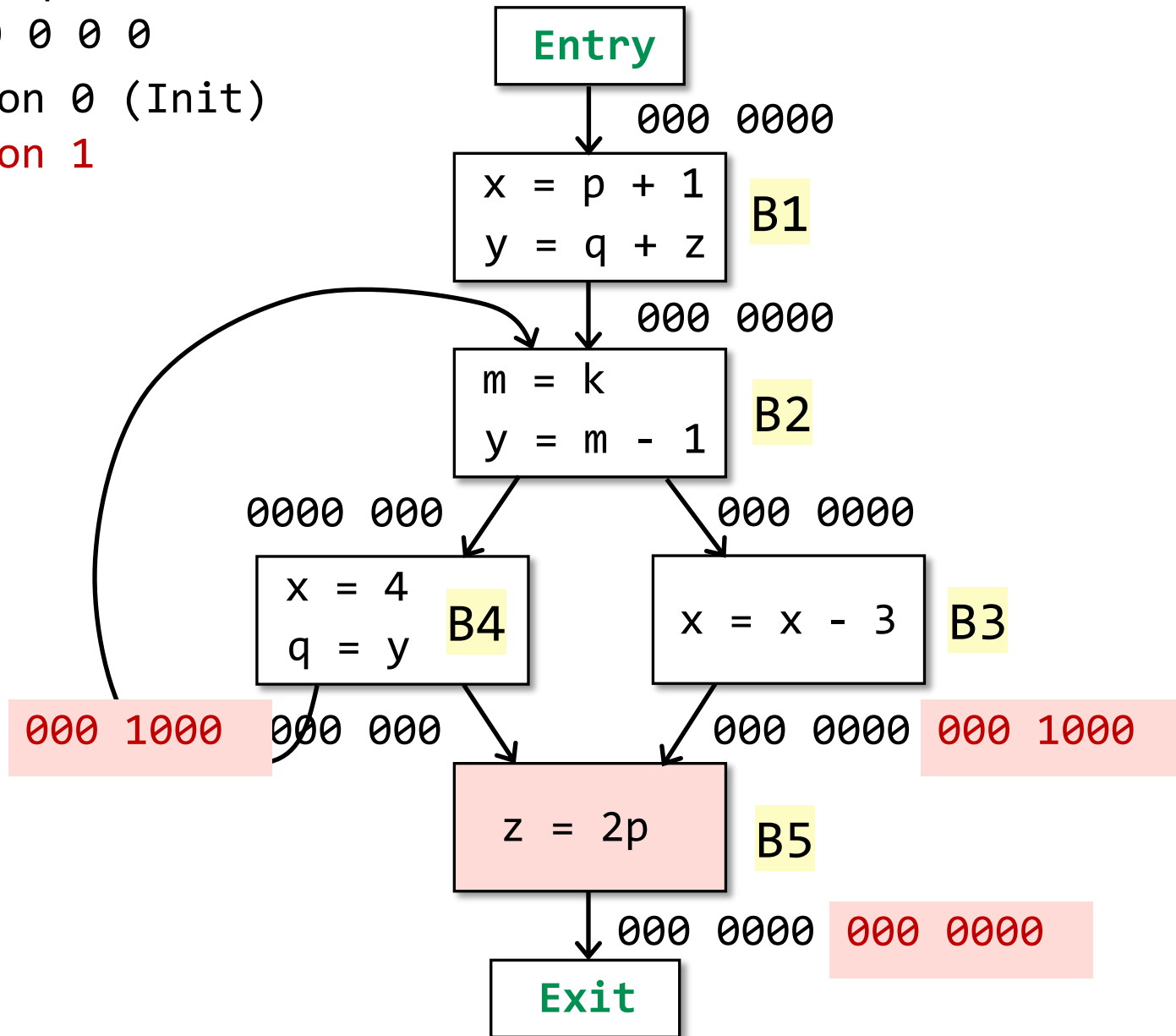
$$IN[B] = use_B \cup (OUT[B] - def_B)$$

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

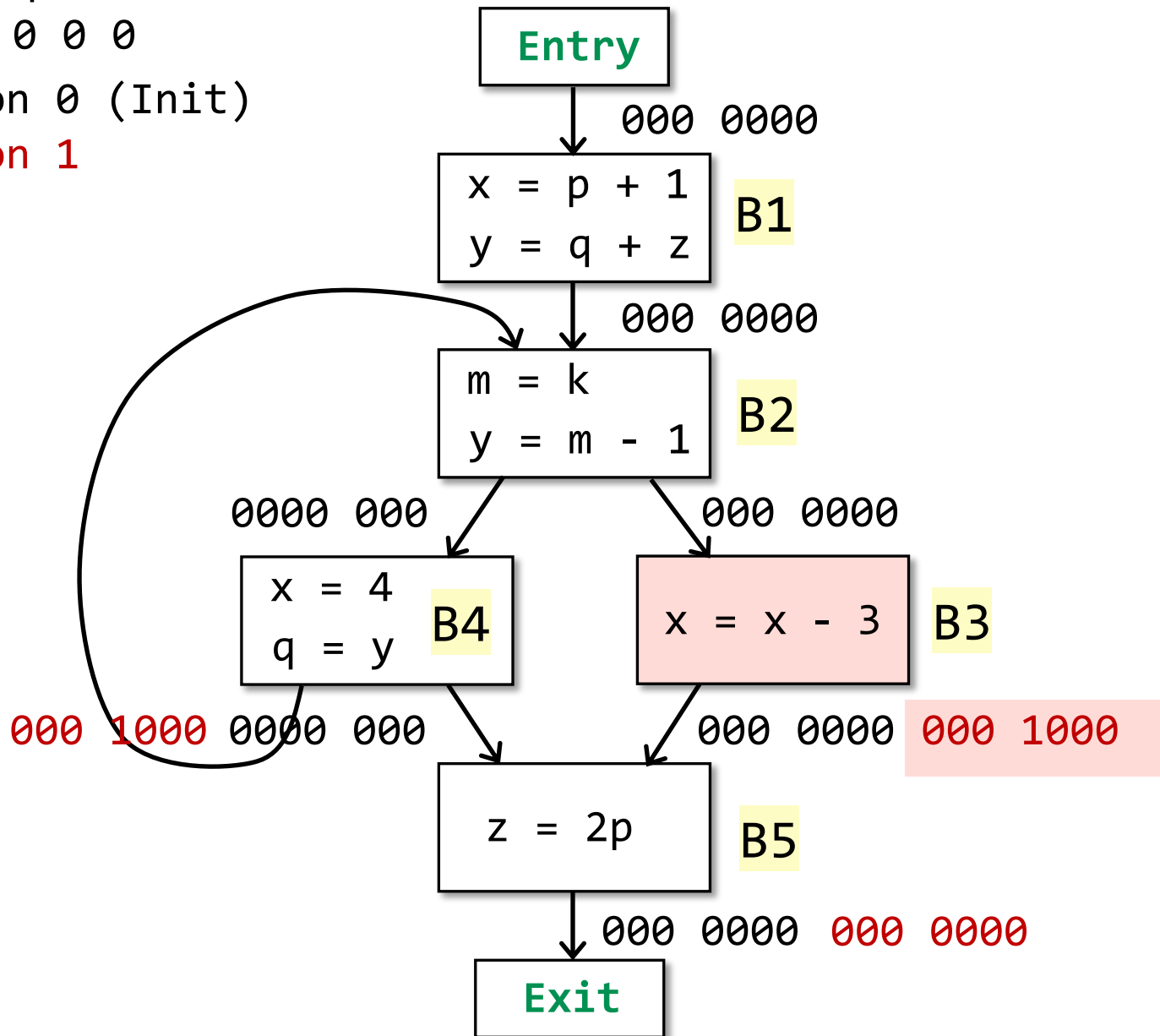


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

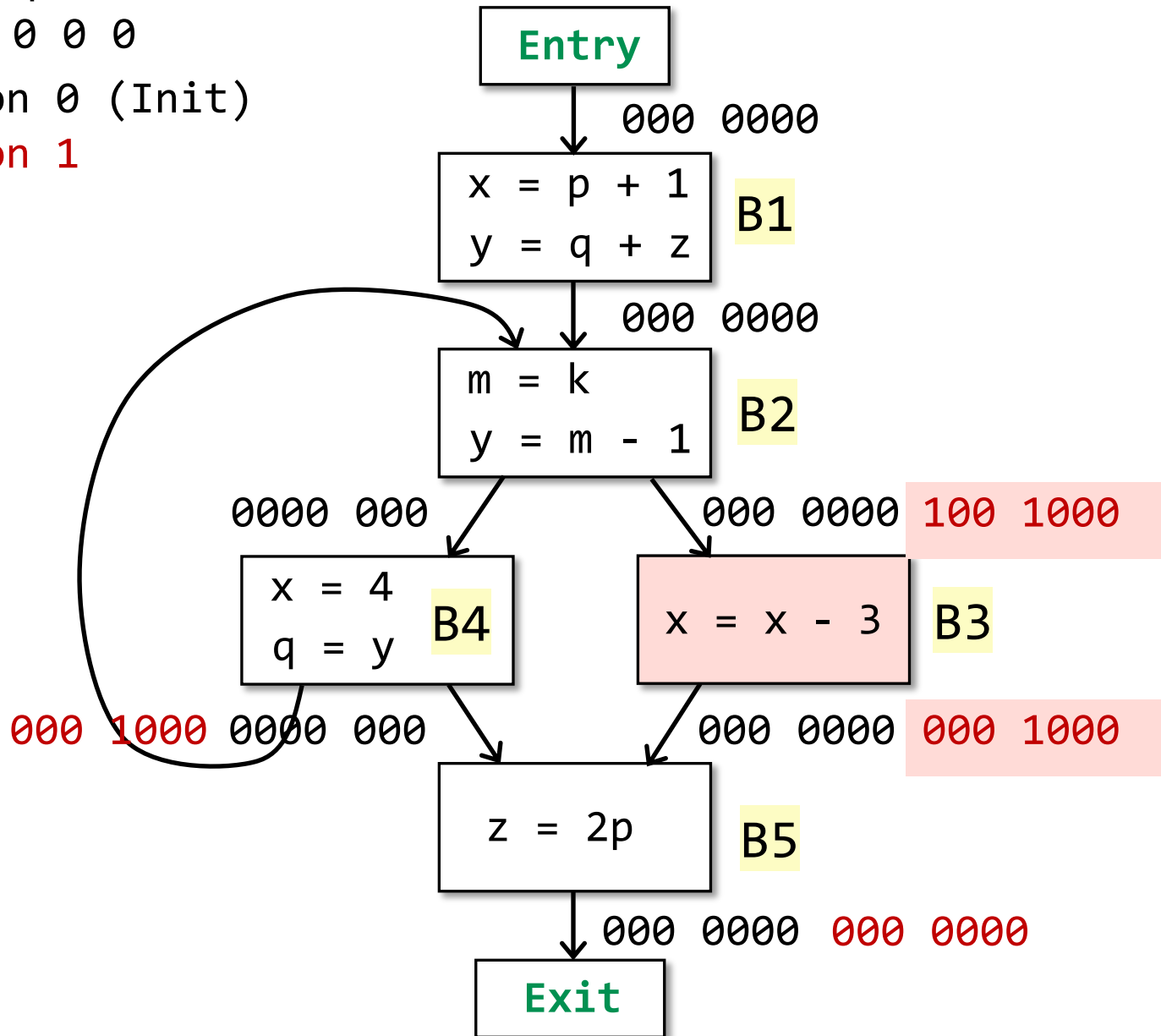


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

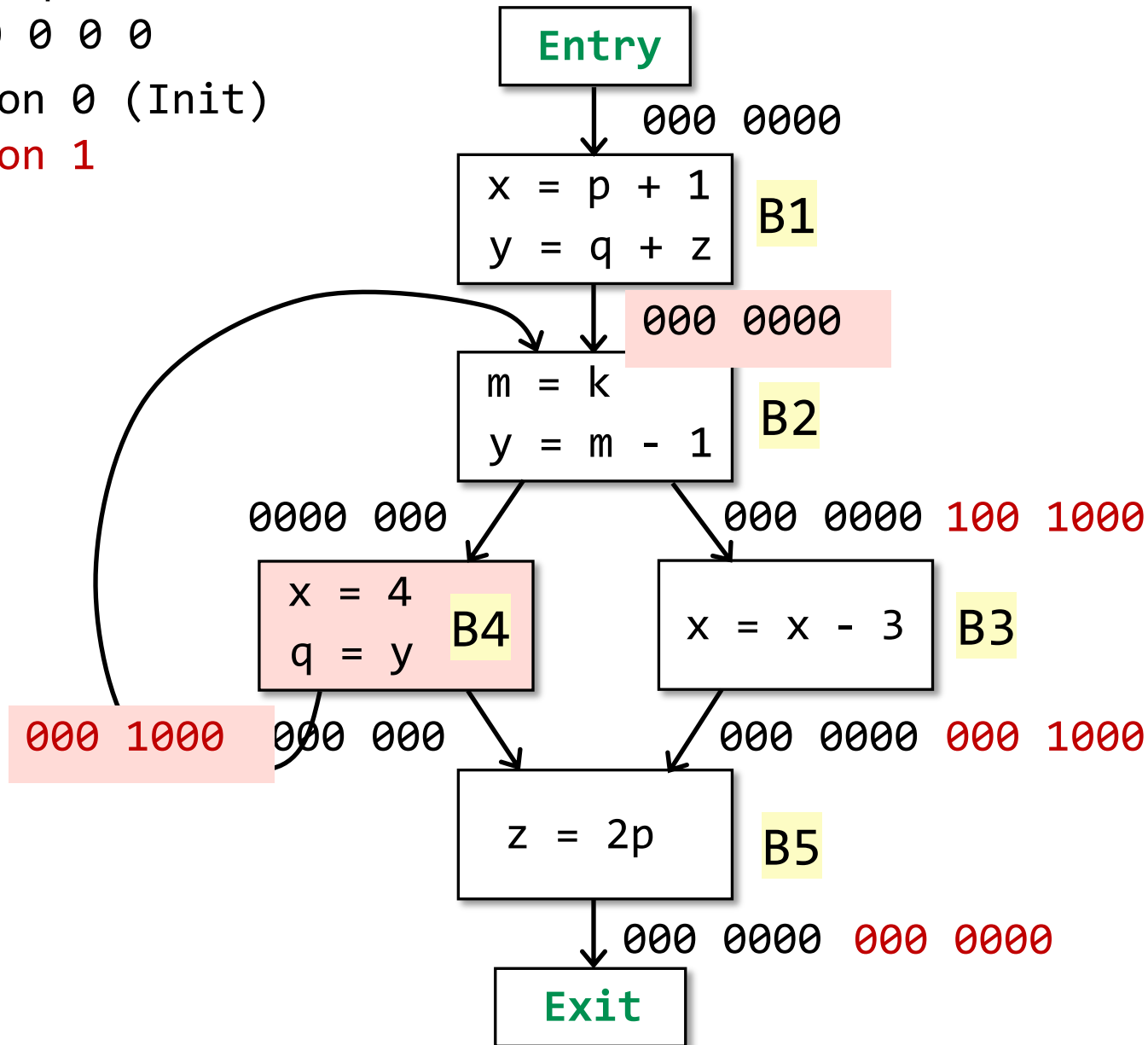


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1



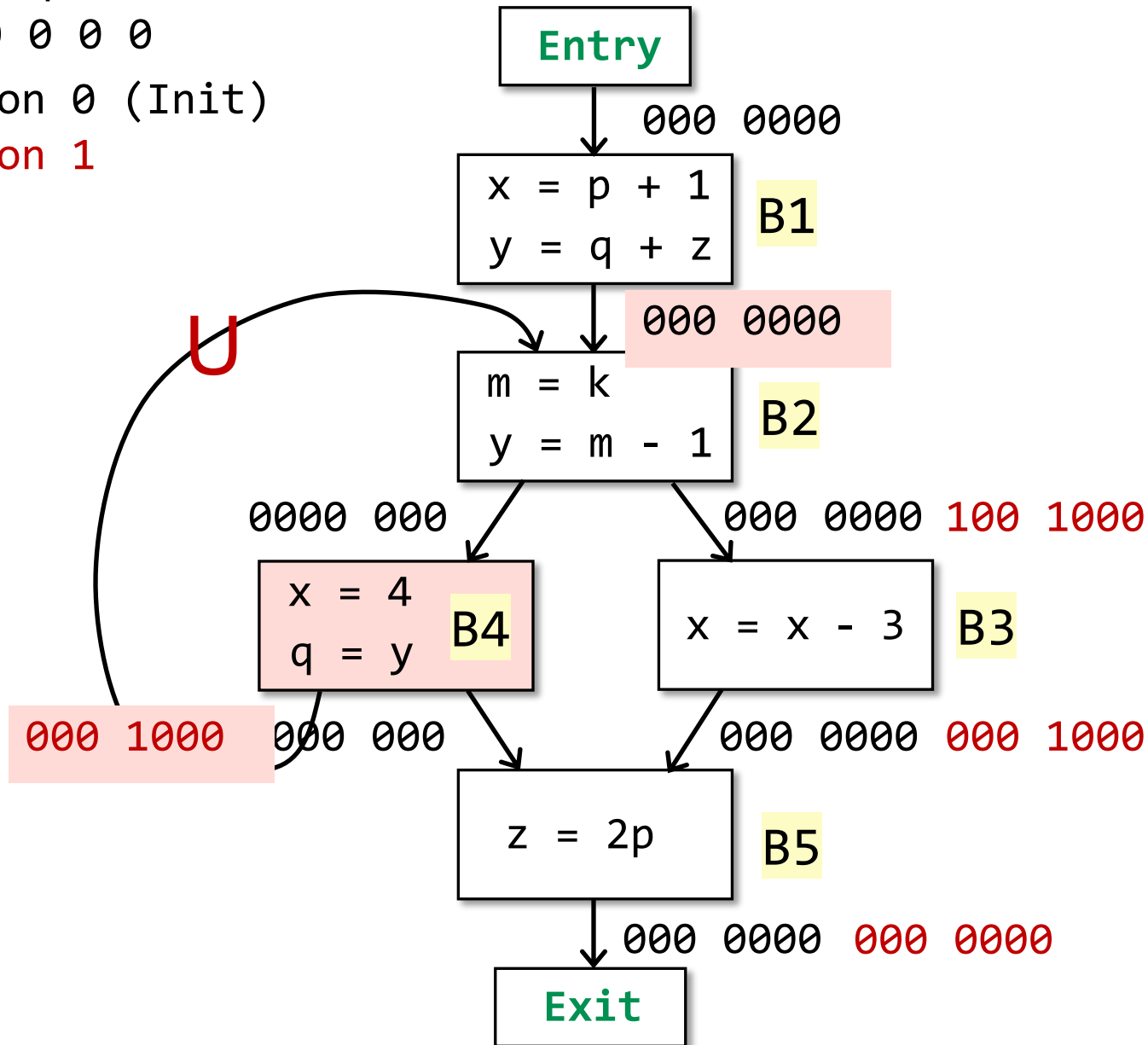
$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

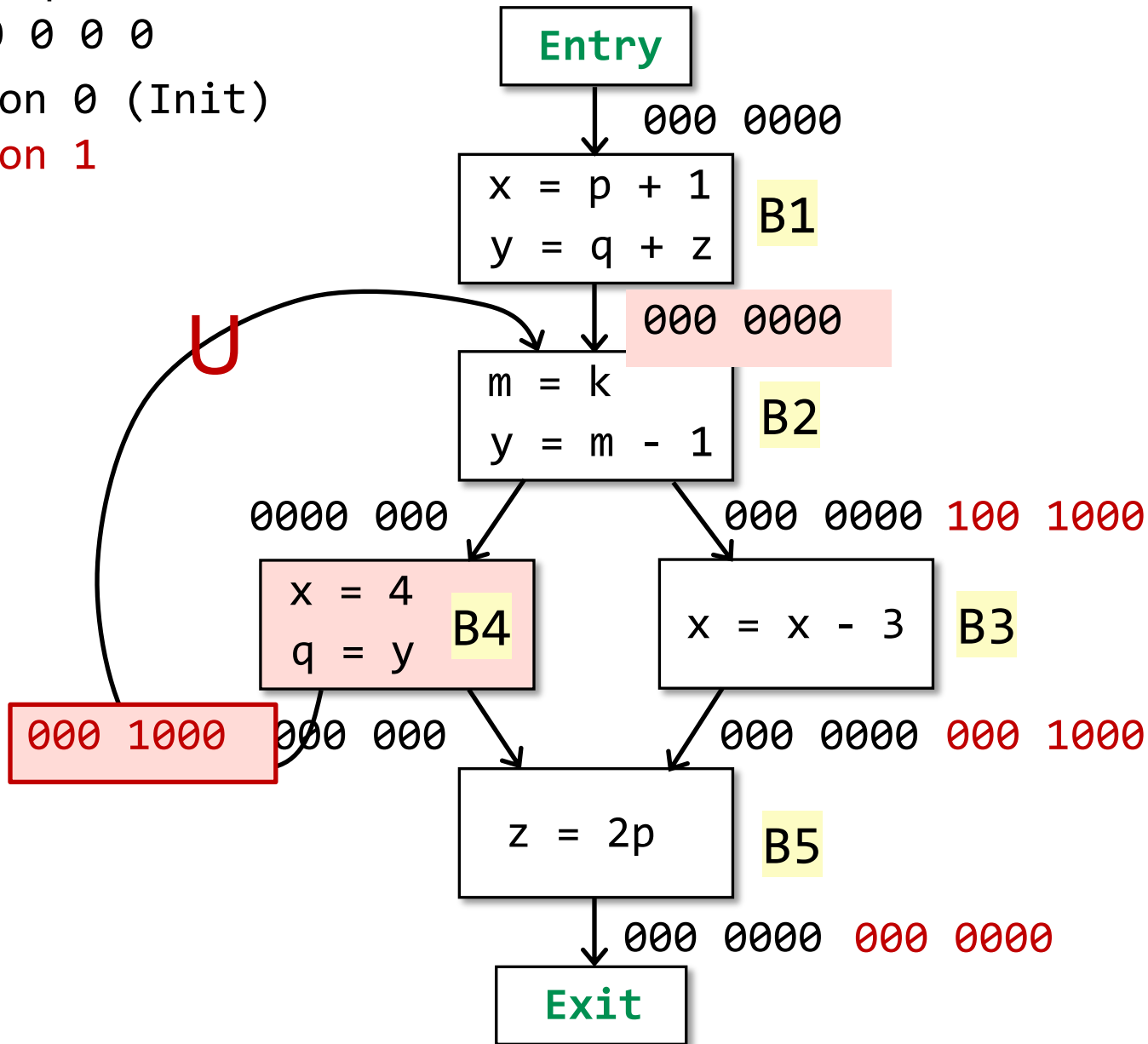


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

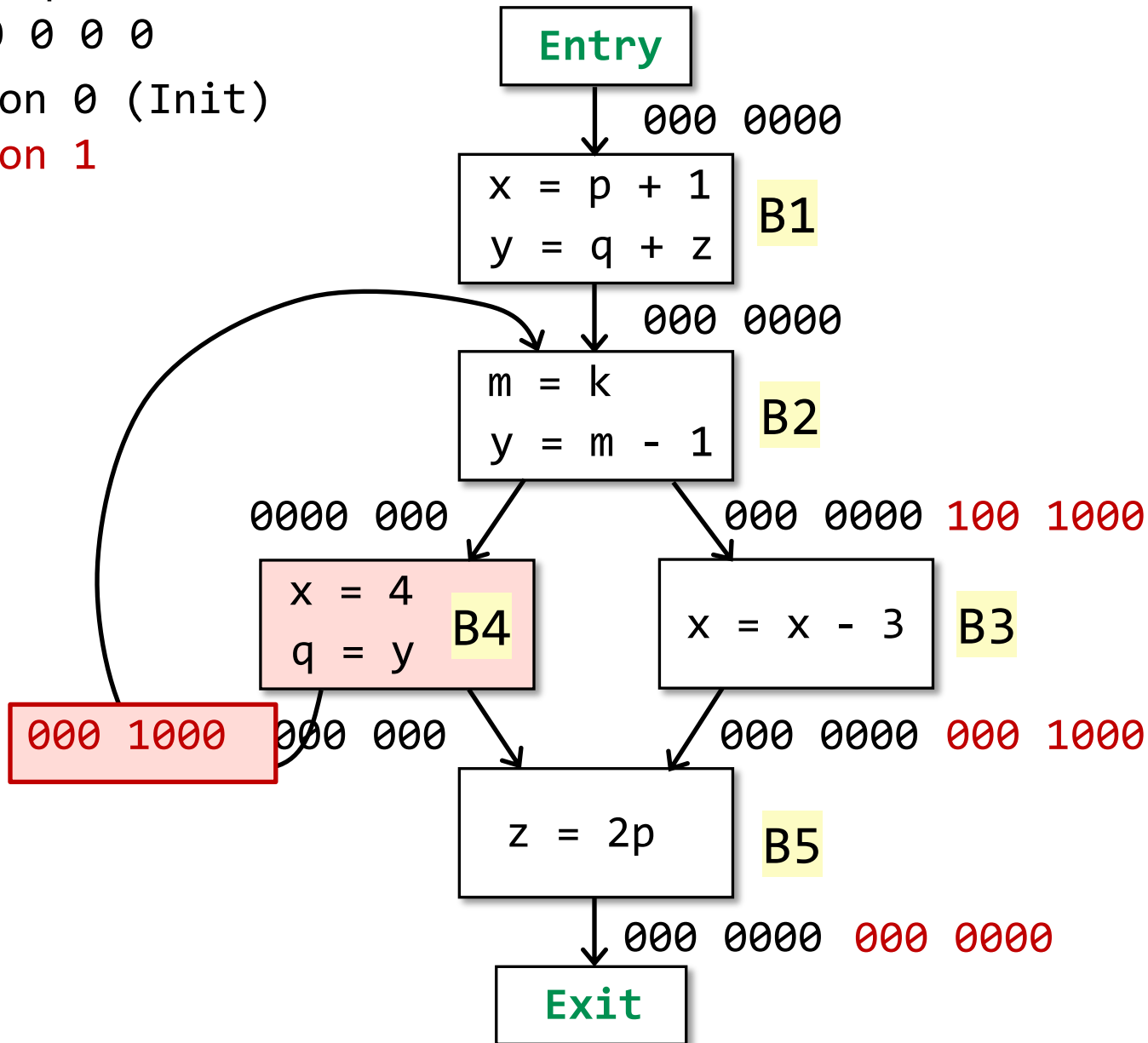


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

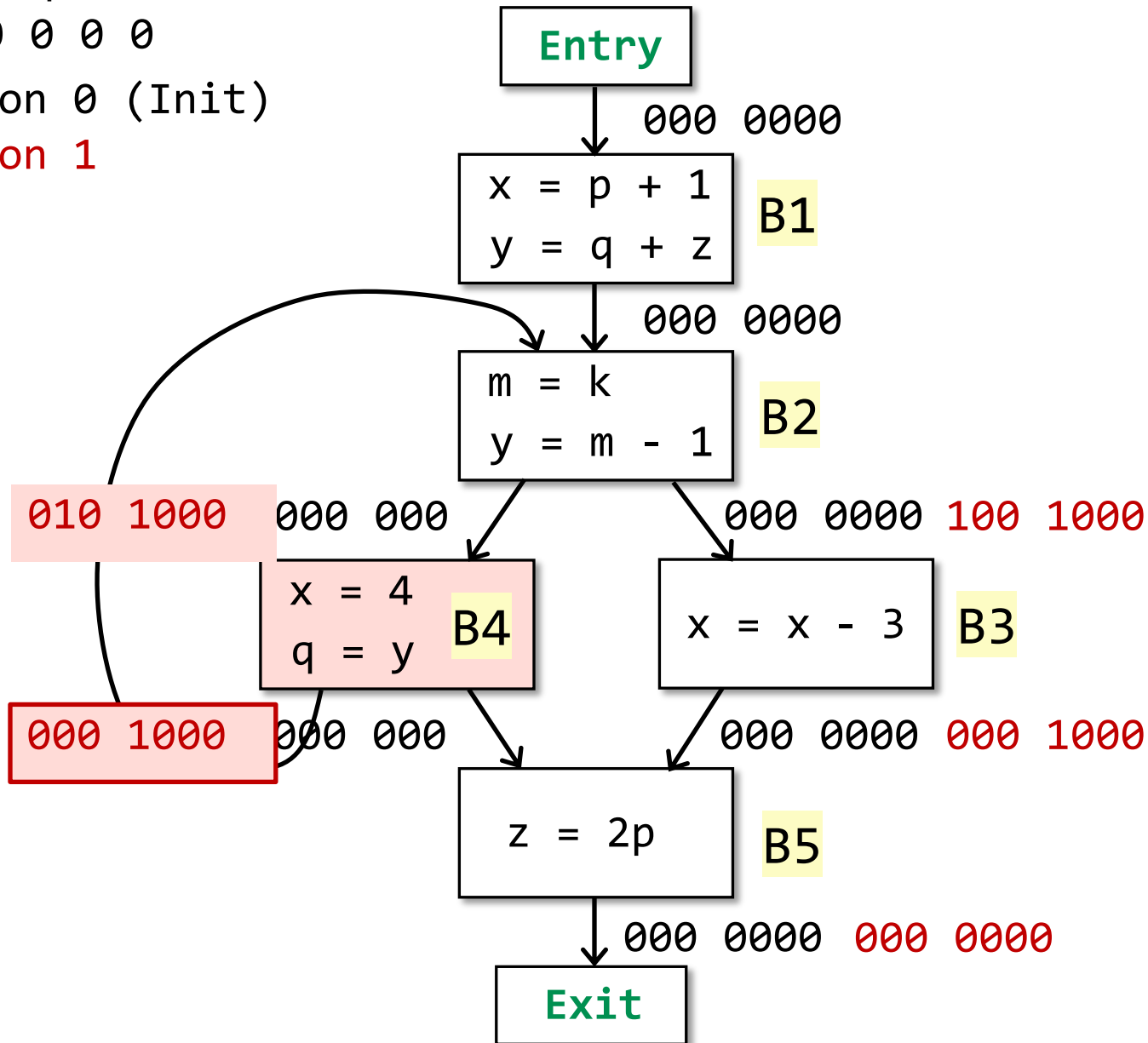


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

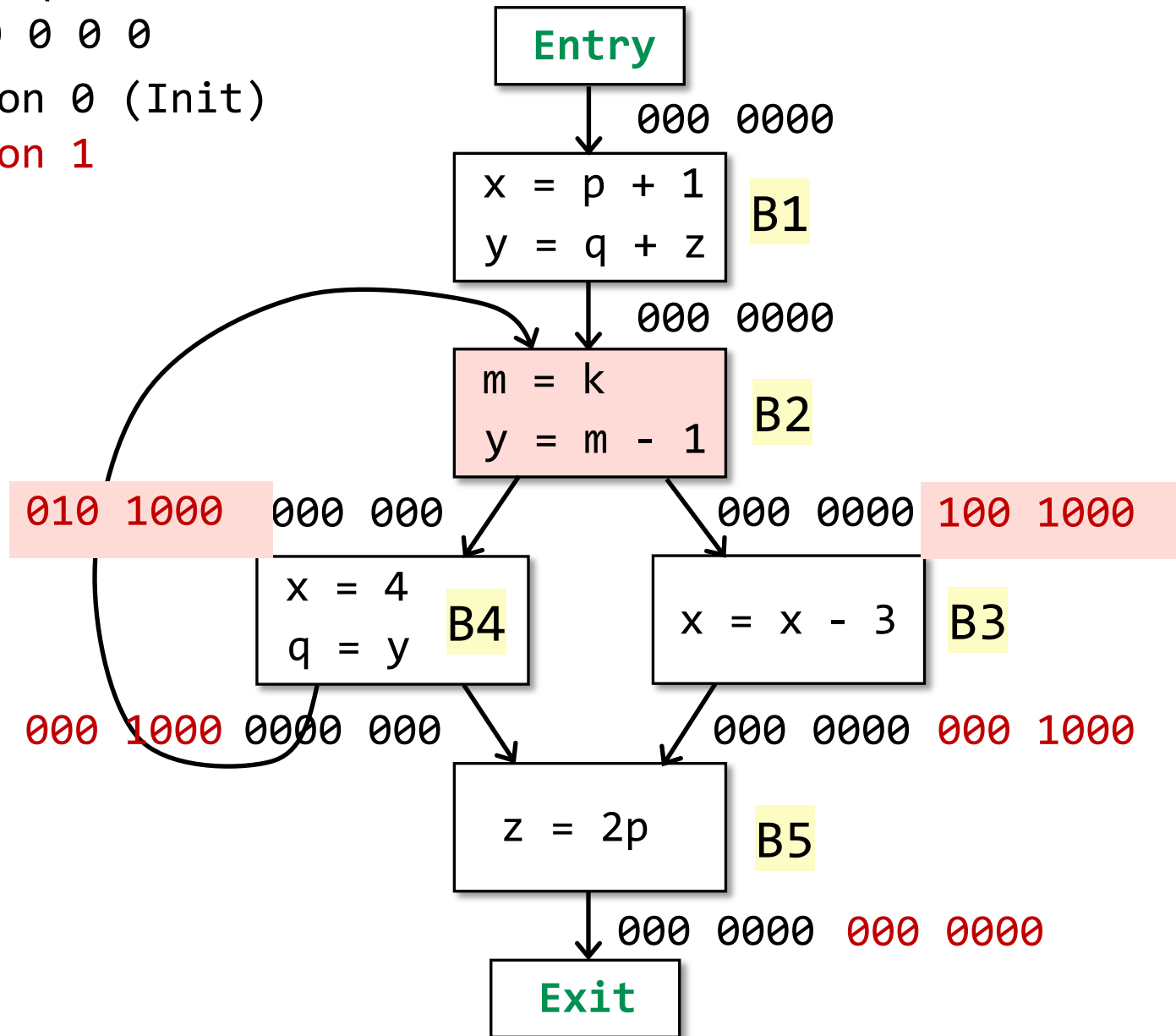


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

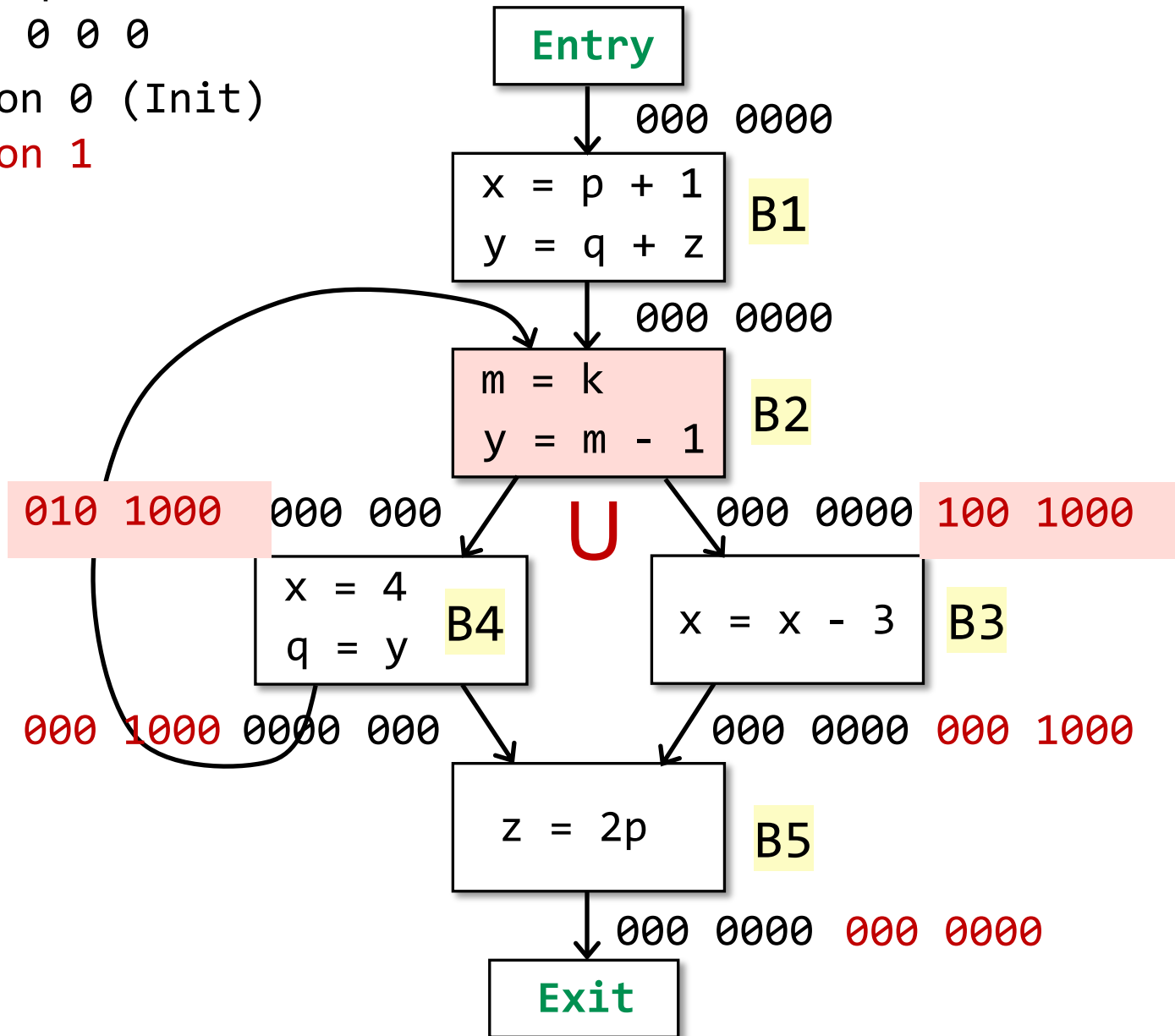


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

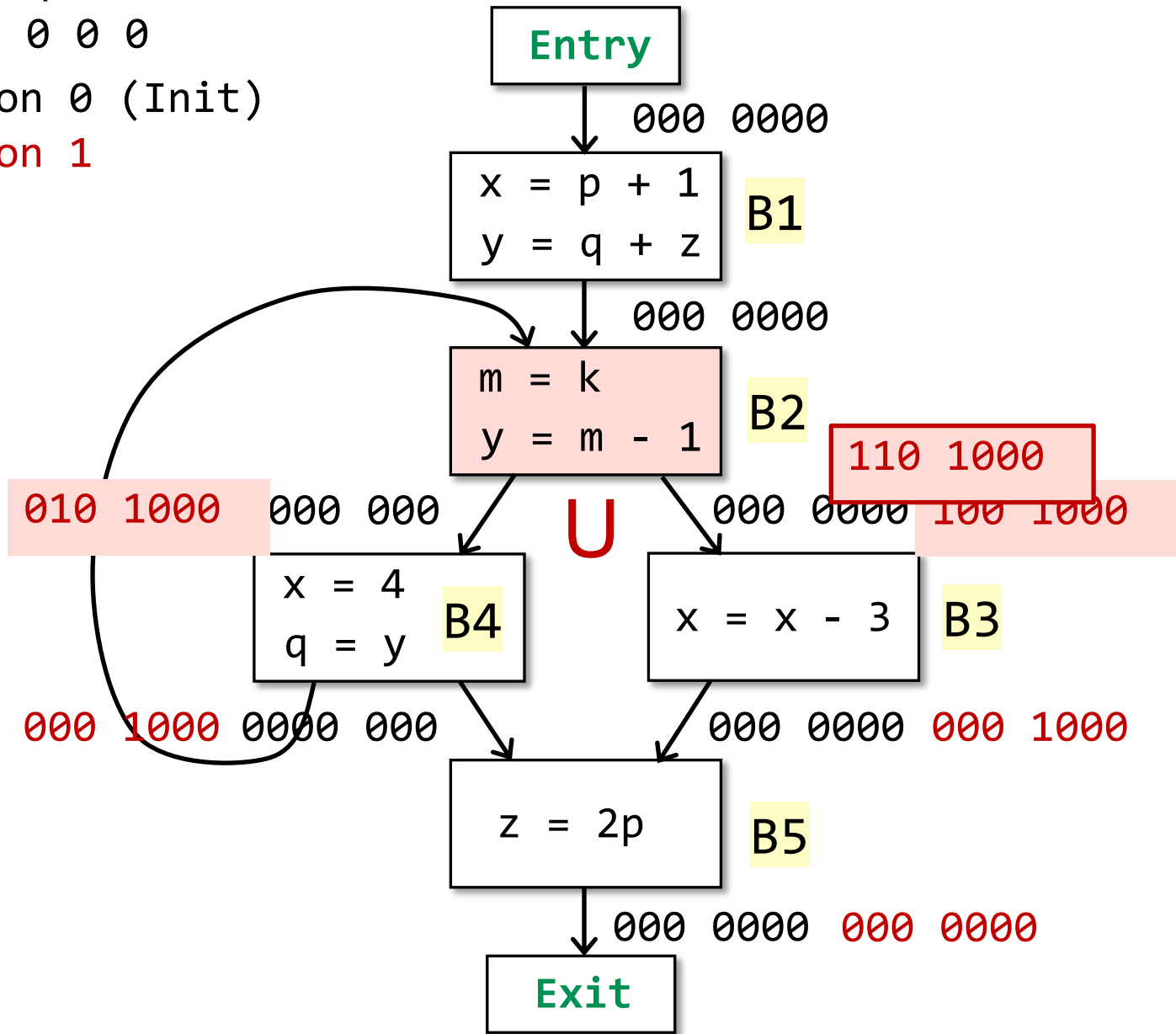
Iteration 1



x	y	z	p	q	m	k
0	0	0	0	0	0	0

Iteration 0 (Init)

Iteration 1

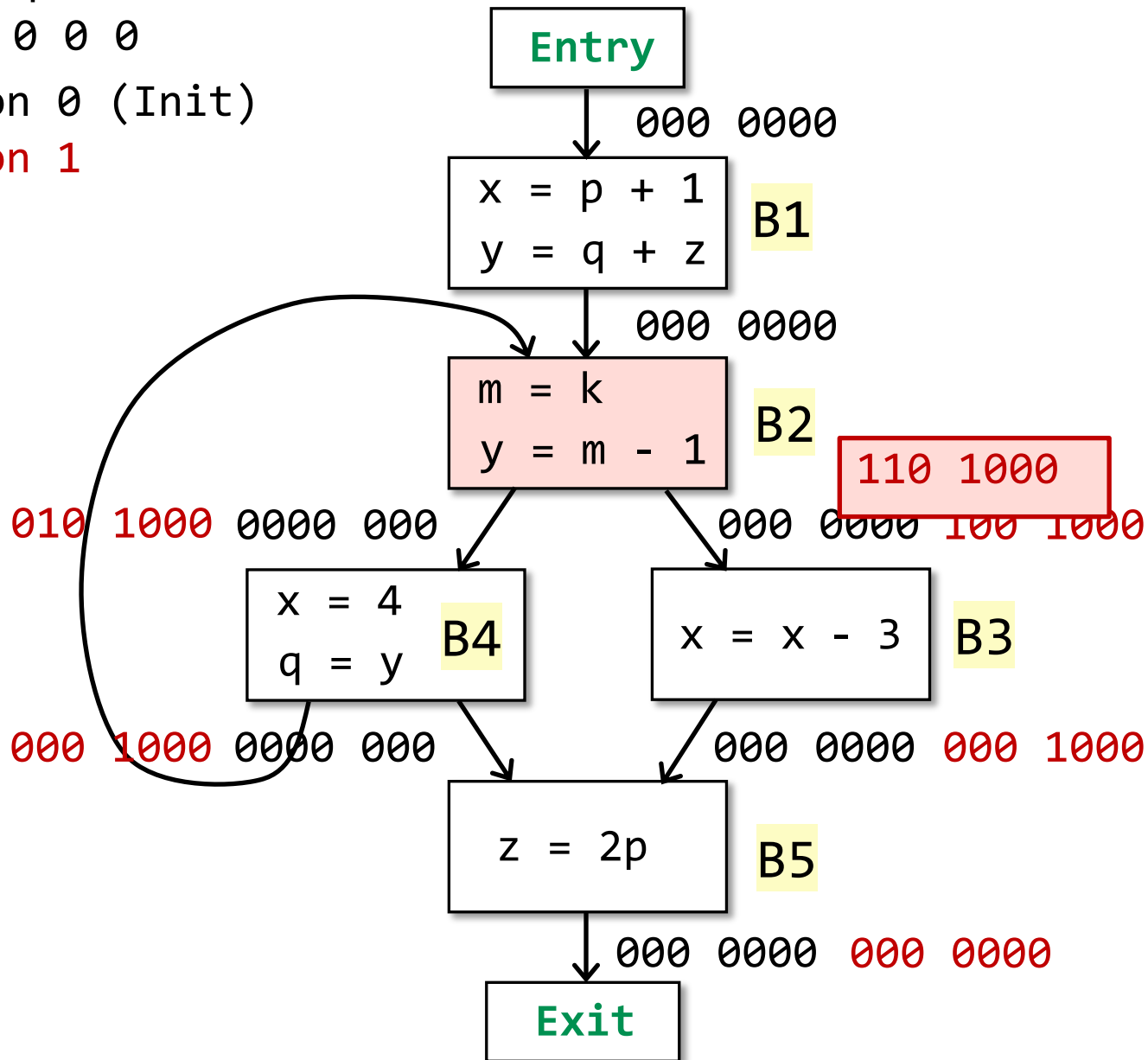


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

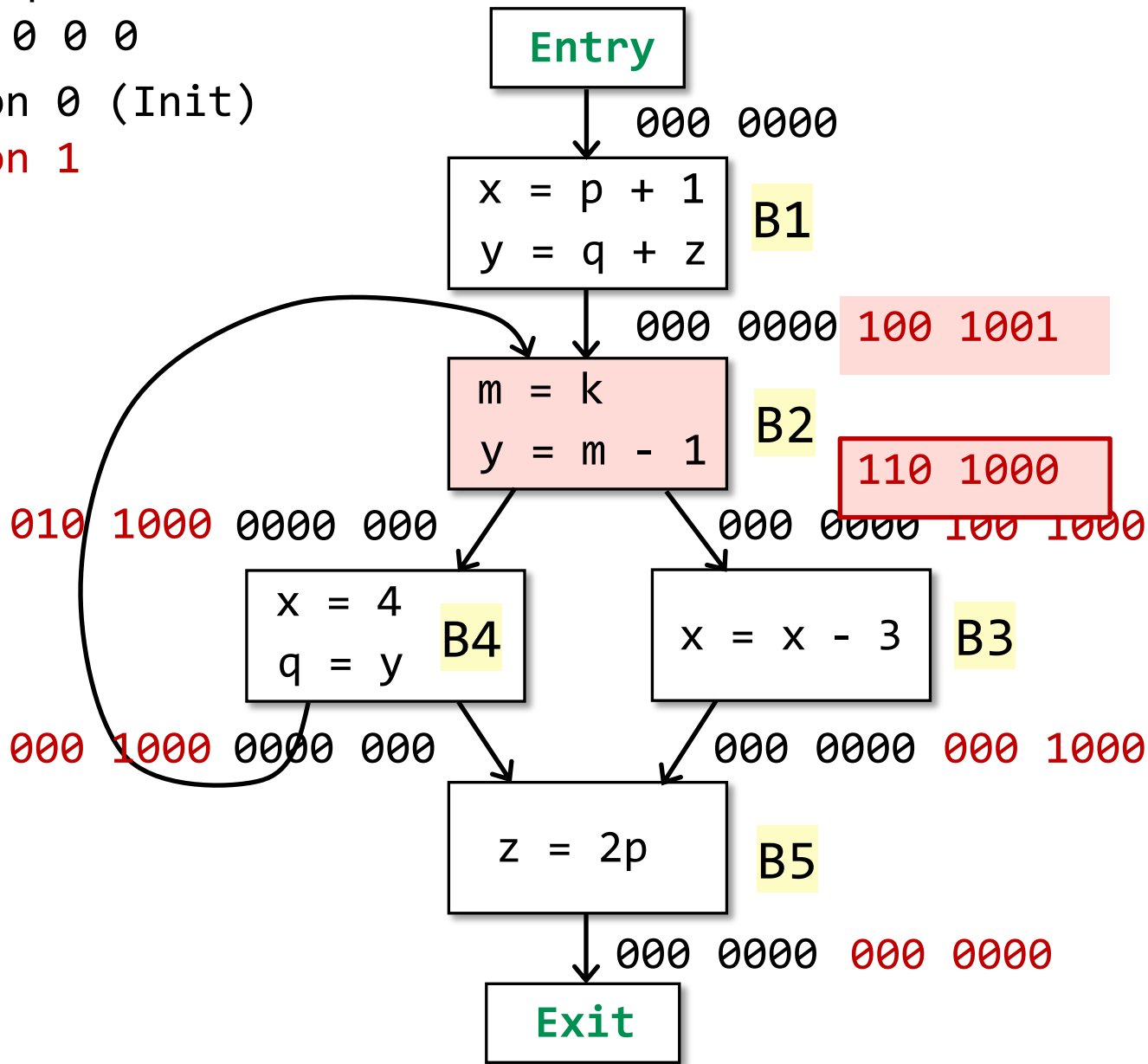
Iteration 1



x	y	z	p	q	m	k
0	0	0	0	0	0	0

Iteration 0 (Init)

Iteration 1

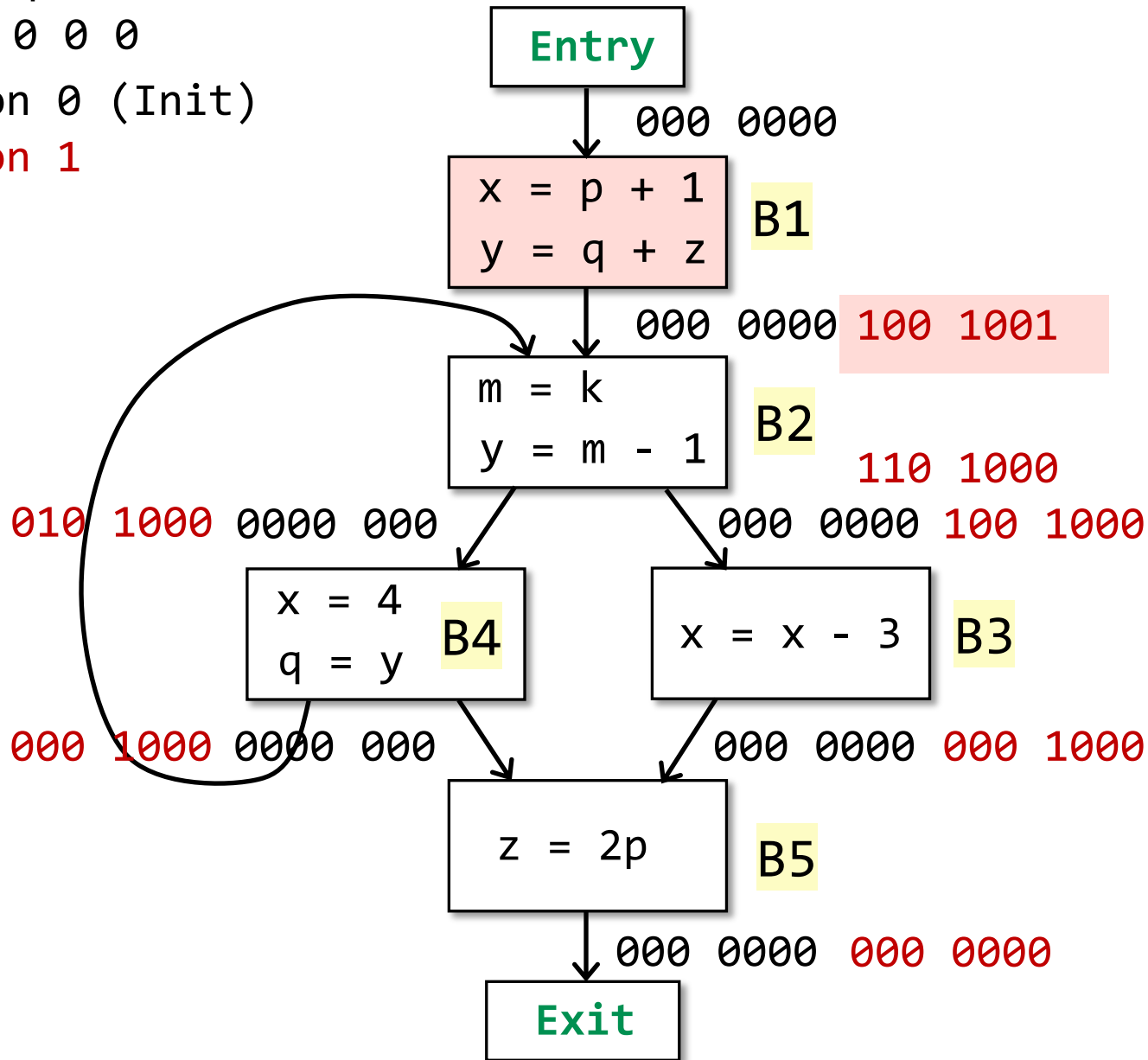


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

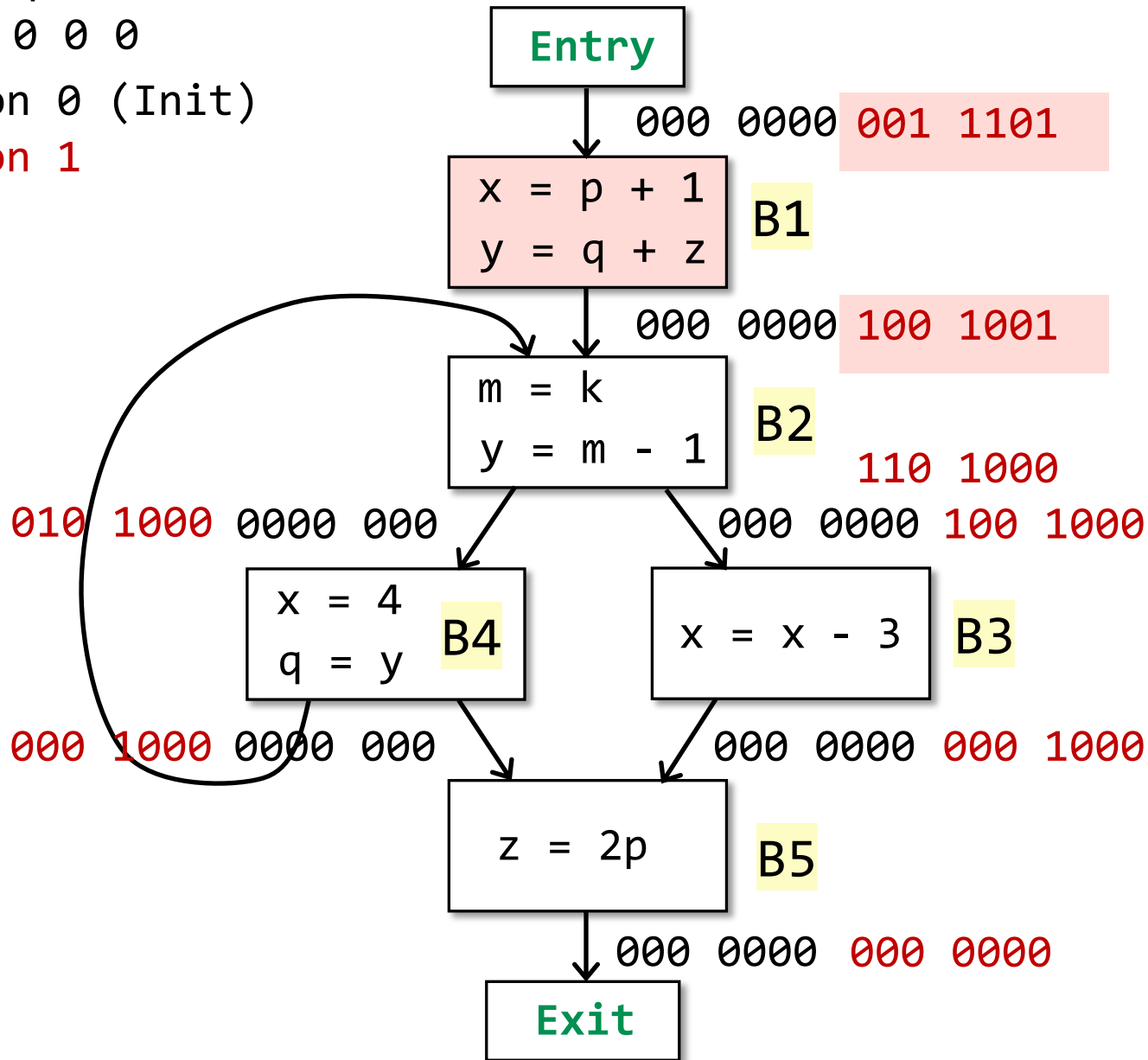


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

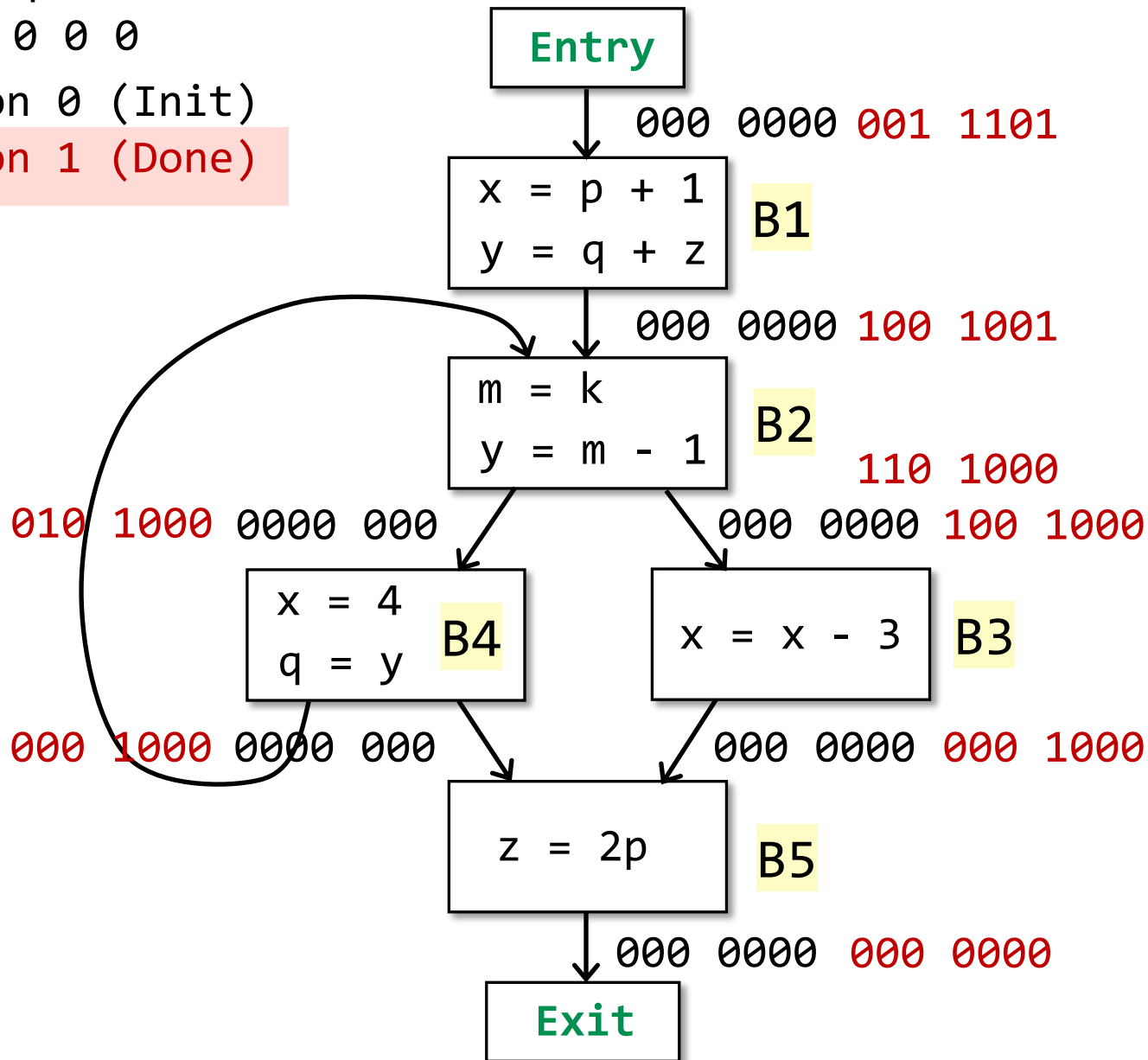


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

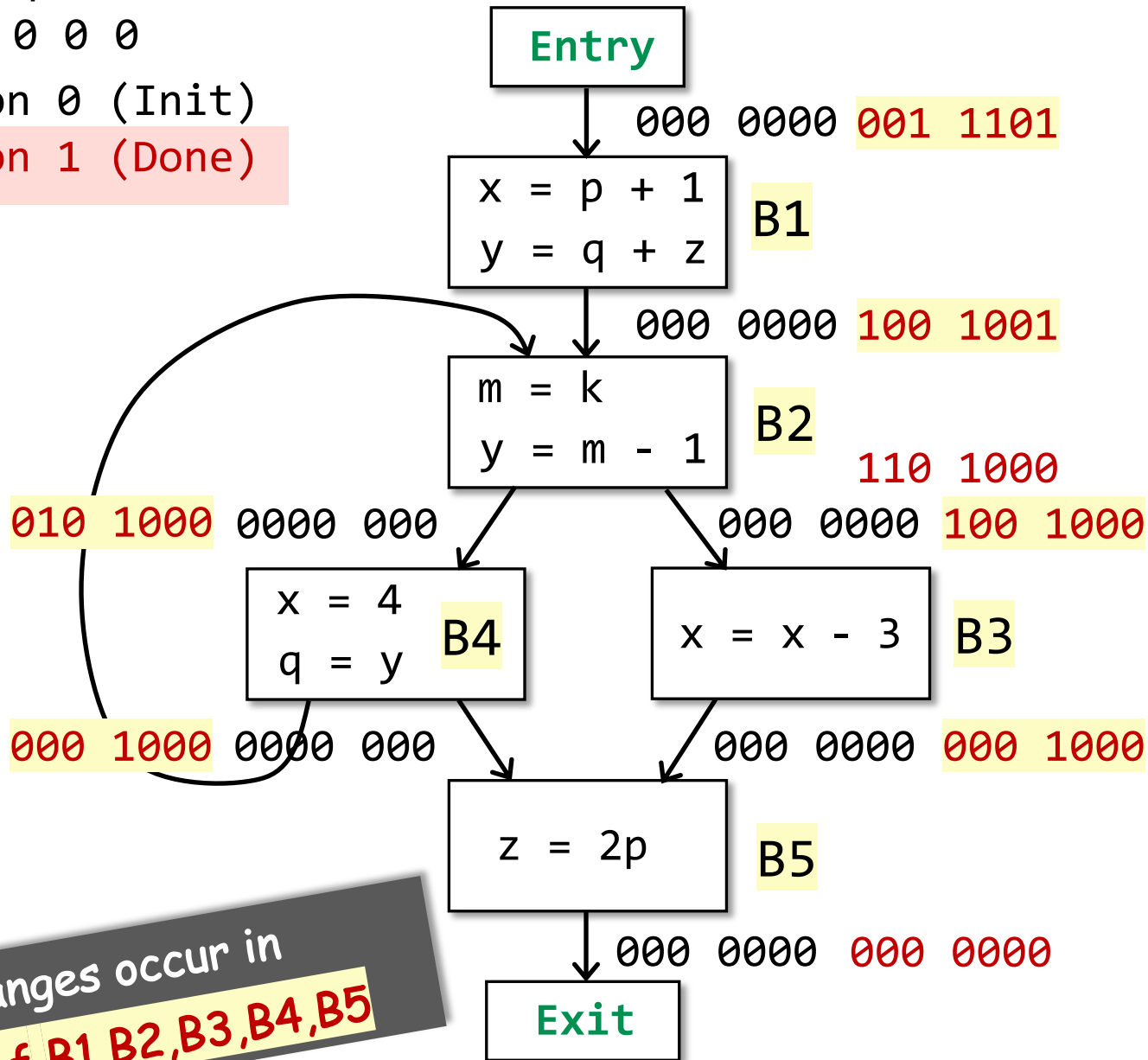


x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)



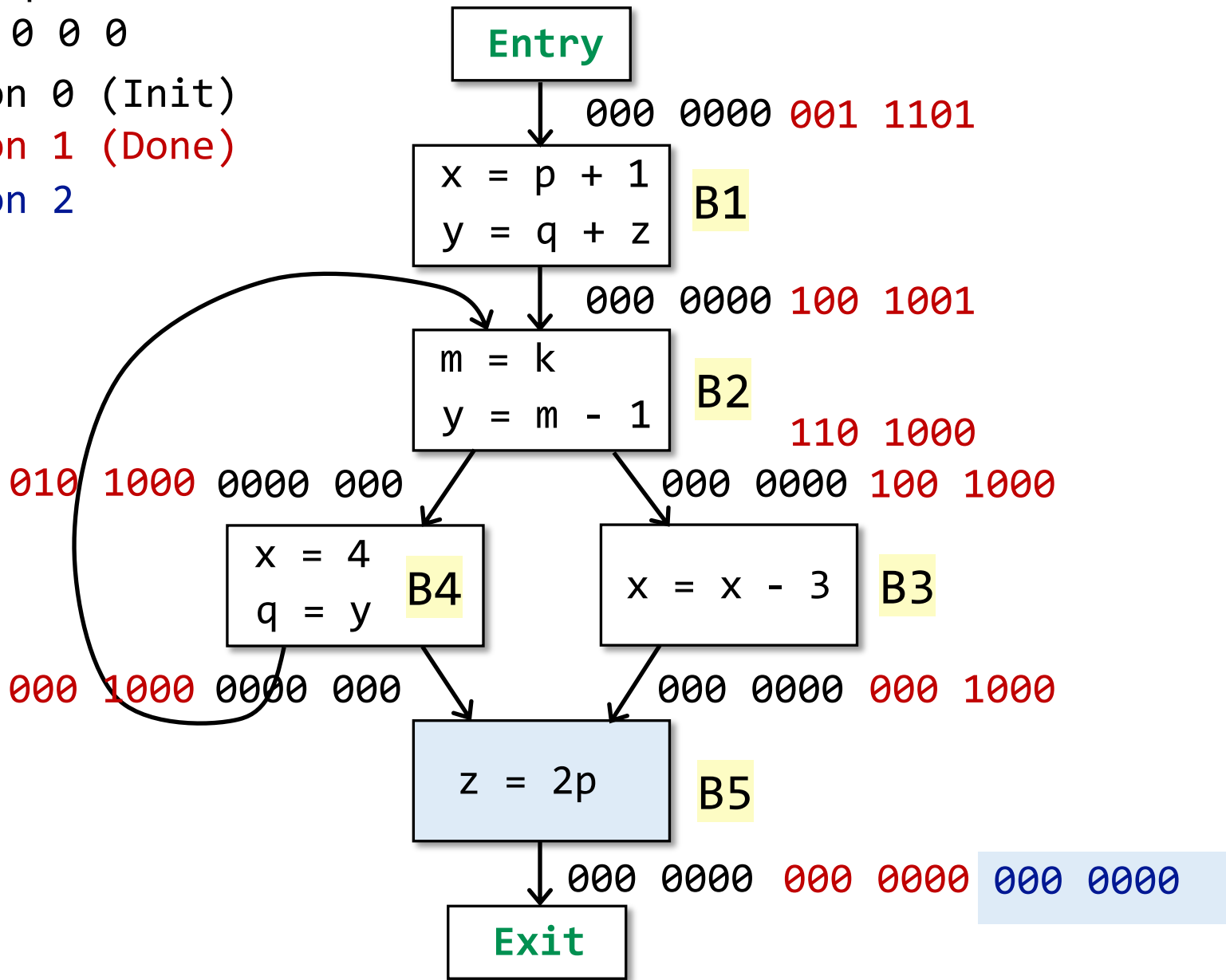
Changes occur in
IN[] of B1, B2, B3, B4, B5

x	y	z	p	q	m	k
0	0	0	0	0	0	0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



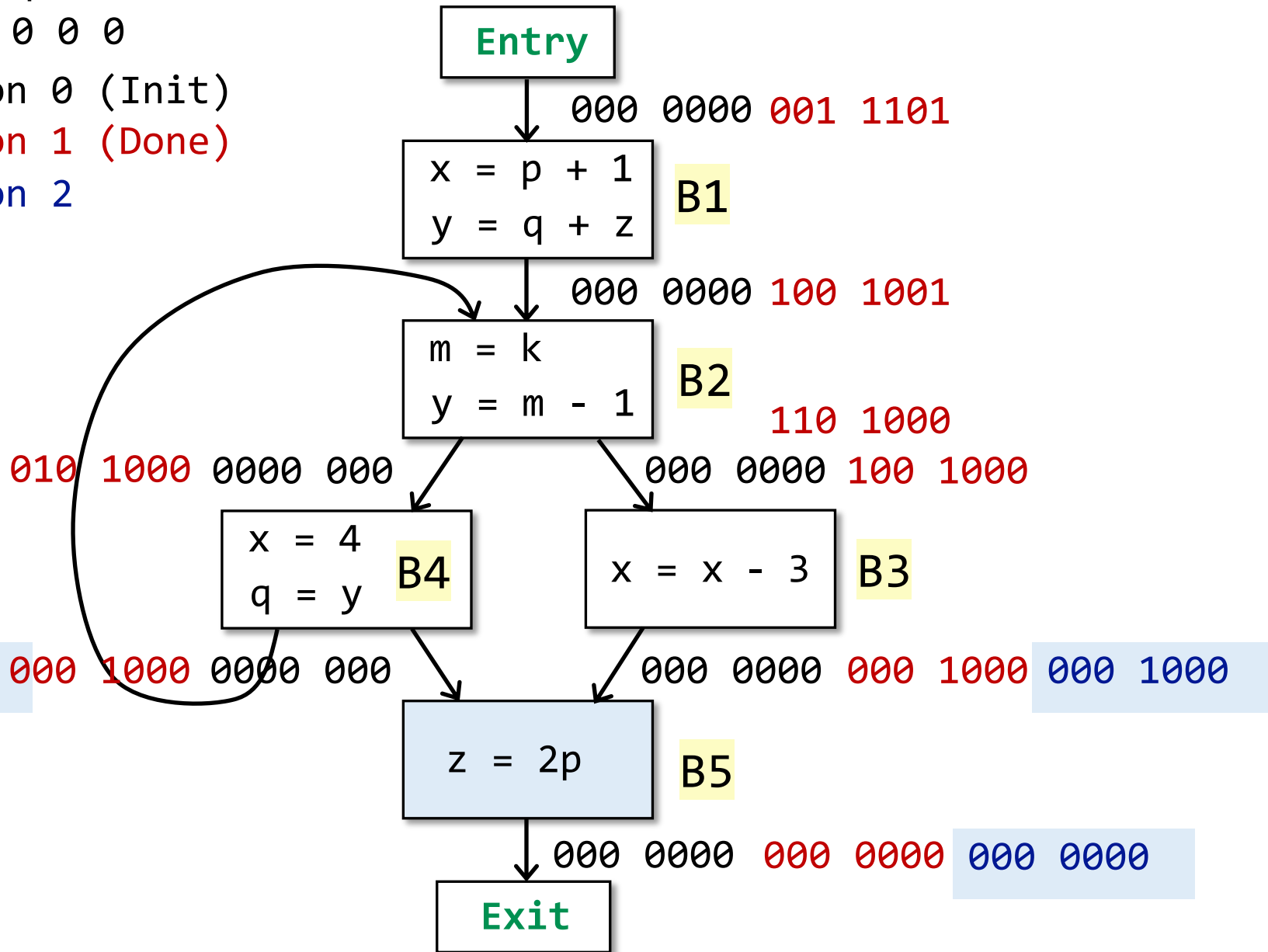
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



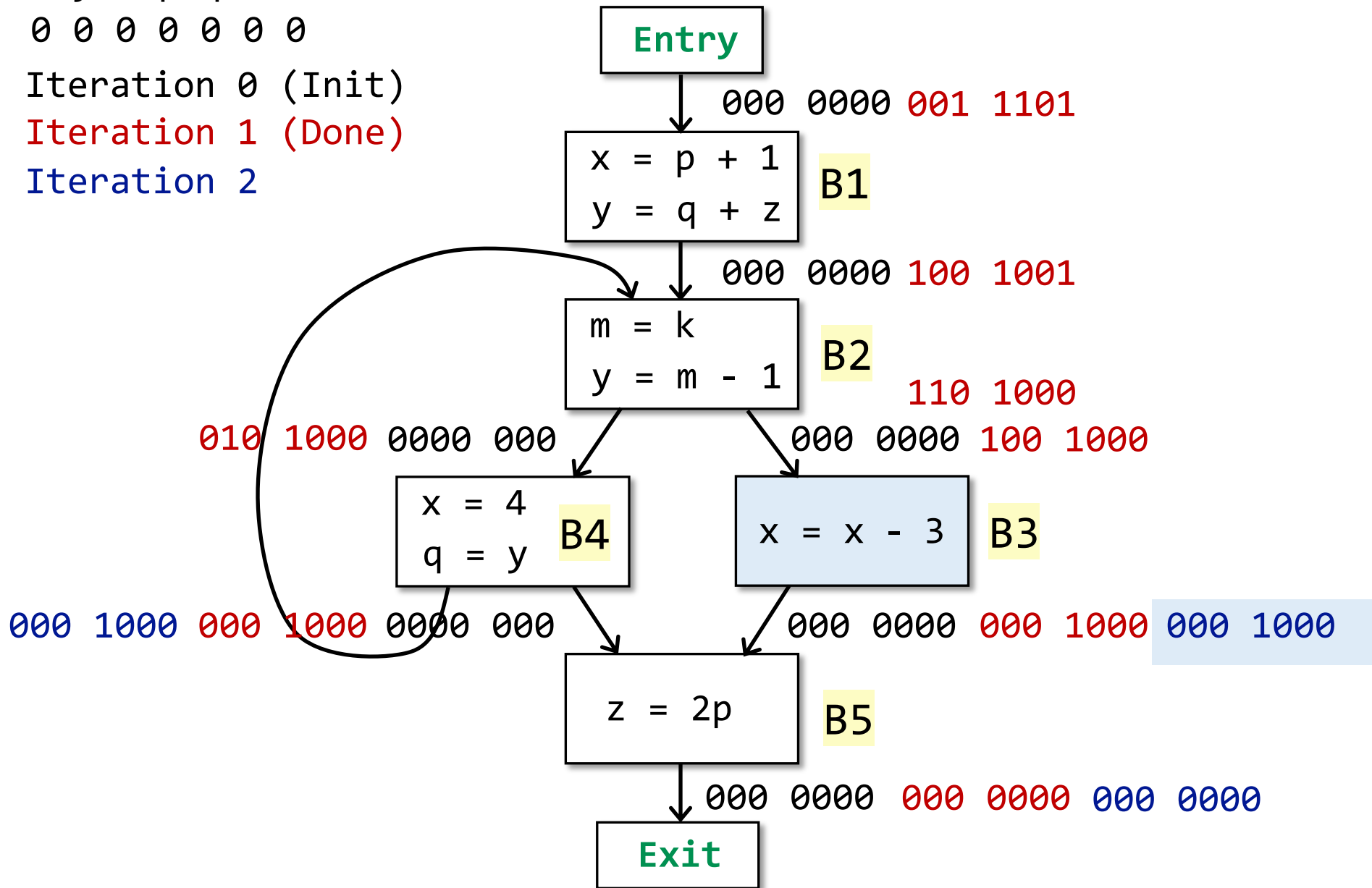
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



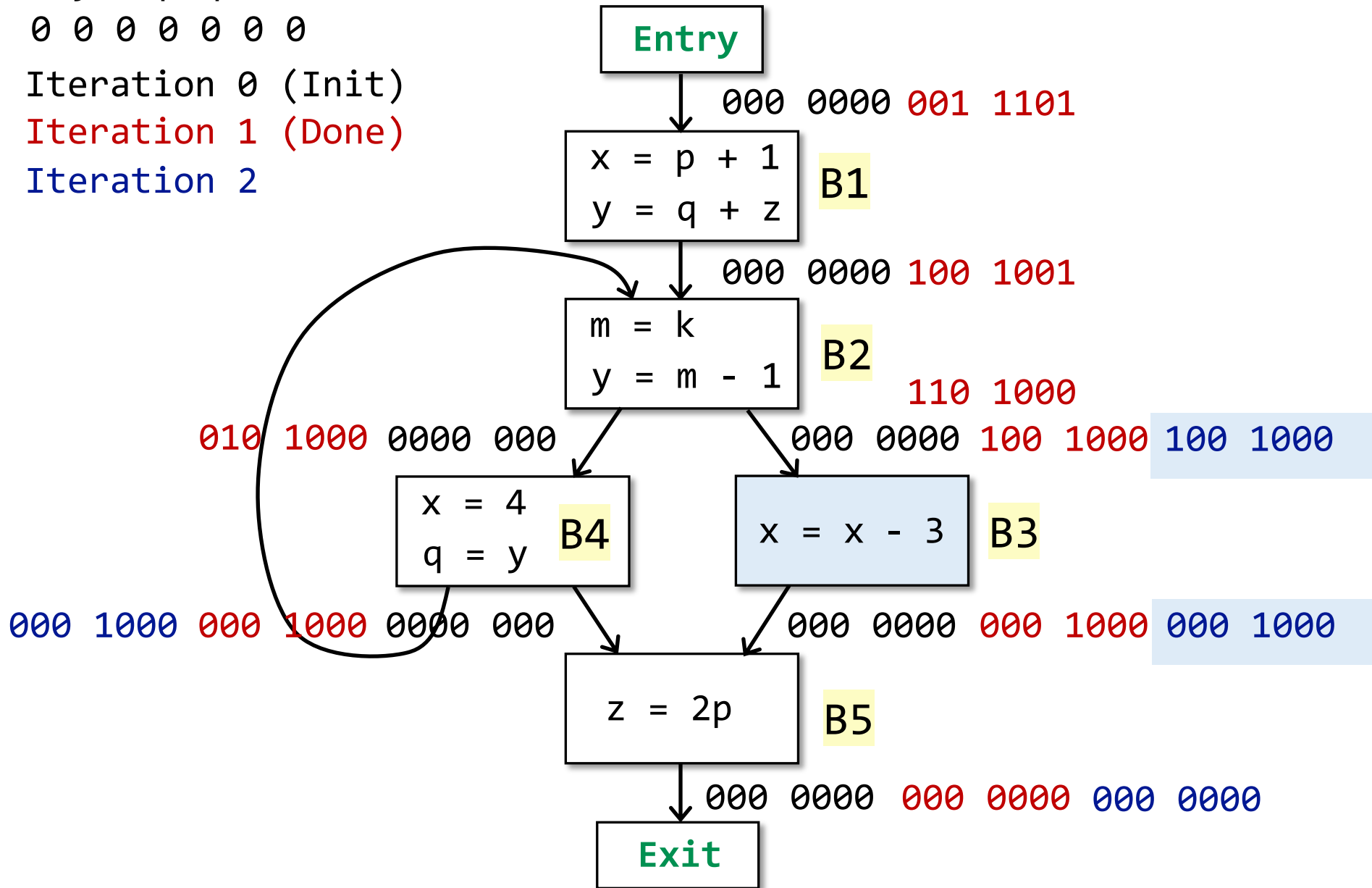
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



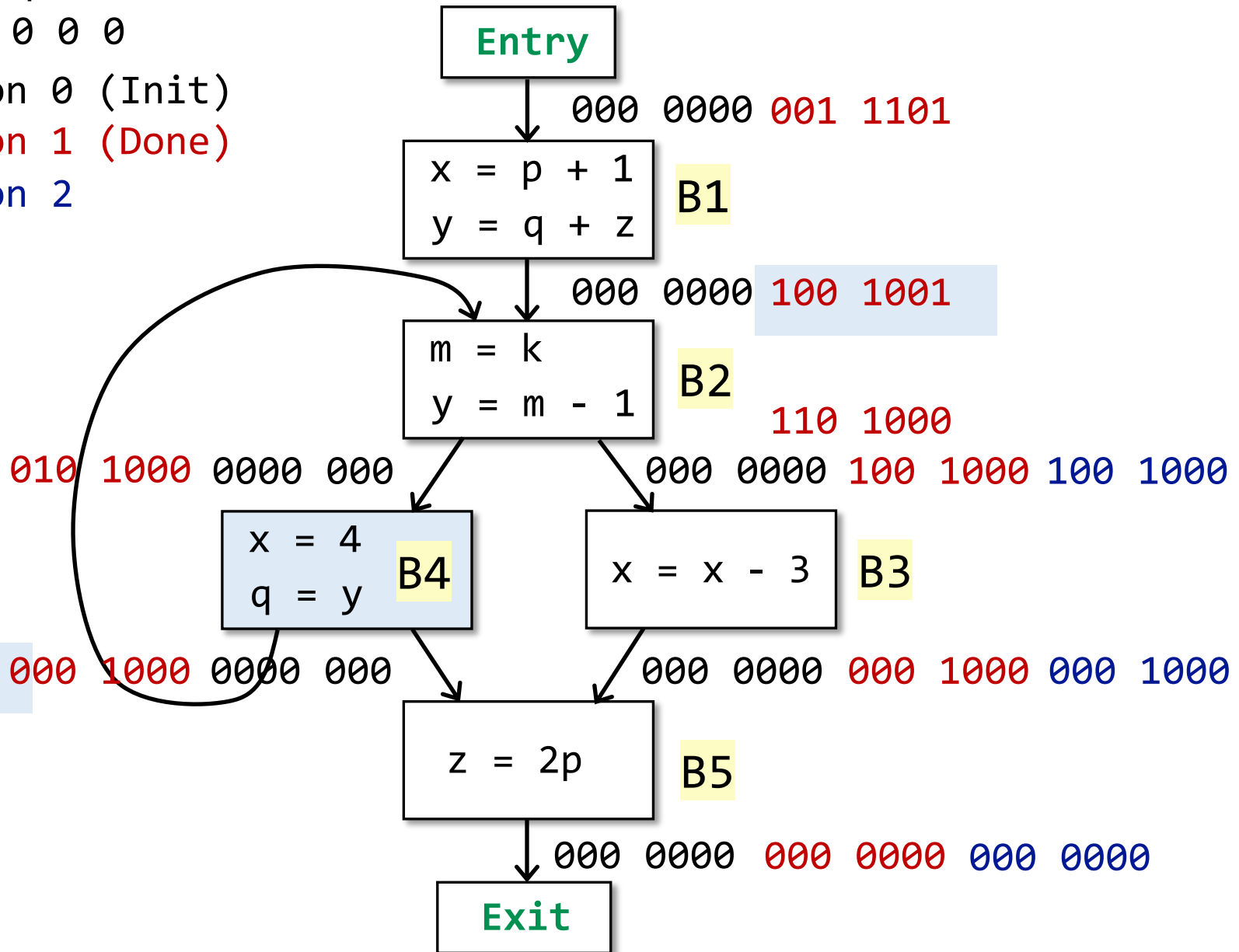
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



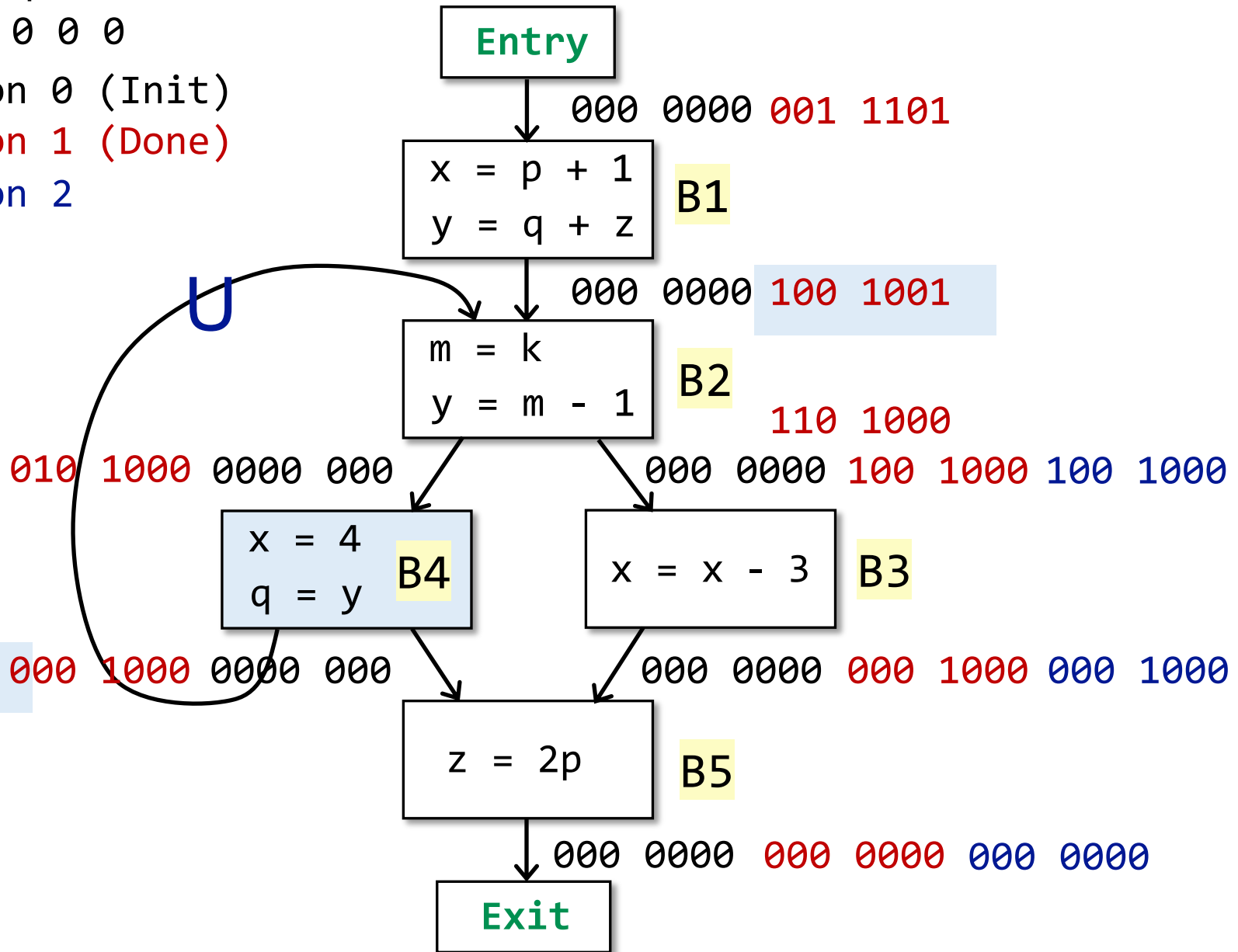
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

Entry

000 0000 001 1101

x = p + 1
y = q + z

B1

000 0000 100 1001

m = k
y = m - 1

B2

110 1000

000 0000 100 1000 100 1000

x = 4
q = y

B4

x = x - 3

B3

000 0000 000 1000 000 1000

z = 2p

B5

000 0000 000 0000 000 0000

Exit

U

010 1000 0000 000

100 1001

000 1000 000 1000 0000 000

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

Entry

000 0000 001 1101

x = p + 1
y = q + z

B1

000 0000 100 1001

m = k
y = m - 1

B2

110 1000

010 1000 0000 000

000 0000 100 1000 100 1000

x = 4
q = y

B4

x = x - 3

B3

100 1001

000 1000 000 1000 0000 000

000 0000 000 1000 000 1000

z = 2p

B5

000 0000 000 0000 000 0000

Exit

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

Entry

000 0000 001 1101

x = p + 1
y = q + z

B1

000 0000 100 1001

m = k
y = m - 1

B2

110 1000

010 1001

10 1000 0000 000

000 0000 100 1000 100 1000

x = 4
q = y

B4

x = x - 3

B3

100 1001

000 1000 000 1000 0000 000

000 0000 000 1000 000 1000

z = 2p

B5

000 0000 000 0000 000 0000

Exit

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

Entry

000 0000 001 1101

x = p + 1
y = q + z

B1

000 0000 100 1001

m = k
y = m - 1

B2

110 1000

010 1001

10 1000 0000 000

000 0000 100 1000

100 1000

x = 4
q = y

B4

x = x - 3

B3

100 1001

000 1000 000 1000 0000 000

000 0000 000 1000 000 1000

z = 2p

B5

000 0000 000 0000 000 0000

Exit

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

Entry

000 0000 001 1101

x = p + 1
y = q + z

B1

000 0000 100 1001

m = k
y = m - 1

B2

110 1000

010 1001

10 1000 0000 000

U

000 0000 100 1000

100 1000

x = 4
q = y

B4

x = x - 3

B3

100 1001

000 1000 000 1000 0000 000

000 0000 000 1000 000 1000

z = 2p

B5

000 0000 000 0000 000 0000

Exit

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

Entry

000 0000 001 1101

x = p + 1
y = q + z

B1

000 0000 100 1001

m = k
y = m - 1

B2

110 1001
110 1000

010 1001

10 1000 0000 000

U

000 0000 100 1000

100 1000

x = 4
q = y

B4

x = x - 3

B3

100 1001

000 1000 000 1000 0000 000

000 0000 000 1000 000 1000

z = 2p

B5

000 0000 000 0000 000 0000

Exit

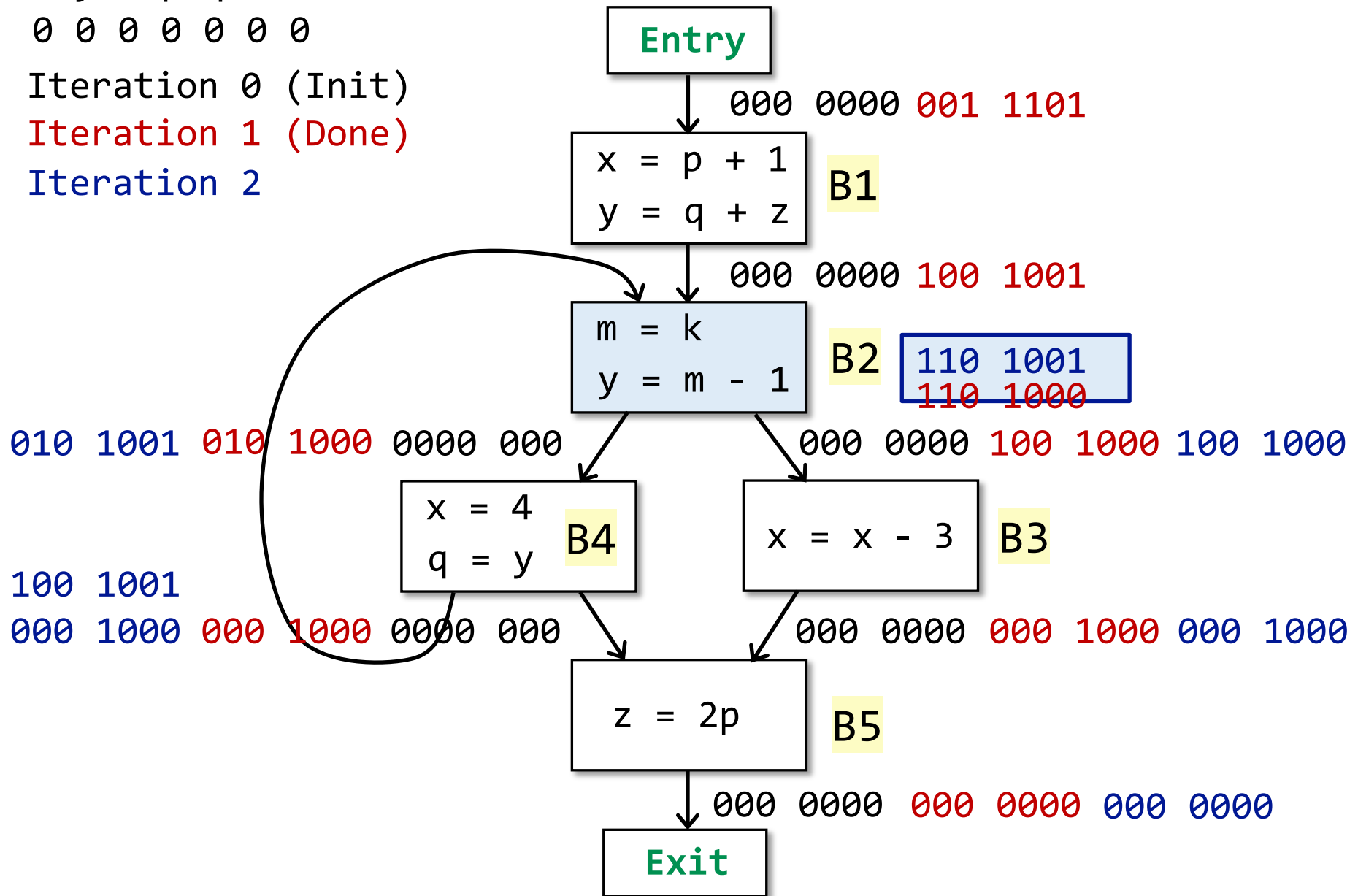
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



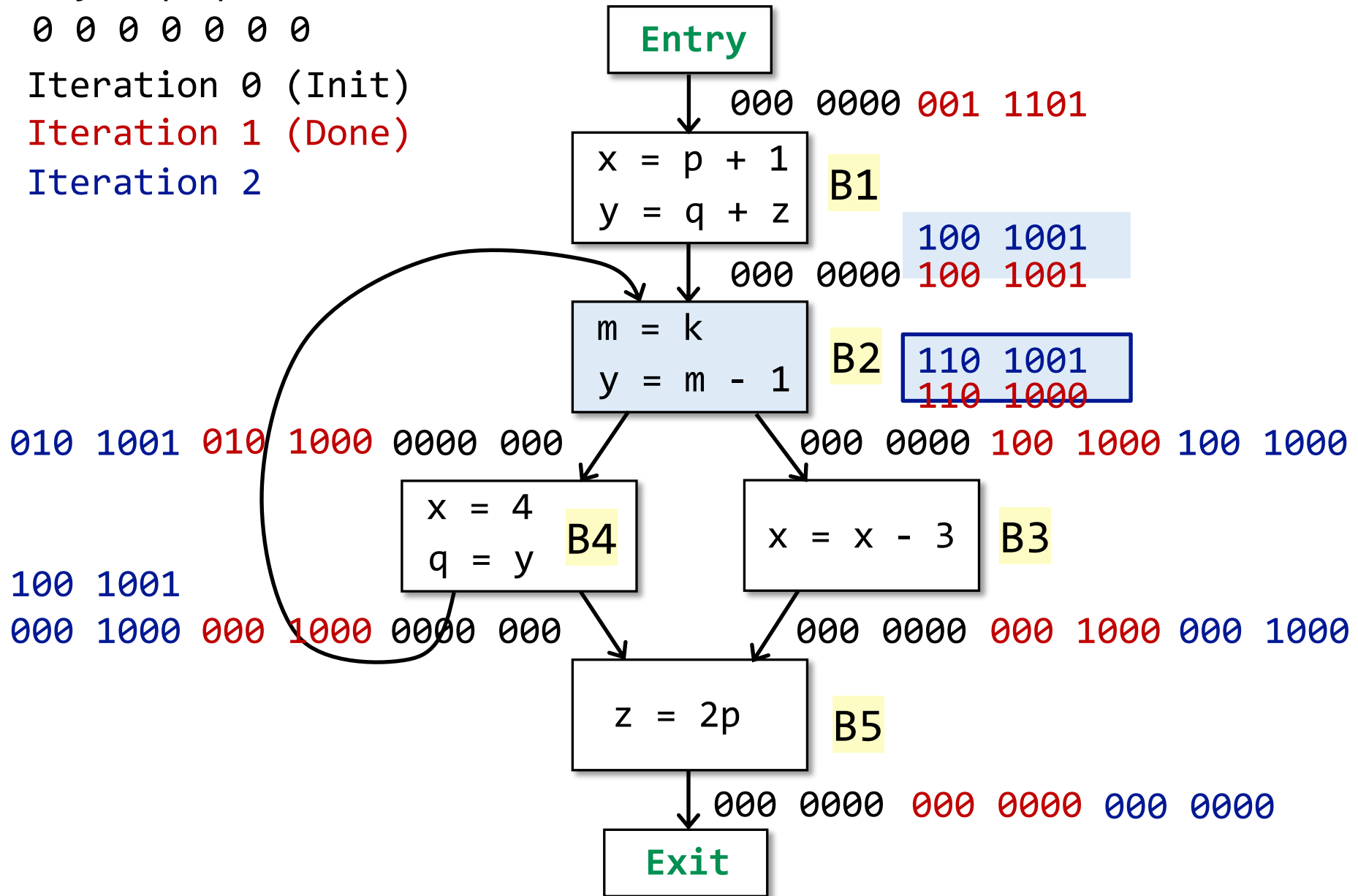
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



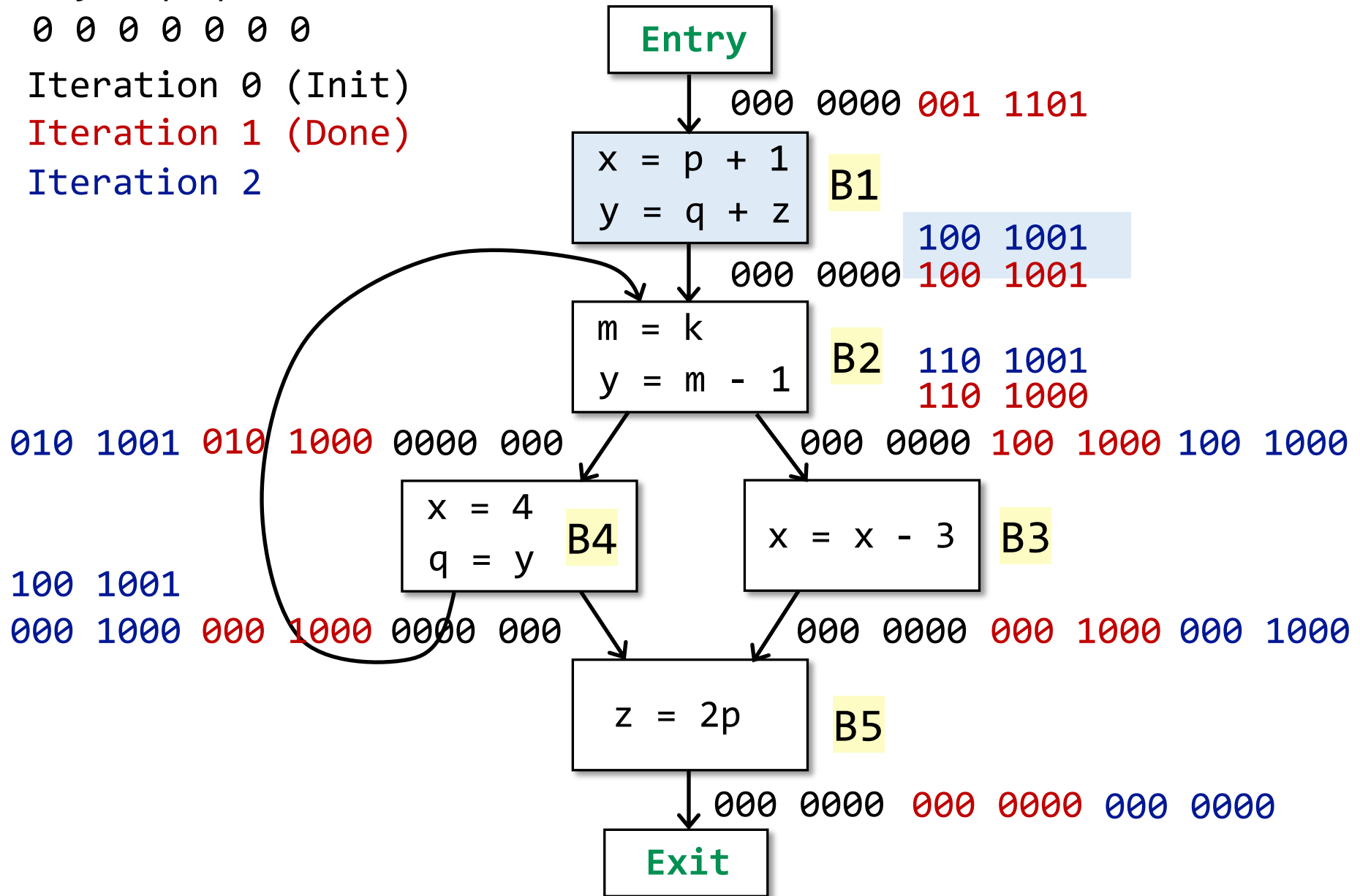
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



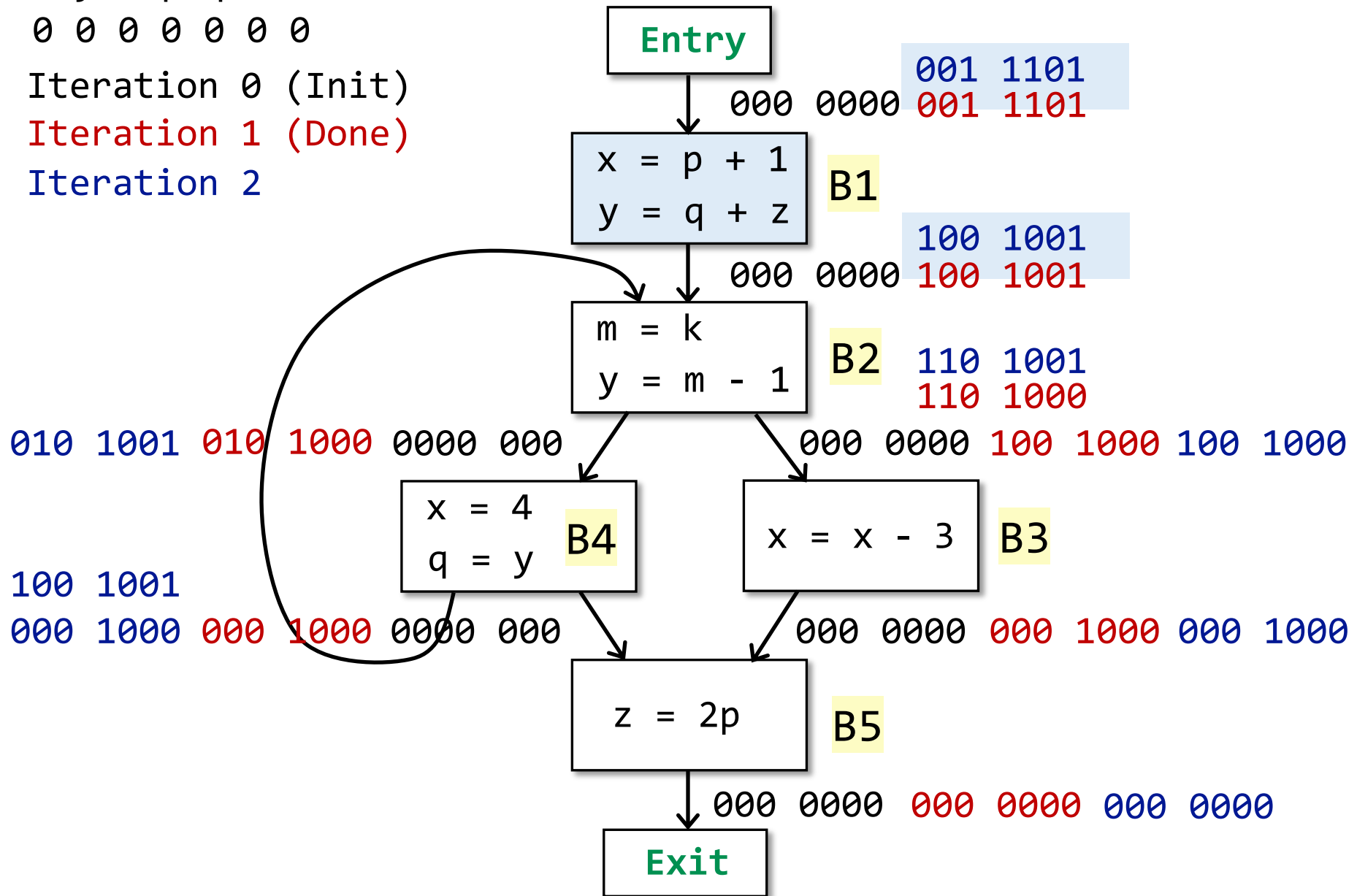
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



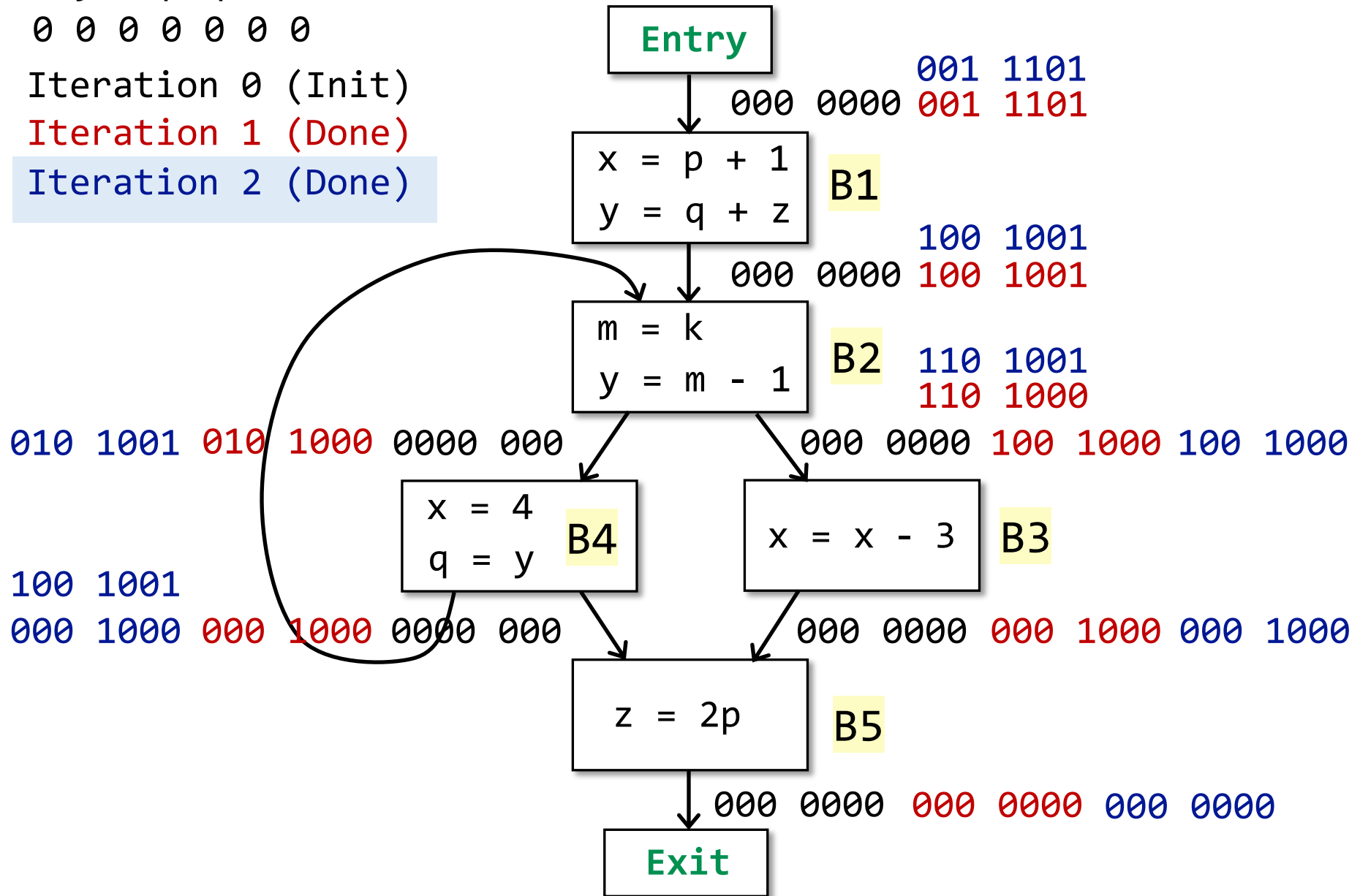
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)



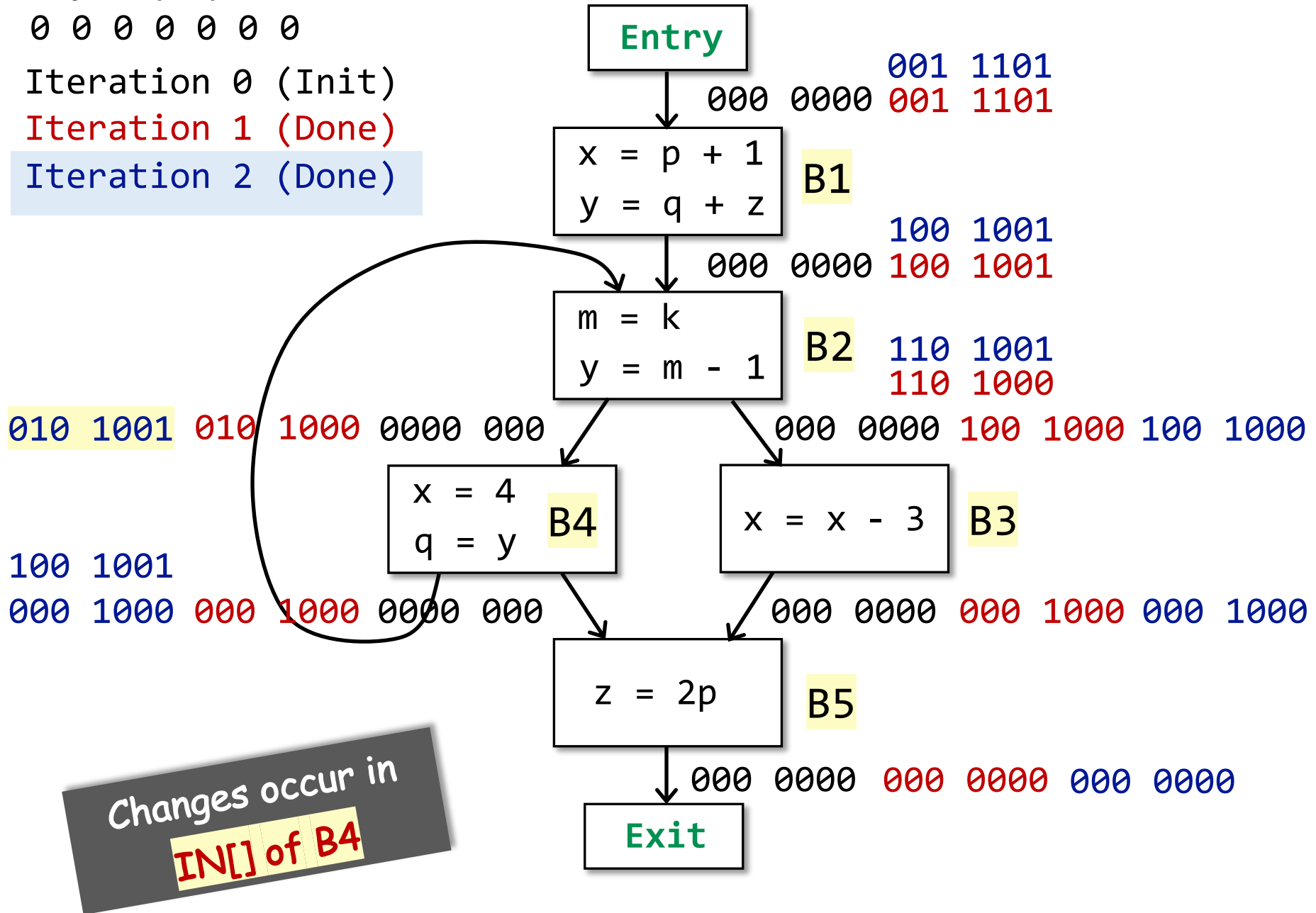
x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)



x y z p q m k

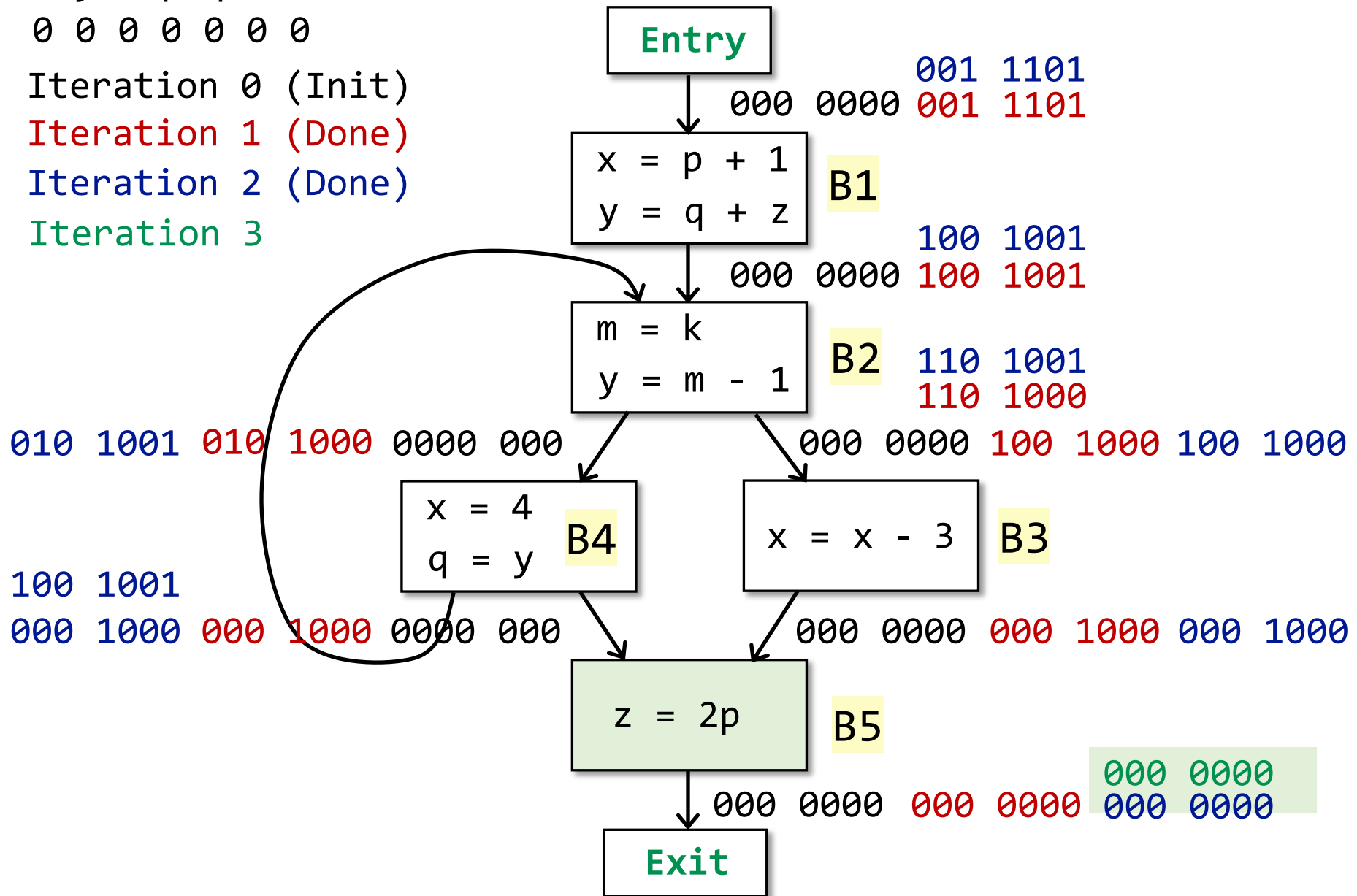
0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



x y z p q m k

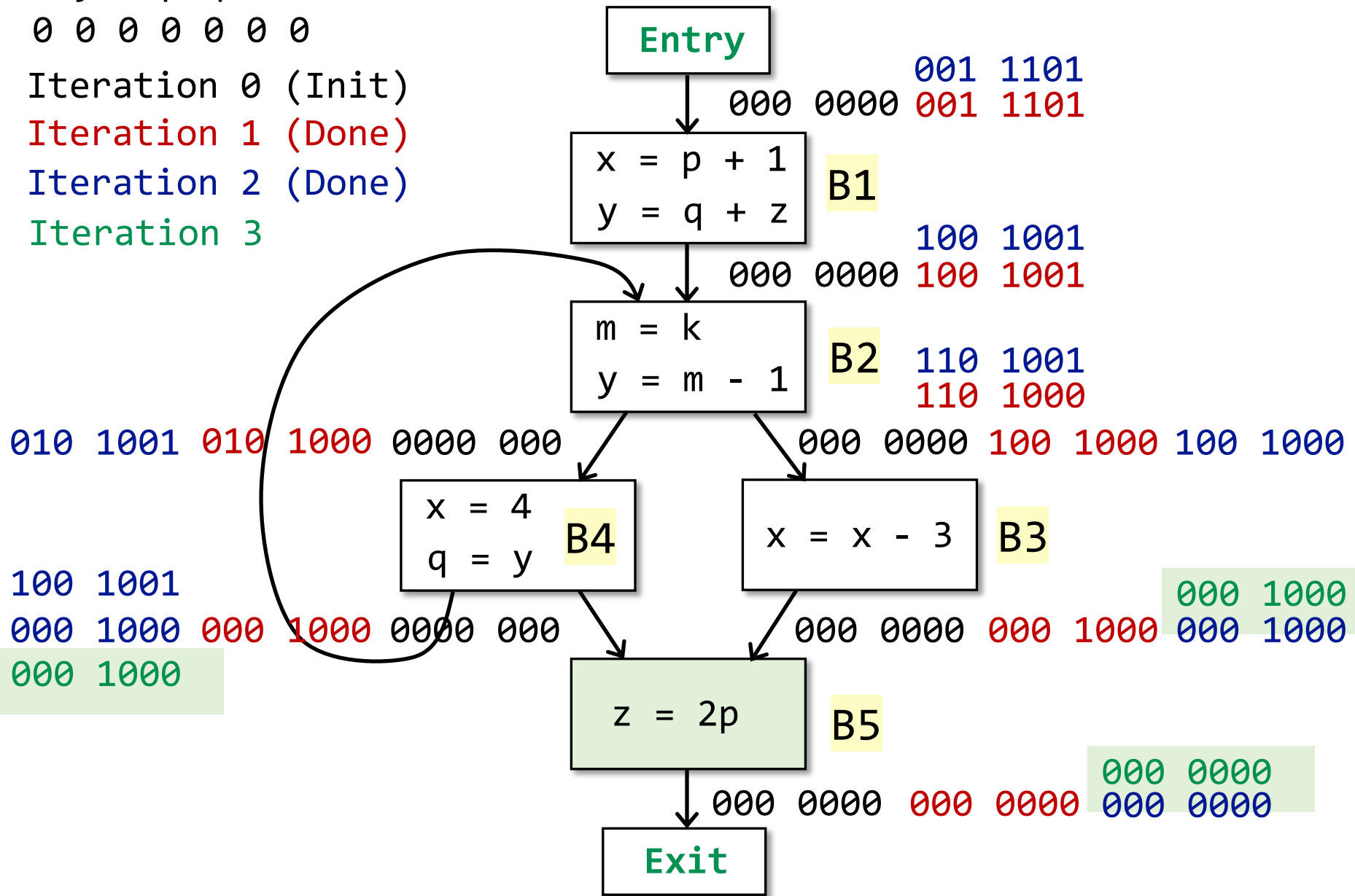
0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



x y z p q m k

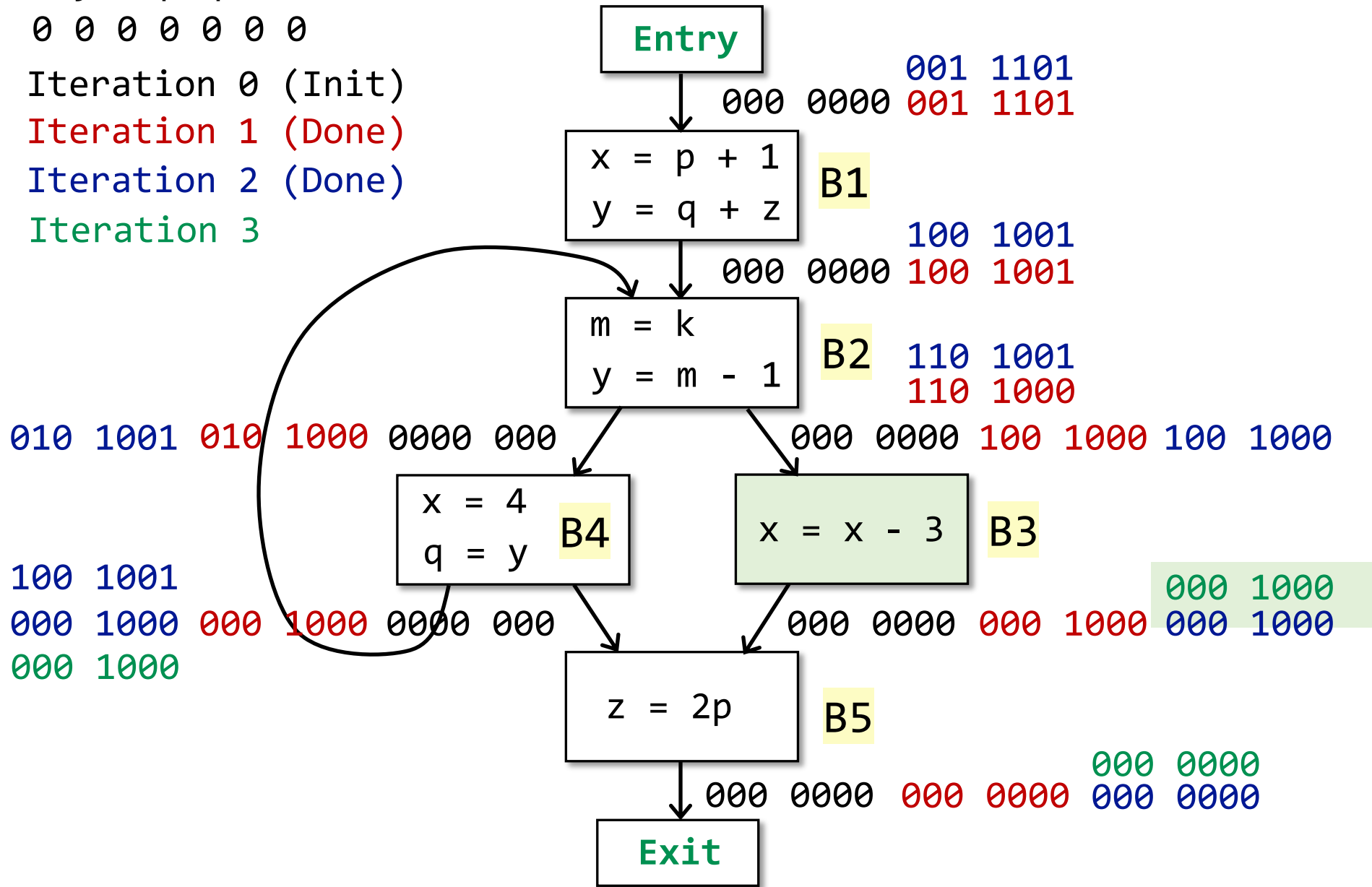
0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



x y z p q m k

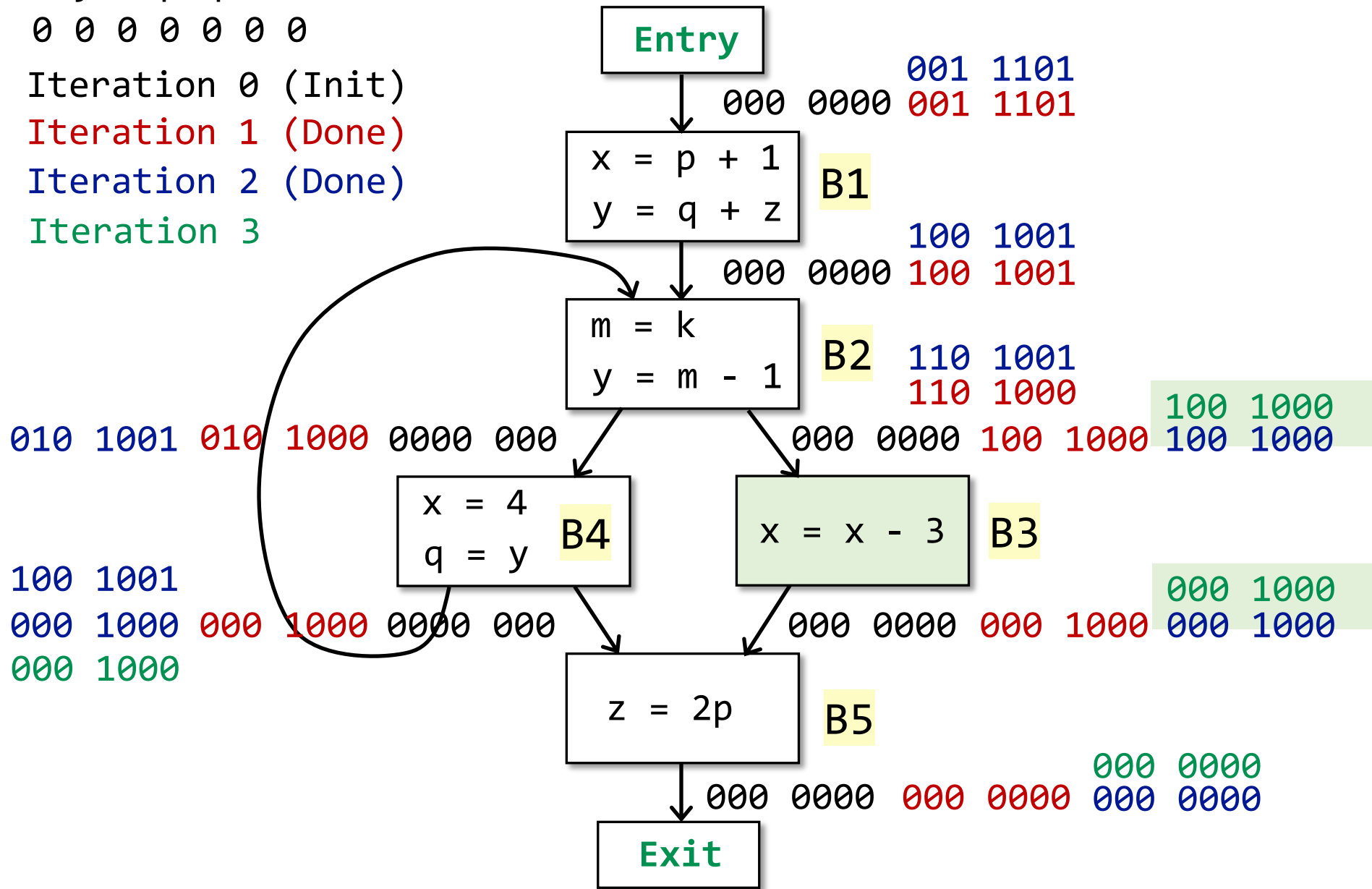
0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



x y z p q m k

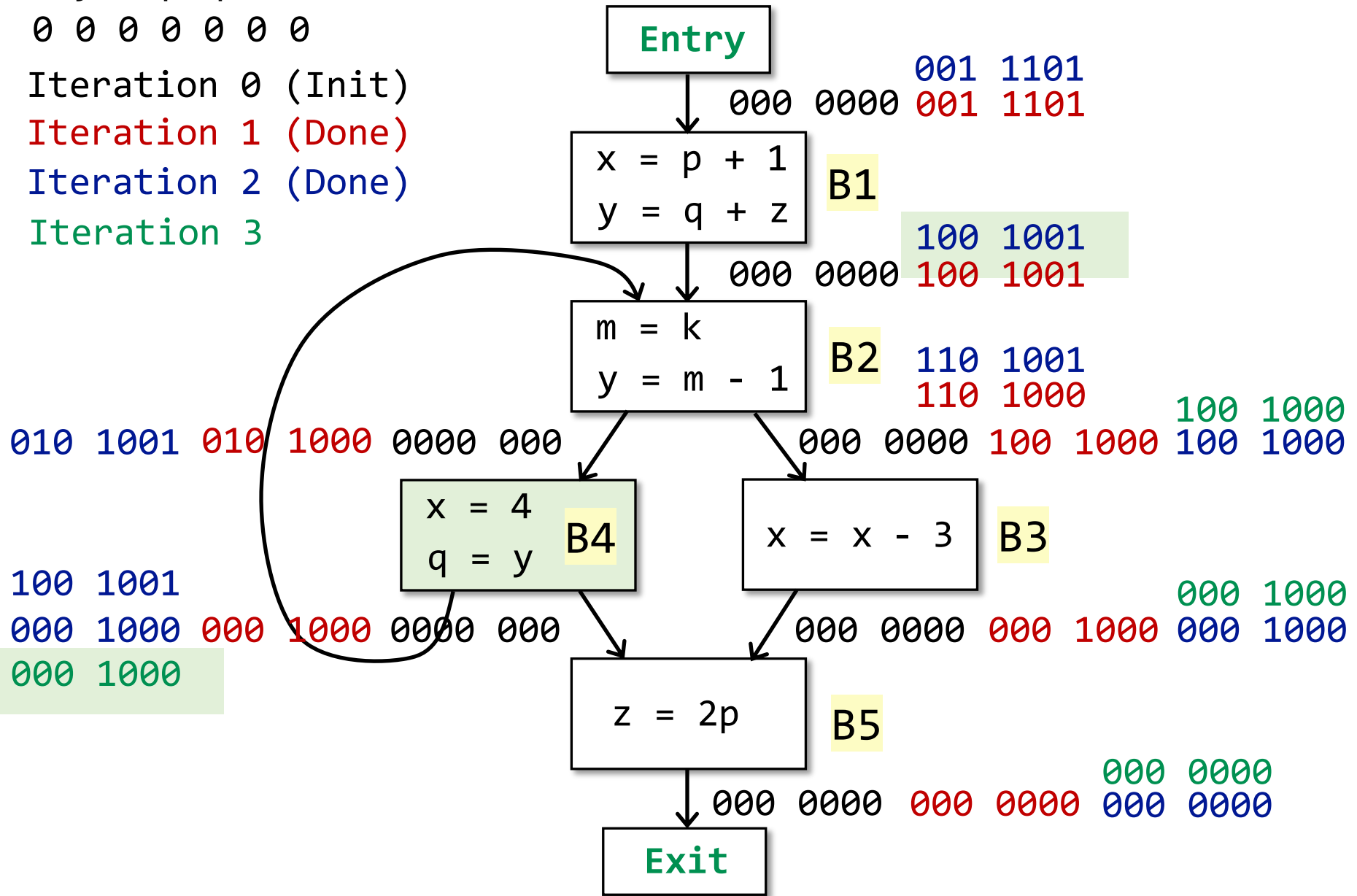
0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



x y z p q m k

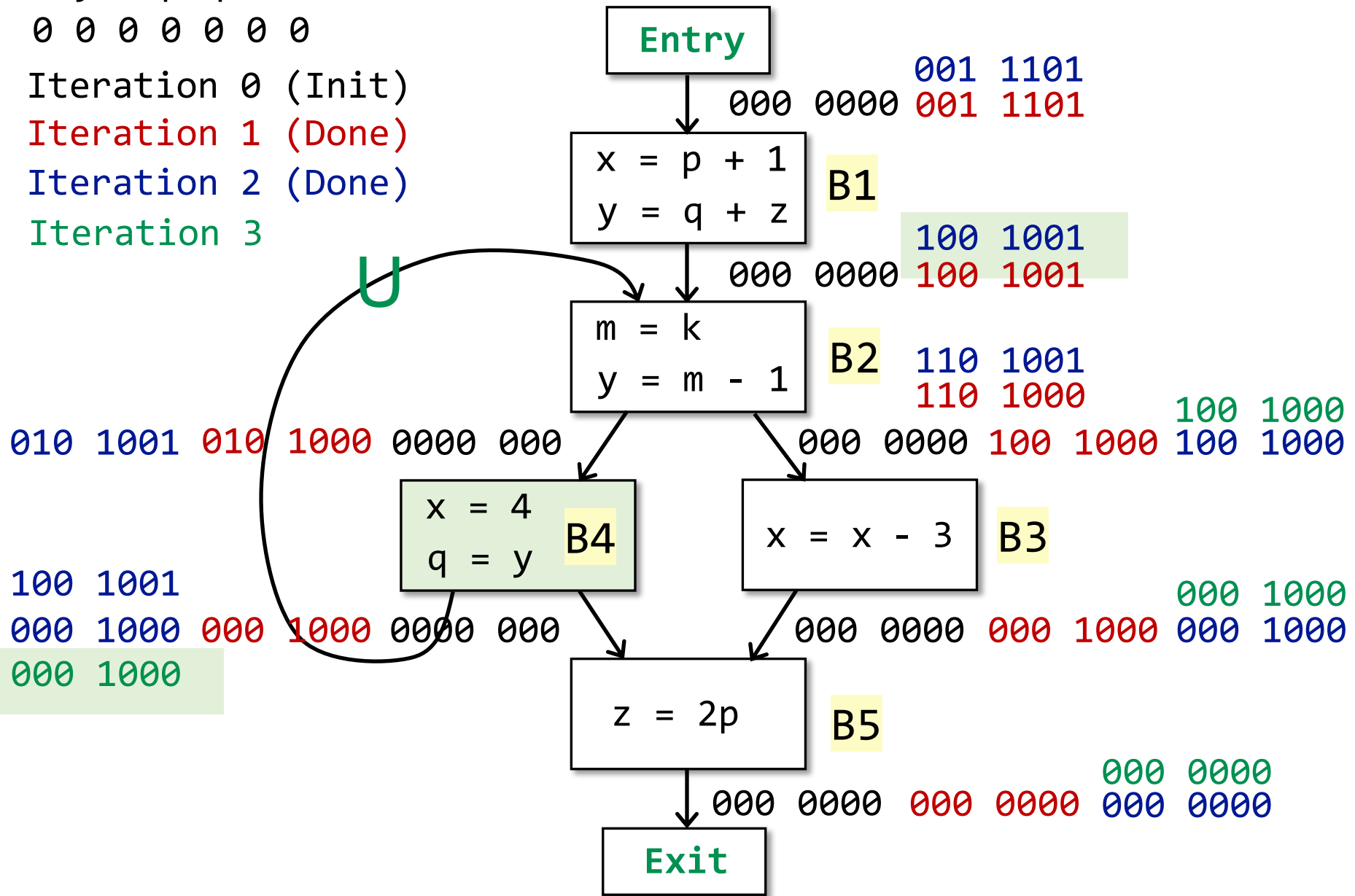
0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

000 0000 001 1101
001 1101

x = p + 1
y = q + z

B1

000 0000 100 1001
100 1001

m = k
y = m - 1

B2

110 1001
110 1000

100 1000
100 1000

010 1001 010 1000 0000 000

000 0000 100 1000

x = 4
q = y

B4

x = x - 3

B3

000 1000
000 1000

100 1001

100 1001

000 1000 000 1000 0000 000

000 0000 000 1000

000 1000

z = 2p

B5

000 0000
000 0000

Exit

U

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

x = p + 1
y = q + z

B1

m = k
y = m - 1

B2

x = 4
q = y

B4

x = x - 3

B3

z = 2p

B5

Exit

000 0000 001 1101
001 1101

100 1001
100 1001

110 1001
110 1000

000 0000 100 1000 100 1000
100 1000

000 0000 000 1000 000 1000
000 1000

000 0000 000 0000 000 0000
000 0000

010 1001 010 1000 0000 000

100 1001

100 1001

000 1000 000 1000 0000 000

000 1000

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

x = p + 1
y = q + z

B1

m = k
y = m - 1

B2

x = 4
q = y

B4

x = x - 3

B3

z = 2p

B5

Exit

000 0000 001 1101
001 1101

100 1001
100 1001

110 1001
110 1000

100 1000
100 1000

000 1000
000 1000

000 0000
000 0000

010 1001
010 1001

100 1001

100 1001
000 1000
000 1000

010 1000 0000 000

000 1000 0000 000

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

x = p + 1
y = q + z

B1

m = k
y = m - 1

B2

x = 4
q = y

B4

x = x - 3

B3

z = 2p

B5

Exit

000 0000 001 1101
001 1101

100 1001
100 1001

110 1001
110 1000

000 0000 100 1000

000 0000 000 1000

000 0000 000 0000 000 0000

010 1001
010 1001

100 1001
100 1001
000 1000
000 1000

100 1000
100 1000

000 1000
000 1000

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

x = p + 1
y = q + z

B1

m = k
y = m - 1

B2

x = 4
q = y

B4

x = x - 3

B3

z = 2p

B5

Exit

U

001 1101
001 1101

100 1001
100 1001

110 1001
110 1000

000 0000 100 1000

100 1000
100 1000

000 0000

000 0000

000 1000

000 1000
000 1000

000 0000

000 0000

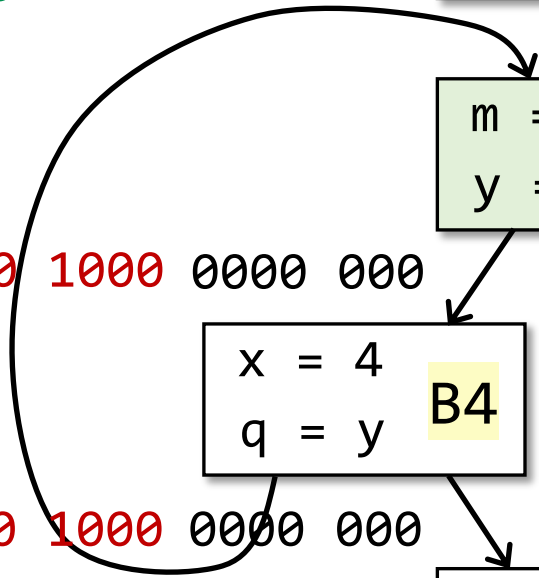
000 0000
000 0000

010 1001
010 1001

100 1001
100 1001
000 1000
000 1000

010 1000 0000 000

000 1000 0000 000



x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

Entry

x = p + 1
y = q + z

B1

m = k
y = m - 1

B2

x = 4
q = y

B4

x = x - 3

B3

z = 2p

B5

Exit

U

001 1101
001 1101

100 1001
100 1001

110 1001
110 1001
110 1000

100 1000
100 1000

000 1000
000 1000

000 0000
000 0000

010 1001
010 1001

100 1001
100 1001
000 1000
000 1000

010 1000 0000 000

000 1000 0000 000

000 0000

000 0000

000 0000

000 0000

000 0000

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

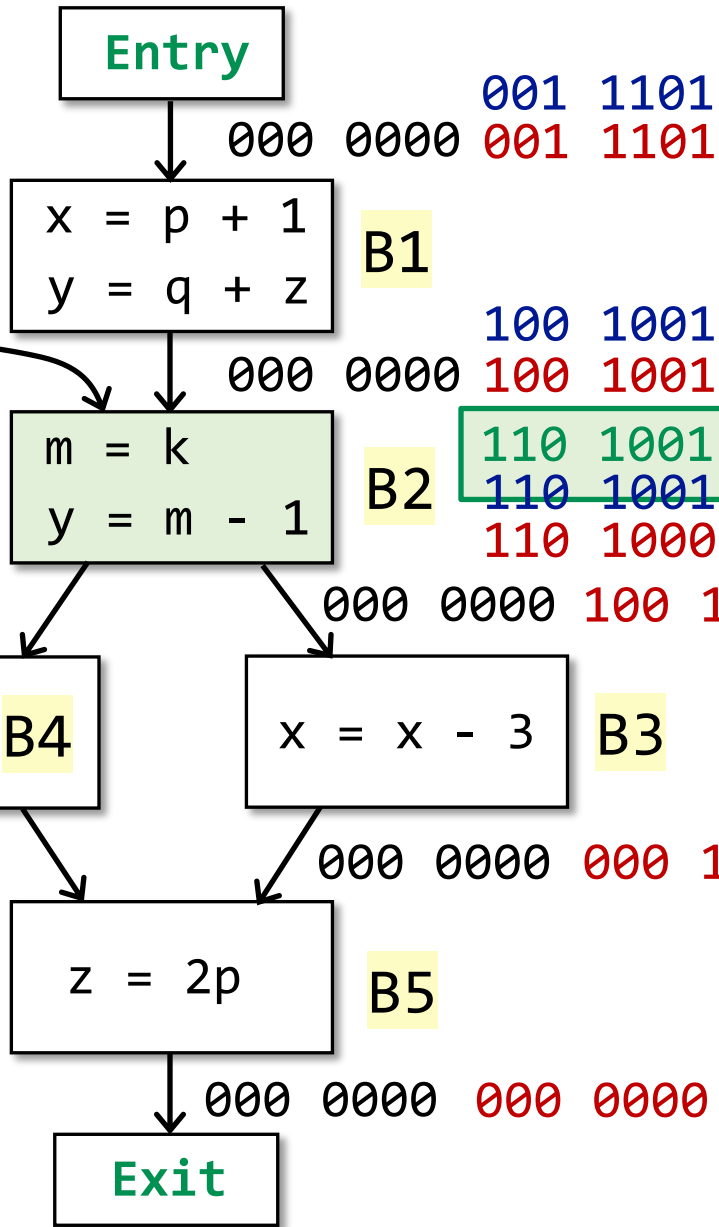
010 1001
010 1001

100 1001
100 1001
000 1000
000 1000

010 1000 0000 000

x = 4
q = y

000 1000 0000 000



000 0000 001 1101
001 1101

100 1001
100 1001

110 1001
110 1001
110 1000

000 0000 100 1000 100 1000

x = x - 3

000 0000 000 1000 000 1000

z = 2p

000 0000 000 0000 000 0000

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

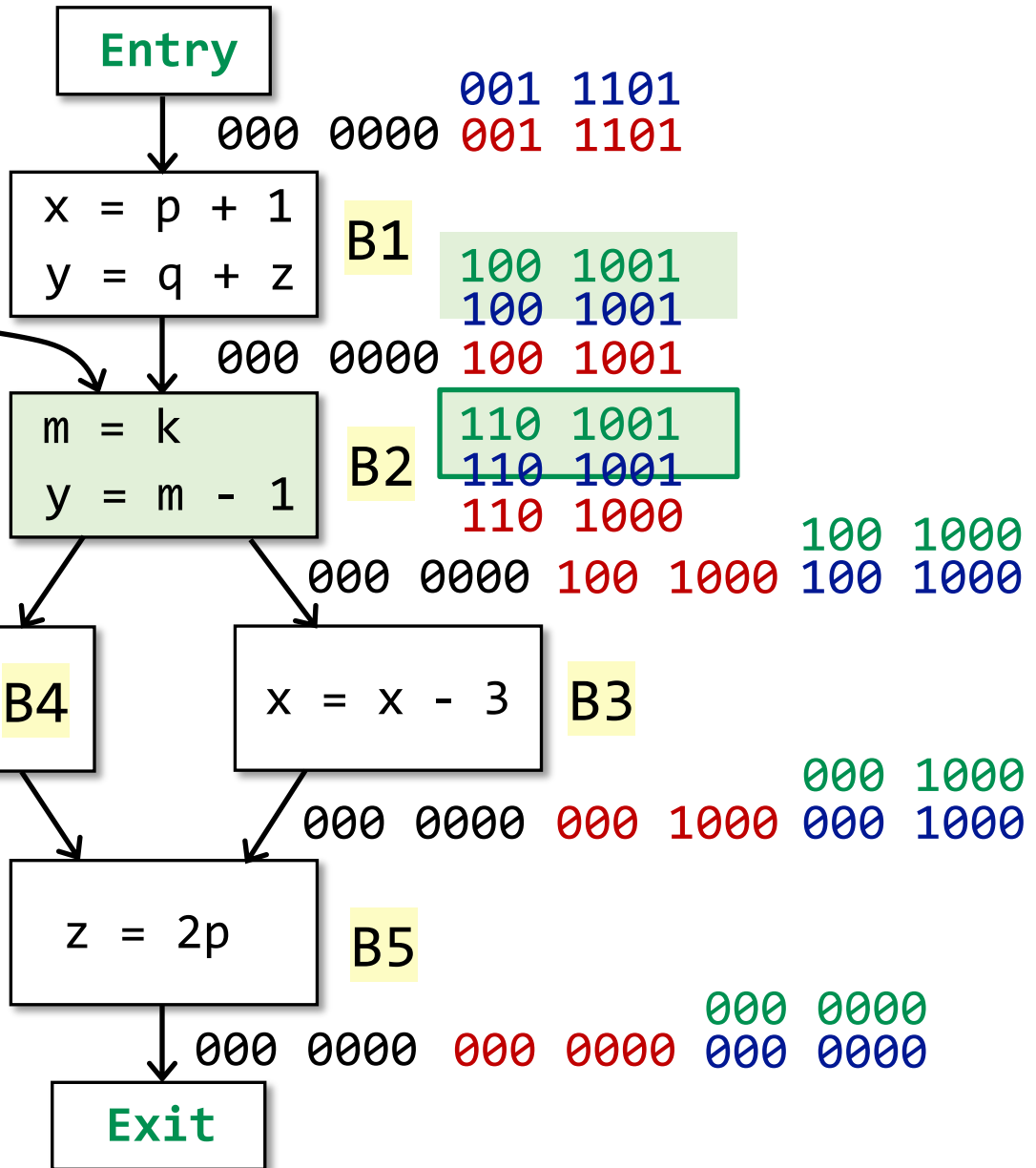
010 1001
010 1001

100 1001
100 1001
000 1000
000 1000

010 1000 0000 000

x = 4
q = y

000 1000 0000 000



x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3

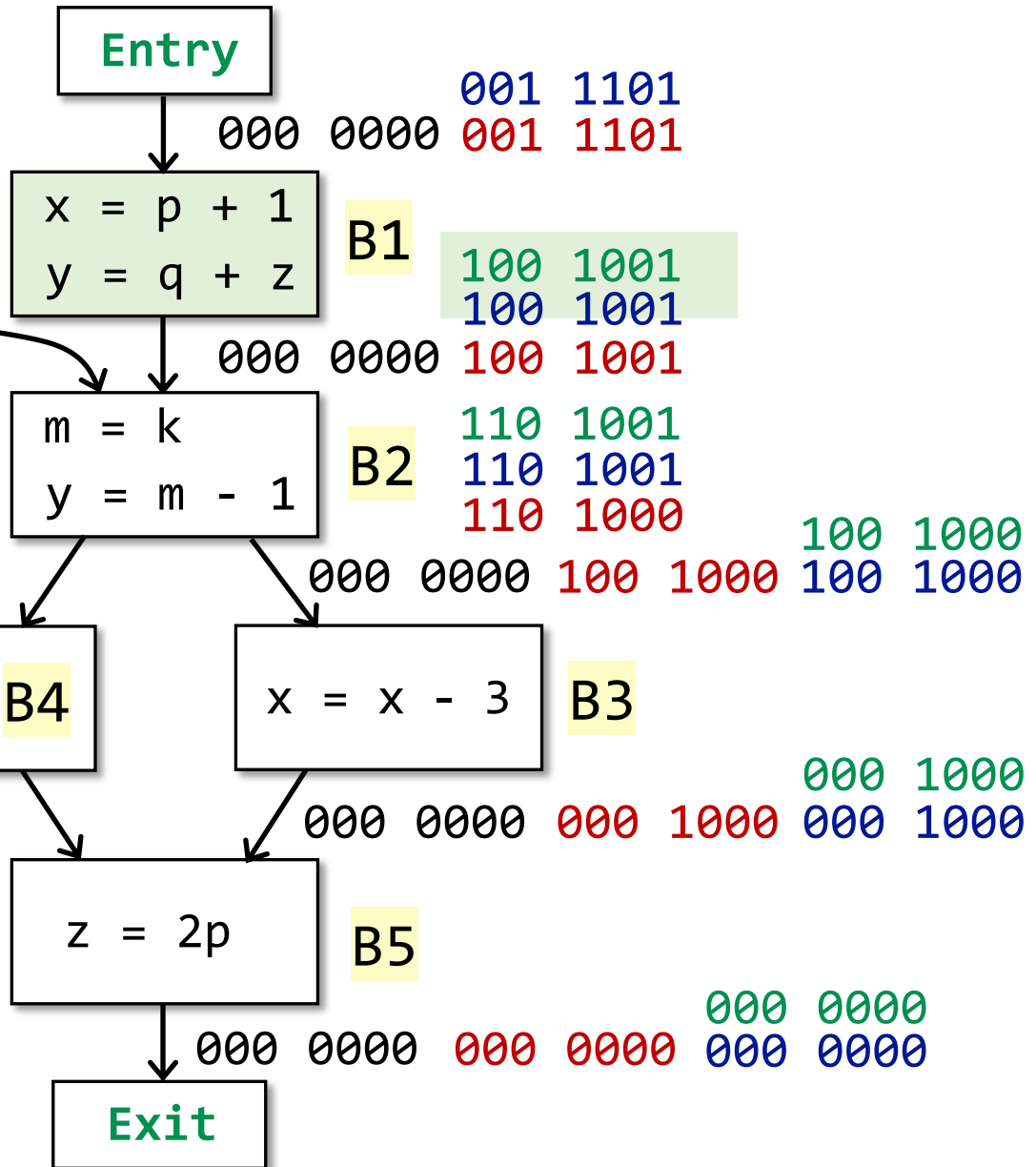
010 1001
010 1001

100 1001
100 1001
000 1000
000 1000

010 1000 0000 000

x = 4
q = y

000 1000 0000 000



x y z p q m k

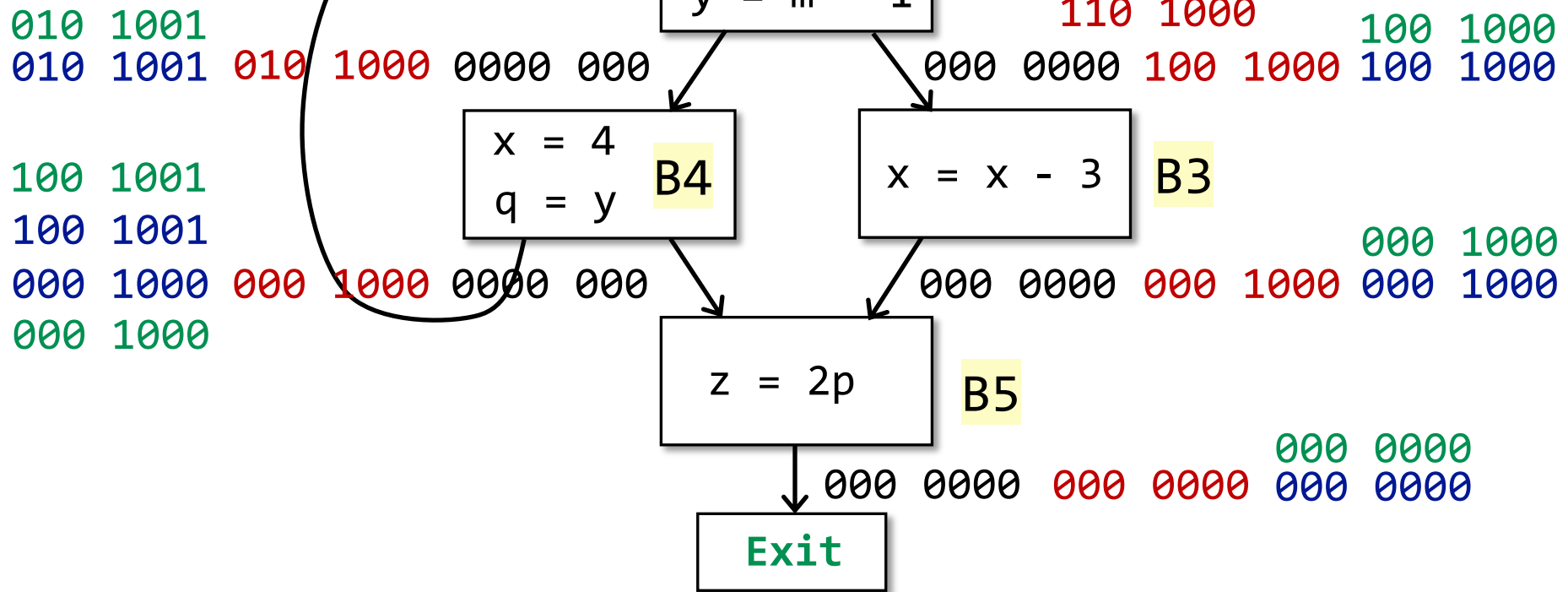
0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3



x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3 (Done)

Entry

000 0000

001 1101
001 1101
001 1101

x = p + 1
y = q + z

B1

000 0000

100 1001
100 1001
100 1001

m = k
y = m - 1

B2

000 0000

110 1001
110 1001
110 1000

x = 4
q = y

B4

x = x - 3

B3

000 0000

100 1000
100 1000
100 1000

z = 2p

B5

000 0000

000 1000
000 1000
000 1000

Exit

000 0000
000 0000

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3 (Done)

Entry

000 0000 001 1101
001 1101
001 1101

x = p + 1
y = q + z

B1

100 1001
100 1001
100 1001

000 0000 100 1001

m = k
y = m - 1

B2

110 1001
110 1001
110 1000

000 0000 100 1000 100 1000
100 1000 100 1000

x = 4
q = y

B4

x = x - 3

B3

000 0000 000 1000 000 1000
000 1000 000 1000

z = 2p

B5

000 0000 000 0000 000 0000
000 0000 000 0000

Exit

No changes occur
in any IN[]

x y z p q m k

0 0 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)

Iteration 3 (Done)

Entry

000 0000

001 1101
001 1101
001 1101

x = p + 1
y = q + z

B1

000 0000

100 1001
100 1001
100 1001

m = k
y = m - 1

B2

000 0000

110 1001
110 1001
110 1000

x = 4
q = y

B4

x = x - 3

B3

000 0000

000 1000

100 1000
100 1000

z = 2p

B5

000 0000

000 0000

000 0000
000 0000

Exit

Final analysis result

Data Flow Analysis Applications

(I) Reaching Definitions Analysis

(II) Live Variables Analysis

(III) Available Expressions Analysis

Available Expressions Analysis

An expression $x \text{ op } y$ is available at program point p if (1) **all** paths from the entry to p **must** pass through the evaluation of $x \text{ op } y$, and (2) after the last evaluation of $x \text{ op } y$, there is no redefinition of x or y

Available Expressions Analysis

An expression $x \text{ op } y$ is available at program point p if (1) **all** paths from the entry to p **must** pass through the evaluation of $x \text{ op } y$, and (2) after the last evaluation of $x \text{ op } y$, there is no redefinition of x or y

- This definition means at program p , we can replace expression $x \text{ op } y$ by the result of its last evaluation
- The information of available expressions can be used for detecting global common subexpressions.

Understanding Available Expressions Analysis

Abstraction

- Data Flow Values/Facts
 - All the expressions in a program
 - Can be represented by bit vectors

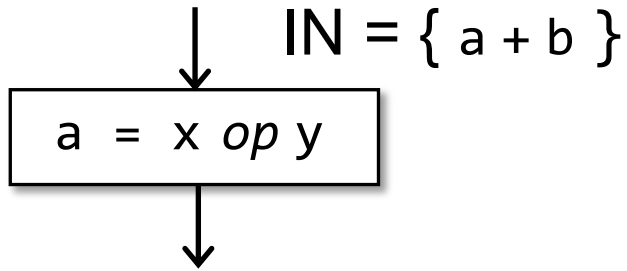
e.g., E1, E2, E3, E4, ..., E100 (100 expressions)

00000...0
└──────────┘
100 bits

Bit i from the left represents expression E_i

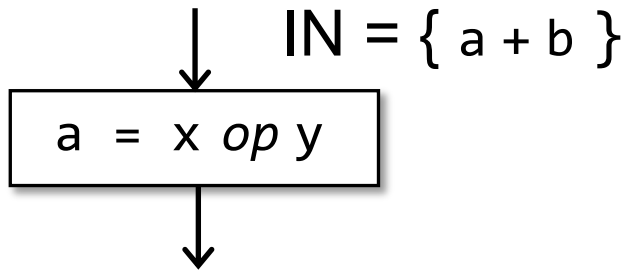
Understanding Available Expressions Analysis

Safe-approximation



Understanding Available Expressions Analysis

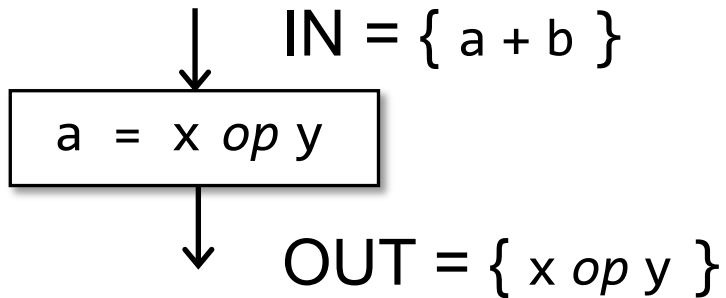
Safe-approximation



- Add to OUT the expression $x \ op \ y$ (gen)
- Delete from IN any expression involving variable a (kill)

Understanding Available Expressions Analysis

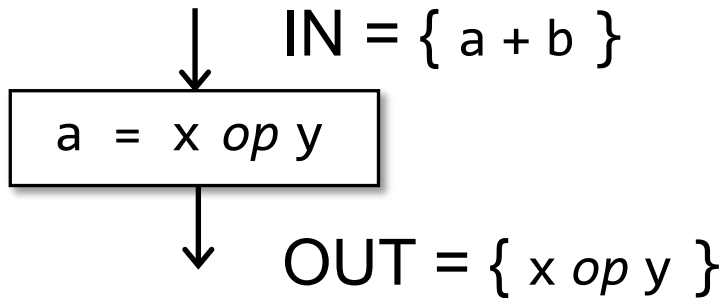
Safe-approximation



- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

Understanding Available Expressions Analysis

Safe-approximation

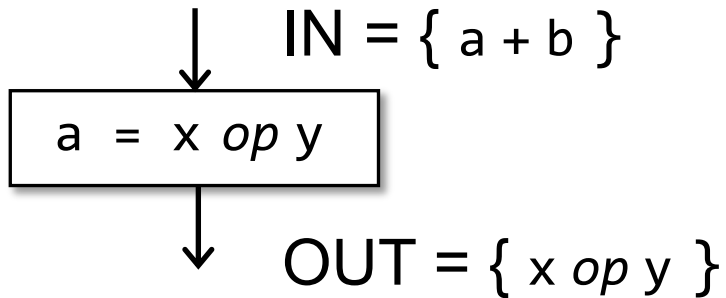


- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = gen_B \cup (IN[B] - kill_B)$$

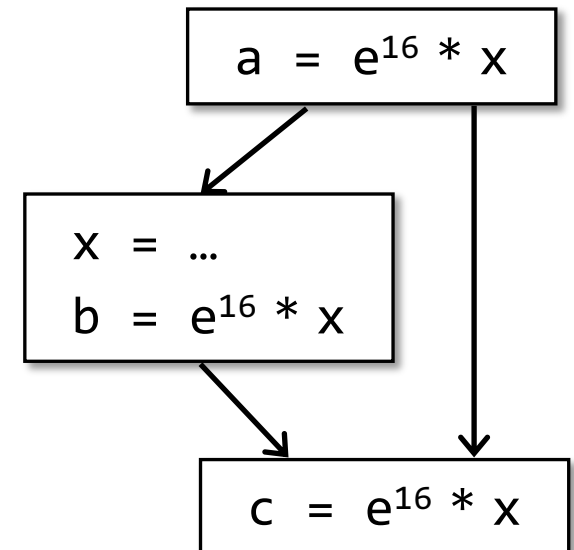
Understanding Available Expressions Analysis

Safe-approximation



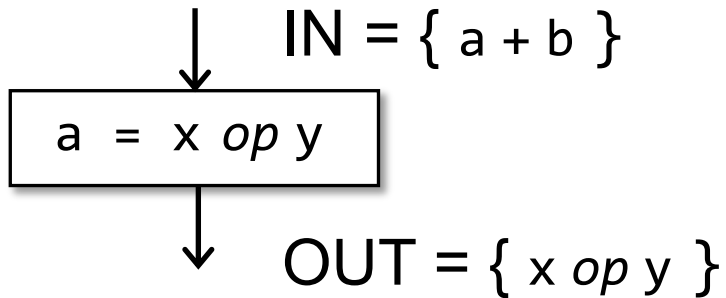
- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$$



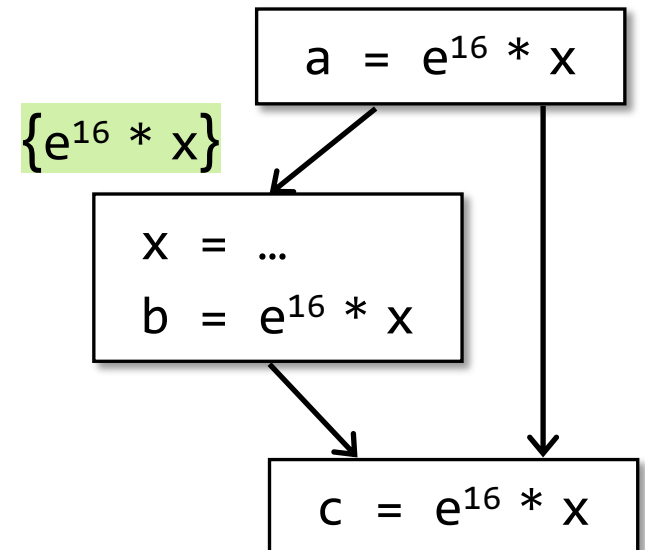
Understanding Available Expressions Analysis

Safe-approximation



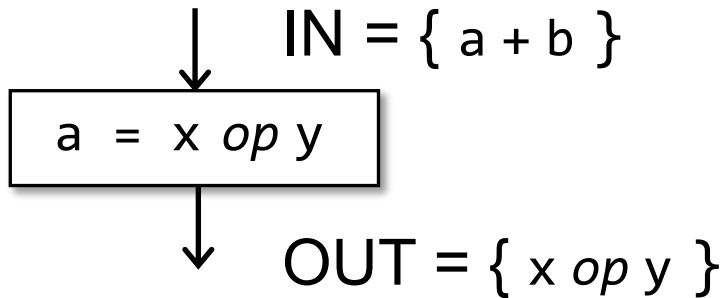
- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$$



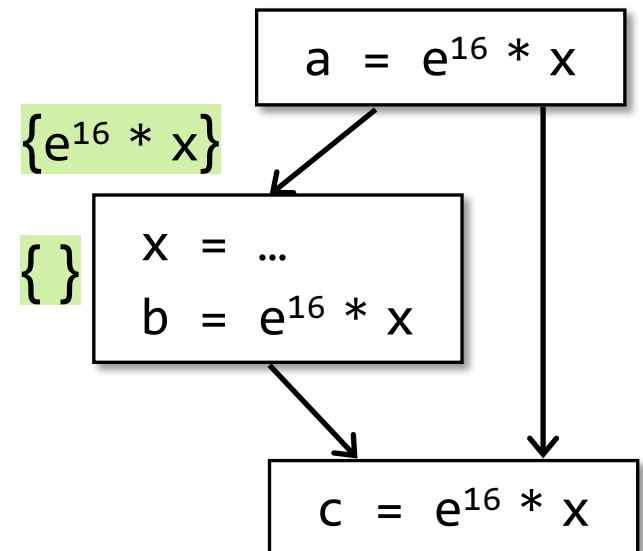
Understanding Available Expressions Analysis

Safe-approximation



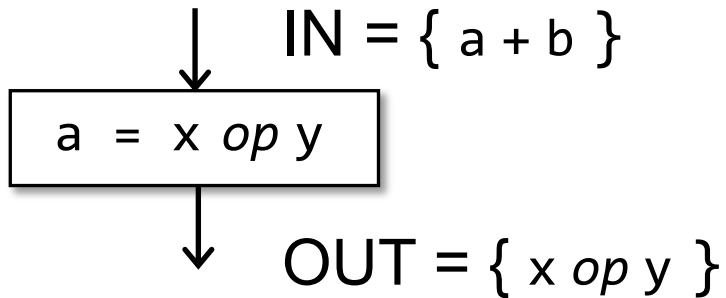
- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$\text{OUT}[B] = \text{gen}_B \cup (\text{IN}[B] - \text{kill}_B)$$



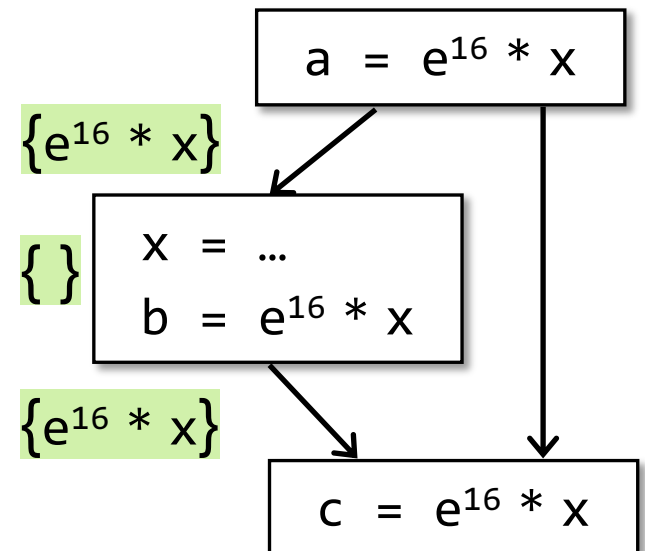
Understanding Available Expressions Analysis

Safe-approximation



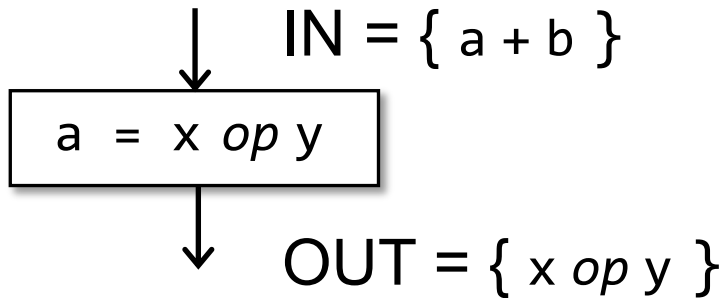
- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = gen_B \cup (IN[B] - kill_B)$$



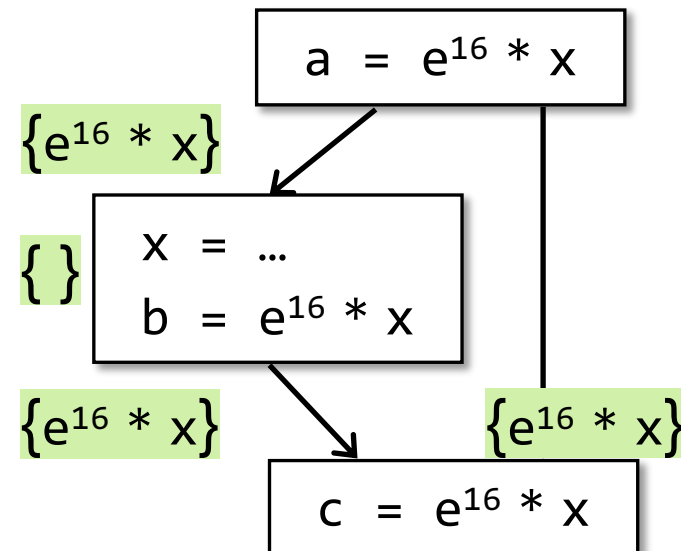
Understanding Available Expressions Analysis

Safe-approximation



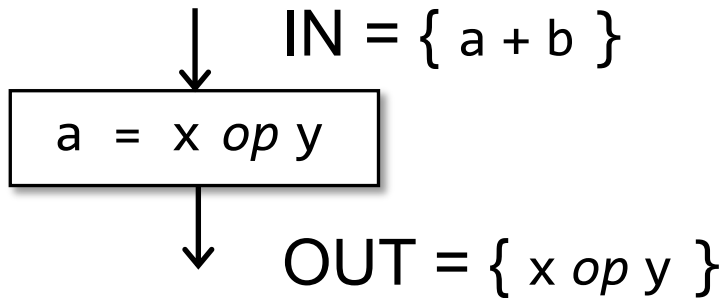
- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$$



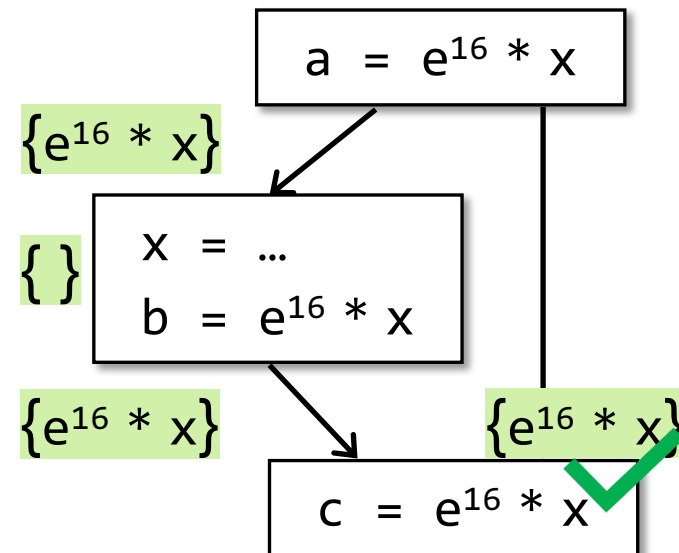
Understanding Available Expressions Analysis

Safe-approximation



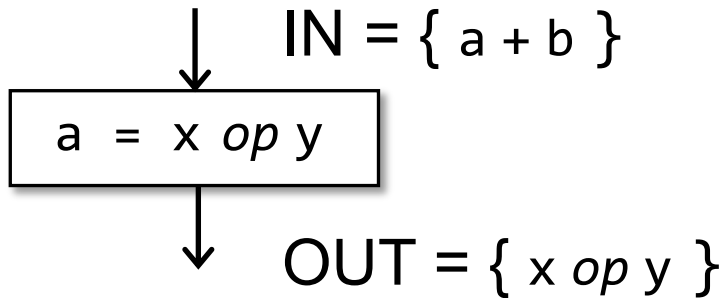
- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = gen_B \cup (IN[B] - kill_B)$$



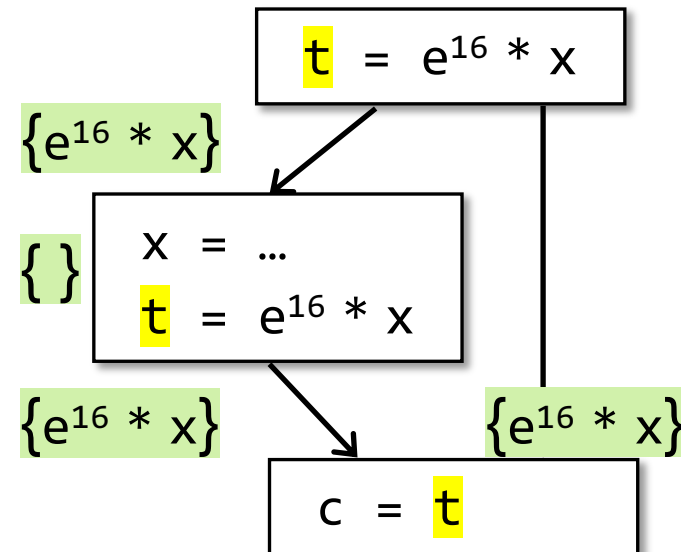
Understanding Available Expressions Analysis

Safe-approximation



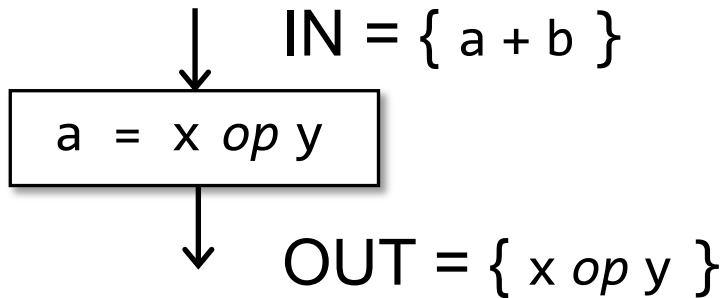
- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = gen_B \cup (IN[B] - kill_B)$$



Understanding Available Expressions Analysis

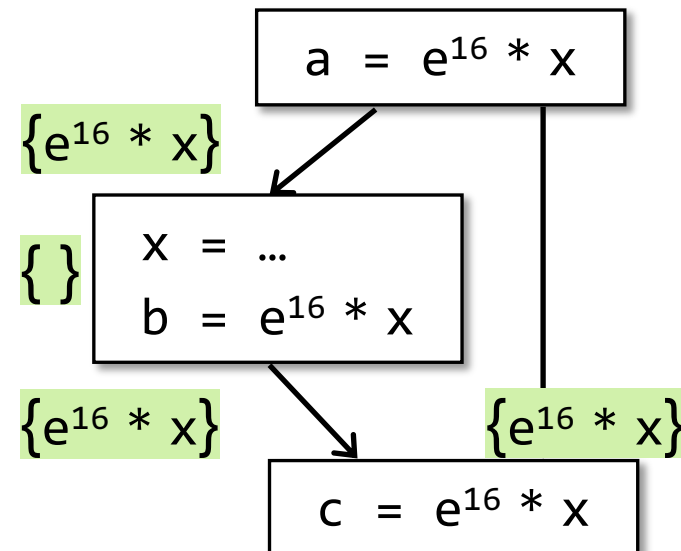
Safe-approximation



- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

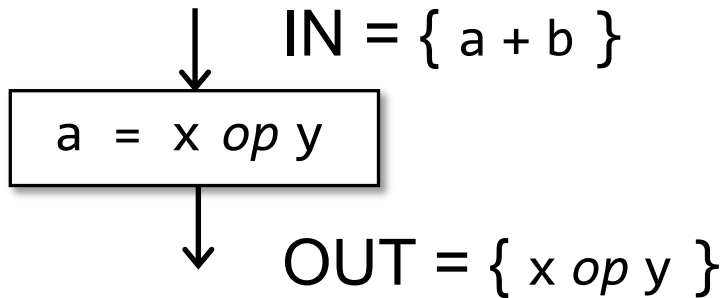
$$\text{OUT}[B] = \text{gen}_B \cup (\text{IN}[B] - \text{kill}_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P]$$



Understanding Available Expressions Analysis

Safe-approximation

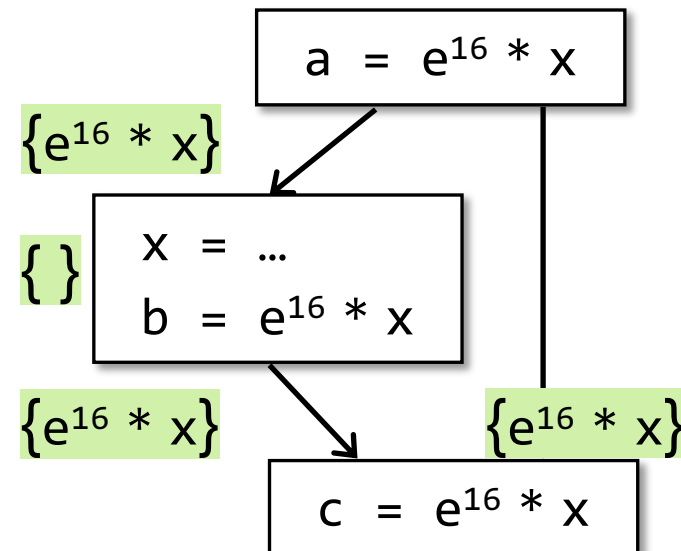


- Add to OUT the expression $x \text{ op } y$ (**gen**)
- Delete from IN any expression involving variable a (**kill**)

$$OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$$

$$IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$$

All paths from entry to point p must pass through the evaluation of $x \text{ op } y$



Understanding Available Expressions Analysis

Safe-approximation

↓ $IN = \{ a + b \}$

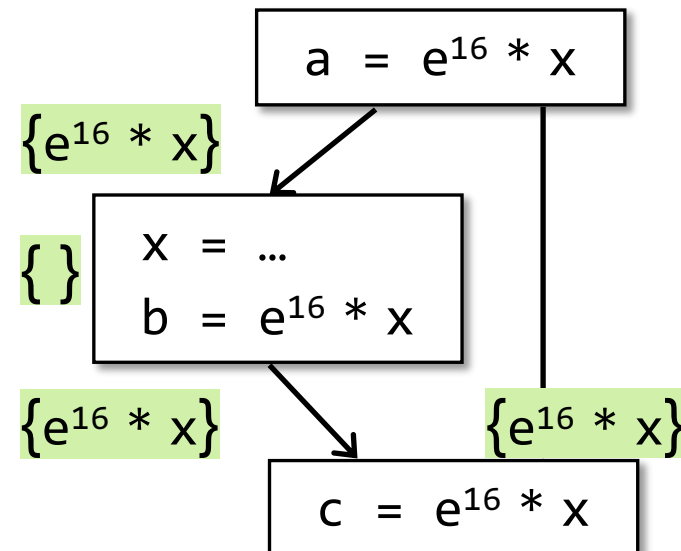
For safety of the analysis, it may report an expression as unavailable even if it is truly available (must analysis $\rightarrow$ under-approximation)

- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$$

$$IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$$

All paths from entry to point p must pass through the evaluation of $x \text{ op } y$



Understanding Available Expressions Analysis

Safe-approximation

↓ $IN = \{ a + b \}$

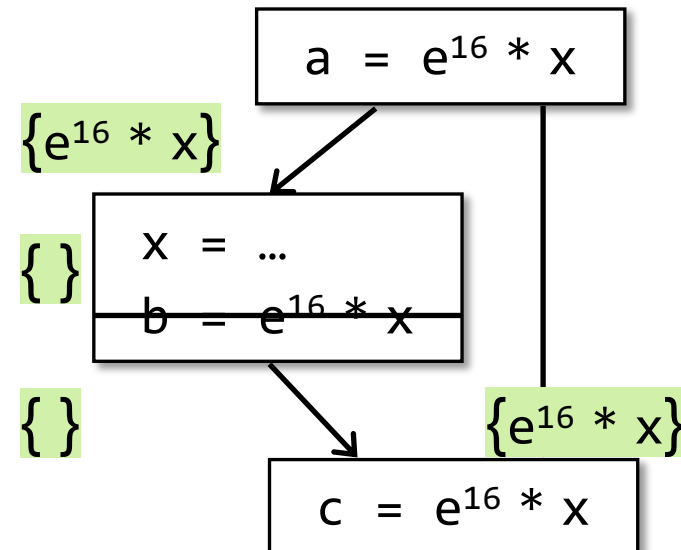
For safety of the analysis, it may report an expression as unavailable even if it is truly available (must analysis $\rightarrow$ under-approximation)

- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$$

$$IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$$

All paths from entry to point p must pass through the evaluation of $x \text{ op } y$



Understanding Available Expressions Analysis

Safe-approximation

↓ $IN = \{ a + b \}$

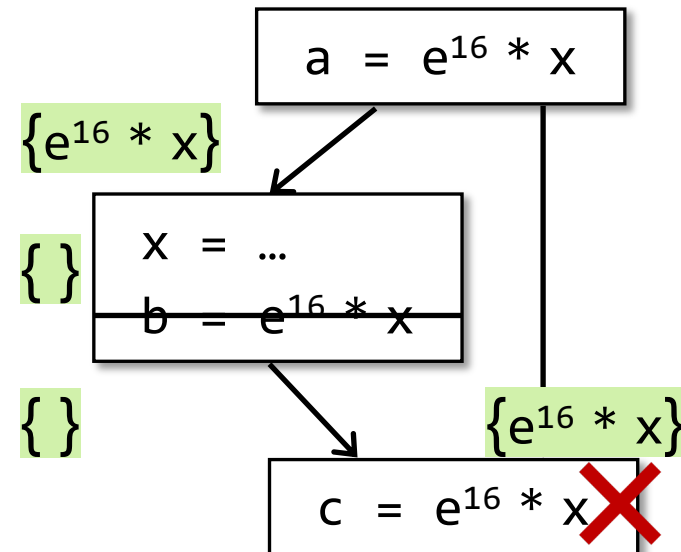
For safety of the analysis, it may report an expression as unavailable even if it is truly available (must analysis $\rightarrow$ under-approximation)

- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$$

$$IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$$

All paths from entry to point p must pass through the evaluation of $x \text{ op } y$



Understanding Available Expressions Analysis

Safe-approximation

↓ $IN = \{ a + b \}$

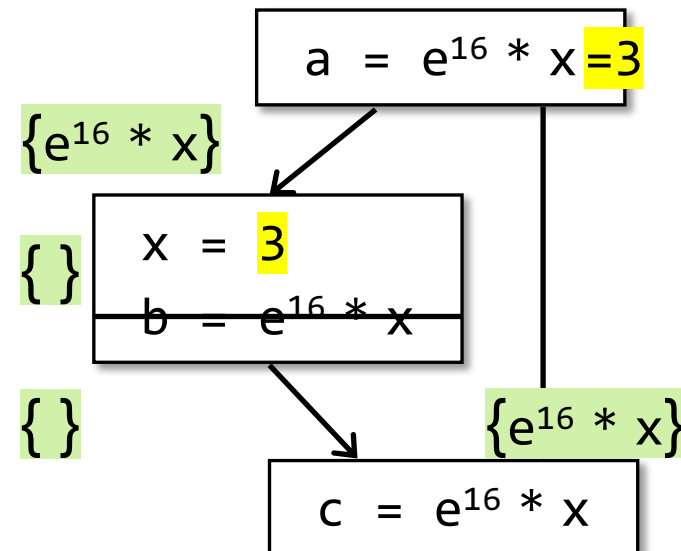
For safety of the analysis, it may report an expression as unavailable even if it is truly available (must analysis $\rightarrow$ under-approximation)

- Add to OUT the expression $x \text{ op } y$ (gen)
- Delete from IN any expression involving variable a (kill)

$$OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$$

$$IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$$

All paths from entry to point p must pass through the evaluation of $x \text{ op } y$



Understanding Available Expressions Analysis

Safe-approximation

↓ $IN = \{ a + b \}$

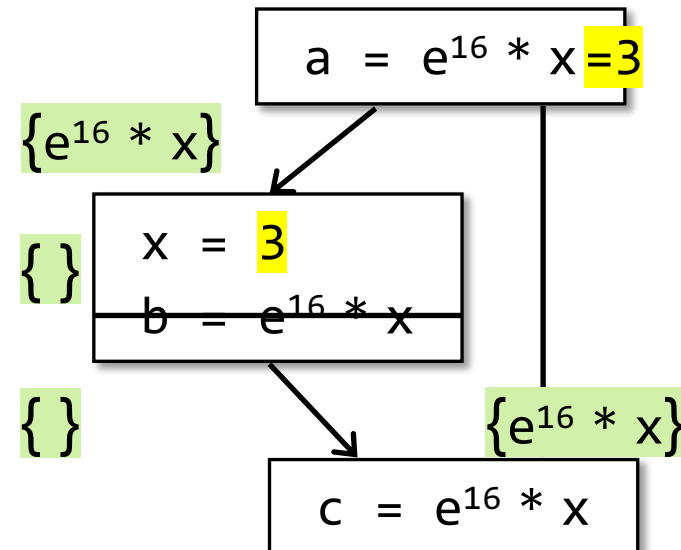
For **safety** of the analysis, it may report an expression as unavailable even if it is truly available (must analysis $\rightarrow$ **under**-approximation)

- Add to OUT the expression $x \text{ op } y$ (**gen**)
- Delete from IN any expression involving variable a (**kill**)

$$OUT[B] = \text{gen}_B \cup (IN[B] - \text{kill}_B)$$

$$IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$$

All paths from entry to point p must pass through the evaluation of $x \text{ op } y$



Algorithm of Available Expressions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ )  
    OUT[B] = U;  
    while (changes to any OUT occur)  
        for (each basic block  $B \setminus entry$ ) {  
             $IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$ ;  
             $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
        }
```

Algorithm of Available Expressions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ )  
    OUT[B] = U;  
while (changes to any OUT occur)  
    for (each basic block  $B \setminus entry$ ) {  
         $IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$ ;  
         $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
    }
```

Algorithm of Available Expressions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ )  
    OUT[B] = U;  
    while (changes to any OUT occur)  
        for (each basic block  $B \setminus entry$ ) {  
             $IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$ ;  
             $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
        }
```

Algorithm of Available Expressions Analysis

INPUT: CFG ($kill_B$ and gen_B computed for each basic block B)

OUTPUT: $IN[B]$ and $OUT[B]$ for each basic block B

METHOD:

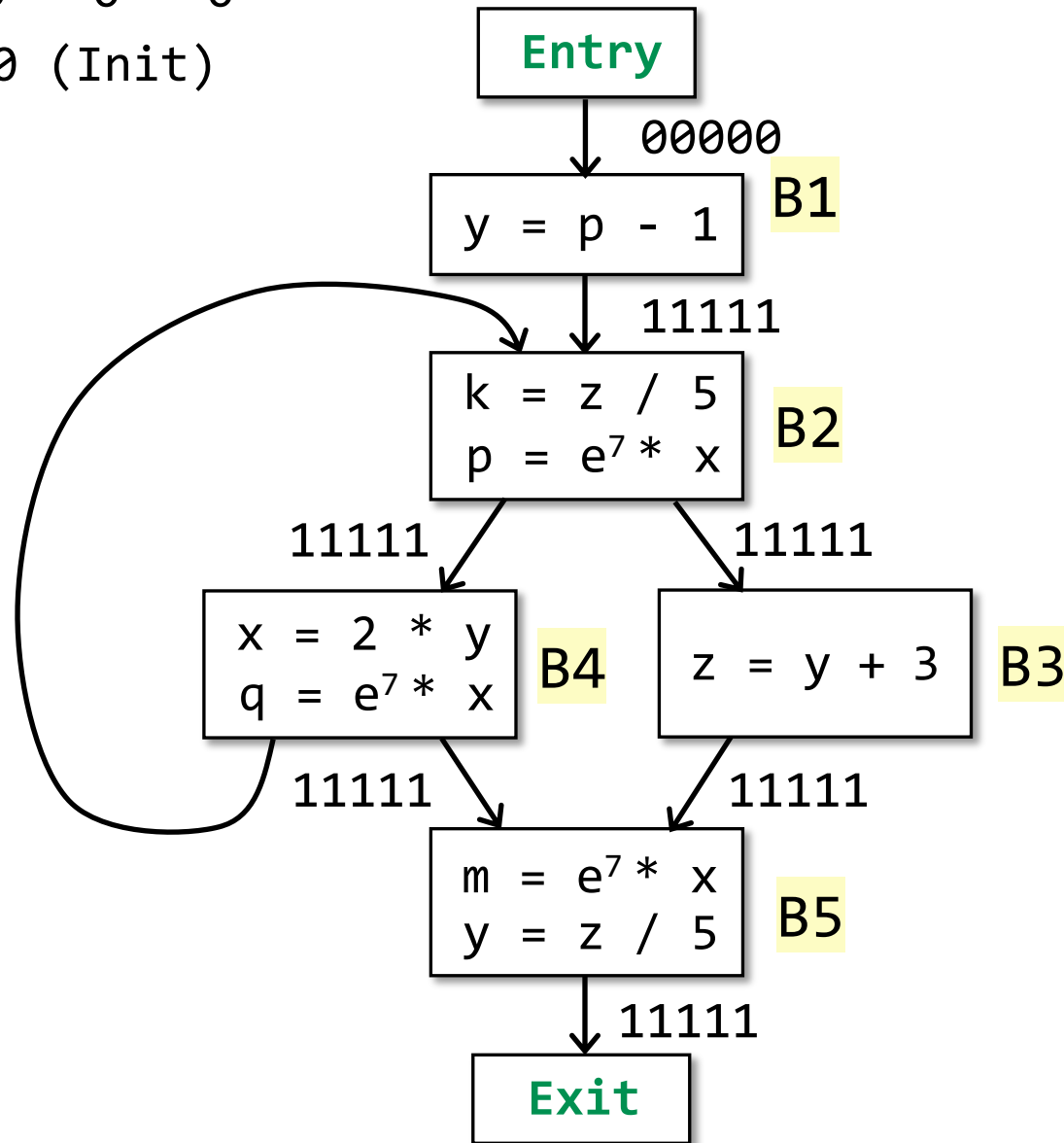
```
OUT[entry] =  $\emptyset$ ;  
for (each basic block  $B \setminus entry$ )  
    OUT[B] = U;  
    while (changes to any OUT occur)  
        for (each basic block  $B \setminus entry$ ) {  
             $IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$ ;  
             $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;  
        }
```



$p-1$ $z/5$ $2*y$ e^7*x $y+3$

0 0 0 0 0

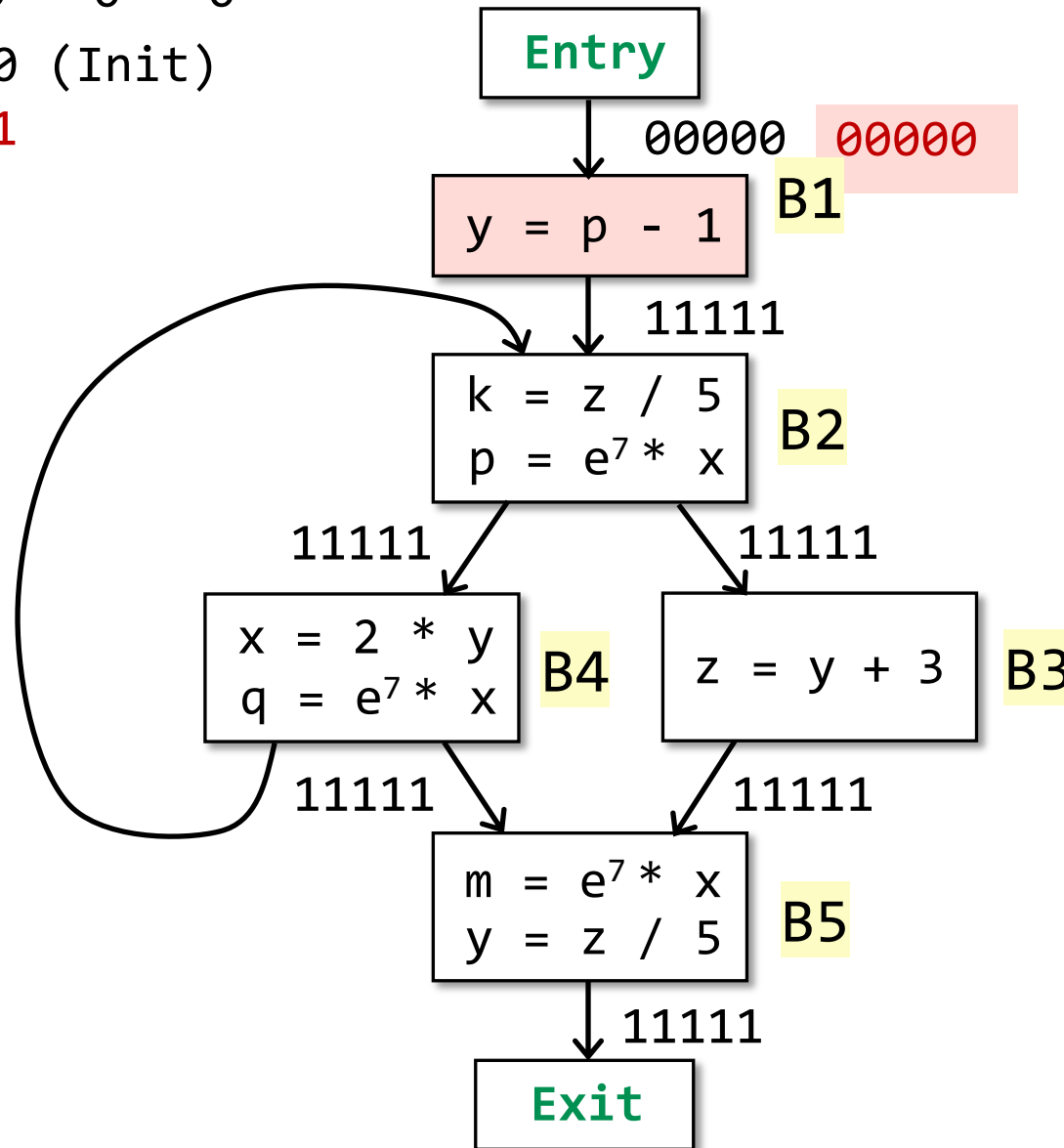
Iteration 0 (Init)



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

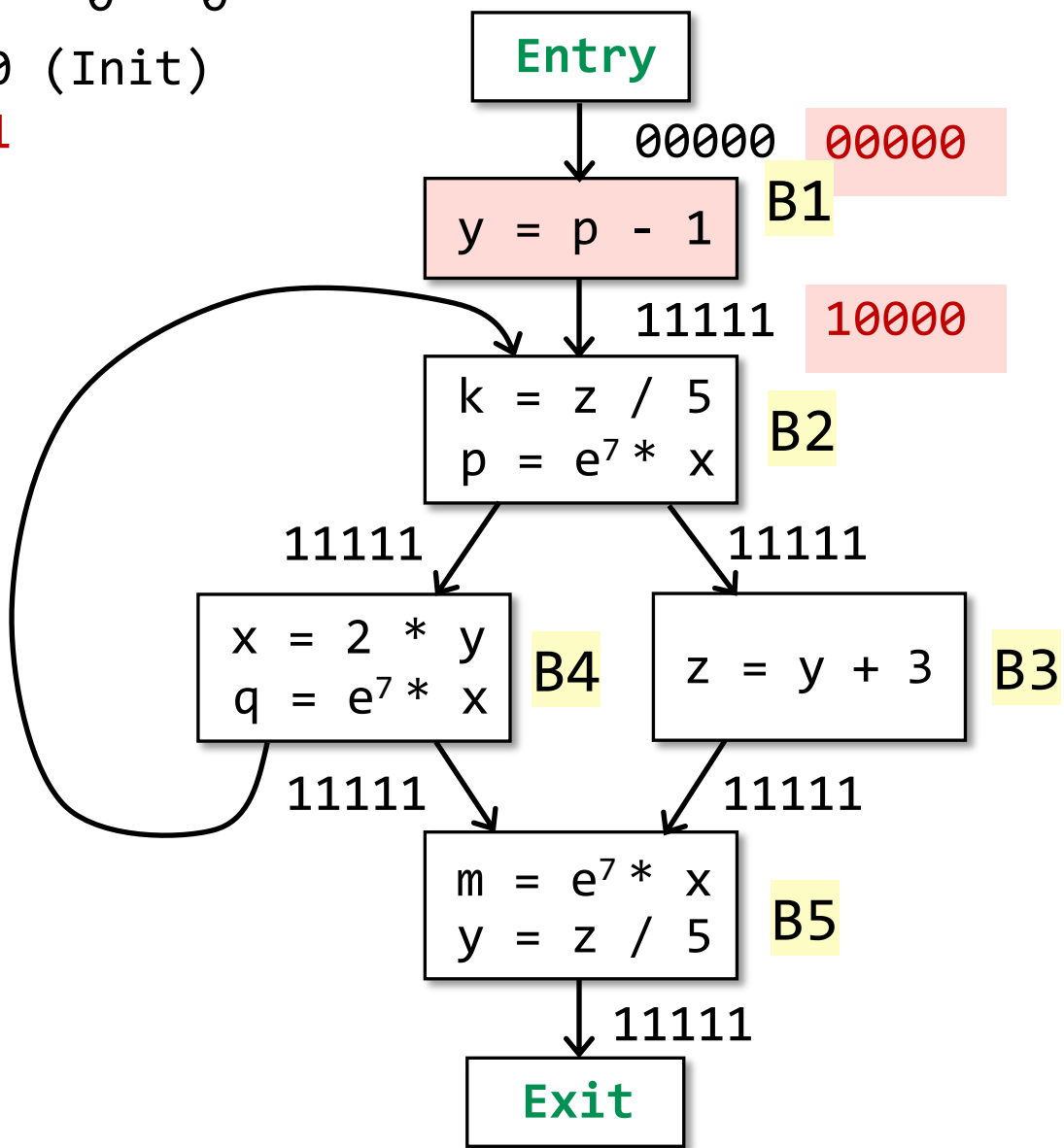
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

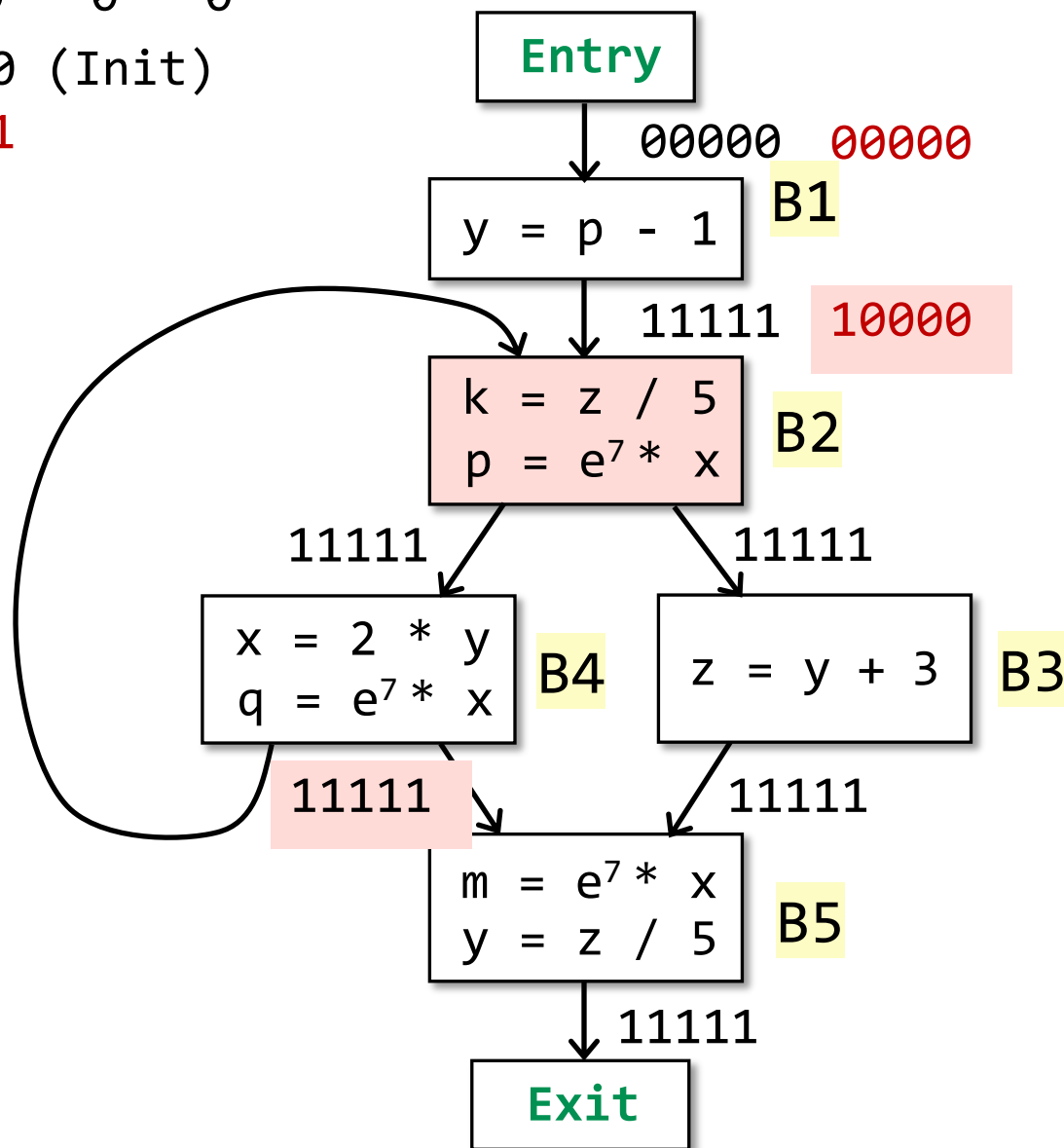
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

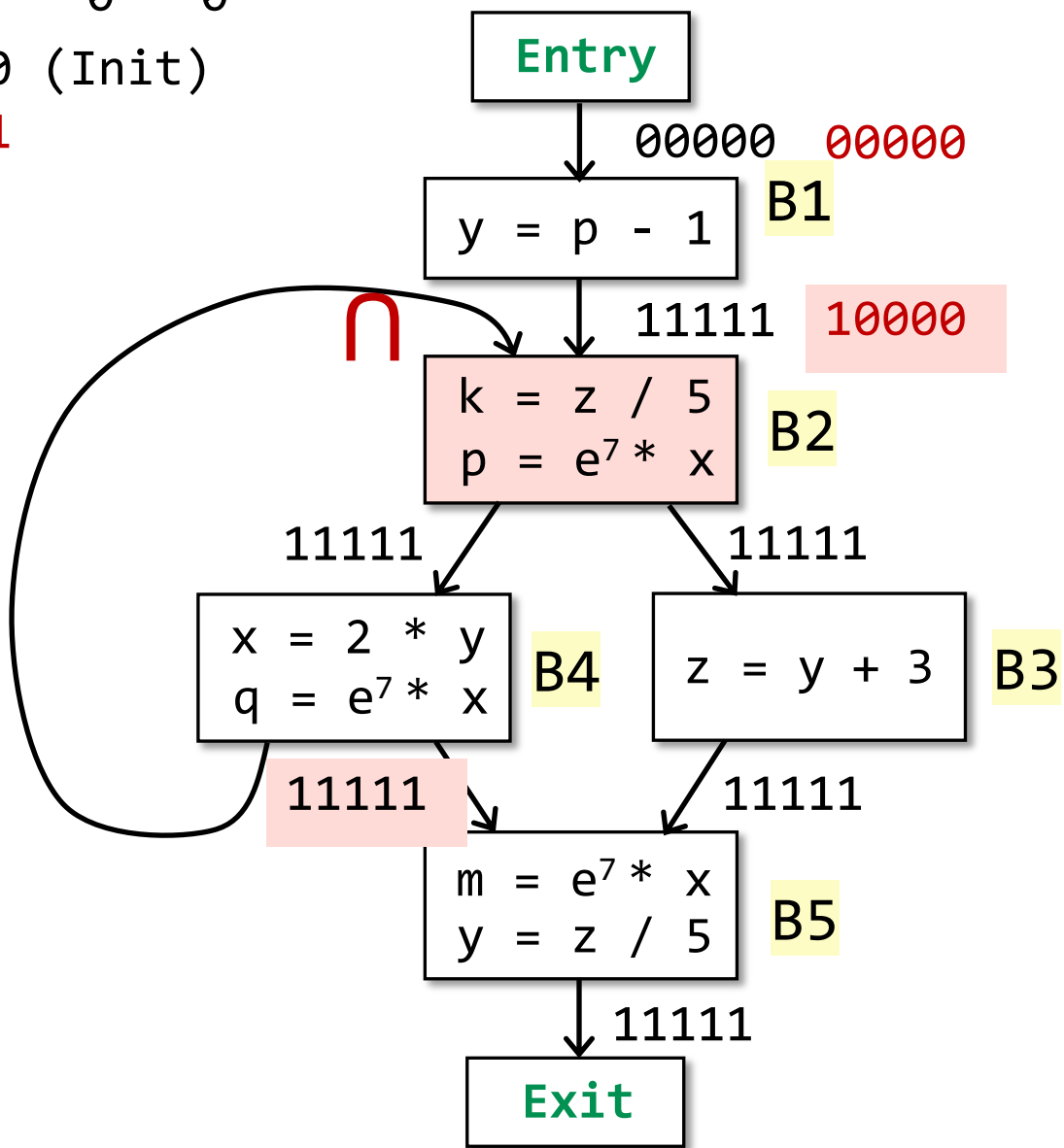
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

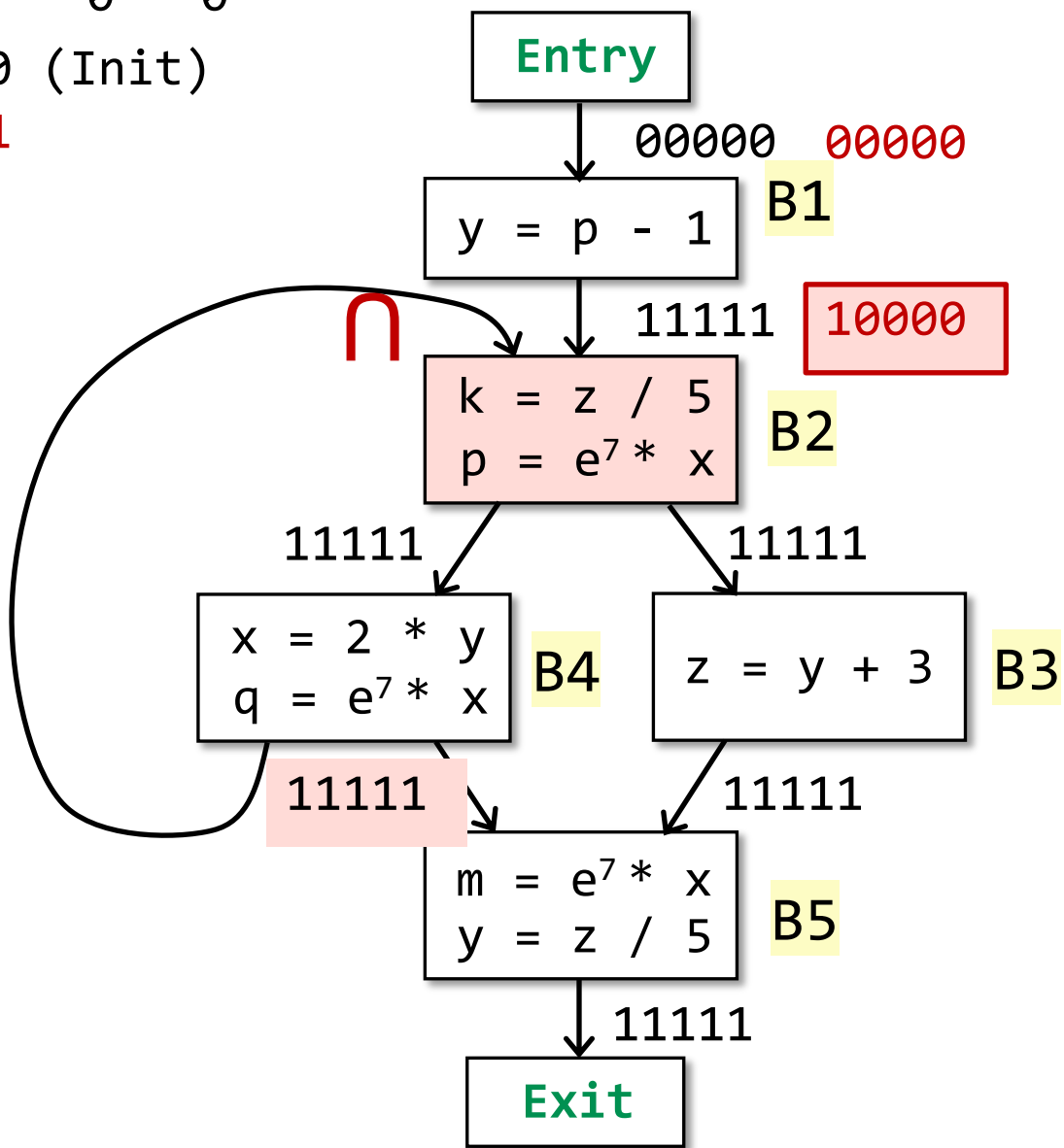
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

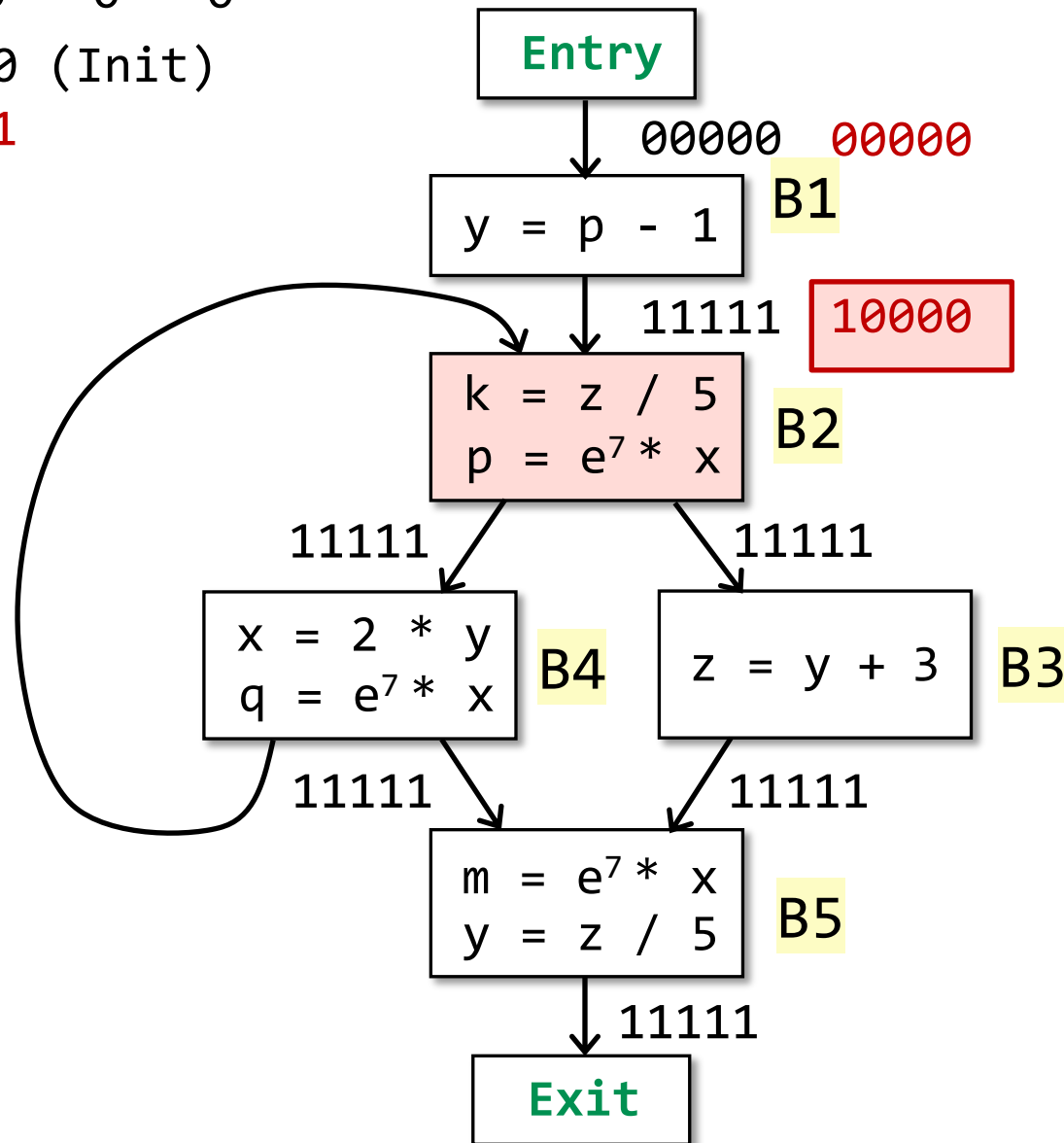
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

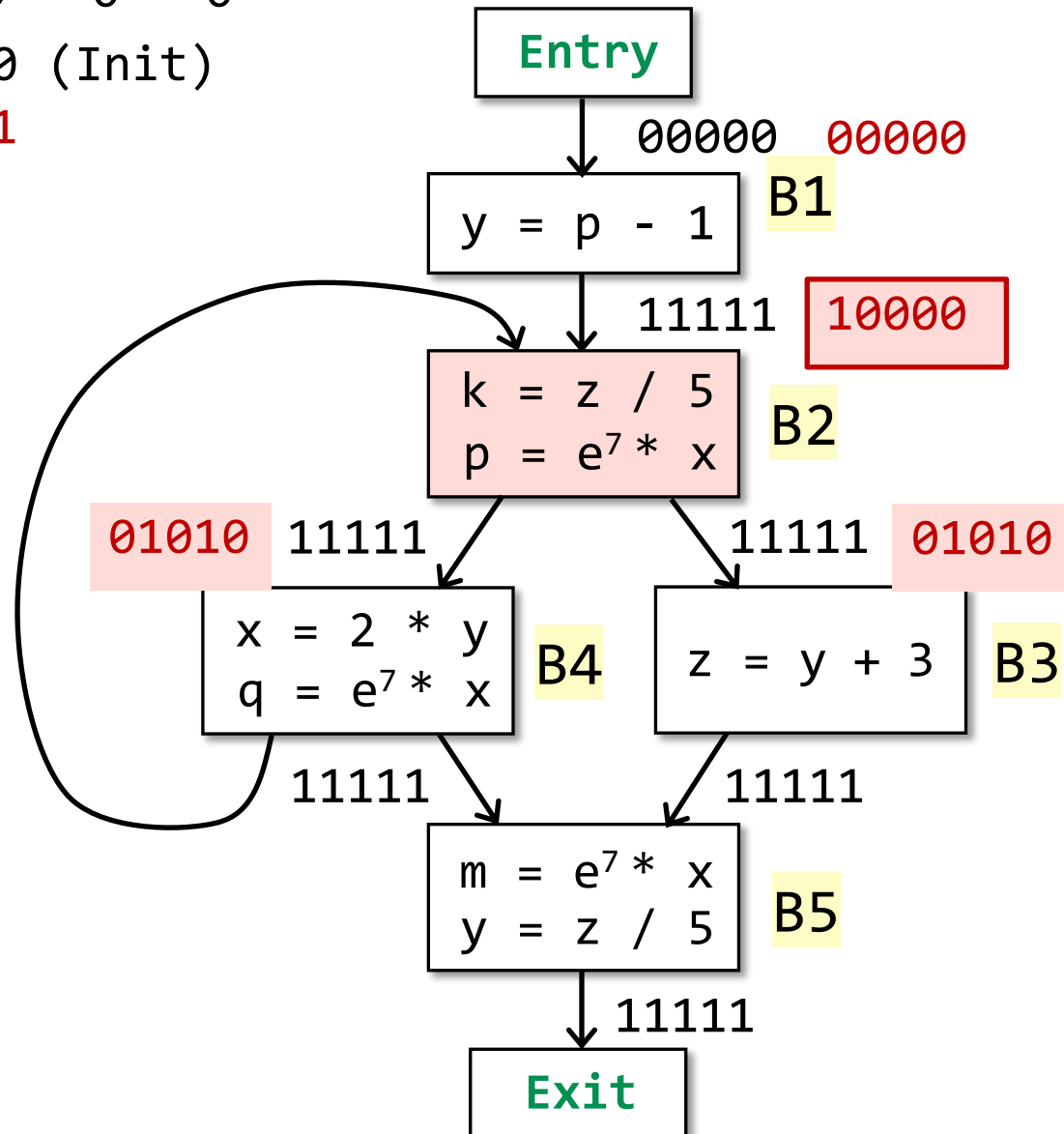
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

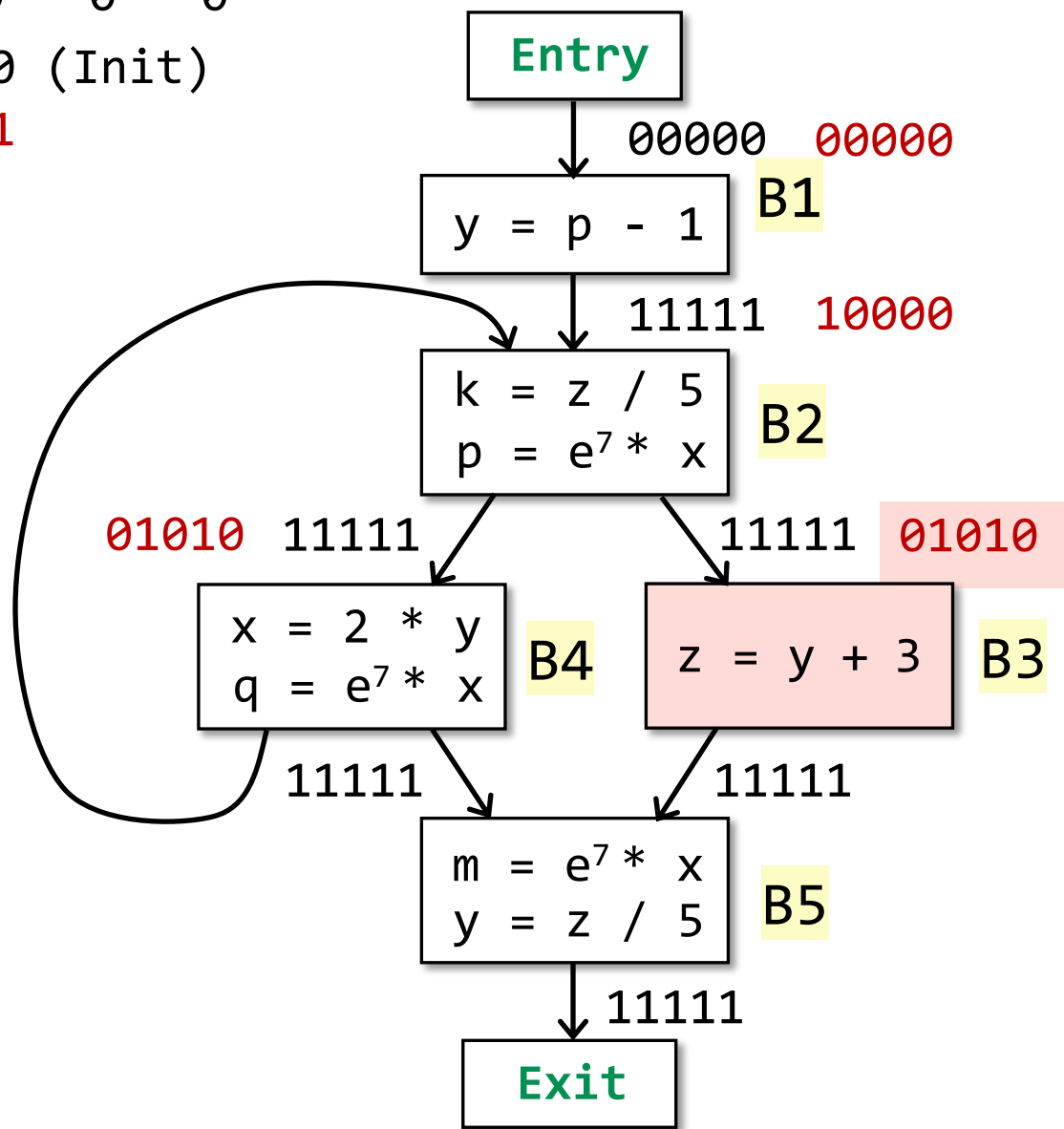
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

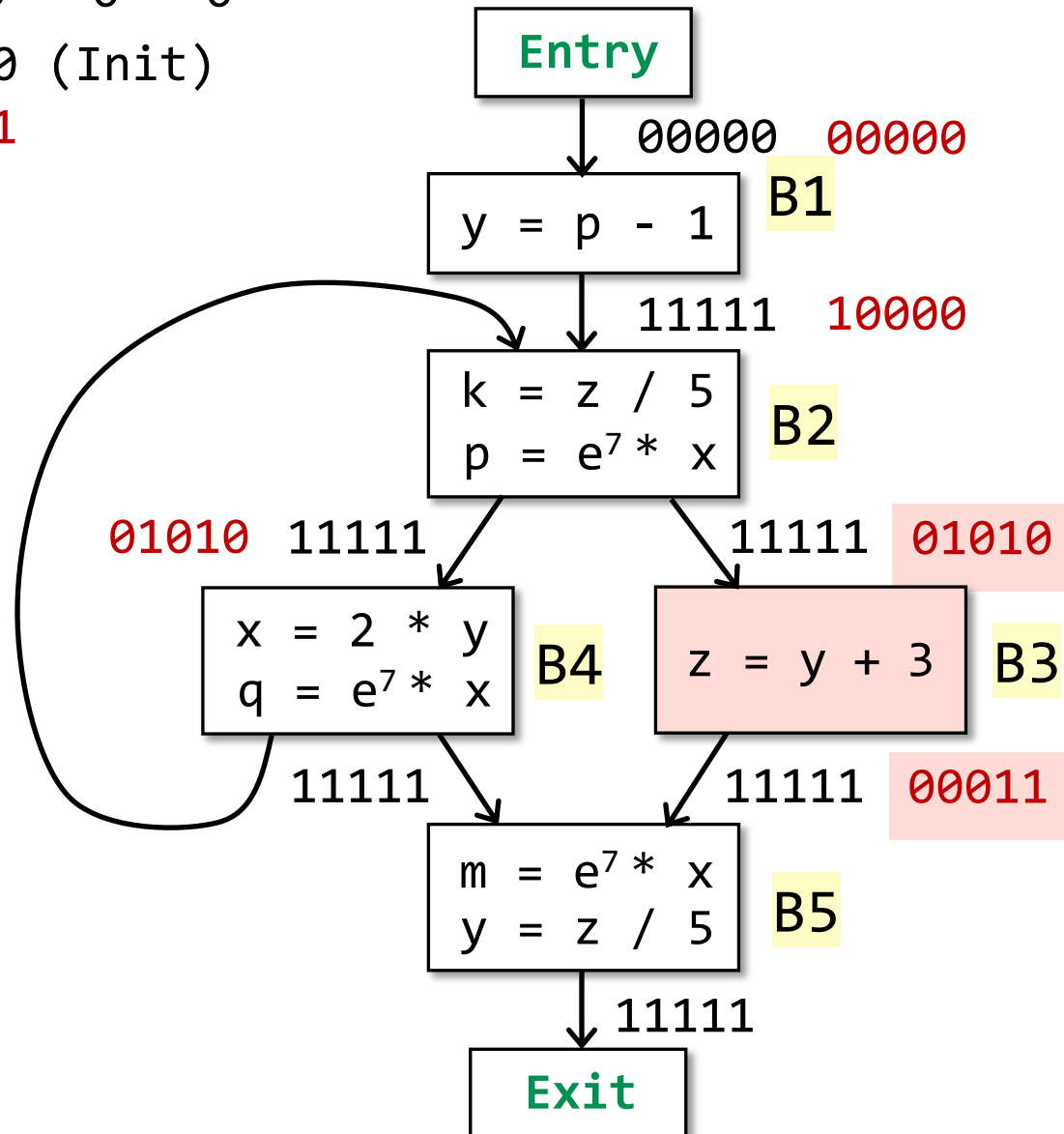
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

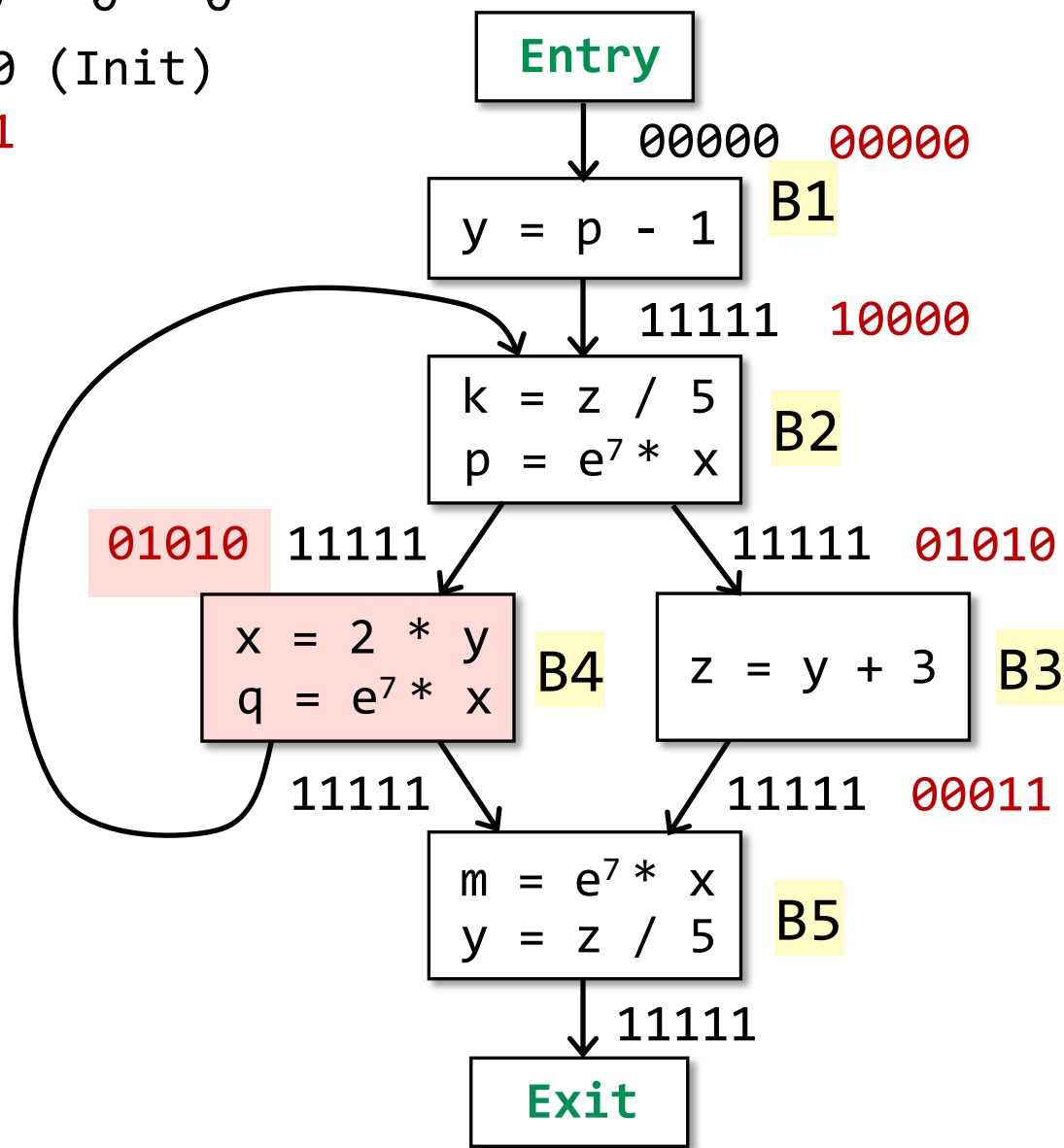
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

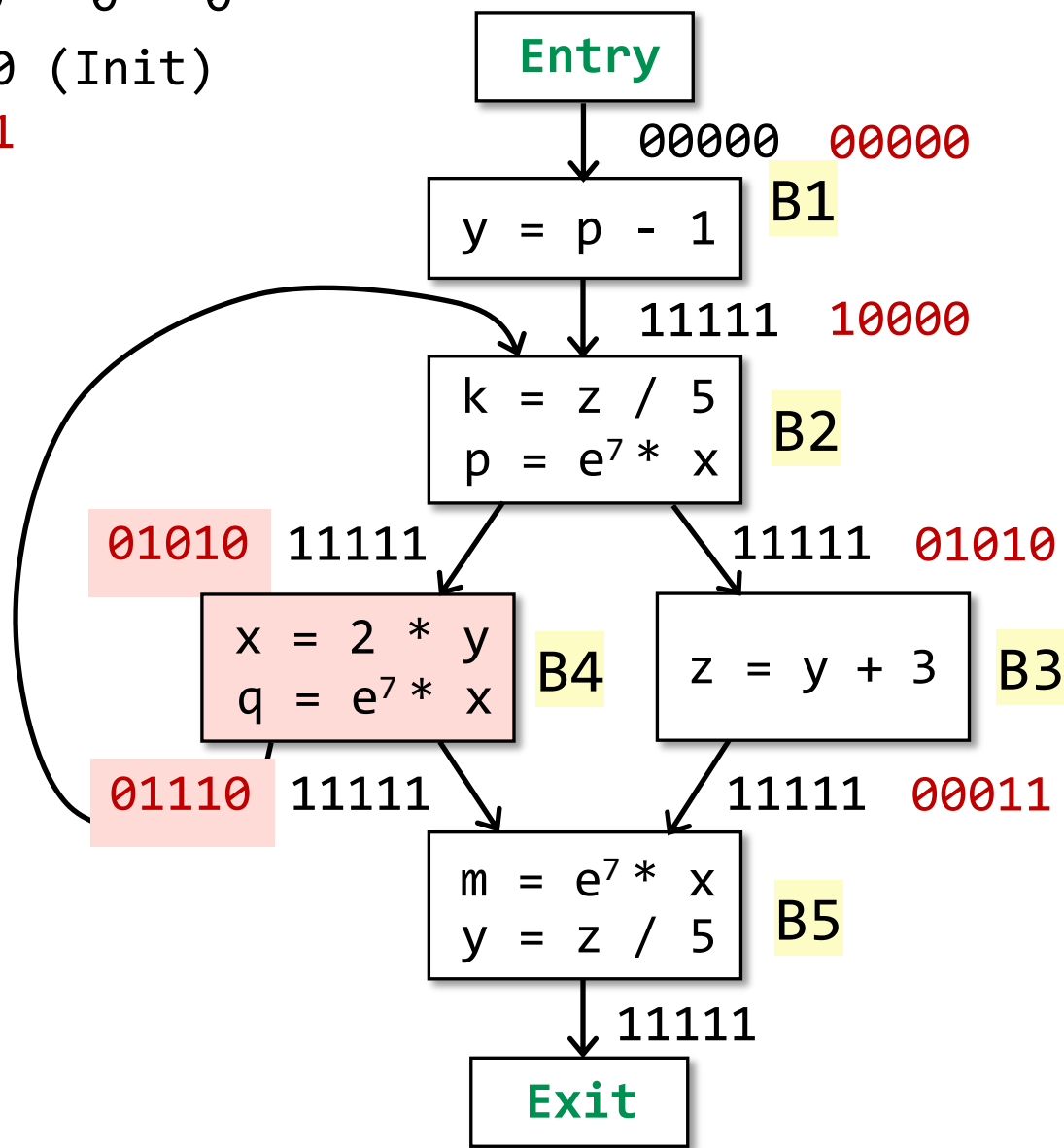
Iteration 1



p-1	z/5	2*y	e ⁷ *x	y+3
0	0	0	0	0

Iteration 0 (Init)

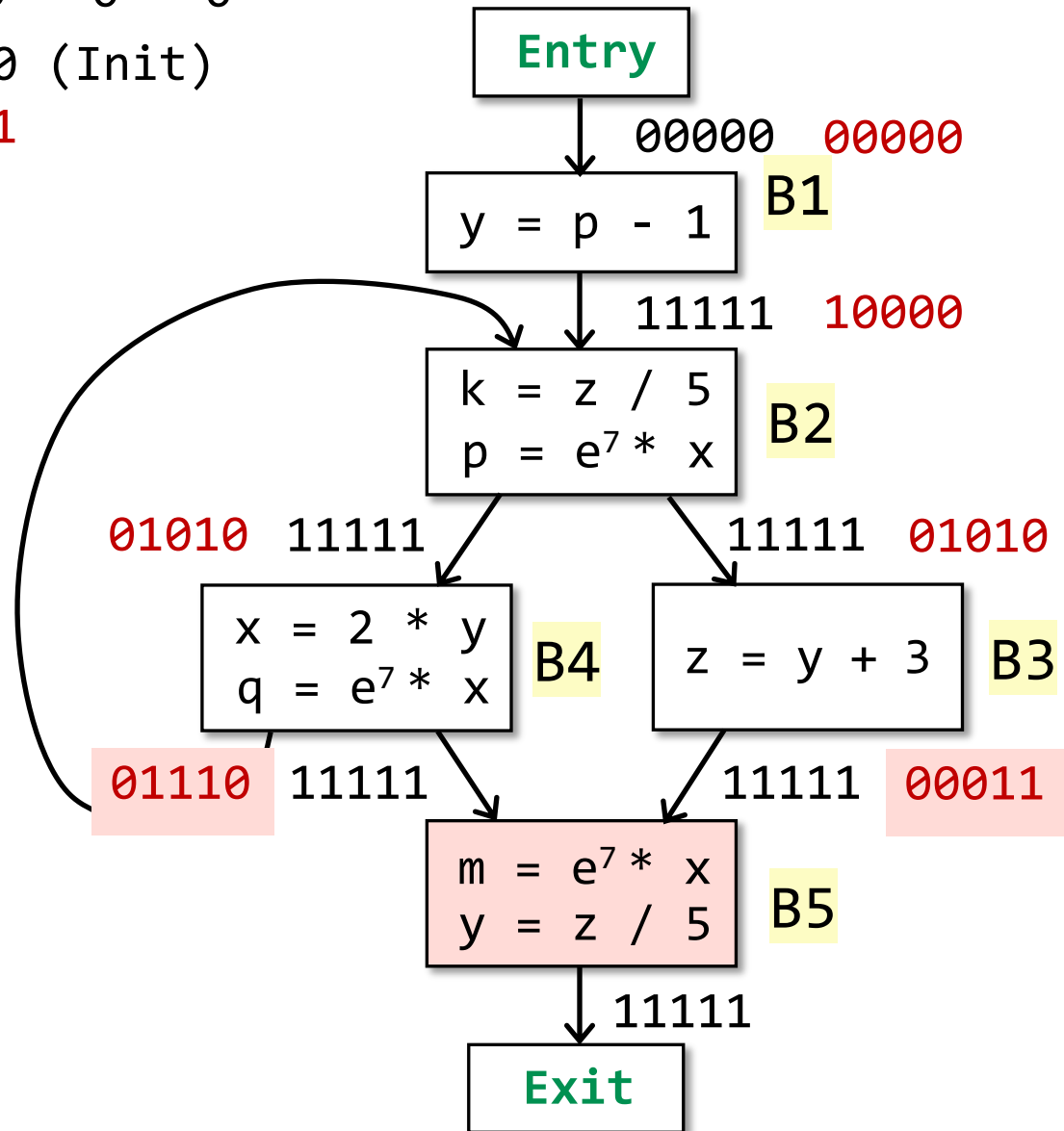
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

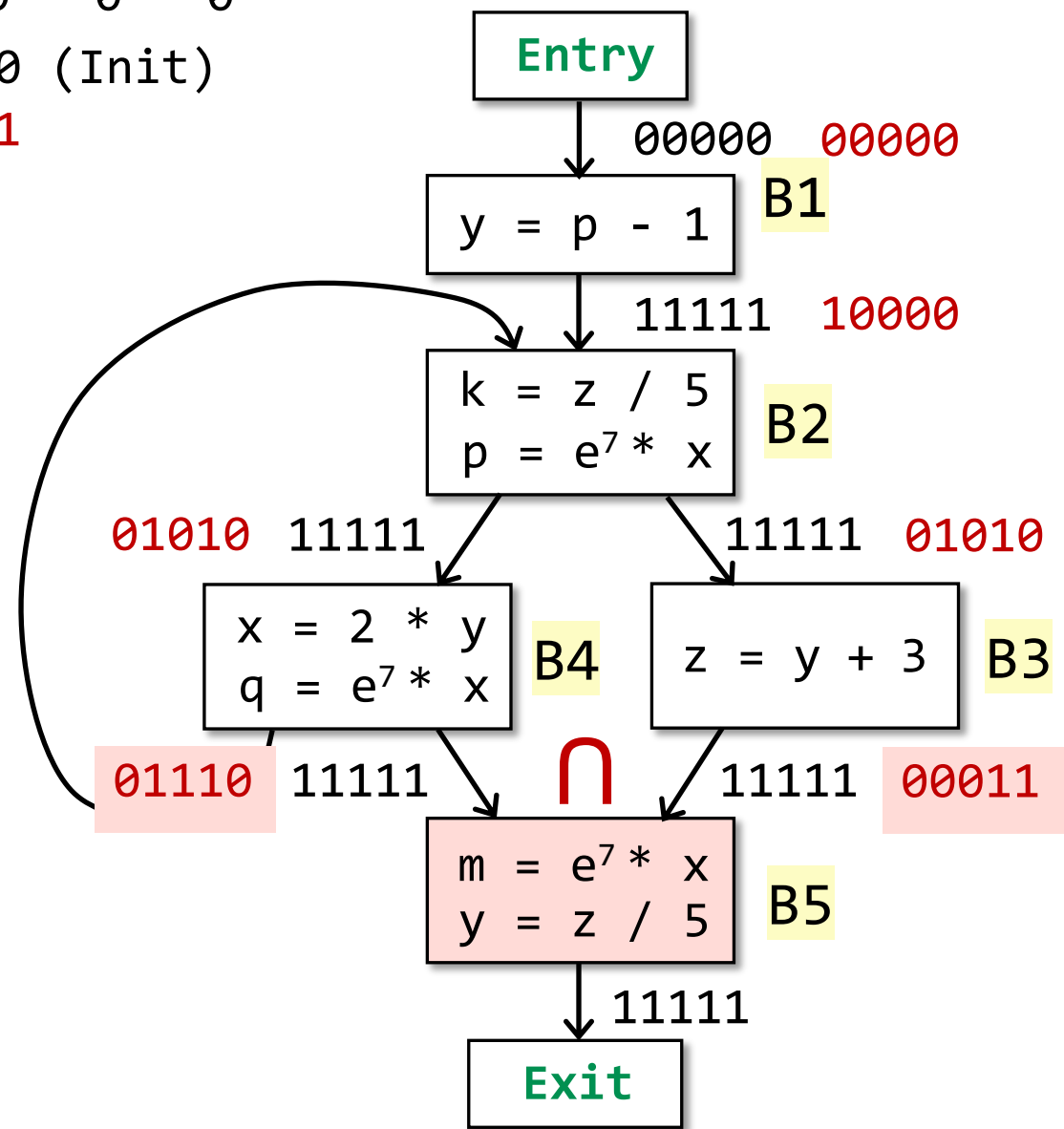
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

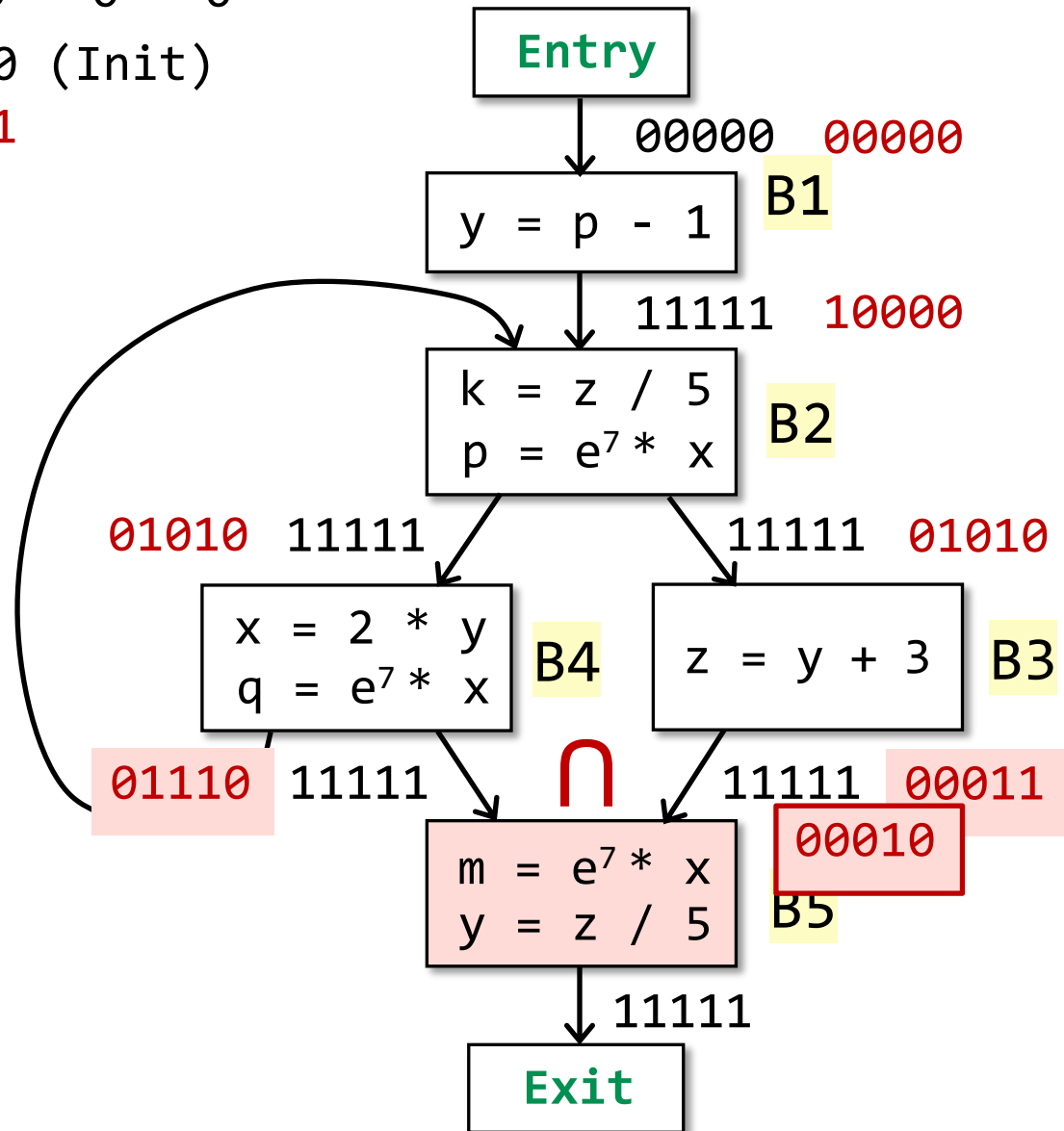
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

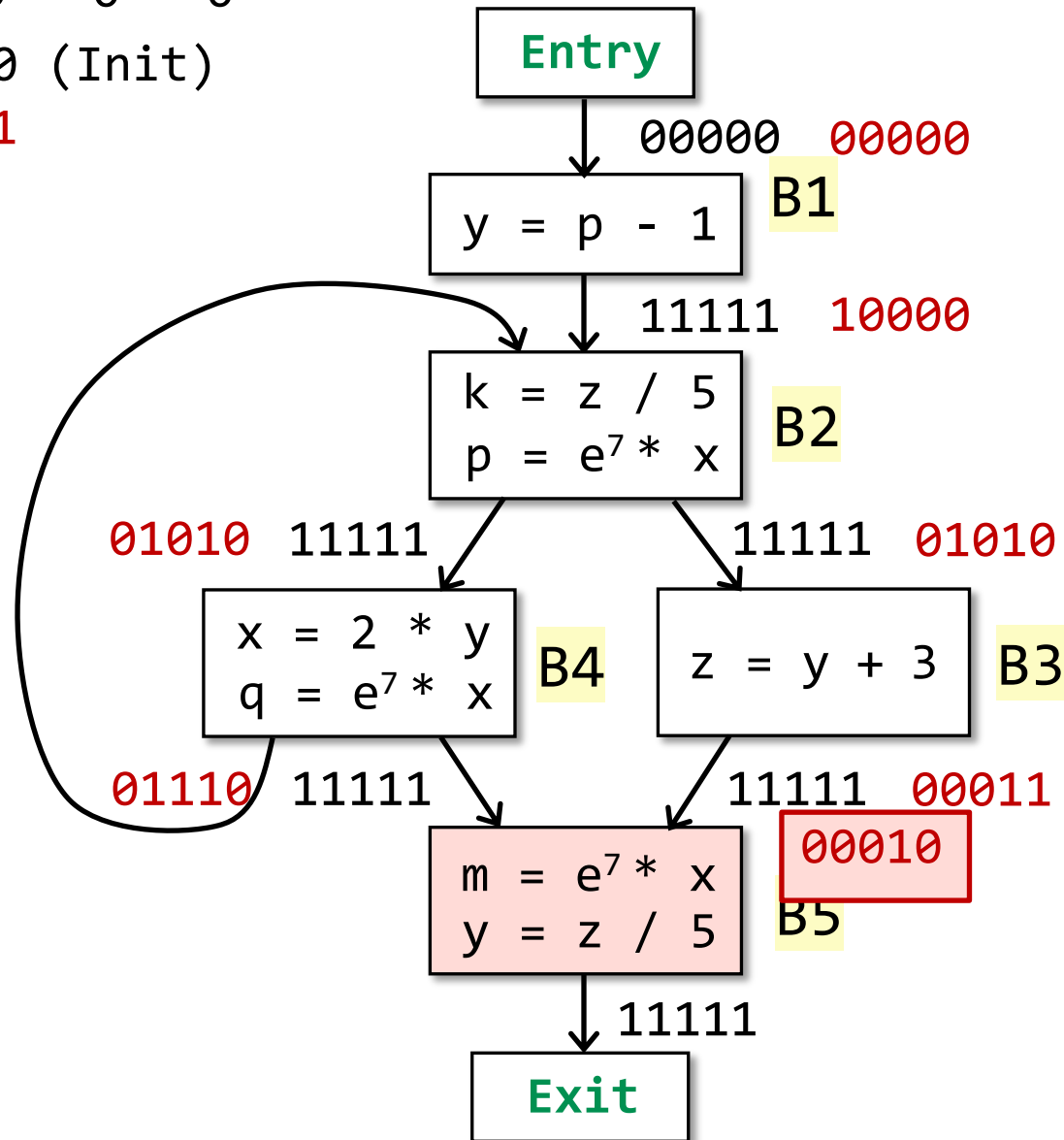
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

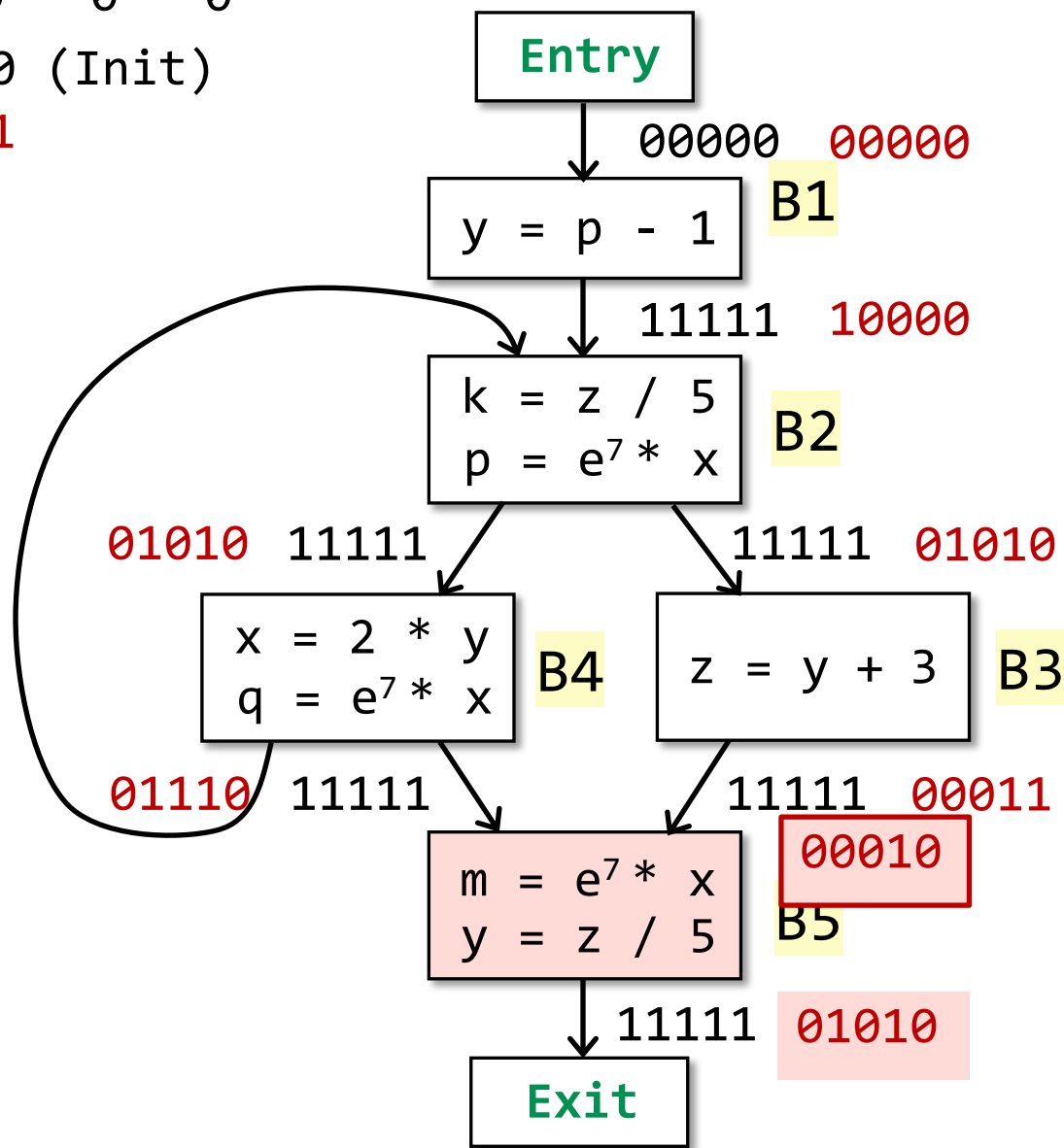
Iteration 1



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

Iteration 1

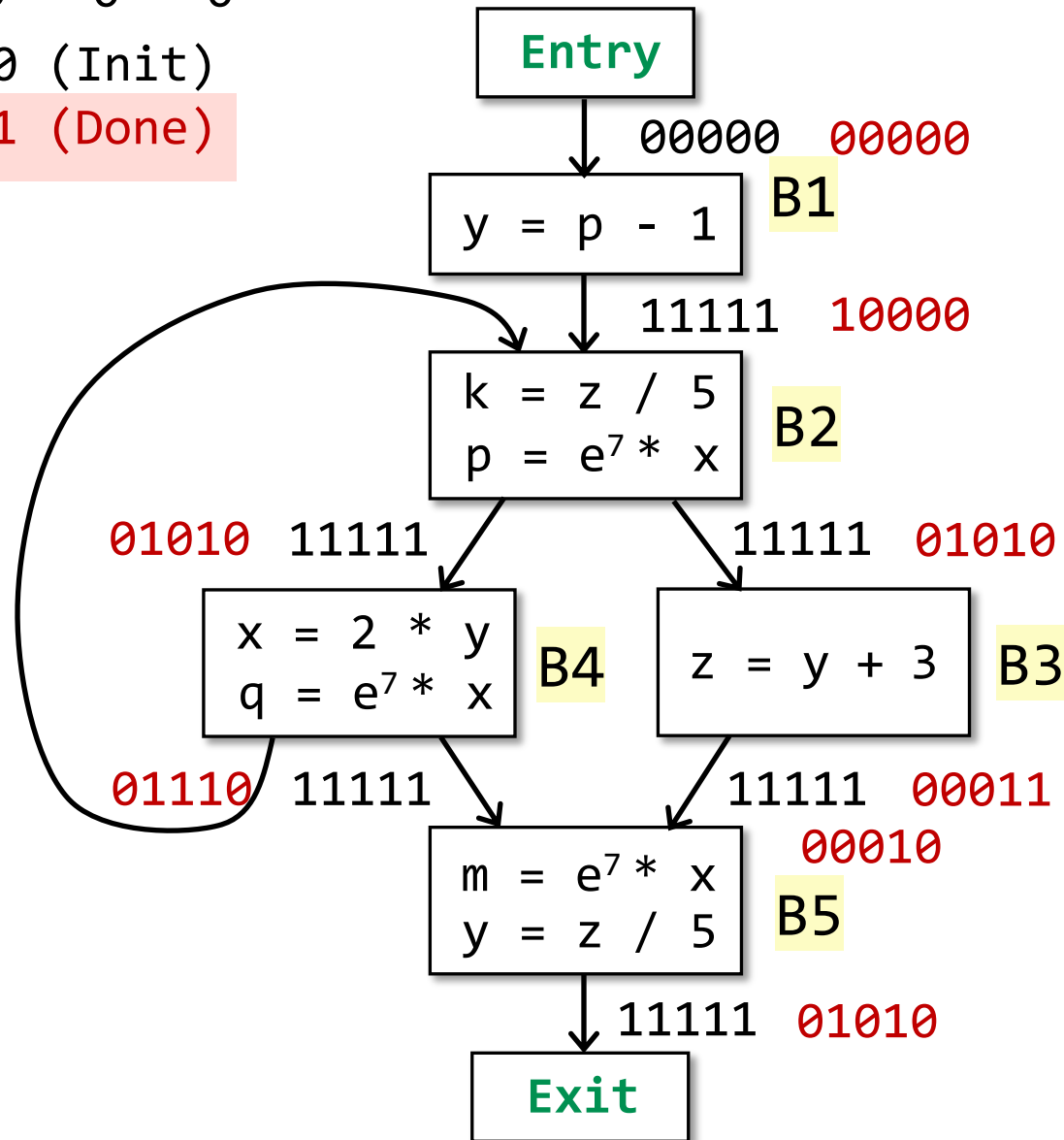


$p-1$ $z/5$ $2*y$ e^7*x $y+3$

0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

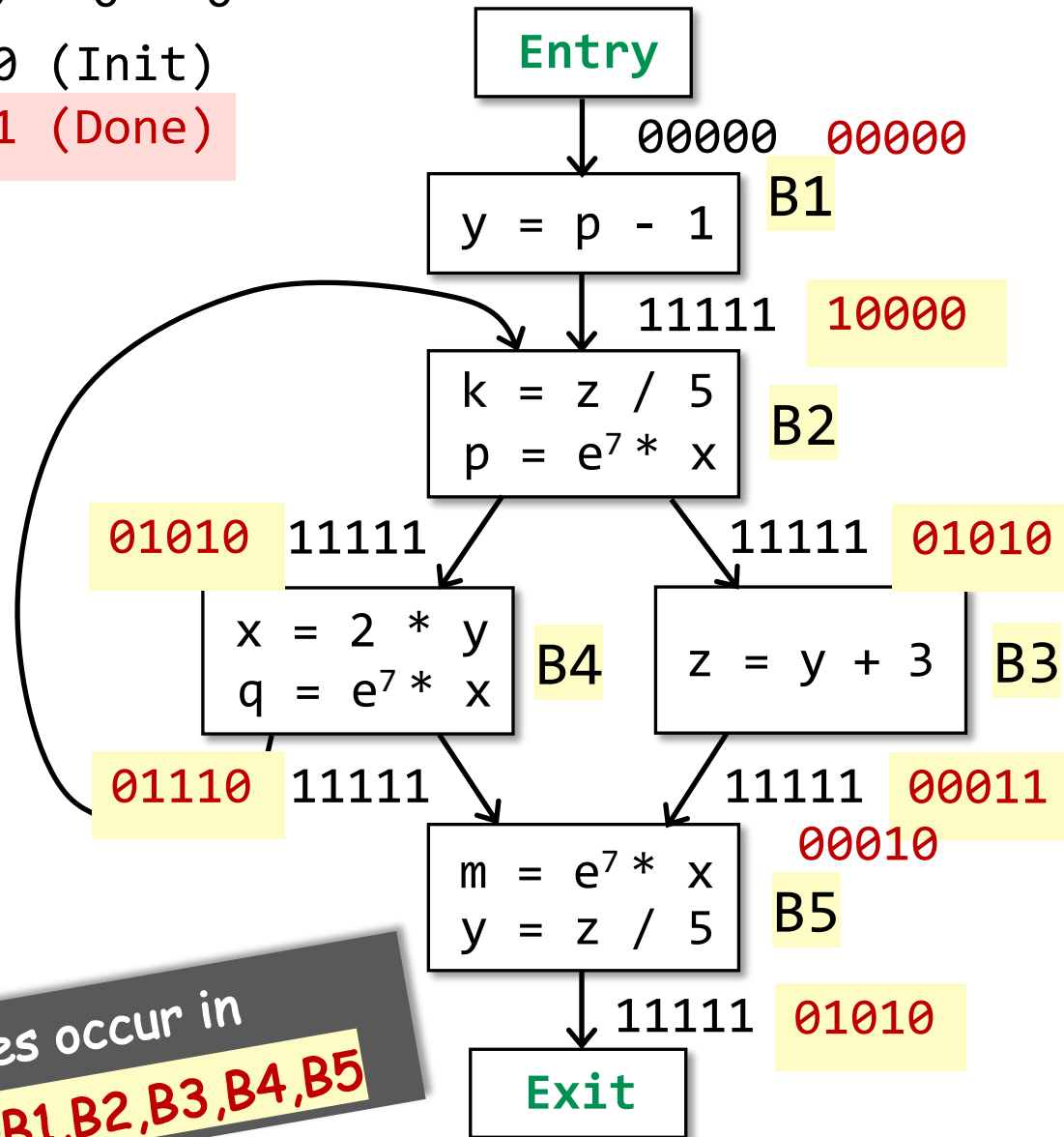


$p-1$ $z/5$ $2*y$ e^7*x $y+3$

0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)



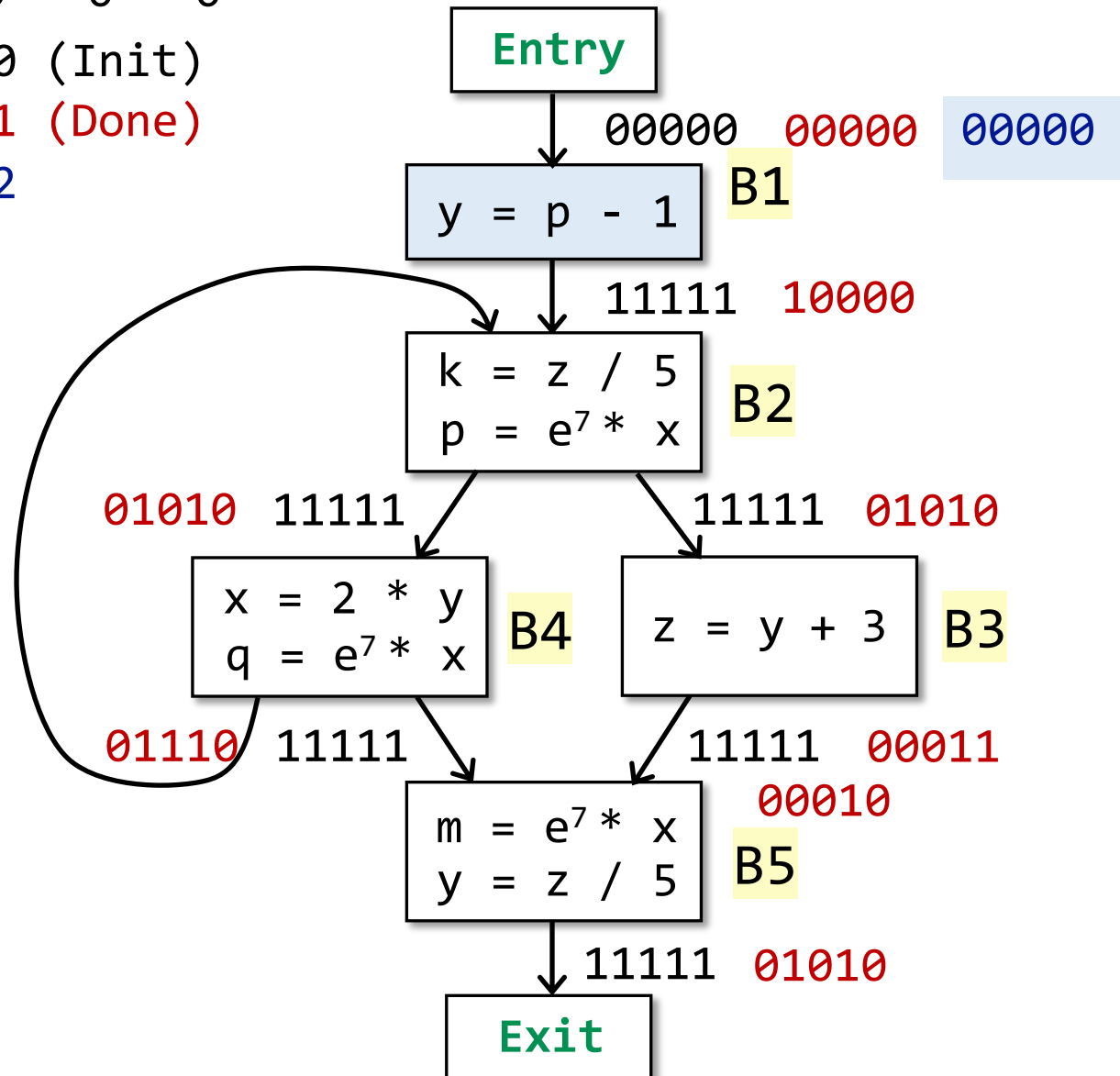
Changes occur in
OUT[] of B1,B2,B3,B4,B5

$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

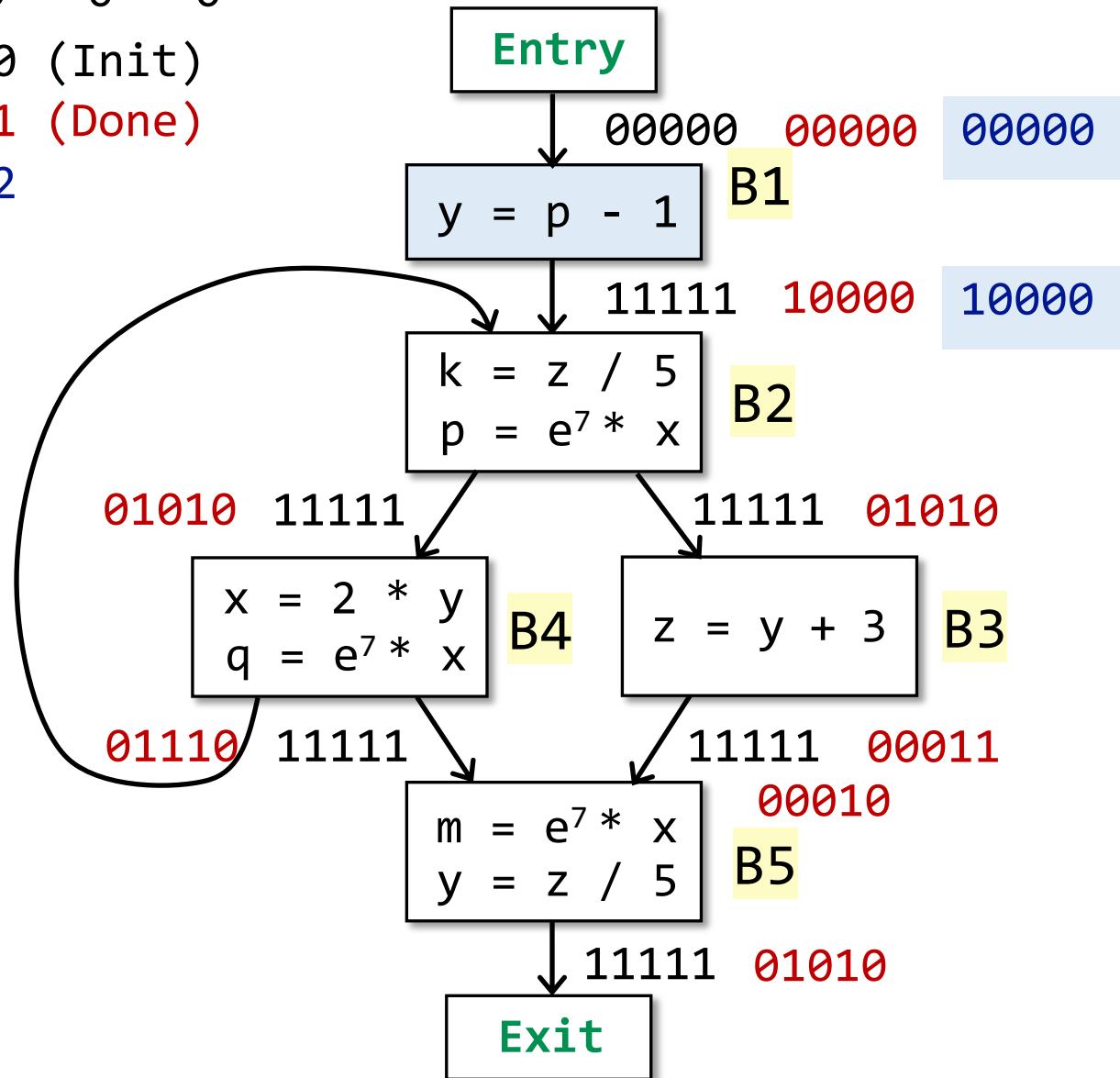


$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

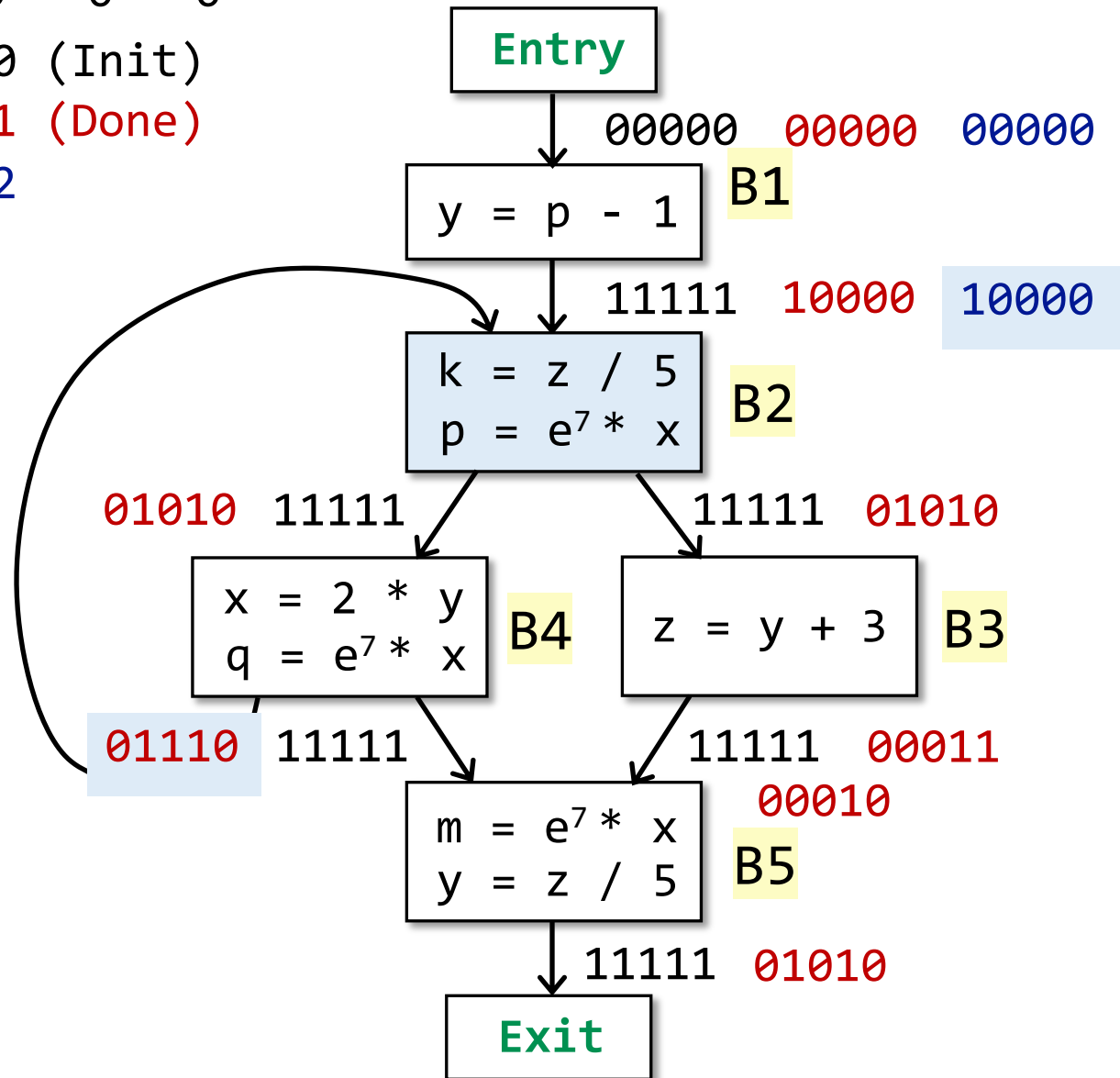


$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

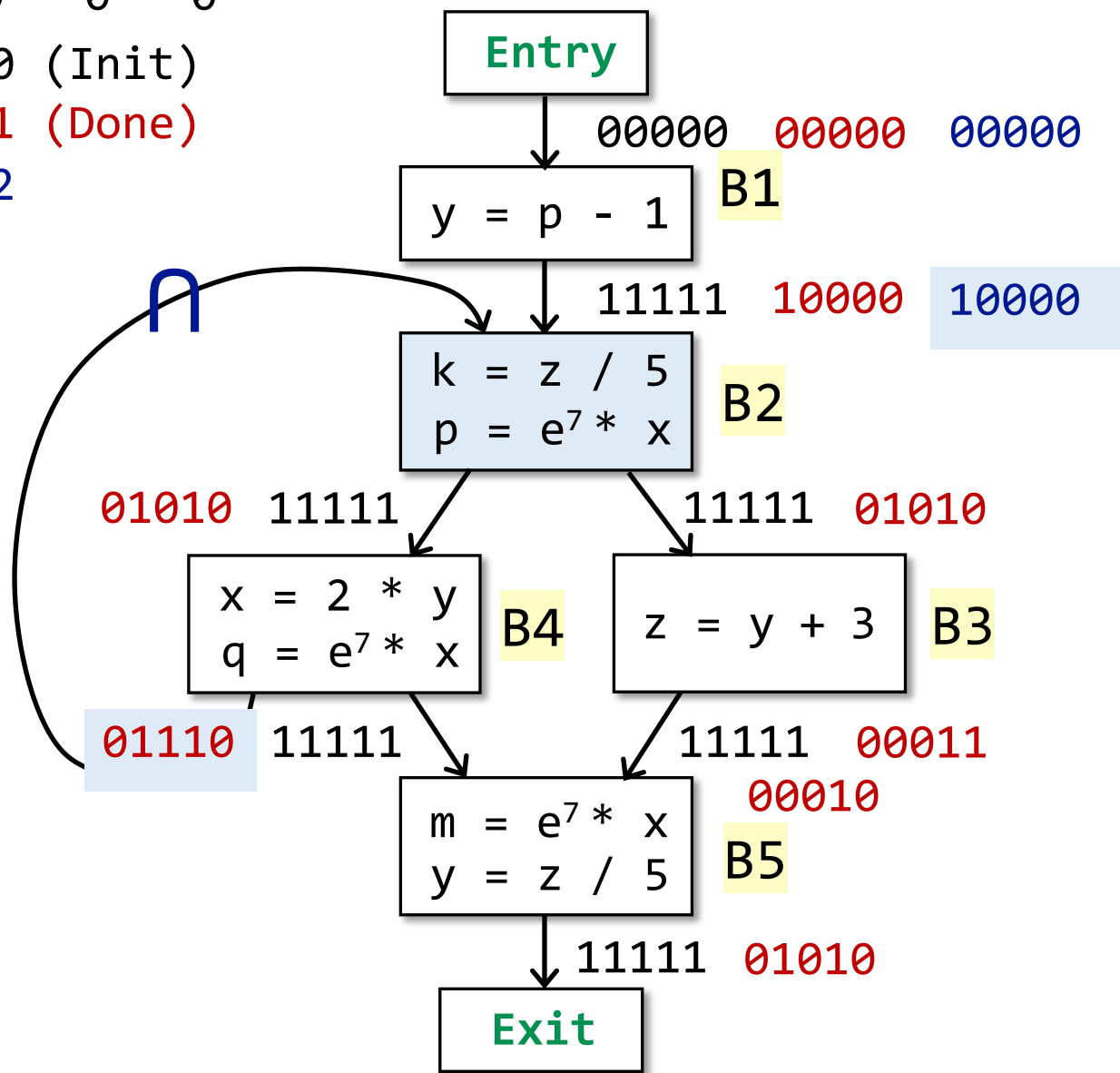
Iteration 1 (Done)

Iteration 2



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)
 Iteration 1 (Done)
 Iteration 2

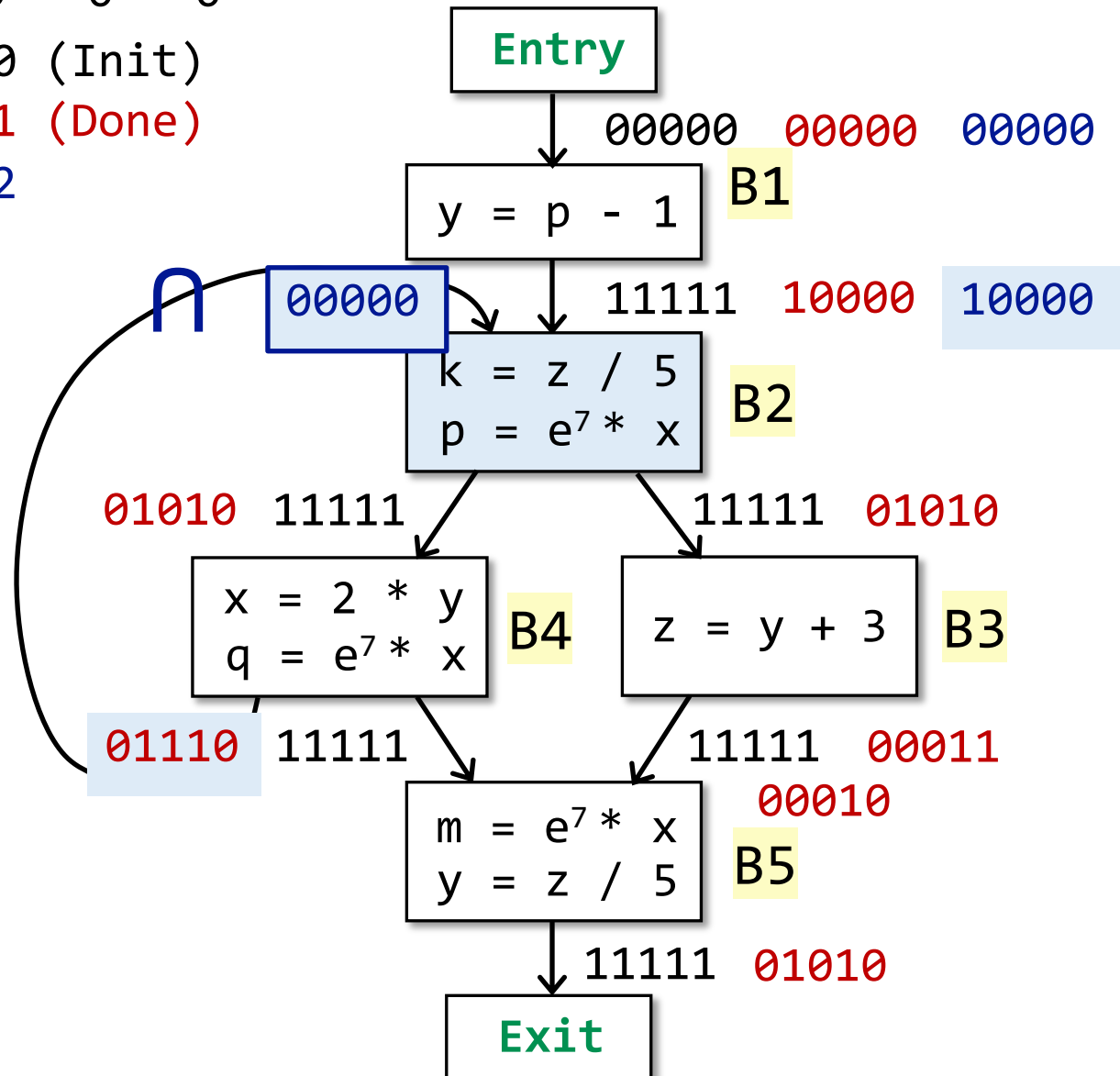


$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

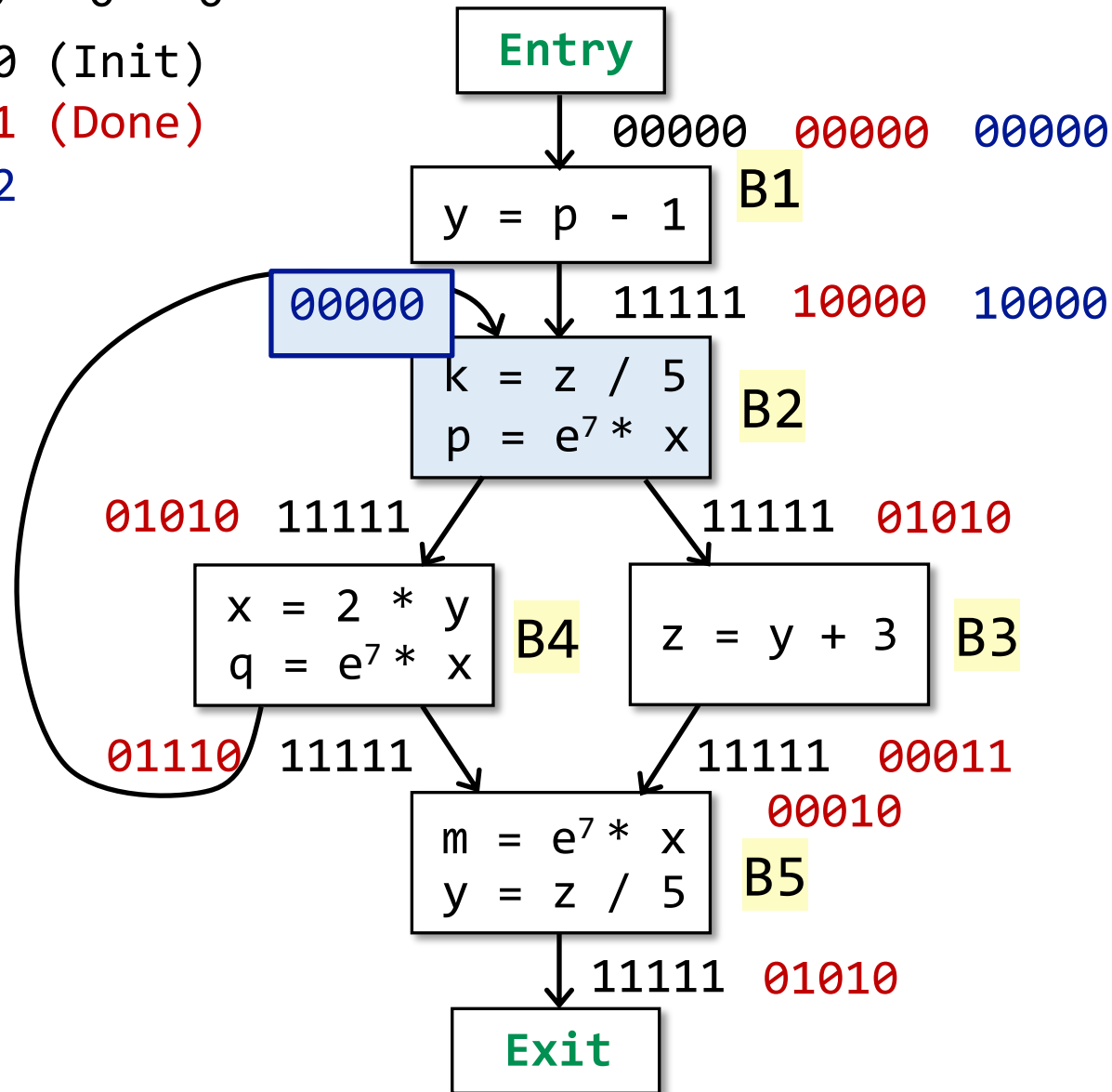


$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

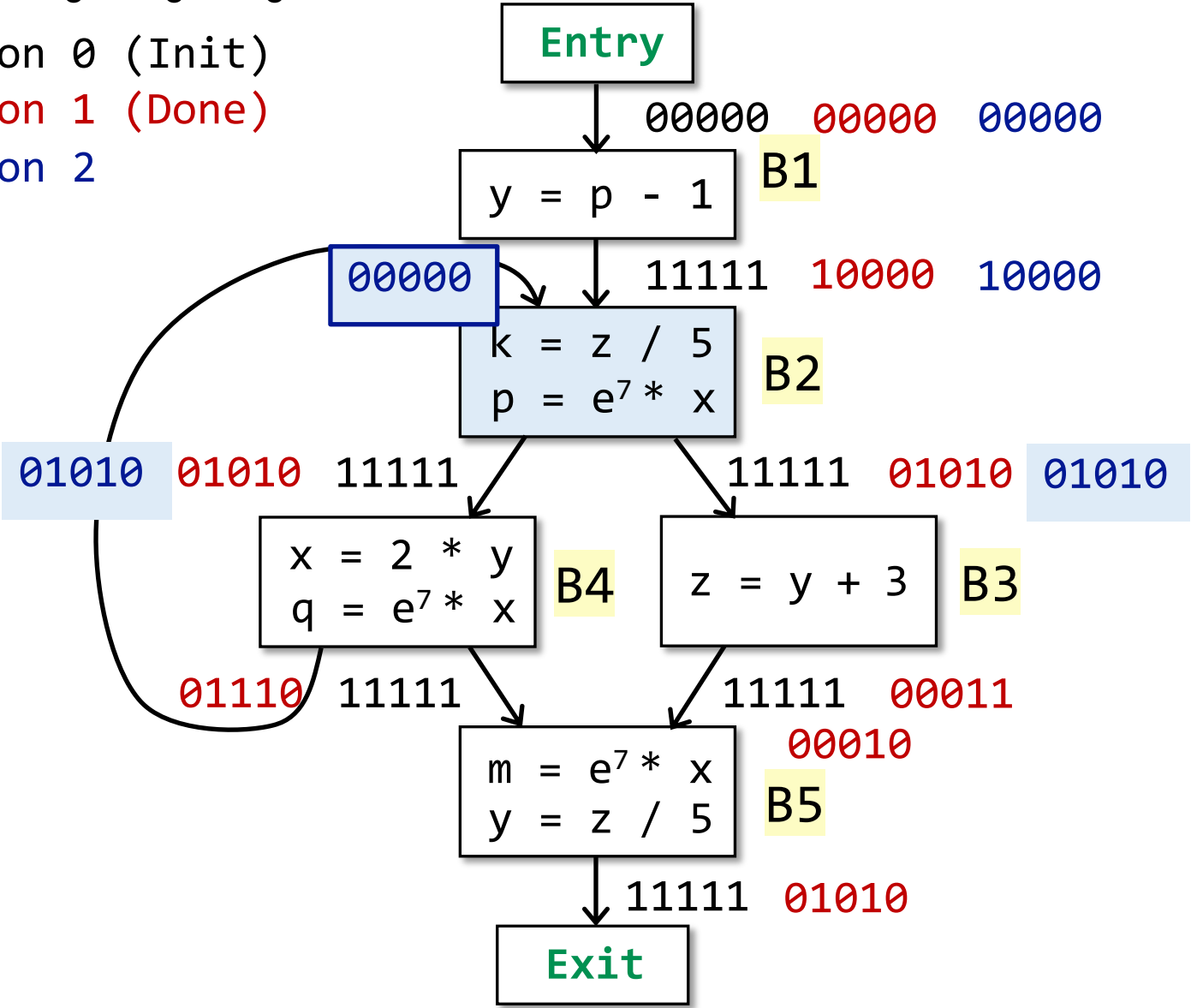
Iteration 1 (Done)

Iteration 2



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)
 Iteration 1 (Done)
 Iteration 2

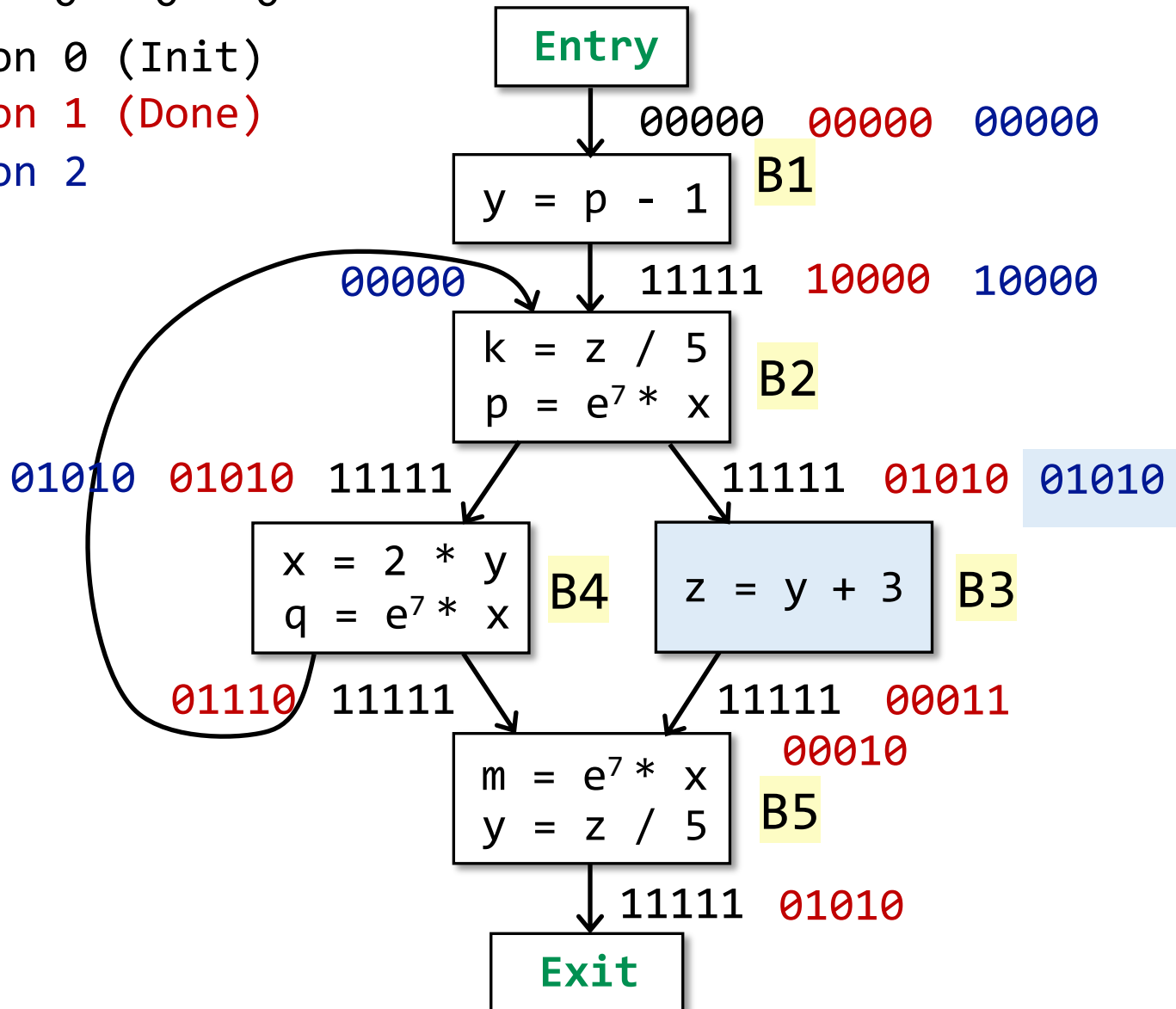


$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

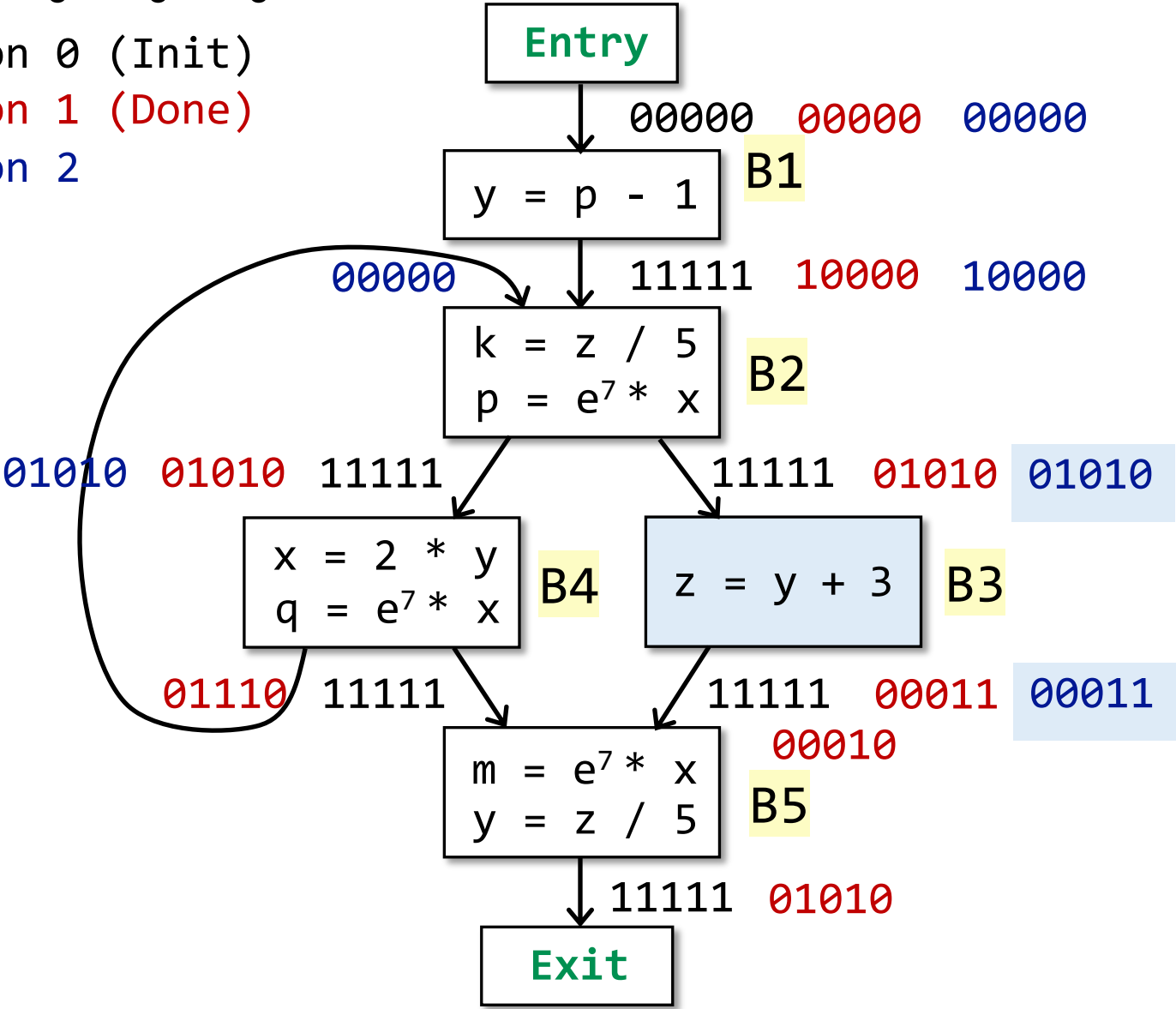
Iteration 1 (Done)

Iteration 2



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)
 Iteration 1 (Done)
 Iteration 2

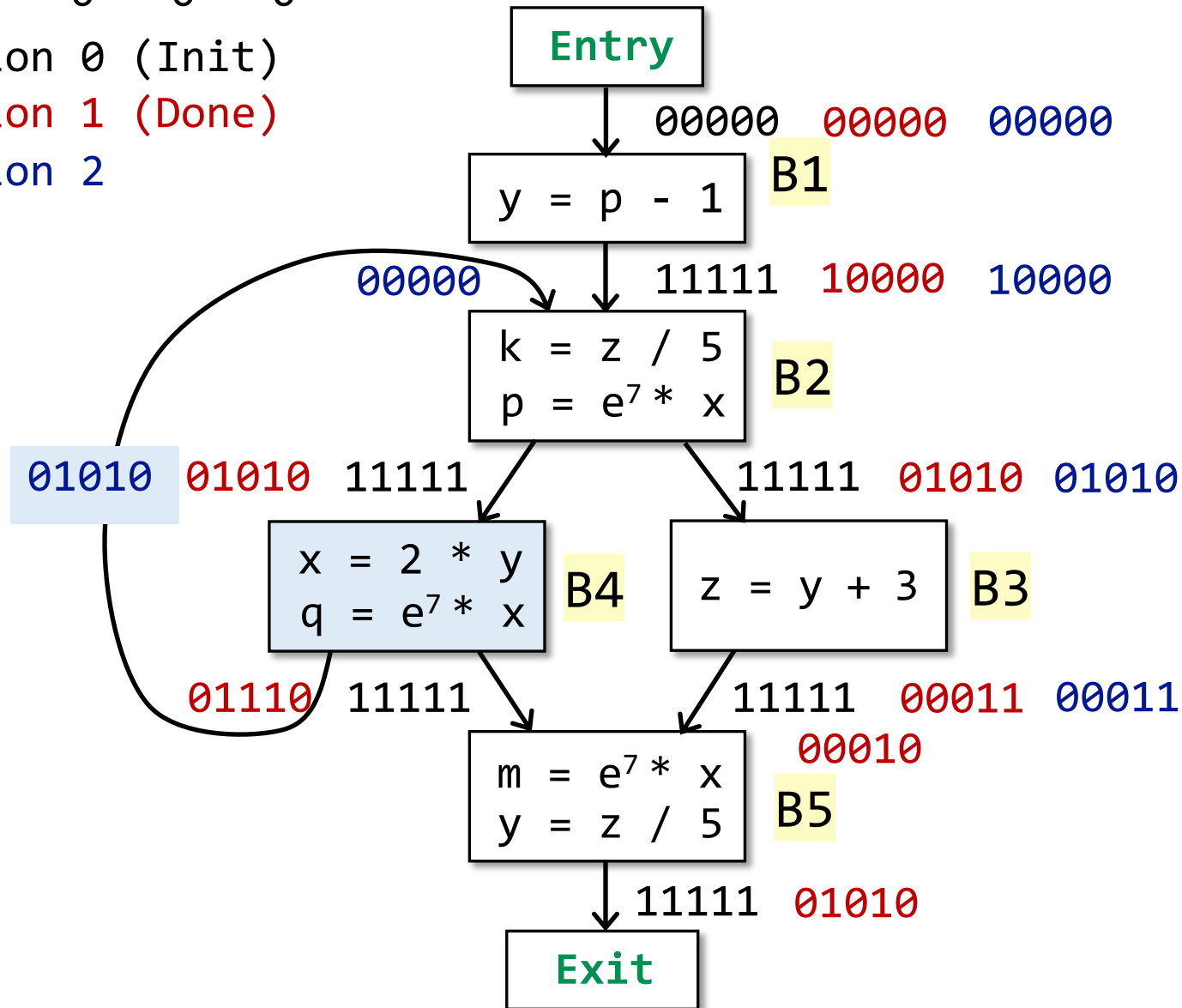


$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

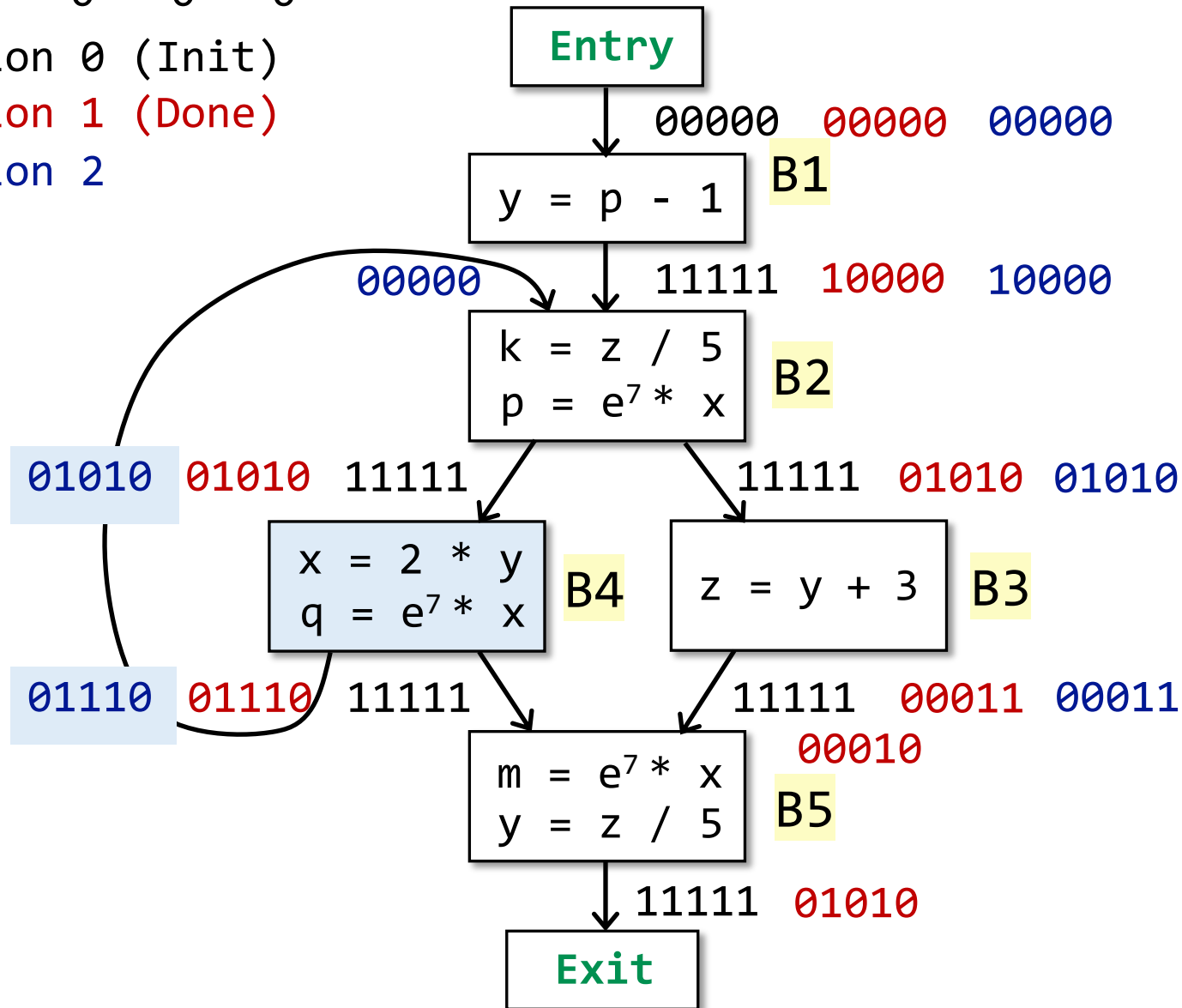


$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

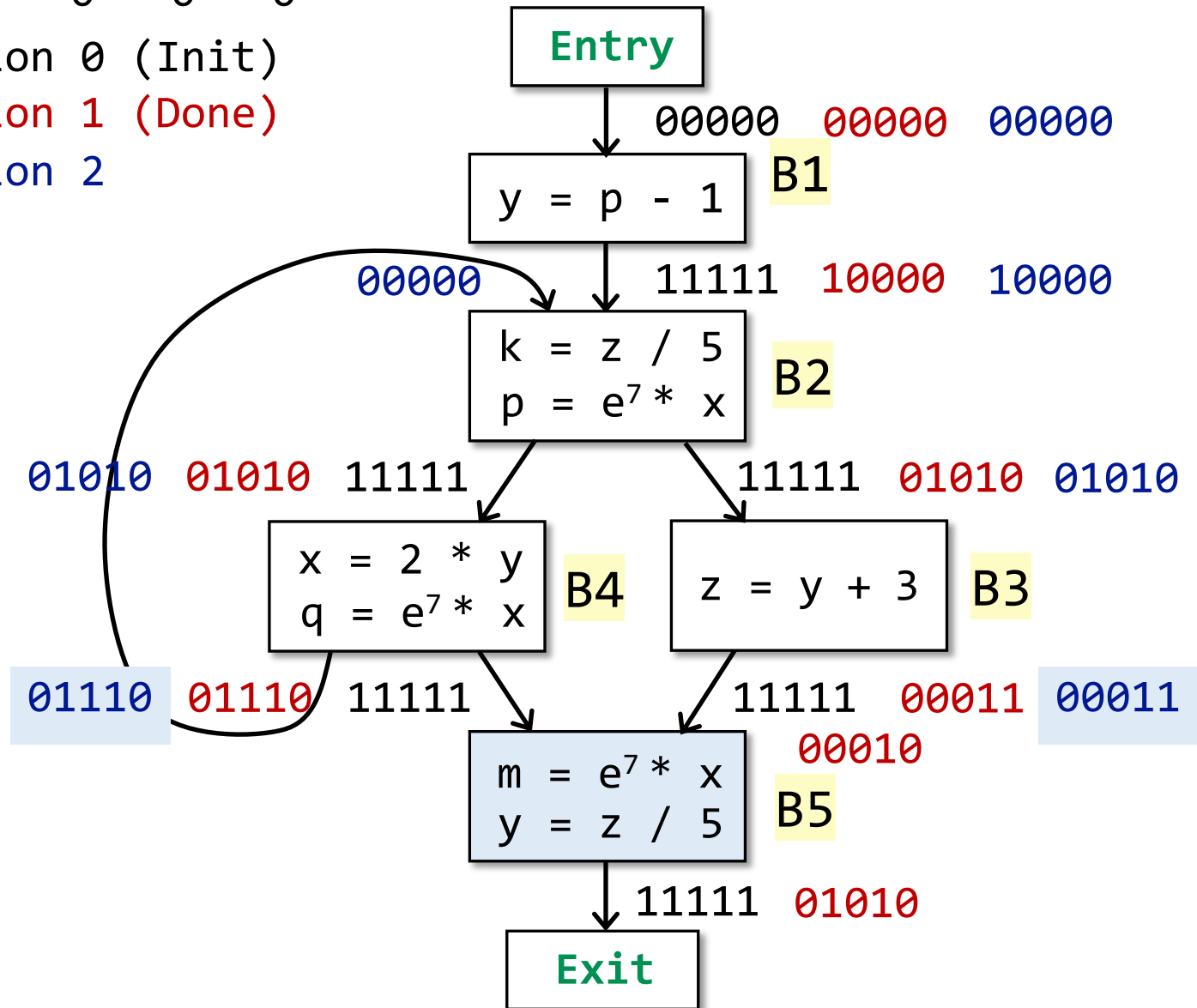


$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

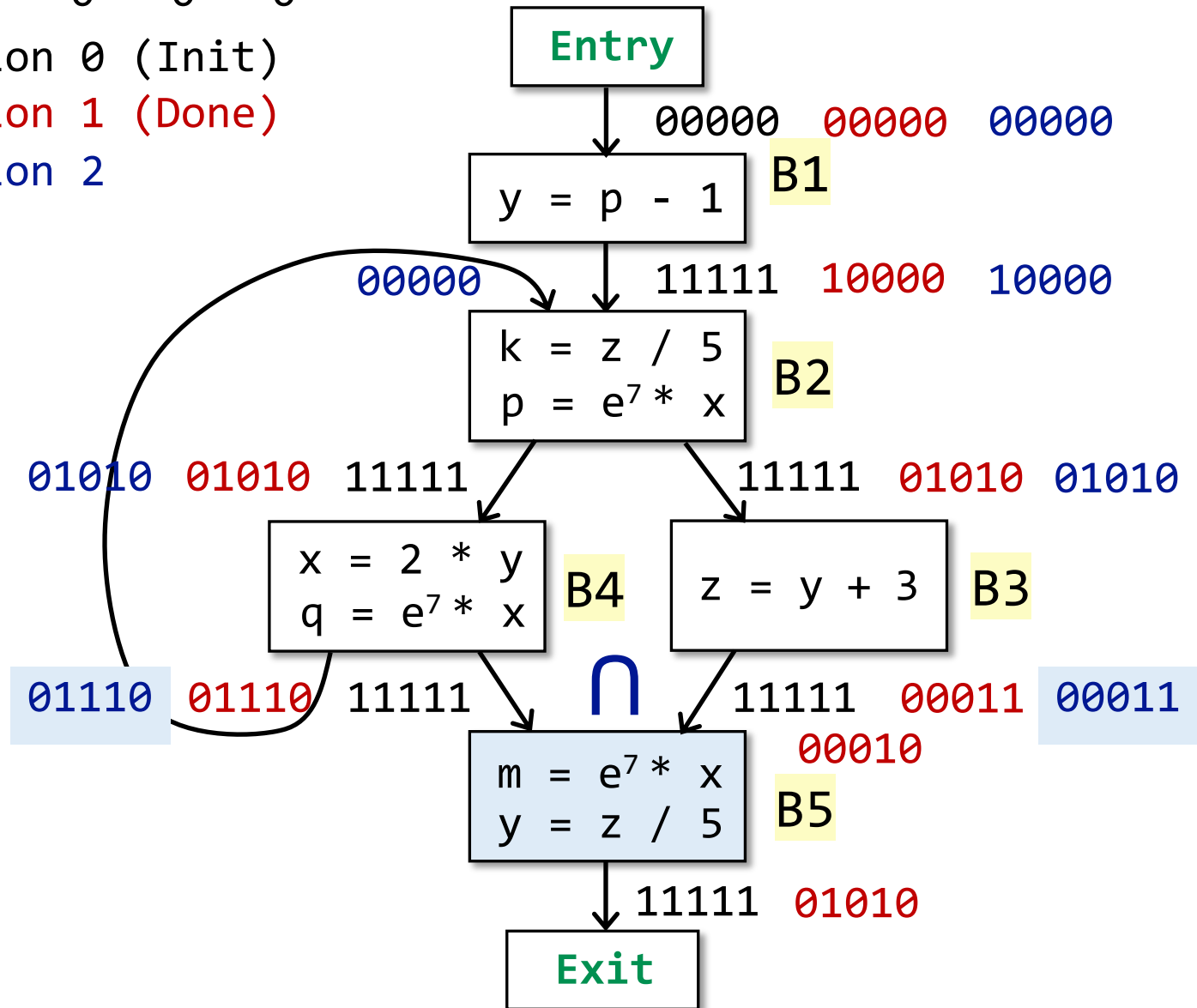


$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2

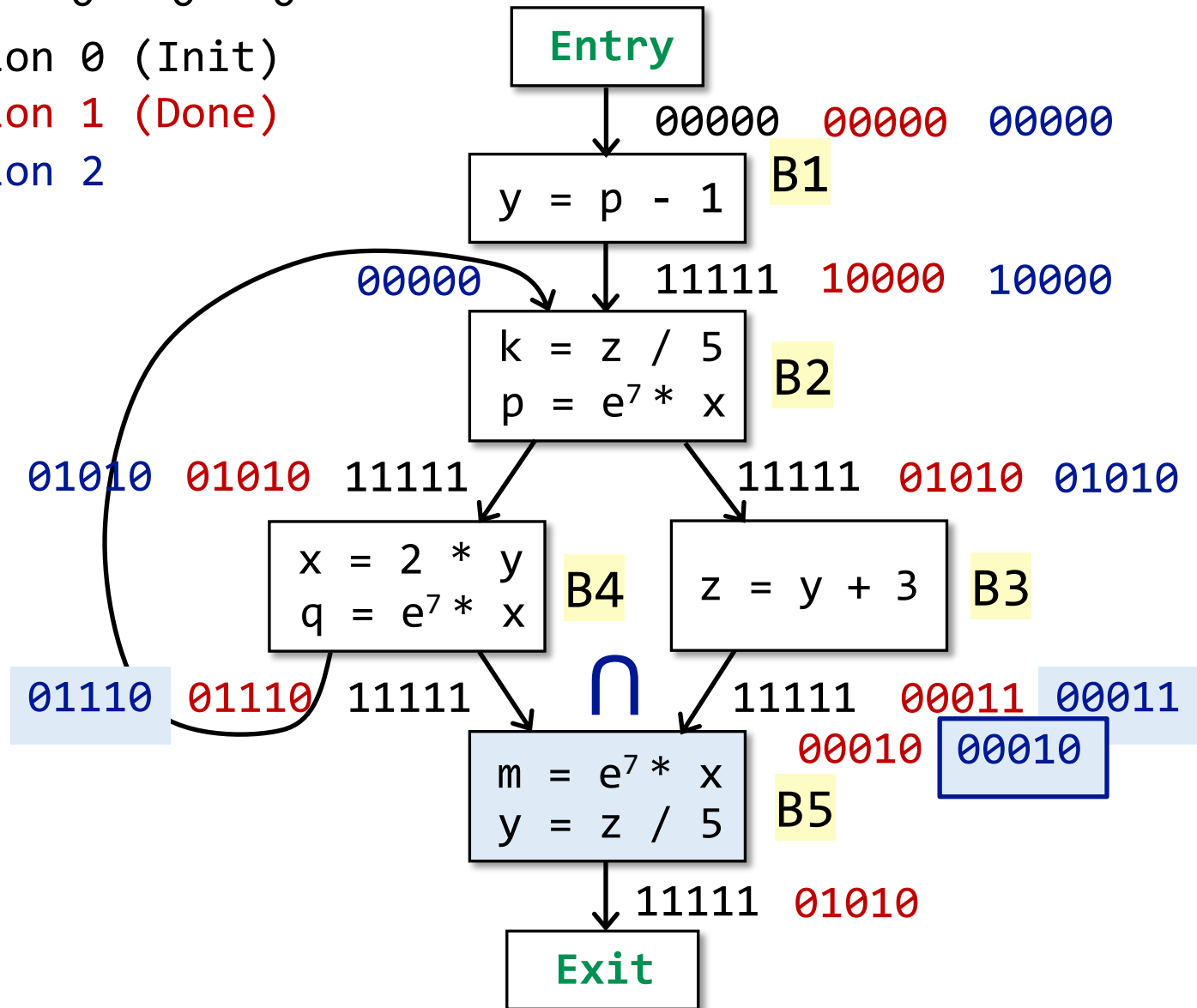


$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2



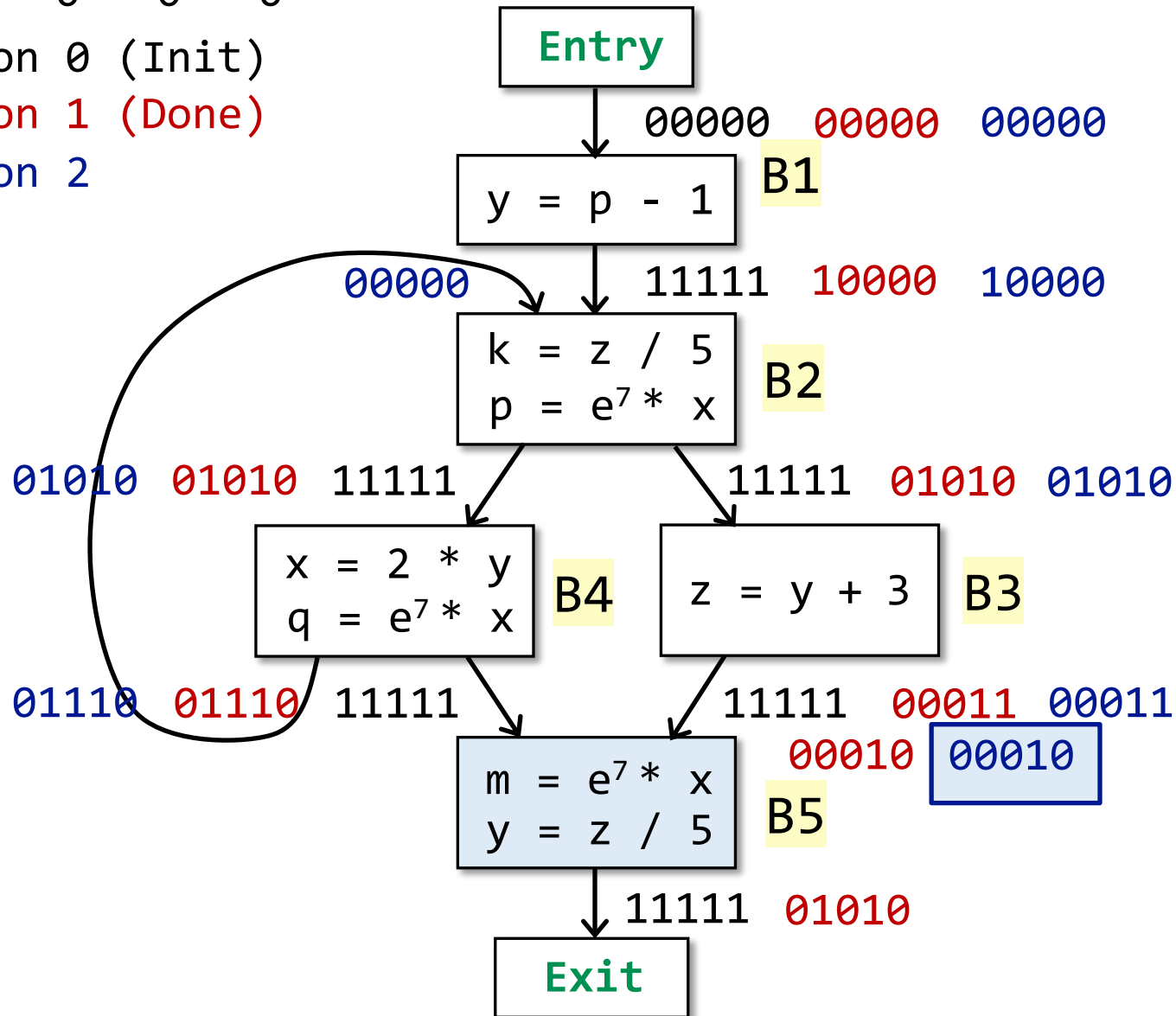
$p-1$ $z/5$ $2*y$ e^7*x $y+3$

0 0 0 0 0

Iteration 0 (Init)

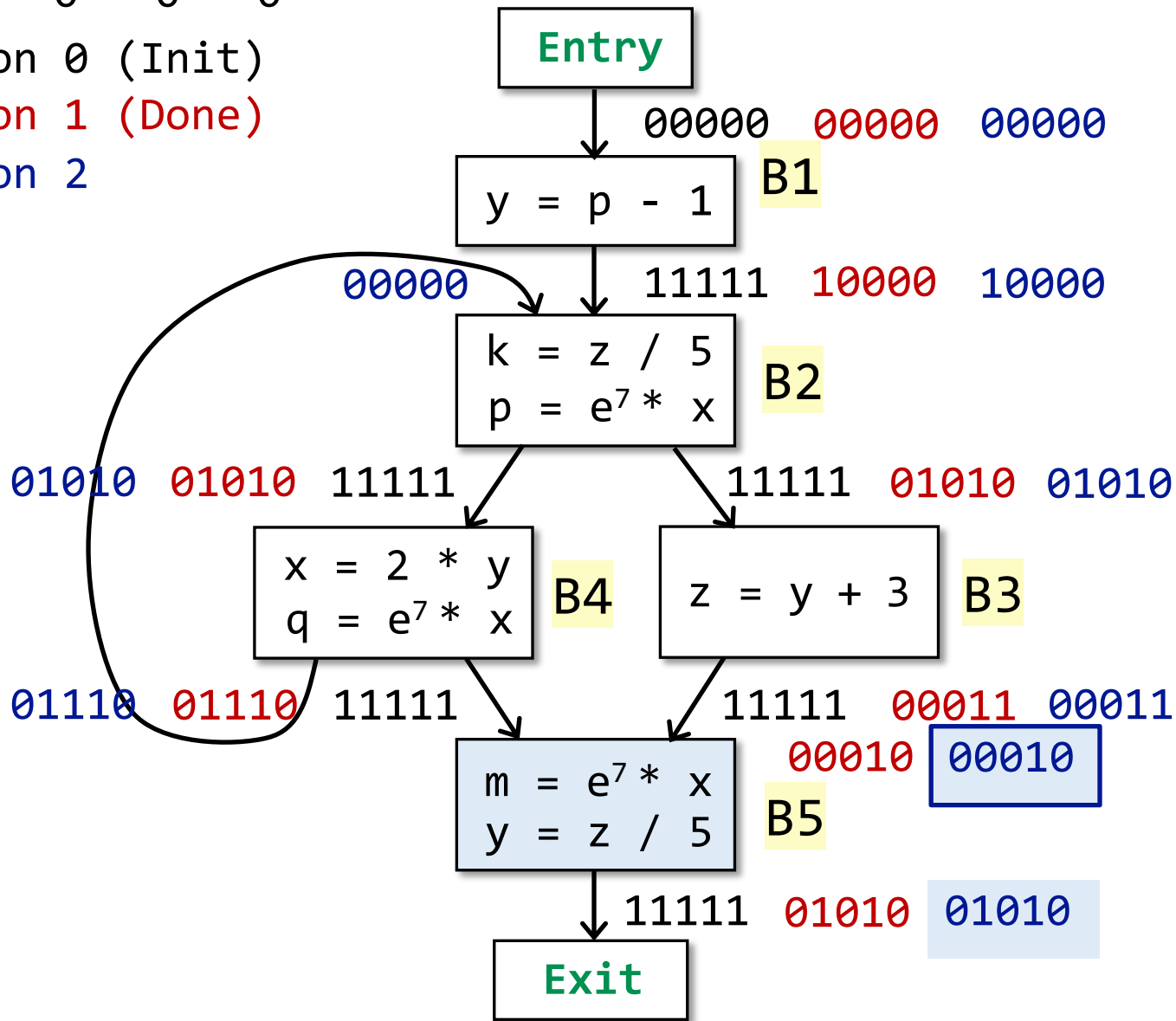
Iteration 1 (Done)

Iteration 2



$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)
 Iteration 1 (Done)
 Iteration 2



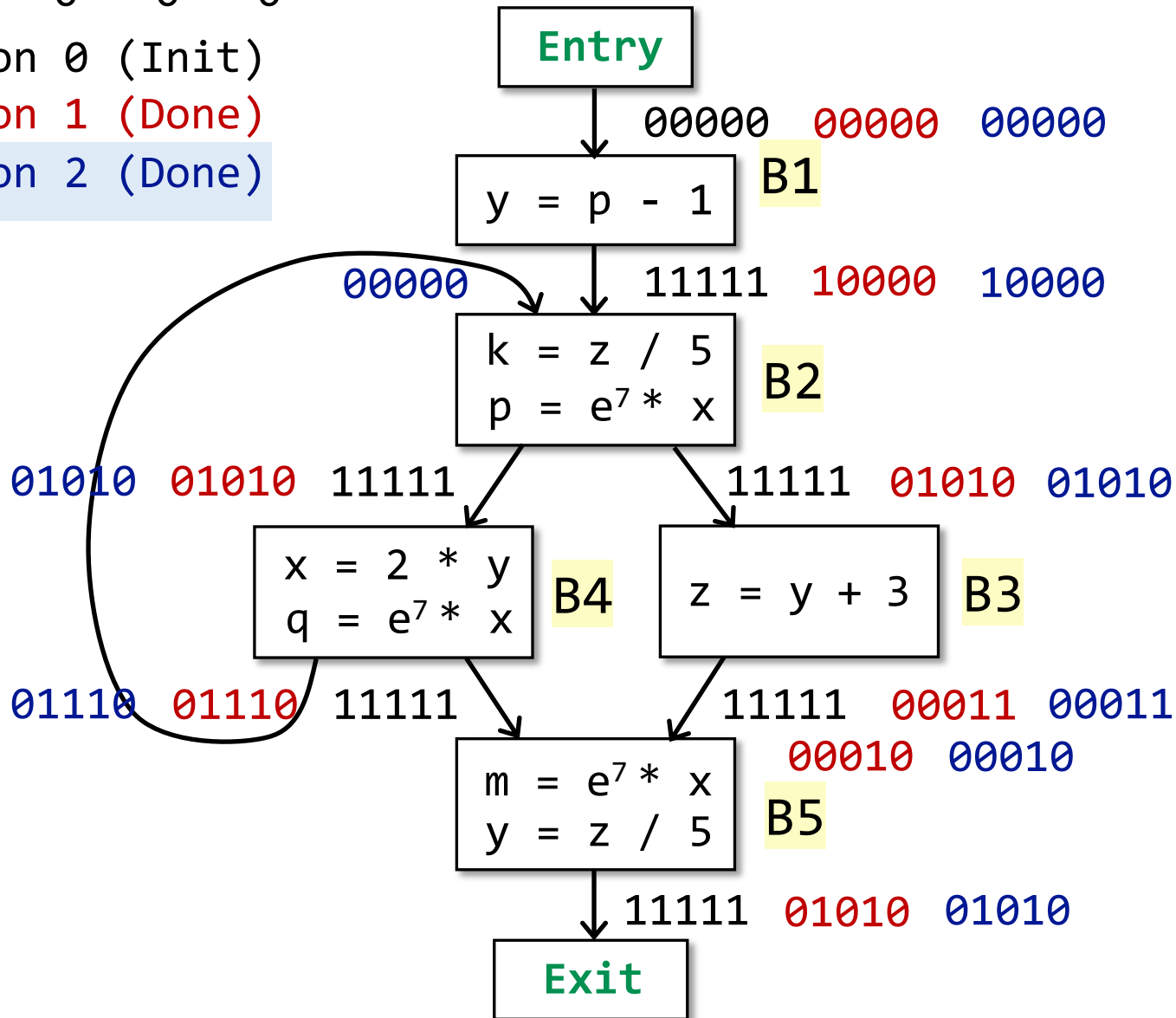
$p-1$ $z/5$ $2*y$ e^7*x $y+3$

0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)



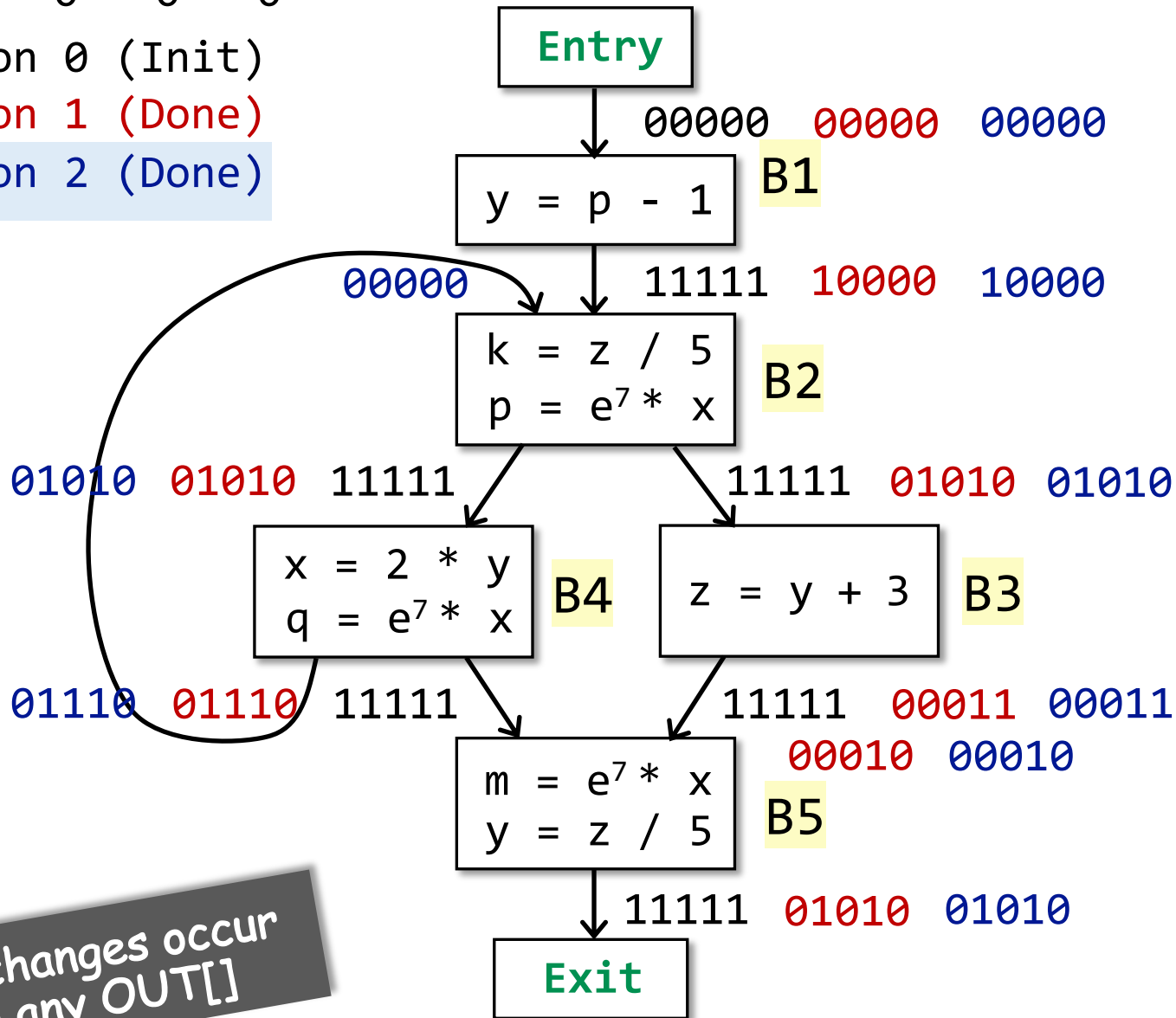
$p-1$ $z/5$ $2*y$ e^7*x $y+3$

0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)



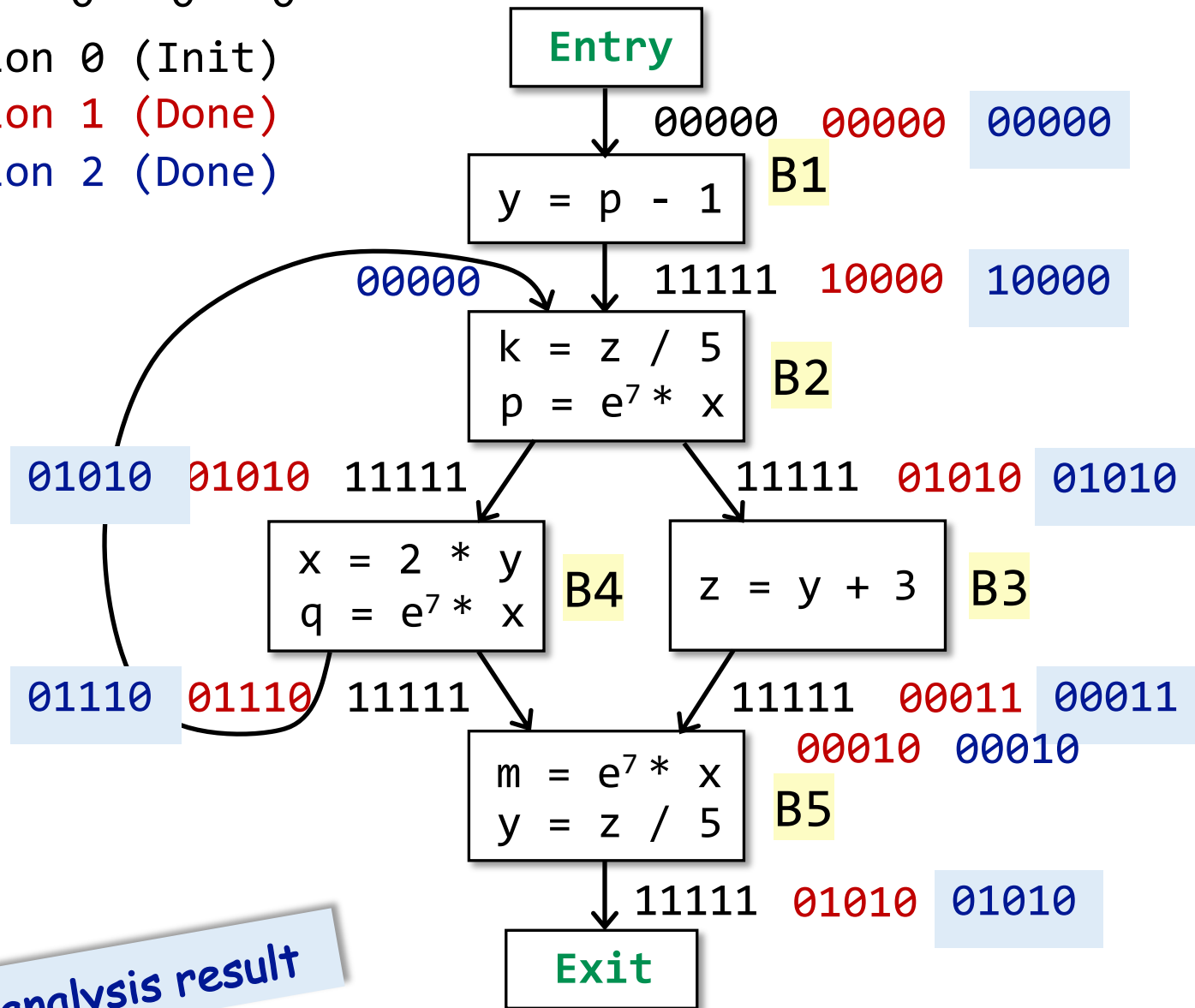
No changes occur
in any OUT[]

$p-1$ $z/5$ $2*y$ e^7*x $y+3$
 0 0 0 0 0

Iteration 0 (Init)

Iteration 1 (Done)

Iteration 2 (Done)



Final analysis result

Analysis Comparison

	Reaching Definitions	Live Variables	Available Expressions
Domain			
Direction			
May/Must			
Boundary			
Initialization			
Transfer function			
Meet			

Analysis Comparison

	Reaching Definitions	Live Variables	Available Expressions
Domain	Set of definitions	Set of variables	Set of expressions
Direction			
May/Must			
Boundary			
Initialization			
Transfer function			
Meet			

Analysis Comparison

	Reaching Definitions	Live Variables	Available Expressions
Domain	Set of definitions	Set of variables	Set of expressions
Direction	Forwards	Backwards	Forwards
May/Must			
Boundary			
Initialization			
Transfer function			
Meet			

Analysis Comparison

	Reaching Definitions	Live Variables	Available Expressions
Domain	Set of definitions	Set of variables	Set of expressions
Direction	Forwards	Backwards	Forwards
May/Must	May	May	Must
Boundary			
Initialization			
Transfer function			
Meet			

Analysis Comparison

	Reaching Definitions	Live Variables	Available Expressions
Domain	Set of definitions	Set of variables	Set of expressions
Direction	Forwards	Backwards	Forwards
May/Must	May	May	Must
Boundary	$\text{OUT}[\text{entry}] = \emptyset$	$\text{IN}[\text{exit}] = \emptyset$	$\text{OUT}[\text{entry}] = \emptyset$
Initialization			
Transfer function			
Meet			

Analysis Comparison

	Reaching Definitions	Live Variables	Available Expressions
Domain	Set of definitions	Set of variables	Set of expressions
Direction	Forwards	Backwards	Forwards
May/Must	May	May	Must
Boundary	$\text{OUT}[\text{entry}] = \emptyset$	$\text{IN}[\text{exit}] = \emptyset$	$\text{OUT}[\text{entry}] = \emptyset$
Initialization	$\text{OUT}[B] = \emptyset$	$\text{IN}[B] = \emptyset$	$\text{OUT}[B] = U$
Transfer function			
Meet			

Analysis Comparison

	Reaching Definitions	Live Variables	Available Expressions
Domain	Set of definitions	Set of variables	Set of expressions
Direction	Forwards	Backwards	Forwards
May/Must	May	May	Must
Boundary	$OUT[entry] = \emptyset$	$IN[exit] = \emptyset$	$OUT[entry] = \emptyset$
Initialization	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$
Transfer function	$OUT/IN = gen\ U\ (IN/OUT - kill)$		
Meet			

Analysis Comparison

	Reaching Definitions	Live Variables	Available Expressions
Domain	Set of definitions	Set of variables	Set of expressions
Direction	Forwards	Backwards	Forwards
May/Must	May	May	Must
Boundary	$OUT[entry] = \emptyset$	$IN[exit] = \emptyset$	$OUT[entry] = \emptyset$
Initialization	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$
Transfer function	$OUT/IN = gen \ U \ (IN/OUT - kill)$		
Meet	U	U	$\cap$

Analysis Comparison

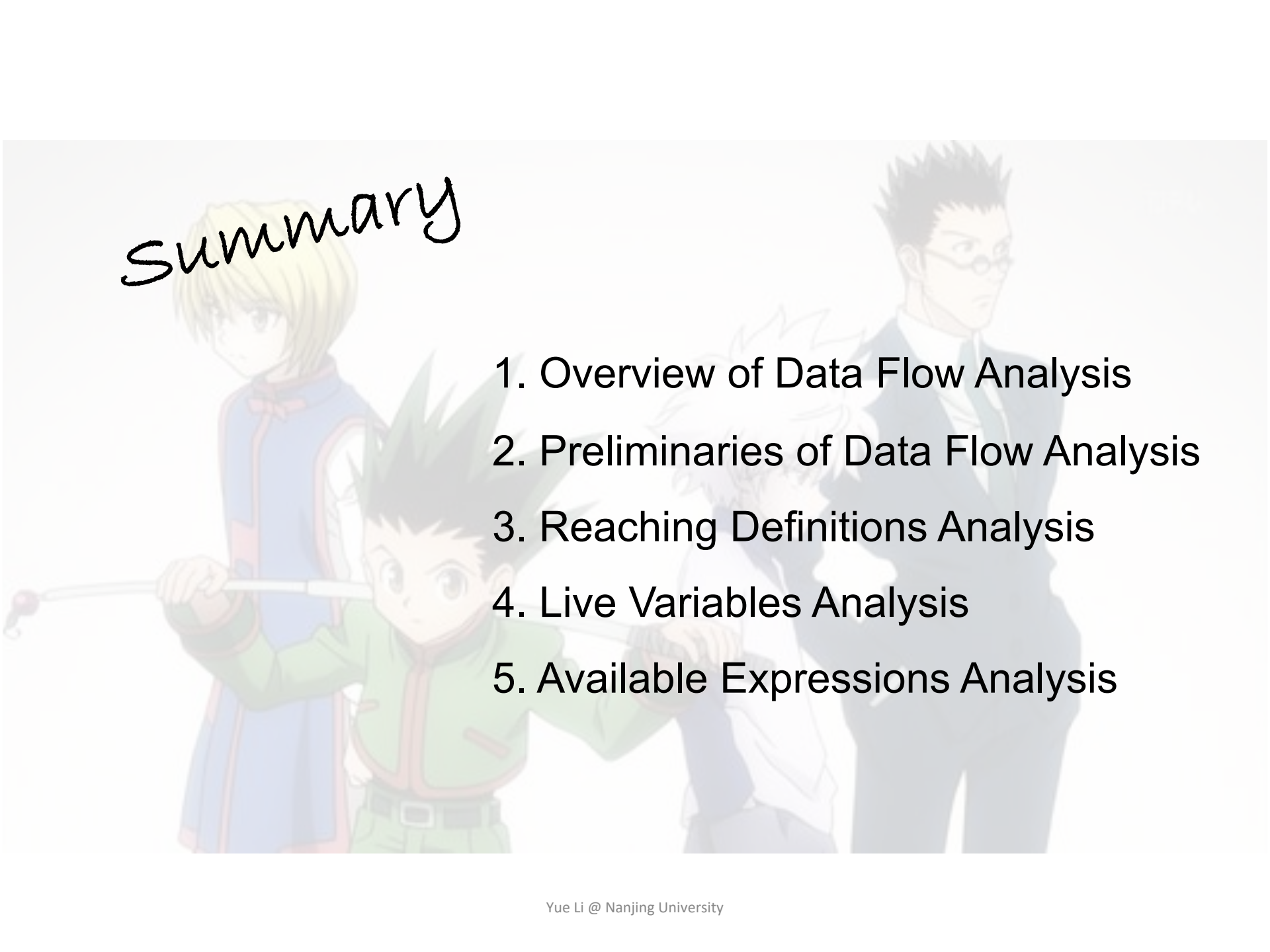
	Reaching Definitions	Live Variables	Available Expressions
Domain	Set of definitions	Set of variables	Set of expressions
Direction	Forwards	Backwards	Forwards
May/Must	May	May	Must
Boundary	$\text{OUT}[\text{entry}] = \emptyset$	$\text{IN}[\text{exit}] = \emptyset$	$\text{OUT}[\text{entry}] = \emptyset$
Initialization	$\text{OUT}[B] = \emptyset$	$\text{IN}[B] = \emptyset$	$\text{OUT}[B] = U$
Transfer function	$\text{OUT/IN} = \text{gen } U \text{ (IN/OUT - kill)}$		
Meet	U	U	$\cap$

Analysis Comparison

According to the meaning of the analysis
We'll draw a theoretical framework to systematically explain them in next lectures

	Reaching Definitions	Live Variables	Available Expressions
Domain	Set of definitions	Set of variables	Set of expressions
Direction	Forwards	Backwards	Forwards
May/Must	May	May	Must
Boundary	$OUT[entry] = \emptyset$	$IN[exit] = \emptyset$	$OUT[entry] = \emptyset$
Initialization	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$
Transfer function	$OUT/IN = gen\ U\ (IN/OUT - kill)$		
Meet	U	U	$\cap$

Summary



1. Overview of Data Flow Analysis
2. Preliminaries of Data Flow Analysis
3. Reaching Definitions Analysis
4. Live Variables Analysis
5. Available Expressions Analysis

The X You Need To Understand in This Lecture

- Understand the three data flow analyses:
 - reaching definitions
 - live variables
 - available expressions
- Can tell the differences and similarities of the three data flow analyses
- Understand the iterative algorithm and can tell why it is able to terminate

注意注意！
划重点了！



Assignment One:
Live variable analysis and iterative solver

Evaluation Criteria

- Coding Assignments 40% <http://tai-e.pascal-lab.net>
- Final Exam 60%

OJ注意事项:

- 用学校邮箱注册
- 填写Student ID
- Nickname用真实姓名
- 抄袭：第一次发现，该次作业0分
第二次发现，课程0分
- 不交：得5%诚信分
- 迟交：扣得分5% x D, D为迟交天数
- 每帮助找出1个新的有效test case, 实验分数+1